DEVOPS ENGINEERING MASTERY

COMPUTER SYSTEMS & LINUX ENGINEERING

কম্পিউটার সিস্টেম, Operating Systems ও Linux Administration

VOLUME 01 • FIRST EDITION

```
nahid@ubuntu-server:~$ uname -a
Linux ubuntu-server 6.8.0 • x86_64 • Ubuntu 24.04 LTS
nahid@ubuntu-server:~$ systemctl status nginx
● nginx.service – active (running)
nahid@ubuntu-server:~$ sudo apt update && sudo apt upgrade -y
```

Ubuntu Server 24.04 LTS (Noble Numbat) — Primary hands-on environment

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

20 Chapters • 15 Guided Labs • 3 Major Projects • 100 MCQs

Computer Systems & Linux Engineering

DEVOPS ENGINEERING MASTERY — Volume 01
কম্পিউটার সিস্টেম, Operating Systems ও Linux Administration

First Edition — Published 8 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS (Noble Numbat, released April 2024, supported until April 2029). All commands in this book were written for Ubuntu 24.04 LTS unless a section explicitly says otherwise. Debian, Fedora/RHEL-family and other distributions are compared where the difference matters. Version-specific instructions are labelled as such; general Linux concepts are distinguished from Ubuntu-specific implementation.

How to use this book. Read Parts I–II in order — they build the mental model everything else rests on. Parts III–IV are best read with a running Ubuntu Server virtual machine beside you: do every hands-on exercise and every lab in Appendix A before moving on. Do not copy commands blindly; read the explanation under each command first. Destructive commands are always flagged with a WARNING box and a safe alternative. Sample terminal outputs are illustrative — your output will differ in dates, usernames, PIDs and sizes, and that is expected.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test VM, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Commands that change a system should first be practised in a virtual machine or a disposable test server, never directly on a production machine. The author and publisher accept no liability for data loss caused by running commands without a backup.

Who this book is for

A Bangladeshi learner who knows some programming but has incomplete computer-systems and Linux foundations, and no prior professional Linux administration experience. A university student or junior software engineer who wants to administer a small Linux server independently — and to understand *why* each step works, not only which command to type.

Learning outcomes

By the end of this book you will be able to: explain computer architecture and how a computer executes programs; explain CPU, RAM, storage, I/O, operating systems, kernels and system calls; explain Linux

architecture, distributions, the boot process and the filesystem; work confidently in the Linux terminal and shell; manage files, users, groups, permissions and processes; install and manage software packages; manage services with systemd; inspect storage, memory, CPU, processes and logs; diagnose common Linux problems systematically; apply basic server security, backups and recovery; and independently administer a small Linux server.

সূচিপত্র — Contents

PART I — Understanding Computer Systems

Chapter 01 — How Computers Actually Work 7

Chapter 02 — CPU, Memory and Storage Architecture 14

Chapter 03 — Operating Systems: From First Principles 20

Chapter 04 — Kernel, User Space and System Calls 27

Chapter 05 — Processes, Threads and Memory Management 33

PART II — Linux Foundations

Chapter 06 — Introduction to Linux and Its Ecosystem 41

Chapter 07 — Linux Installation, Distributions and Boot Process 47

Chapter 08 — Linux Filesystem Hierarchy 53

Chapter 09 — Terminal, Shell and Command-Line Fundamentals 59

Chapter 10 — Essential Linux Commands and Text Processing 66

PART III — Linux Administration

Chapter 11 — Users, Groups and Permission Management 74

Chapter 12 — File Ownership, ACLs and Sudo 80

Chapter 13 — Process and Job Management 86

Chapter 14 — Package Management and Software Installation 92

Chapter 15 — systemd, Services and Boot Management 99

PART IV — Operating and Troubleshooting Linux

Chapter 16 — Disk, Filesystem and Storage Management 106

Chapter 17 — CPU, RAM and System Resource Monitoring 112

Chapter 18 — Logs, Diagnostics and Troubleshooting 117

Chapter 19 — Linux Security and System Maintenance 123

Chapter 20 — Backup, Recovery and Practical Server Administration 129

Appendices

Appendix A — Guided Labs (15 Labs) 136

Appendix B — Independent Assignments (10) 157

Appendix C — Major Projects (A, B, C — TechNova) 160

Appendix D — Final Examination & Assessment Rubric 165

Appendix E — Glossary (117 Terms) 168

Appendix F — References 174

Chapter-end quizzes (100 MCQs), short-answer and command-writing exercises, and the troubleshooting case studies are inside each chapter and Appendix D. Their answers — with explanations, not just option letters — are in the separate companion volume, *Quiz & Answer Guide*. The hands-on material is also collected in the separate *Practical Lab & Assignment Workbook*.

PART I • DEVOPS ENGINEERING MASTERY

Understanding Computer Systems

কম্পিউটার কীভাবে কাজ করে — প্রথম নীতি থেকে: হার্ডওয়্যার, CPU, মেমোরি, অপারেটিং সিস্টেম, কার্নেল ও প্রসেস।

অধ্যায় ১ • Part I — Understanding Computer Systems

Chapter 01: How Computers Actually Work — কম্পিউটার আসলে কীভাবে কাজ করে

শুরুতে: একটি বাস্তব দৃশ্য

ঢাকার একটি বিশ্ববিদ্যালয়ের ছাত্র রাফি তার ল্যাপটপে ব্রাউজার খুলে একটি ওয়েবসাইটে ঢুকল। সে মাউসে ক্লিক করল, কীবোর্ডে কিছু লিখল, আর কয়েক সেকেন্ডের মধ্যে পর্দায় ছবি, লেখা আর ভিডিও ভেসে উঠল। বাইরে থেকে দেখলে মনে হয় যাদু। কিন্তু ভেতরে যা ঘটছে তা যাদু নয় — তা হলো কোটি কোটি ক্ষুদ্র বৈদ্যুতিক সুইচের সুশৃঙ্খল নাচ, যা প্রতি সেকেন্ডে বিলিয়ন বার অবস্থা বদলাচ্ছে। এই অধ্যায়ের লক্ষ্য সেই যাদুর পেছনের যুক্তিটি বোঝা: কম্পিউটার আসলে কী, সে কীভাবে তথ্য ধারণ করে, কীভাবে নির্দেশনা পড়ে এবং কীভাবে একটি সাধারণ ক্লিক শেষ পর্যন্ত পর্দায় ছবি হয়ে ওঠে।

অনেক শিক্ষার্থী প্রোগ্রামিং শেখে কিন্তু কম্পিউটার কীভাবে কাজ করে তা জানে না। ফলে তারা কোড লিখতে পারে, কিন্তু কোড ধীর কেন চলে, মেমোরি কেন শেষ হয়ে যায়, বা প্রোগ্রাম কেন হঠাৎ বন্ধ হয়ে যায় — তা ব্যাখ্যা করতে পারে না। একজন DevOps engineer-এর জন্য এই ভিত্তি অপরিহার্য, কারণ পরের অধ্যায়গুলোতে আমরা CPU, মেমোরি, অপারেটিং সিস্টেম আর Linux নিয়ে কাজ করব। ভিত্তি দুর্বল হলে সার্ভার পরিচালনার সিদ্ধান্তগুলো অনুমানের উপর দাঁড়াবে।

> **CONCEPT:** কম্পিউটার কোনো বুদ্ধিমান প্রাণী নয়। সে শুধু দুটি কাজ অসাধারণ দ্রুততায় করতে পারে — সংখ্যা মনে রাখা এবং খুব সহজ নির্দেশনা অনুসরণ করা। সব জটিলতা আসে এই দুটি সহজ ক্ষমতার কোটি কোটি পুনরাবৃত্তি থেকে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- কম্পিউটার, hardware ও software-এর সংজ্ঞা নিজের ভাষায় ব্যাখ্যা করতে।
- Input, processing, output ও storage — এই চারটি ধাপ দিয়ে যেকোনো কম্পিউটিং কাজ বিশ্লেষণ করতে।
- Bit ও byte ব্যাখ্যা করতে এবং binary সংখ্যা decimal-এ রূপান্তর করতে।
- একটি প্রোগ্রাম কীভাবে লেখা থেকে execution পর্যন্ত যায়, তার ধাপগুলো বর্ণনা করতে।
- ব্যবহারকারী অ্যাপ্লিকেশন খুললে ভেতরে কী কী ঘটে, তার ক্রম ব্যাখ্যা করতে।
- Data, information, instruction ও program-এর পার্থক্য চিহ্নিত করতে।

Prerequisites — পূর্বজ্ঞান

- কম্পিউটার বা ল্যাপটপ ব্যবহারের সাধারণ অভিজ্ঞতা (ফাইল খোলা, ব্রাউজার চালানো)।
- দশমিক সংখ্যা পদ্ধতি (decimal) সম্পর্কে প্রাথমিক ধারণা।
- কোনো একটি প্রোগ্রামিং ভাষায় `print` বা `hello world` লেখার অভিজ্ঞতা থাকলে ভালো, না থাকলেও চলবে।

১.১ কম্পিউটার আসলে কী?

সহজ সংজ্ঞা: কম্পিউটার হলো এমন একটি যন্ত্র যা **input নেয়, নির্দেশনা অনুযায়ী তা প্রক্রিয়া করে, ফলাফল output দেয় এবং প্রয়োজনে তথ্য সংরক্ষণ করে**। এই সংজ্ঞার প্রতিটি শব্দ গুরুত্বপূর্ণ। ক্যালকুলেটরও কম্পিউটার — সে সংখ্যা নেয়,

যোগ করে, ফল দেখায়। আপনার ফোন, ল্যাপটপ, রাউটার, এমনকি আধুনিক ওয়াশিং মেশিনের ভেতরের চিপও একই সংজ্ঞায় পড়ে।

একটি উপমা ভাবুন: একটি রান্নাঘর। রান্নাঘরে কাঁচামাল আসে (input), রাঁধুনি রেসিপি অনুযায়ী রান্না করে (processing), খাবার পরিবেশন করা হয় (output), আর অবশিষ্ট উপকরণ ফ্রিজে রাখা হয় (storage)। কম্পিউটারেও একই কাঠামো। পার্থক্য শুধু গতিতে আর নির্ভুলতায় — রাঁধুনি একটি রেসিপি অনুসরণ করতে মিনিট নেয়, কম্পিউটার প্রতি সেকেন্ডে কোটি নির্দেশনা অনুসরণ করে এবং কখনো ক্লান্ত হয় না, তবে রেসিপিতে ভুল থাকলে সে ভুলটিই নিখুঁতভাবে করবে।

এখানেই প্রথম গুরুত্বপূর্ণ সত্য: **কম্পিউটার নিজে কিছু বোঝে না, কিছু সিদ্ধান্ত নেয় না।** সে শুধু মানুষের লেখা নির্দেশনা (program) অক্ষরে অক্ষরে পালন করে। Artificial Intelligence-এর ক্ষেত্রেও একই কথা — মডেল যত জটিলই হোক, ভেতরে চলছে সংখ্যার গণনা আর নির্দেশনার অনুসরণ।

১.২ Hardware ও Software

Hardware হলো কম্পিউটারের ভৌত অংশ — যা ছোঁয়া যায়: CPU, RAM, hard disk, কীবোর্ড, মনিটর, motherboard। **Software** হলো নির্দেশনার সমষ্টি — যা ছোঁয়া যায় না: operating system, ব্রাউজার, গেম, আপনার লেখা Python প্রোগ্রাম।

দুটির সম্পর্ক বোঝার সবচেয়ে ভালো উপায়: hardware হলো শরীর, software হলো মন বা নির্দেশনা। শরীর ছাড়া মন কাজ করতে পারে না, আবার নির্দেশনা ছাড়া শরীর অচল। নতুন ল্যাপটপ কিনে operating system ছাড়া চালু করলে পর্দায় শুধু একটি বার্তা আসবে — bootable device পাওয়া যায়নি। আবার operating system আছে কিন্তু RAM নষ্ট, তাহলে কম্পিউটার চালুই হবে না। DevOps-এ এই পার্থক্য প্রতিদিন কাজে লাগে: কোনো সমস্যা hardware-এর নাকি software-এর, তা আলাদা করতে না পারলে troubleshooting অন্ধকারে ঢিল ছোঁড়া হয়ে যায়।

বিষয়	Hardware	Software
সংজ্ঞা	ভৌত যন্ত্রাংশ	নির্দেশনা ও তথ্যের সমষ্টি
উদাহরণ	CPU, RAM, SSD, motherboard	Ubuntu, Firefox, VS Code
নষ্ট হলে	বদলাতে হয়, মেরামত খরচ বেশি	আবার install বা update করা যায়
দেখা যায়	চোখে দেখা ও ছোঁয়া যায়	শুধু তার প্রভাব দেখা যায়

১.৩ Input, Processing, Output ও Storage

প্রতিটি কম্পিউটিং কাজ চার ধাপে ভাঙা যায়। এই মডেলকে **IPO+S model** বলা যায়:

1. **Input:** তথ্য কম্পিউটারে ঢোকে — কীবোর্ডে টাইপ, মাউসের ক্লিক, মাইক্রোফোনের শব্দ, নেটওয়ার্ক থেকে আসা data packet, সেন্সরের পাঠ।
2. **Processing:** CPU নির্দেশনা অনুযায়ী তথ্য নিয়ে গণনা করে, সিদ্ধান্ত নেয় (if-else), তুলনা করে।
3. **Output:** ফলাফল বাইরে যায় — পর্দায় ছবি, স্পিকারে শব্দ, প্রিন্টারে কাগজ, নেটওয়ার্কে পাঠানো উত্তর।
4. **Storage:** তথ্য পরে ব্যবহারের জন্য রাখা হয় — RAM-এ সাময়িকভাবে, disk-এ স্থায়ীভাবে।

উদাহরণ: আপনি Facebook-এ একটি ছবিতে like দিলেন। Input — মাউসের ক্লিক ও তার অবস্থান। Processing — ব্রাউজার বুঝল কোন বোতামে ক্লিক হয়েছে, সার্ভারকে জানাল, সার্ভারের CPU ডেটাবেসে like-এর সংখ্যা এক বাড়াল।

Output — পর্দায় বোতামটি নীল হয়ে গেল, সংখ্যা বদলে গেল। Storage — like-টি ডেটাবেসে স্থায়ীভাবে সংরক্ষিত হলো, যাতে আগামীকালও দেখা যায়।

IPO+S চক্র

Input (কীবোর্ড / মাউস / নেটওয়ার্ক) → Processing (CPU) → Output (মনিটর / স্পিকার / নেটওয়ার্ক)

↑ ↓

Storage (RAM — সাময়িক, Disk — স্থায়ী) — Processing-এর দুই পাশে কাজ করে

১.৪ Binary: কম্পিউটার কেন শুধু ০ আর ১ বোঝে

কম্পিউটারের ভেতরে কোনো অক্ষর, ছবি বা শব্দ সরাসরি থাকে না। থাকে শুধু বৈদ্যুতিক অবস্থা — সুইচ অন অথবা অফ, ভোল্টেজ বেশি অথবা কম। এই দুটি অবস্থাকে আমরা লিখি **0** আর **1** দিয়ে। একটি 0 বা 1-কে বলে **bit** (binary digit)। আটটি bit মিলে হয় একটি **byte**।

কেন দুটি অবস্থা? কারণ দুটি অবস্থা নির্ভরযোগ্যভাবে আলাদা করা সহজ। ভোল্টেজ একটু কম-বেশি হলেও অন নাকি অফ, তা বোঝা যায়। দশটি অবস্থা ব্যবহার করলে সামান্য নয়েজেই ভুল হতো। এই সরলতাই কম্পিউটারকে নির্ভরযোগ্য করেছে।

Binary সংখ্যা পড়ার নিয়ম: প্রতিটি অবস্থানের মান ২-এর ঘাত। ডান থেকে বামে: ১, ২, ৪, ৮, ১৬, ৩২, ৬৪, ১২৮। যেমন **01000001** = ৬৪ + ১ = ৬৫। ASCII নামের একটি পুরনো মান অনুযায়ী ৬৫ মানে বড় হাতের **A**। অর্থাৎ A অক্ষরটি কম্পিউটারে সংরক্ষিত হয় 01000001 হিসেবে। একটি ছবি, একটি গান, একটি ভিডিও — সবকিছু শেষ পর্যন্ত ০ আর ১-এর দীর্ঘ সারি। পার্থক্য শুধু এই সারিকে কীভাবে ব্যাখ্যা করতে হবে, সেই নিয়মে (encoding)।

একক	মান	বাস্তব ধারণা
1 bit	0 বা 1	একটি সুইচ
1 byte	8 bit	একটি অক্ষর (যেমন A)
1 KB	1,024 byte	এক পৃষ্ঠা লেখা
1 MB	1,024 KB	একটি ছোট ছবি
1 GB	1,024 MB	একটি সিনেমার অংশ / কয়েকশো ছবি
1 TB	1,024 GB	একটি ল্যাপটপের পুরো disk

WARNING: দোকানে লেখা 1 TB আর operating system-এ দেখানো 931 GB — দুটোই সঠিক, কিন্তু একক আলাদা। নির্মাতারা 1 TB = 1,000 GB ধরে, আর কম্পিউটার 1 TiB = 1,024 GiB ধরে। এটি প্রতারণা নয়, এককের পার্থক্য। বইয়ের পরের অধ্যায়ে disk দেখার সময় এটি মনে রাখবেন।

১.৫ প্রোগ্রাম কীভাবে চলে: Source Code থেকে Execution

আপনি Python-এ লিখলেন `print("Hello")`। কম্পিউটার এই ইংরেজি-সদৃশ লেখা সরাসরি বোঝে না। তাকে বুঝতে হয় **machine code** — ০ আর ১-এর নির্দেশনা, যা CPU সরাসরি পড়তে পারে। মাঝখানে কয়েকটি ধাপ আছে:

1. **লেখা (Source code):** মানুষের পড়ার মতো ভাষায় নির্দেশনা।
2. **অনুবাদ:** Compiler (C, Go) পুরো প্রোগ্রাম একবারে machine code-এ বদলায়; Interpreter (Python) লাইন ধরে ধরে চালানোর সময় অনুবাদ করে।
3. **Load:** Operating system প্রোগ্রামটিকে disk থেকে RAM-এ তোলে।
4. **Execute:** CPU RAM থেকে একের পর এক নির্দেশনা পড়ে ও পালন করে — এই চক্রকে বলে **fetch–decode–execute cycle**।
5. **Output:** ফলাফল পর্দায় বা ফাইলে যায়।

Fetch–decode–execute চক্রটি পুরো কম্পিউটিং-এর হৃদয়। CPU memory থেকে পরের নির্দেশনা আনে (fetch), বোঝে সেটি কী করতে বলছে (decode), তারপর করে (execute)। তারপর পরের নির্দেশনা। প্রতি সেকেন্ডে এটি কোটি কোটি বার ঘটে। একটি ভিডিও চালানো, একটি ওয়েবসাইট খোলা — সবই এই একই চক্রের পুনরাবৃত্তি, শুধু নির্দেশনার সংখ্যা আর জটিলতা আলাদা।

Fetch–Decode–Execute চক্র

Memory থেকে নির্দেশনা আনা (Fetch) → নির্দেশনা বোঝা (Decode) → কাজটি করা (Execute) → পরের নির্দেশনায় যাওয়া ↺

এই চক্র প্রতি সেকেন্ডে বিলিয়ন বার ঘোরে।

১.৬ ব্যবহারকারী অ্যাপ্লিকেশন খুললে আসলে কী ঘটে?

একটি সম্পূর্ণ walkthrough — আপনি Ubuntu-তে Firefox আইকনে ক্লিক করলেন:

1. মাউস ক্লিক একটি electrical signal হয়ে operating system-এর কাছে যায়। OS বোঝে ক্লিকটি কোন আইকনে হয়েছে।
2. OS disk থেকে Firefox-এর program file খুঁজে বের করে RAM-এ load করে। একই সঙ্গে প্রয়োজনীয় library গুলোও load হয়।
3. CPU Firefox-এর প্রথম নির্দেশনা থেকে execution শুরু করে। Firefox OS-কে বলে: আমাকে একটি window দাও, নেটওয়ার্কে সংযোগ দাও।
4. OS window তৈরি করে, পর্দায় আঁকার দায়িত্ব নেয়, নেটওয়ার্ক card-কে নির্দেশ দেয়।
5. Firefox ইন্টারনেট থেকে ওয়েবসাইটের তথ্য আনে, তা ব্যাখ্যা করে পর্দায় সাজায়।
6. আপনি স্ক্রল করলে প্রতিটি স্ক্রল আবার input হয়ে একই চক্র ঘোরে — শুধু এবার প্রোগ্রামটি RAM-এ already আছে, তাই দ্রুত মনে হয়।

এখানে লক্ষ করুন: অ্যাপ্লিকেশন একা কাজ করে না। প্রতিটি পদক্ষেপে সে operating system-এর সাহায্য নেয় — ফাইল পড়তে, পর্দায় আঁকতে, নেটওয়ার্ক ব্যবহার করতে। এই সাহায্য চাওয়ার ব্যবস্থার নাম **system call**, যা Chapter 04-এ বিস্তারিত আসবে।

REAL WORLD: বাংলাদেশের একটি ছোট software company-তে junior developer-এর ল্যাপটপে একটি web app ধীর চলছিল। কারণ খুঁজতে গিয়ে দেখা গেল RAM প্রায় পূর্ণ, আর browser-এ ৩০টি tab খোলা। প্রতিটি tab একটি আলাদা process হিসেবে RAM দখল করে রেখেছে। সমস্যাটি কোডের নয় — resource-এর। এই বইয়ের Chapter 05 ও 17-তে এই বিশ্লেষণ শেখা হবে।

১.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Computer	Input নিয়ে নির্দেশনা অনুযায়ী প্রক্রিয়া করে output দেওয়া যন্ত্র	১
Hardware	কম্পিউটারের ভৌত অংশ	১
Software / Program	নির্দেশনার সমষ্টি	১
Bit / Byte	তথ্যের ক্ষুদ্রতম একক (0/1); ৮ bit = ১ byte	১
Binary	শুধু ০ ও ১ দিয়ে সংখ্যা প্রকাশের পদ্ধতি	১
Machine Code	CPU সরাসরি বুঝতে পারে এমন ০/১ নির্দেশনা	১
Fetch–Decode–Execute	CPU-র নির্দেশনা পড়া, বোঝা ও পালন করার চক্র	১
Input / Output	কম্পিউটারে তথ্য ঢোকা / ফলাফল বাইরে যাওয়া	১

১.৮ হাতে-কলমে অনুশীলন

LAB: আপনার কম্পিউটারকে চেনা — কোনো install ছাড়াই। Ubuntu থাকলে terminal খুলুন; Windows/macOS থাকলে পরে Chapter 07-এর VM ব্যবহার করবেন।

1. Terminal খুলুন। `echo $0` লিখে Enter চাপুন — কোন shell চলছে দেখাবে।
2. `uname -a` লিখুন — operating system ও kernel-এর পরিচয় দেখাবে।
3. `ls /` লিখুন — root directory-র ভেতরে কী কী আছে দেখুন (Chapter 08-এ প্রতিটির মানে শিখবেন)।
4. একটি text file বানান: `echo "Hello Computer" > hello.txt`, তারপর `cat hello.txt` দিয়ে পড়ুন।
5. Binary অনুশীলন: কাগজে 01001000 01100101 01101100 01101100 01101111-কে decimal-এ বদলে ASCII table মিলিয়ে দেখুন — এটি কোন শব্দ?

উদাহরণ আউটপুট (illustrative):

```
$ uname -a
Linux ubuntu-server 6.8.0-31-generic #31-Ubuntu SMP x86_64 GNU/Linux
$ ls /
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr
var
```

```
$ cat hello.txt
Hello Computer
```

ব্যাখ্যা: `uname -a` জানাচ্ছে এটি Linux, kernel version 6.8.0-31। `ls /`-এর তালিকায় প্রতিটি নাম একটি directory — এগুলো Chapter 08-এ একে একে চেনা হবে। Binary অনুশীলনের উত্তর: 72 101 108 108 111 = **Hello**।

১.৯ Common Mistakes ও Troubleshooting

- **ভুল:** "RAM বেশি মানেই কম্পিউটার দ্রুত" — না। RAM বেশি হলে একসঙ্গে বেশি প্রোগ্রাম চলতে পারে, কিন্তু CPU ধীর হলে গণনা ধীরই থাকবে। Bottleneck খুঁজতে হয় (Chapter 02)।
- **ভুল:** Storage আর memory এক ভাবা। Disk-এ ফাইল থাকে স্থায়ীভাবে; RAM-এ থাকে শুধু চলমান কাজ। কম্পিউটার বন্ধ করলে RAM খালি হয়ে যায়।
- **ভুল:** Binary-কে শুধু গণিতের কৌতূহল ভাবা। বাস্তবে প্রতিটি file size, প্রতিটি network packet, প্রতিটি ছবি binary-র ব্যাখ্যা।
- **সমস্যা:** প্রোগ্রাম চালু হচ্ছে না — প্রথমে দেখুন ফাইলটি আছে কি না (`ls`), তারপর permission আছে কি না (Chapter 11)।

১.১০ Security Considerations

এই পর্যায়ে একটি মূল নীতি: **কম্পিউটার নির্দেশনা অন্ধভাবে পালন করে।** তাই ক্ষতিকর program (malware) চালালে সেও একই নিষ্ঠায় ক্ষতি করবে। অচেনা file download করে চালানো, অচেনা লিংকে ক্লিক করা — এগুলো technical সমস্যা নয়, trust-এর সমস্যা। DevOps-এ পরে শিখবেন: শুধু বিশ্বস্ত উৎস থেকে software install করতে হয় (Chapter 14)।

১.১১ Best Practices

- কোনো সমস্যা দেখলে প্রথমে IPO+S দিয়ে ভাঙুন: input ঠিক আছে? processing কোথায় আটকাচ্ছে? output কী বলছে? storage পূর্ণ কি না?
- File size-এর একক (KB/MB/GB) আর binary হিসাবের পার্থক্য মনে রাখুন।
- নতুন ধারণা শিখলে নিজের ভাষায় একবার লিখুন — না পারলে বোঝা হয়নি।

CHECK YOUR UNDERSTANDING: নিচের ৫টি প্রশ্নের উত্তর নিজে ভাবুন। উত্তর এই বইয়ের Answer Guide-এ আছে — আগে নিজে চেষ্টা করুন।

1. কম্পিউটারের চারটি মৌলিক কাজ কী কী? A. Input, processing, output, storage B. Typing, clicking, printing, saving C. CPU, RAM, disk, monitor D. Coding, compiling, running, debugging
2. 1 byte সমান কত bit? A. 2 bit B. 4 bit C. 8 bit D. 16 bit
3. Binary সংখ্যা 1010-এর decimal মান কত? A. 8 B. 10 C. 12 D. 20
4. Fetch–decode–execute চক্রটি কে সম্পন্ন করে? A. Hard disk B. CPU C. RAM D. Monitor
5. নিচের কোনটি software? A. RAM B. Keyboard C. Ubuntu operating system D. Motherboard

Knowledge Check — জ্ঞান যাচাই (উত্তর Answer Guide-এ)

1. কম্পিউটার তথ্য ধারণ করে কীভাবে? A) দশমিক সংখ্যায় B) ০ ও ১ (বাইনারি) দিয়ে C) অক্ষরে D) শব্দে

2. 1 byte সমান কত bit? A) 4 bit B) 8 bit C) 16 bit D) 1024 bit
3. প্রোগ্রাম চালু হলে তা কোথায় লোড হয়? A) HDD-তে থেকে যায় B) RAM-এ আসে C) CPU cache-এ স্থায়ী হয় D) মনিটরে
4. কীবোর্ড Input–Processing–Output–Storage মডেলে কোনটি? A) Output B) Storage C) Input D) Processing
5. CPU-র প্রধান কাজ কী? A) তথ্য স্থায়ীভাবে রাখা B) নির্দেশনা পড়ে কার্যকর করা C) পর্দায় ছবি আঁকা D) বিদ্যুৎ সরবরাহ করা

১.১২ Practical Assignment

Assignment 1.1: আপনার নিজের কম্পিউটার বা ফোনের জন্য IPO+S বিশ্লেষণ লিখুন (এক পৃষ্ঠা): একটি কাজ বাছুন (যেমন YouTube-এ ভিডিও দেখা), তার input, processing, output ও storage ধাপগুলো লিখুন, আর প্রতিটি ধাপে কোন hardware ও software জড়িত তা চিহ্নিত করুন। সঙ্গে binary-তে আপনার নামের প্রথম অক্ষরটি লিখুন।

১.১৩ Chapter Summary

কম্পিউটার হলো input নিয়ে নির্দেশনা অনুযায়ী প্রক্রিয়া করে output দেওয়া ও তথ্য সংরক্ষণ করা যন্ত্র। তার ভেতরে সব তথ্য binary — ০ আর ১। Program হলো নির্দেশনার সমষ্টি, যা source code থেকে machine code-এ অনুবাদ হয়ে RAM-এ load হয়, আর CPU fetch–decode–execute চক্রে তা পালন করে। অ্যাপ্লিকেশন খোলা থেকে ক্লিকের উত্তর — সবকিছু এই একই চক্রের পুনরাবৃত্তি, যেখানে operating system প্রতিটি পদক্ষেপে সাহায্য করে।

Revision Checklist

- ☐ IPO+S মডেল দিয়ে যেকোনো কাজ ভাঙতে পারি।
- ☐ Bit, byte, KB, MB, GB-এর সম্পর্ক বলতে পারি।
- ☐ Binary থেকে decimal রূপান্তর করতে পারি।
- ☐ Source code থেকে execution পর্যন্ত ধাপগুলো বলতে পারি।
- ☐ Fetch–decode–execute চক্র ব্যাখ্যা করতে পারি।

অধ্যায় ২ • Part I — Understanding Computer Systems

Chapter 02: CPU, Memory and Storage Architecture — CPU, মেমোরি ও স্টোরেজের স্থাপত্য

শুরুতে: একটি বাস্তব দৃশ্য

চট্টগ্রামের একটি startup-এ দুজন developer একই দামের দুটি ল্যাপটপ কিনল। একটিতে 16 GB RAM আর পুরনো hard disk, অন্যটিতে 8 GB RAM আর NVMe SSD। প্রথমজন ভাবল বেশি RAM মানেই দ্রুত। কিন্তু বাস্তবে দ্বিতীয় ল্যাপটপটি boot হয় ১০ সেকেন্ডে, project খোলে মুহূর্তে, আর প্রথমটি প্রতিটি কাজে অপেক্ষা করায়। কারণ performance কখনো একটি উপাদানের নয় — তা পুরো ব্যবস্থার ভারসাম্যের ফল। এই অধ্যায়ে আমরা কম্পিউটারের ভেতরের প্রতিটি প্রধান উপাদান — CPU, cache, RAM, storage — কীভাবে কাজ করে, কীভাবে একে অপরের সঙ্গে কথা বলে, আর কোথায় গতি আটকে যায়, তা শিখব।

> **CONCEPT:** কম্পিউটারের গতি নির্ধারণ করে সবচেয়ে ধীর অংশটি, সবচেয়ে দ্রুতটি নয়। একে বলে bottleneck। দ্রুত CPU-র পাশে ধীর disk থাকলে CPU বেশিরভাগ সময় অপেক্ষা করবে — যেমন দ্রুত রাঁধুনির পাশে ধীর সরবরাহকারী থাকলে রান্না আটকে যায়।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- CPU-র প্রধান অংশ (ALU, Control Unit, registers) ও তাদের কাজ ব্যাখ্যা করতে।
- Core, thread, clock speed ও cache-এর পার্থক্য ও প্রভাব বিশ্লেষণ করতে।
- RAM, virtual memory ও swap-এর সম্পর্ক ব্যাখ্যা করতে।
- HDD, SATA SSD ও NVMe-এর মধ্যে গতি ও ব্যবহারের পার্থক্য বলতে।
- Storage hierarchy (register → cache → RAM → disk) আঁকতে ও ব্যাখ্যা করতে।
- একটি বাস্তব সিস্টেমে bottleneck চিহ্নিত করতে `top`, `free` ও `df`-এর প্রাথমিক output পড়তে।

Prerequisites — পূর্বজ্ঞান

- Chapter 01: binary, fetch–decode–execute চক্র, IPO+S মডেল।
- একটি কম্পিউটারে ফাইল খোলা ও program চালানোর অভিজ্ঞতা।

২.১ CPU: কম্পিউটারের মস্তিষ্ক — তবে মস্তিষ্কের মতো নয়

CPU (Central Processing Unit) হলো সেই chip যা নির্দেশনা পালন করে। উপমা হিসেবে মস্তিষ্ক বলা হয়, তবে উপমাটি বিভ্রান্তিকর: মস্তিষ্ক বোঝে ও সিদ্ধান্ত নেয়, CPU শুধু গণনা করে ও নির্দেশনা অনুসরণ করে। CPU-র ভেতরে তিনটি প্রধান অংশ:

- **ALU (Arithmetic Logic Unit):** গণনা ও তুলনার কাজ করে — যোগ, বিয়োগ, AND/OR, ছোট-বড় তুলনা।

- **Control Unit (CU):** নির্দেশনা পড়ে, বোঝে কোন অংশকে কী করতে হবে, আর কাজের ক্রম নিয়ন্ত্রণ করে। সে নিজে গণনা করে না — সে পরিচালক।
- **Registers:** CPU-র ভেতরের অতি ক্ষুদ্র, অতি দ্রুত স্মৃতি। এখানে থাকে এখনই যেসব সংখ্যা নিয়ে কাজ চলছে। Register-এর সংখ্যা কম (সাধারণত ১৬–৩২টি সাধারণ register), কিন্তু গতি সবচেয়ে বেশি।

একটি নির্দেশনা পালনের ধাপ: Control Unit memory থেকে নির্দেশনা আনে, decode করে, প্রয়োজনে register থেকে সংখ্যা নিয়ে ALU-কে গণনা করতে বলে, ফলাফল আবার register বা memory-তে রাখে। এই চক্র Chapter 01-এর fetch–decode–execute-ই, এখন আমরা তার ভেতরের অংশগুলো চিনছি।

২.২ Core, Thread ও Clock Speed

Core হলো CPU-র ভেতরের একটি স্বতন্ত্র processing unit — প্রায় একটি ছোট CPU। ৪-core CPU মানে একসঙ্গে চারটি কাজ সত্যিকার অর্থে সমান্তরালে চলতে পারে। **Thread** দুই অর্থে ব্যবহৃত হয়: hardware thread (একটি core একসঙ্গে দুটি কাজের ভান করতে পারলে, যেমন Intel Hyper-Threading) এবং software thread (Chapter 05-এ বিস্তারিত)। আপাতত মনে রাখুন: বেশি core মানে একসঙ্গে বেশি কাজ, বেশি clock speed মানে প্রতিটি কাজ দ্রুত।

Clock speed মাপা হয় GHz-এ (gigahertz)। 3 GHz মানে প্রতি সেকেন্ডে ৩ বিলিয়ন clock tick। প্রতিটি tick-এ CPU একটি ছোট ধাপ এগোতে পারে। তবে শুধু GHz দিয়ে CPU তুলনা করা ভুল — আধুনিক CPU প্রতি tick-এ পুরনো CPU-র চেয়ে বেশি কাজ করে (একে বলে IPC, instructions per cycle)। তাই 2.5 GHz-এর নতুন CPU 3.5 GHz-এর দশ বছরের পুরনো CPU-র চেয়ে দ্রুত হতে পারে। ল্যাপটপ কেনার সময় শুধু GHz নয়, প্রজন্ম (generation) ও core সংখ্যাও দেখতে হয়।

> **PRO TIP:** DevOps-এ সার্ভার বাছার সময় প্রশ্ন করুন: কাজটি কি একটানা গণনা-নির্ভর (বেশি clock speed দরকার) নাকি অনেক একসঙ্গে কাজ (বেশি core দরকার)? একটি web server সাধারণত বেশি core চায়; একটি একক ভারী গণনা বেশি clock speed চায়।

২.৩ Cache: ছোট, দ্রুত, অপরিহার্য

CPU অত্যন্ত দ্রুত, RAM তার তুলনায় ধীর — প্রায় ১০০ গুণ ধীর। CPU প্রতিবার RAM-এর জন্য অপেক্ষা করলে তার বেশিরভাগ সময় নষ্ট হতো। সমাধান: CPU-র ভেতরে ও পাশে ছোট ছোট দ্রুত স্মৃতি — **cache**। L1 cache সবচেয়ে ছোট ও দ্রুত (CPU-র ভেতরে), L2 তার চেয়ে বড় ও কিছুটা ধীর, L3 আরও বড়। CPU প্রথমে L1-এ খোঁজে, না পেলে L2, তারপর L3, তারপর RAM।

কেন এটি কাজ করে? কারণ প্রোগ্রাম একই তথ্য বারবার ব্যবহার করে (locality of reference)। একটি loop-এর ভেতরের সংখ্যা বারবার লাগে, তাই একবার cache-এ এলে বারবার RAM-এ যেতে হয় না। উপমা: রাঁধুনির হাতের কাছে রাখা মসলা (L1), রান্নাঘরের তাক (L2), ভাঁড়ার ঘর (RAM), বাজার (disk)। হাতের কাছের মসলা শেষ হলে তাকে যেতে হয় দূরে — সময় বাড়ে।

২.৪ RAM ও Virtual Memory

RAM (Random Access Memory) হলো চলমান কাজের জায়গা। Disk থেকে program ও তথ্য RAM-এ আসে, CPU সেখান থেকে কাজ করে। RAM **volatile** — বিদ্যুৎ গেলে তার সব তথ্য মুছে যায়। তাই কাজ শেষে ফাইল disk-এ save করতে হয়। RAM বেশি হলে একসঙ্গে বেশি program খোলা যায়; RAM কম হলে operating system বাধ্য হয়ে disk-কে

অস্থায়ী RAM হিসেবে ব্যবহার করে — একে বলে **swap**। Swap disk-এর উপর চলে বলে অত্যন্ত ধীর, আর সিস্টেম তখন লক্ষণীয়ভাবে আটকে যায়।

Virtual memory হলো operating system-এর একটি কৌশল: প্রতিটি program-কে মনে করানো হয় তার নিজের অনেক বড় একটি memory আছে, যদিও বাস্তবে RAM ভাগ করে ব্যবহার হচ্ছে। OS memory-কে **page** নামের ছোট টুকরায় ভাগ করে (সাধারণত 4 KB), আর প্রয়োজনমতো page RAM ও disk (swap)-এর মধ্যে আনা-নেওয়া করে। এই ব্যবস্থার ফলে একটি program অন্য program-এর memory নষ্ট করতে পারে না — নিরাপত্তা ও স্থিতির ভিত্তি। বিস্তারিত Chapter 05-এ।

২.৫ Storage: HDD, SSD, NVMe

Storage হলো স্থায়ী স্মৃতি — বিদ্যুৎ গেলেও তথ্য থাকে। তিনটি প্রধান প্রযুক্তি:

প্রযুক্তি	কীভাবে কাজ করে	গতি (প্রায়)	বৈশিষ্ট্য
HDD	ঘূর্ণায়মান চৌম্বক disk, চলন্ত মাথা তথ্য পড়ে	80–160 MB/s	সস্তা, বেশি ধারণক্ষমতা, ধীর, চলন্ত অংশ আছে
SATA SSD	Flash memory, চলন্ত অংশ নেই	500 MB/s পর্যন্ত	দ্রুত, নীরব, HDD-র চেয়ে দামি
NVMe SSD	Flash memory, PCIe লাইনে সরাসরি CPU-র সঙ্গে	3,000–7,000 MB/s	সবচেয়ে দ্রুত, আধুনিক ল্যাপটপ ও সার্ভারের মান

গতির পার্থক্য শুধু সংখ্যায় নয়, অনুভূতিতে। HDD-তে Ubuntu boot হতে পারে এক মিনিট, NVMe-তে ১০ সেকেন্ড। DevOps-এ database server-এর disk ধীর হলে পুরো application ধীর হয়ে যায়, কারণ প্রতিটি query disk পড়তে অপেক্ষা করে। তাই production server-এ প্রায় সবসময় SSD/NVMe ব্যবহার হয়।

Storage Hierarchy — গতি বনাম ধারণক্ষমতা

Registers (সবচেয়ে দ্রুত, কয়েক KB) → L1/L2/L3 Cache (খুব দ্রুত, কয়েক MB) → RAM (দ্রুত, কয়েক GB) → NVMe/SSD (মাঝারি, শত GB–TB) → HDD / Network storage (ধীর, TB)

উপরে গেলে গতি বাড়ে, ধারণক্ষমতা কমে, প্রতি byte-এর দাম বাড়ে।

২.৬ I/O ও Bottleneck

I/O (Input/Output) বলতে CPU-র বাইরের জগতের সঙ্গে সব আদান-প্রদান বোঝায় — disk পড়া, নেটওয়ার্কে পাঠানো, কীবোর্ড থেকে নেওয়া। CPU গণনায় দ্রুত কিন্তু I/O-তে তাকে অপেক্ষা করতে হয়। একটি web request-এর বেশিরভাগ সময় কাটে অপেক্ষায়: database-র উত্তরের অপেক্ষা, disk পড়ার অপেক্ষা, নেটওয়ার্কের অপেক্ষা। তাই ভালো engineer শুধু CPU দ্রুত করেন না — অপেক্ষা কমান।

Bottleneck চেনার সহজ নিয়ম: যে resource ১০০%-এর কাছাকাছি ব্যবহৃত হচ্ছে অথচ কাজ শেষ হচ্ছে না, সেখানেই আটকে আছে। CPU ১০০% মানে গণনা বেশি; RAM পূর্ণ মানে memory কম; disk ব্যস্ত মানে I/O বেশি। Chapter 17-তে এই নির্ণয়ের পদ্ধতি বিস্তারিত শেখা হবে।

একটি অনুরোধের যাত্রা (Web request)

ব্যবহারকারী → নেটওয়ার্ক → Web server (CPU) → প্রয়োজনে Disk / Database পড়া (I/O — অপেক্ষা) → ফলাফল তৈরি (CPU) → নেটওয়ার্কে উত্তর → ব্যবহারকারীর পর্দা

মোট সময়ের বড় অংশ প্রায়ই I/O অপেক্ষায় কাটে, CPU গণনায় নয়।

২.৭ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা	অধ্যায়
CPU / Core	নির্দেশনা পালনকারী chip / তার ভেতরের স্বতন্ত্র processing unit	২
ALU / Control Unit	গণনার অংশ / পরিচালনার অংশ	২
Register / Cache	CPU-র ভেতরের অতি দ্রুত স্মৃতি / CPU-র পাশের দ্রুত স্মৃতি	২
Clock Speed (GHz)	প্রতি সেকেন্ডে clock tick-এর সংখ্যা	২
RAM / Volatile	চলমান কাজের স্মৃতি / বিদ্যুৎ গেলে মুছে যায়	২
Virtual Memory / Swap	প্রতিটি program-কে বড় memory-র ভান করানো কৌশল / disk-কে অস্থায়ী RAM হিসেবে ব্যবহার	২
HDD / SSD / NVMe	চৌম্বক disk / flash storage / PCIe-তে সরাসরি flash storage	২
Bottleneck / I/O	সবচেয়ে ধীর অংশ যা গতি আটকায় / CPU-র বাইরের জগতের সঙ্গে আদান-প্রদান	২

২.৮ হাতে-কলমে অনুশীলন: নিজের সিস্টেমের সম্পদ দেখা

Ubuntu terminal-এ (VM বা সার্ভারে) নিচের command গুলো চালান। প্রতিটি command-এর মানে Chapter 09–10 ও 17-তে বিস্তারিত আসবে; এখন শুধু output পড়তে শিখুন।

```
lscpu
free -h
df -h
lsblk
```

উদাহরণ আউটপুট (illustrative):

```
$ lscpu | head -20
Architecture:            x86_64
CPU(s):                  4
Model name:              Intel(R) Core(TM) i5-1135G7
CPU MHz:                 2419.000
L1d cache:               192 KiB
L2 cache:                5 MiB
L3 cache:                8 MiB

$ free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:           7.7Gi       2.1Gi       3.2Gi       0.2Gi       2.4Gi       5.2Gi
Swap:          2.0Gi       0.0Gi       2.0Gi

$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        50G   12G   36G  25% /

$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   50G  0 disk
├─sda1   8:1    0  512M  0 part /boot/efi
└─sda2   8:2    0 49.5G  0 part /
```

ব্যাখ্যা: `lscpu` বলছে ৪টি CPU, cache-এর আকার। `free -h`-এ মোট RAM 7.7 GiB, ব্যবহৃত 2.1 GiB, আর Swap ব্যবহৃত 0 — ভালো লক্ষণ, swap ব্যবহার হচ্ছে না। `df -h` বলছে disk ২৫% পূর্ণ। `lsblk` দেখাচ্ছে disk-এর partition গঠন। `available` কলামটিই আসল — নতুন program-এর জন্য কত RAM সত্যিই খালি আছে।

২.৯ Common Mistakes ও Troubleshooting

- **ভুল:** "CPU ১০০% মানেই CPU খারাপ" — না, হতে পারে program-এ infinite loop আছে, বা কাজটিই ভারী। আগে কোন process CPU খাচ্ছে দেখুন (`top`)।
- **ভুল:** Swap দেখলেই আতঙ্ক। অল্প swap ব্যবহার স্বাভাবিক; ক্রমাগত বেশি swap মানেই RAM কম।
- **ভুল:** Disk পূর্ণ হলে শুধু ফাইল মুছে ফেলা। আগে `du` দিয়ে দেখুন কোন directory বড় হচ্ছে — log file হঠাৎ বড় হওয়া একটি সাধারণ কারণ (Chapter 16, 18)।
- **সমস্যা:** কম্পিউটার হঠাৎ ধীর — `free -h` ও `df -h` দিয়ে RAM ও disk পরীক্ষা করুন; দুটোই ঠিক থাকলে CPU দেখুন।

২.১০ Security Considerations

Swap ও disk-এ সংবেদনশীল তথ্য (যেমন password, key) থেকে যেতে পারে, কারণ RAM-এর তথ্য swap-এ লেখা হয়। Production server-এ swap-এ সংবেদনশীল তথ্য যাওয়া ঠেকাতে encrypted swap বা swap বন্ধ রাখার সিদ্ধান্ত নেওয়া হয়। একইভাবে পুরনো disk ফেলে দেওয়ার আগে তা মুছে ফেলা (wipe) জরুরি — delete করলেই তথ্য যায় না, শুধু তার ঠিকানা মুছে যায়।

২.১১ Best Practices

- সার্ভার কেনার আগে workload বুঝুন: বেশি core, নাকি দ্রুত disk, নাকি বেশি RAM — কোনটি বেশি দরকার?
- Monitoring-এর অভ্যাস করুন: CPU, RAM, disk — তিনটিরই নিয়মিত চিত্র রাখুন, সমস্যার সময় তুলনা করতে পারবেন।
- কখনো disk ৯০%-এর বেশি পূর্ণ হতে দেবেন না — অনেক service disk পূর্ণ হলে লেখা বন্ধ করে দেয়।

CHECK YOUR UNDERSTANDING:

1. CPU-র কোন অংশ গণনা করে? A. Control Unit B. ALU C. Register D. Cache

2. 4-core CPU-র অর্থ কী? A. ৪ গুণ clock speed B. একসঙ্গে ৪টি কাজ সমান্তরালে চলতে পারে C. ৪ গুণ RAM D. ৪টি disk
3. RAM-এর প্রধান বৈশিষ্ট্য কোনটি? A. বিদ্যুৎ গেলে তথ্য থাকে B. Volatile — বিদ্যুৎ গেলে তথ্য মুছে যায় C. সবচেয়ে ধীর storage D. শুধু operating system থাকে
4. কোন storage সবচেয়ে দ্রুত? A. HDD B. SATA SSD C. NVMe SSD D. USB pen drive
5. Swap কী? A. CPU-র একটি অংশ B. Disk-কে অস্থায়ী RAM হিসেবে ব্যবহার C. একটি file system D. নেটওয়ার্কের গতি বাড়ানোর উপায়

Knowledge Check — জ্ঞান যাচাই (উত্তর Answer Guide-এ)

1. Fetch–Decode–Execute চক্রের প্রথম ধাপ কোনটি? A) Execute B) Fetch C) Store D) Decode
2. সবচেয়ে দ্রুত মেমোরি কোনটি? A) HDD B) RAM C) CPU register ও cache D) SSD
3. Clock speed বাড়লে কী হয়? A) সবসময় দ্বিগুণ গতি পাওয়া যায় B) প্রতি সেকেন্ডে বেশি চক্র হয়, তবে এটিই একমাত্র মাপকাঠি নয় C) RAM বেড়ে যায় D) ডিস্ক দ্রুত হয়
4. RAM পূর্ণ হলে operating system কী করে? A) কম্পিউটার বন্ধ করে দেয় B) Swap / virtual memory ব্যবহার করে C) ফাইল মুছে ফেলে D) CPU বন্ধ করে দেয়
5. NVMe SSD, SATA SSD-র চেয়ে কেন দ্রুত? A) বেশি জায়গা থাকে বলে B) PCIe দিয়ে CPU-র সঙ্গে সরাসরি যুক্ত থাকে বলে C) বেশি বিদ্যুৎ খায় বলে D) আকার ছোট বলে

২.১২ Practical Assignment

Assignment 2.1: আপনার নিজের কম্পিউটারের (বা VM-এর) `lscpu`, `free -h`, `df -h` ও `lsblk` output সংগ্রহ করুন। একটি সংক্ষিপ্ত report লিখুন: কত core, কত RAM, কত disk, swap ব্যবহৃত হচ্ছে কি না, আর bottleneck কোথায় হতে পারে বলে মনে করেন — যুক্তিসহ।

২.১৩ Chapter Summary

CPU নির্দেশনা পালন করে — ALU গণনা করে, Control Unit পরিচালনা করে, register ও cache গতি দেয়। Core বাড়লে সমান্তরাল কাজ বাড়ে, clock speed বাড়লে প্রতিটি কাজ দ্রুত হয়। RAM চলমান কাজের জায়গা, virtual memory প্রতিটি program-কে আলাদা ও নিরাপদ memory দেয়, আর swap হলো RAM কম পড়লে disk-এর ধীর বিকল্প। Storage-এ HDD ধীর ও সস্তা, SSD দ্রুত, NVMe সবচেয়ে দ্রুত। Performance নির্ধারণ করে সবচেয়ে ধীর অংশ — bottleneck।

Revision Checklist

- ☐ ALU, Control Unit, register, cache-এর কাজ বলতে পারি।
- ☐ Core, thread, clock speed-এর পার্থক্য ব্যাখ্যা করতে পারি।
- ☐ RAM, virtual memory, swap-এর সম্পর্ক বলতে পারি।
- ☐ HDD, SSD, NVMe-র পার্থক্য ও ব্যবহার বলতে পারি।
- ☐ Storage hierarchy আঁকতে পারি ও bottleneck চিহ্নিত করতে পারি।

অধ্যায় ৩ • Part I — Understanding Computer Systems

Chapter 03: Operating Systems: From First Principles — অপারেটিং সিস্টেম: প্রথম নীতি থেকে

শুরুতে: একটি বাস্তব দৃশ্য

ধরুন একটি অফিসে একটি মাত্র প্রিন্টার, কিন্তু দশজন কর্মী একসঙ্গে print করতে চায়। কেউ যদি প্রিন্টার দখল করে রাখে, বাকিরা আটকে যাবে। কেউ ভুল করে অন্যের কাগজ নিয়ে নেবে। আবার কেউ প্রিন্টারের ভেতরের যন্ত্রপাতি নষ্ট করে ফেলতে পারে। এই বিশৃঙ্খলা ঠেকাতে দরকার একজন ব্যবস্থাপক — যিনি সিদ্ধান্ত নেবেন কে কখন প্রিন্টার ব্যবহার করবে, কার কাজ আগে হবে, আর কেউ যাতে অন্যের কাজ নষ্ট করতে না পারে। Operating system ঠিক সেই ব্যবস্থাপক। কম্পিউটারে CPU, RAM, disk, নেটওয়ার্ক — সবকিছু সীমিত, আর program অনেক। OS সেই সীমিত সম্পদ ভাগ করে দেয়, শৃঙ্খলা রাখে, আর প্রতিটি program-কে একটি সহজ, নিরাপদ জগৎ দেখায়।

> **CONCEPT:** Operating system কোনো অ্যাপ্লিকেশন নয় — তা হলো কম্পিউটারের ব্যবস্থাপক ও মধ্যস্থতাকারী। ব্যবহারকারী program লেখে, OS ঠিক করে সেই program কখন CPU পাবে, কত RAM পাবে, কোন ফাইল পড়তে পারবে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Operating system কেন দরকার, তা প্রথম নীতি থেকে ব্যাখ্যা করতে।
- OS-এর মূল দায়িত্বগুলো (resource management, abstraction, protection, coordination) তালিকা করতে।
- Hardware abstraction বলতে কী বোঝায়, উদাহরণসহ ব্যাখ্যা করতে।
- Process, memory ও file management-এর মৌলিক ধারণা দিতে।
- Kernel ও operating system-এর পার্থক্য বলতে।
- Windows, Linux ও macOS-এর মধ্যে স্থাপত্য ও ব্যবহারের পার্থক্য তুলনা করতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 01–02: hardware, CPU, RAM, storage, program execution।
- একাধিক program একসঙ্গে চালানোর অভিজ্ঞতা (যেমন ব্রাউজার + গান + editor)।

৩.১ OS ছাড়া কম্পিউটার কেমন হতো?

কল্পনা করুন OS নেই। প্রতিটি program-কে নিজে জানতে হবে: disk-এর কোন অংশে কী আছে, কীবোর্ড থেকে কীভাবে পড়তে হয়, পর্দায় কীভাবে আঁকতে হয়, নেটওয়ার্ক card-কে কীভাবে চালাতে হয়। প্রতিটি program-কে প্রতিটি hardware-এর জন্য আলাদা code লিখতে হতো। একটি program চলার সময় অন্য program চলতে পারত না — কারণ কে CPU ছাড়বে, তা ঠিক করার কেউ নেই। একটি program ভুল করে অন্য program-এর memory মুছে দিলে পুরো সিস্টেম ভেঙে পড়ত।

এই অবস্থা ১৯৫০-এর দশকের প্রথম কম্পিউটারগুলোতে ছিল। Program-এরাই hardware সরাসরি চালাত। OS-এর জন্ম এই সমস্যাগুলো সমাধানের জন্য — একটি সাধারণ স্তর, যা hardware-এর জটিলতা লুকিয়ে program-কে সহজ উপায় দেয়, আর সম্পদ ভাগ করে দেয়। এই ধারণার নাম **abstraction**: জটিল বাস্তবতাকে সহজ রূপে দেখানো।

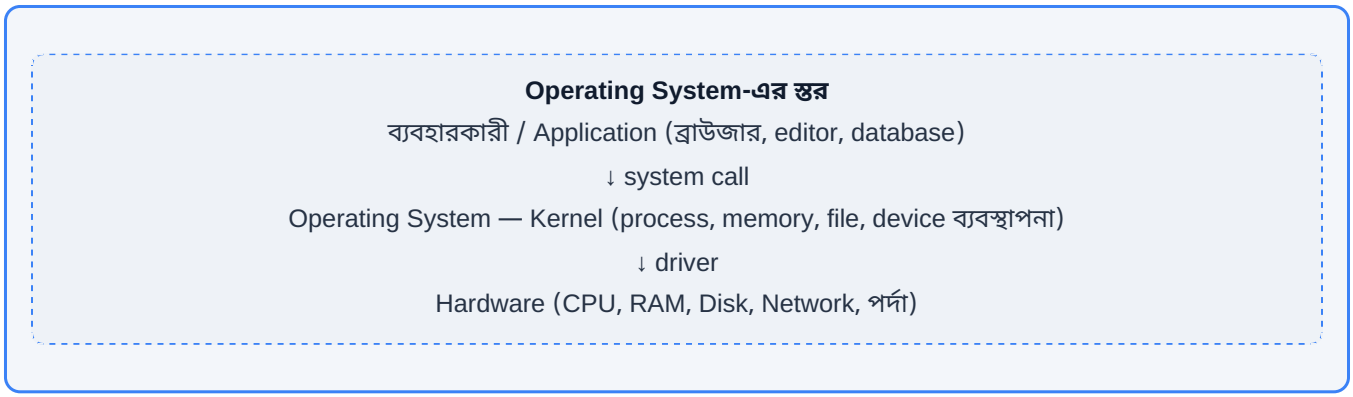
৩.২ OS-এর মূল দায়িত্ব

- **Resource management:** CPU, RAM, disk, নেটওয়ার্ক — কে কতটুকু পাবে, কখন পাবে, তা ঠিক করা।
- **Hardware abstraction:** ভিন্ন ভিন্ন hardware-কে একই সহজ উপায়ে ব্যবহারের ব্যবস্থা করা। Program-কে জানতে হয় না disk-টি HDD নাকি SSD — সে শুধু ফাইল পড়তে বলে।
- **Process management:** অনেক program একসঙ্গে চালানো, তাদের মধ্যে CPU ভাগ করা, একটি আটকে গেলে অন্যগুলোকে বাঁচানো।
- **Memory management:** প্রতিটি program-কে আলাদা memory দেওয়া, একটি যাতে অন্যের memory নষ্ট করতে না পারে।
- **File system management:** ফাইল সংরক্ষণ, খোঁজা, permission নিয়ন্ত্রণ।
- **Security ও protection:** কে কোন ফাইল পড়তে পারবে, কোন program কী করতে পারবে, তা নিয়ন্ত্রণ করা।
- **User interface:** ব্যবহারকারী ও program-এর সঙ্গে যোগাযোগের উপায় দেওয়া — command line বা graphical interface।

৩.৩ Hardware Abstraction: একটি উদাহরণ

আপনি Python-এ লেখেন `open("note.txt").read()`। এই এক লাইনের পেছনে OS কী করে? সে জানে ফাইলটি disk-এর কোথায় আছে, কোন file system-এ আছে, disk controller-কে কী নির্দেশ দিতে হবে, তথ্য RAM-এ এনে আপনার program-কে দিতে হবে। আপনি কিছুই জানেন না — disk-টি কোন কোম্পানির, কত RPM-এ ঘোরে, partition কোথায়। এই লুকিয়ে রাখাই abstraction।

এই abstraction-এর ফল: একই program ভিন্ন ভিন্ন কম্পিউটারে চলে। আপনার laptop-এর disk আর সার্ভারের disk আলাদা হলেও `open()` একইভাবে কাজ করে। DevOps-এ এর গুরুত্ব অসীম — একই application laptop, VM ও cloud server — তিন জায়গায় চলে, কারণ OS hardware-এর পার্থক্য সামলে নেয়।



৩.৪ Process ও Memory Management — সংক্ষিপ্ত পরিচয়

একটি program disk-এ থাকা অবস্থায় শুধু নির্দেশনার সমষ্টি। যখন OS তাকে RAM-এ তুলে চালাতে শুরু করে, তখন সে হয় একটি **process** — চলমান program, যার নিজের memory, নিজের অবস্থা আছে। আপনার ব্রাউজার খোলা থাকলে সেটি

একটি (বা একাধিক) process। গান চালু থাকলে আরেকটি process। OS প্রতিটি process-কে CPU-র সময় ভাগ করে দেয় — এত দ্রুত ভাগ করে যে মনে হয় সবকিছু একসঙ্গে চলছে। এই ভাগাভাগির নাম **scheduling** (Chapter 05-এ বিস্তারিত)।

Memory management-এর মূল কাজ: প্রতিটি process-কে তার নিজের memory দেওয়া, আর একটি process যাতে অন্যের memory পড়তে বা বদলাতে না পারে। এই আলাদা রাখার ব্যবস্থা না থাকলে একটি খারাপ program পুরো সিস্টেম ভেঙে দিতে পারত। আধুনিক OS-এ প্রতিটি process নিজের আলাদা virtual memory জগতে চলে — Chapter 02-এর virtual memory ধারণার প্রয়োগ এখানেই।

৩.৫ File System: তথ্যের সংগঠন

Disk হলো তথ্যের বিশাল গুদাম। File system হলো সেই গুদামের তালিকা ও নিয়ম — কোন ফাইল কোথায় আছে, তার নাম কী, কে পড়তে পারবে, কত বড়। File system ছাড়া disk শুধু ০/১-এর সমুদ্র; file system সেই সমুদ্রকে ফাইল ও directory-তে সাজায়। Linux-এ সবকিছু ফাইল হিসেবে দেখা হয় — এমনকি hardware device-ও (`/dev`)। এই ধারণা Chapter 08-এ বিস্তারিত আসবে।

৩.৬ Kernel বনাম Operating System

অনেকেই kernel আর operating system-কে এক ভাবে। পার্থক্য গুরুত্বপূর্ণ:

- **Kernel** হলো OS-এর মূল অংশ — যা hardware-এর সঙ্গে সরাসরি কথা বলে, CPU, memory, device পরিচালনা করে। Kernel ছাড়া OS অচল।
- **Operating system** হলো kernel + তার চারপাশের সবকিছু — shell, file utilities, system programs, user interface। Ubuntu হলো একটি operating system; তার ভেতরের Linux হলো kernel।

উপমা: kernel হলো সরকারের মূল প্রশাসন, operating system হলো পুরো রাষ্ট্র — প্রশাসনসহ রাস্তা, বিদ্যুৎ, অফিস সবকিছু। Chapter 04-এ kernel-এর ভেতরটা বিস্তারিত দেখা হবে।

৩.৭ Windows, Linux ও macOS তুলনা

বিষয়	Windows	Linux (Ubuntu)	macOS
Kernel	Windows NT kernel	Linux kernel	XNU (Darwin)
Source	Proprietary (বন্ধ)	Open source	আংশিক open
প্রধান ব্যবহার	Desktop, office, gaming	Server, cloud, DevOps, desktop	Desktop, creative কাজ
খরচ	License লাগে	সাধারণত বিনামূল্যে	Apple hardware-এর সঙ্গে
Command line	PowerShell / CMD	Bash — সবচেয়ে শক্তিশালী	Unix-ভিত্তিক shell
Server-এ	কিছু ক্ষেত্রে	বিশ্বের বেশিরভাগ server	প্রায় নেই

DevOps-এর দুনিয়ায় Linux প্রাধান্য পায় কেন? তিনটি কারণ: (১) বিনামূল্যে ও open source — কোড দেখা ও বদলানো যায়; (২) command line-কেন্দ্রিক — automation সহজ; (৩) server ও cloud-এর মান — বেশিরভাগ cloud server

Linux চালায়। এই বই Linux-কেন্দ্রিক হলেও OS-এর মূল ধারণাগুলো সব OS-এ একই — Windows বা macOS-এও process, memory, file system আছে, শুধু নাম ও উপায় আলাদা।

একই সময়ে অনেক process — OS কী করে?

Process A (ব্রাউজার) → OS CPU সময় ভাগ করে দেয় → Process B (গান) → Process C (editor)

প্রতিটি process ভাবে CPU শুধু তার, আসলে OS মিলিসেকেন্ডে ভাগ করে দিচ্ছে।

একটি process আটকে গেলেও OS অন্যগুলোকে চালিয়ে নেয়।

৩.৭ক Batch, Multiprogramming ও Multitasking — একটু ইতিহাস

প্রথম দিকের OS-গুলো **batch processing** করত — একটি কাজ শেষ হলে পরেরটি। এতে CPU অনেক সময় অলস থাকত, কারণ একটি কাজ disk-এর অপেক্ষায় থাকলে CPU বসে থাকত। তারপর এল **multiprogramming**: একাধিক program RAM-এ রাখা হলো, একটি অপেক্ষায় গেলে আরেকটি CPU পেল। এরপর **multitasking** বা time-sharing: CPU সময়কে ছোট ছোট টুকরায় ভাগ করে প্রতিটি program-কে একটু একটু করে দেওয়া হলো, যাতে ব্যবহারকারীর মনে হয় সবকিছু একসঙ্গে চলছে। আপনার laptop-এ ব্রাউজার, গান ও editor একসঙ্গে চলা এই time-sharing-এরই ফল। এই ইতিহাস বোঝা জরুরি, কারণ আজকের scheduler, process ও memory ব্যবস্থাপনা এই তিন ধাপের সমস্যা সমাধানের জন্য তৈরি হয়েছে। একটি OS-কে বিচার করার মানদণ্ডও এখান থেকে আসে: CPU কতটা ব্যস্ত থাকে, ব্যবহারকারী কত দ্রুত সাড়া পায়, আর একটি program অন্যকে আটকাতে পারে কি না।

৩.৮ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Operating System (OS)	hardware পরিচালনা ও program-কে সেবা দেওয়া ব্যবস্থাপক software	৩
Kernel	OS-এর মূল অংশ, hardware-এর সঙ্গে সরাসরি কথা বলে	৩
Abstraction	জটিল বাস্তবতাকে সহজ রূপে দেখানো	৩
Process	চলমান program, নিজের memory ও অবস্থাসহ	৩
Scheduling	process-গুলোর মধ্যে CPU সময় ভাগ করে দেওয়া	৩
File System	disk-এ ফাইল সাজানো ও খোঁজার নিয়ম	৩
System Call	program-এর OS-এর কাছে সেবা চাওয়ার উপায়	৩

৩.৯ হাতে-কলমে অনুশীলন: OS-কে চেনা

```
uname -a
ps aux | head -20
ls /etc/os-release && cat /etc/os-release
```

উদাহরণ আউটপুট (illustrative):

```
$ cat /etc/os-release
NAME="Ubuntu"
VERSION="24.04 LTS (Noble Numbat)"
ID=ubuntu
VERSION_ID="24.04"

$ ps aux | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  23712  3120 ?        Ss   10:00   0:01 /sbin/init
nahid       1200  0.5  1.2  500000 90000 ?       Sl   10:05   0:10 firefox
```

ব্যাখ্যা: `ps aux` চলমান process-গুলো দেখায়। PID 1 হলো `init` (systemd) — সব process-এর জনক, Chapter 15-এ আসবে। প্রতিটি লাইন একটি process: কে চালাচ্ছে (USER), কত CPU/RAM খাচ্ছে, কী command। `/etc/os-release` জানাচ্ছে এটি Ubuntu 24.04 LTS।

> **REAL WORLD:** ঢাকার একটি hosting company-র সার্ভারে একটি web app হঠাৎ বন্ধ হয়ে গেল। দেখা গেল একটি process memory খেয়ে শেষ করে দিয়েছে, আর OS বাধ্য হয়ে তাকে বন্ধ করেছে (OOM killer)। সমস্যাটি app-এর memory leak — OS ঠিক কাজই করেছে: একটি খারাপ process-কে পুরো সার্ভার মারতে দেয়নি। Chapter 05 ও 17-তে এই নির্ণয় শেখা হবে।

৩.১০ Common Mistakes ও Troubleshooting

- **ভুল:** "OS আর kernel একই" — না। Ubuntu একটি OS, Linux তার kernel।
- **ভুল:** "একটি program আটকে গেলে পুরো কম্পিউটার নষ্ট" — আধুনিক OS-এ একটি process আটকে গেলে অন্যগুলো চলে; শুধু সেই process-টি বন্ধ করলেই হয়।
- **ভুল:** "Linux মানেই Ubuntu" — Ubuntu একটি distribution; Linux kernel ব্যবহার করে আরও শত শত distribution আছে (Chapter 06)।
- **সমস্যা:** কম্পিউটার চালু হচ্ছে না — OS-এর সমস্যা নাকি hardware-এর, তা আলাদা করুন: BIOS/UEFI পর্দা আসছে কি না দেখুন (Chapter 07)।

৩.১১ Security Considerations

OS-এর সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা কাজ হলো **আলাদা রাখা (isolation)**: একটি program যাতে অন্য program-এর memory বা ফাইল পড়তে না পারে, একজন user যাতে অন্যের ফাইল বদলাতে না পারে। এই আলাদা রাখার ব্যবস্থা ভেঙে গেলে (যেমন ভুল permission) নিরাপত্তা ভেঙে যায়। তাই Chapter 11–12-এর permission শেখা শুধু নিয়ম নয় — OS-এর নিরাপত্তা মডেলের মূল।

৩.১২ Best Practices

- OS-কে নিয়মিত update করুন — update-এ নিরাপত্তা ত্রুটির সমাধান থাকে (Chapter 14, 19)।
- প্রতিটি service-কে শুধু তার প্রয়োজনীয় permission দিন (least privilege)।
- কোনো program আটকে গেলে পুরো সিস্টেম restart করার আগে শুধু সেই process-টি বন্ধ করে দেখুন।

CHECK YOUR UNDERSTANDING:

1. Operating system-এর প্রধান কাজ কোনটি? A. গেম খেলা B. Hardware পরিচালনা ও program-কে সেবা দেওয়া C. শুধু ফাইল সংরক্ষণ D. ইন্টারনেট চালানো
2. Kernel কী? A. পুরো operating system B. OS-এর মূল অংশ যা hardware-এর সঙ্গে কথা বলে C. একটি application D. একটি file system
3. Abstraction বলতে কী বোঝায়? A. তথ্য মুছে ফেলা B. জটিল hardware-কে সহজ উপায়ে ব্যবহারের ব্যবস্থা C. Program দ্রুত করা D. Disk ভাগ করা
4. Process কী? A. Disk-এ থাকা program B. চলমান program, নিজের memorysহ C. একটি ফাইল D. একটি hardware
5. DevOps-এ Linux বেশি ব্যবহৃত হয় কেন? A. শুধু দেখতে সুন্দর বলে B. Open source, command-line কেন্দ্রিক ও server-এর মান বলে C. গেম বেশি চলে বলে D. License দামি বলে

Knowledge Check — জ্ঞান যাচাই (উত্তর Answer Guide-এ)

1. Operating system-এর প্রধান কাজ কী? A) গেম চালানো B) Hardware পরিচালনা ও resource ভাগ করে দেওয়া C) ইন্টারনেট দেওয়া D) ফাইল তৈরি করা
2. Kernel কী? A) পুরো operating system B) OS-এর কেন্দ্র, যা hardware-এর সঙ্গে কথা বলে C) একটি application D) একটি filesystem
3. একসঙ্গে অনেক প্রোগ্রাম চলা কীভাবে সম্ভব হয়? A) প্রত্যেকের আলাদা CPU থাকে বলে B) OS CPU-র সময় ভাগ করে দেয় (multitasking) C) RAM বাড়িয়ে D) ডিস্ক দ্রুত করে
4. Windows ও macOS-এর তুলনায় Linux-এর বৈশিষ্ট্য কোনটি? A) শুধু GUI থাকে B) Open-source kernel ও অনেক distribution থাকে C) শুধু Apple hardware-এ চলে D) কোনো package manager নেই
5. Hardware abstraction-এর সুবিধা কী? A) প্রোগ্রামকে প্রতিটি hardware-এর খুঁটিনাটি জানতে হয় না B) কম্পিউটার ধীর হয় C) ফাইল মুছে যায় D) RAM কম লাগে

৩.১৩ Practical Assignment

Assignment 3.1: আপনার কম্পিউটারে `ps aux` চালিয়ে চলমান process-গুলোর তালিকা দেখুন। পাঁচটি process বাছুন, প্রতিটির জন্য লিখুন: এটি কী program, কে চালাচ্ছে, কত CPU/RAM ব্যবহার করছে, আর এটি বন্ধ হলে কী হবে বলে মনে করেন।

৩.১৪ Chapter Summary

Operating system হলো hardware ও program-এর মাঝখানের ব্যবস্থাপক। সে সম্পদ ভাগ করে, hardware-এর জটিলতা লুকিয়ে সহজ abstraction দেয়, process ও memory পরিচালনা করে, ফাইল সাজায় ও নিরাপত্তা দেয়। Kernel হলো তার মূল অংশ; operating system হলো kernelসহ পুরো ব্যবস্থা। Windows, Linux ও macOS একই মূল কাজ করে ভিন্ন ভিন্ন উপায়ে — আর server ও DevOps-এর দুনিয়ায় Linux-ই মান।

Revision Checklist

- ☐ OS কেন দরকার, প্রথম নীতি থেকে ব্যাখ্যা করতে পারি।

- ☐ OS-এর সাতটি মূল দায়িত্ব বলতে পারি।
- ☐ Abstraction উদাহরণসহ ব্যাখ্যা করতে পারি।
- ☐ Kernel ও OS-এর পার্থক্য বলতে পারি।
- ☐ Windows, Linux, macOS তুলনা করতে পারি।

অধ্যায় ৪ • Part I — Understanding Computer Systems

Chapter 04: Kernel, User Space and System Calls — Kernel, User Space ও System Call

শুরুতে: একটি বাস্তব দৃশ্য

একটি ব্যাংকের ভল্টের কথা ভাবুন। গ্রাহকরা ভল্টে সরাসরি ঢুকতে পারে না — তারা কাউন্টারে গিয়ে অনুরোধ করে, আর ভেতরের কর্মচারী নিয়ম মেনে টাকা বের করে দেয়। কেন? কারণ ভল্টে সরাসরি ঢুকতে দিলে কেউ ভুল করে বা ইচ্ছে করে অন্যের টাকা নিয়ে নিতে পারে। কম্পিউটারেও একই ব্যবস্থা। Program সরাসরি hardware ছুঁতে পারে না — তাকে kernel-এর কাছে অনুরোধ করতে হয়। এই অনুরোধের নাম **system call**। আর ভল্টের বাইরের জগৎ হলো **user space**, ভেতরের জগৎ **kernel space**। এই অধ্যায়ে আমরা সেই সীমানা, তার কারণ, আর একটি অনুরোধের সম্পূর্ণ যাত্রা শিখব।

> **CONCEPT:** Kernel হলো একমাত্র অংশ যা hardware-কে সরাসরি নিয়ন্ত্রণ করে। বাকি সব program — ব্রাউজার থেকে আপনার Python code পর্যন্ত — kernel-এর কাছে অনুরোধ করে। এই সীমানাই কম্পিউটারকে স্থিতিশীল ও নিরাপদ রাখে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Kernel space ও user space-এর পার্থক্য ও সীমানার কারণ ব্যাখ্যা করতে।
- System call কী, তা উদাহরণসহ বর্ণনা করতে।
- Privilege level (ring) ও mode switch-এর ধারণা দিতে।
- Interrupt ও context switching কীভাবে কাজ করে, তা ব্যাখ্যা করতে।
- Device driver-এর ভূমিকা বলতে।
- একটি program ফাইল পড়তে চাইলে ভেতরে কী কী ঘটে, তার ধাপগুলো বর্ণনা করতে।
- Linux kernel-এর ধরন (monolithic) ও প্রধান উপাদানগুলো চিহ্নিত করতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 03: OS-এর দায়িত্ব, kernel বনাম OS, process-এর ধারণা।
- Chapter 01–02: CPU, RAM, file-এর ধারণা।

৪.১ কেন সীমানা দরকার? প্রথম নীতি

ধরুন যেকোনো program সরাসরি disk-এ লিখতে পারত, যেকোনো memory পড়তে পারত। তাহলে একটি ভুল program (বা ক্ষতিকর program) পুরো disk মুছে দিতে পারত, অন্য program-এর password পড়ে নিতে পারত, বা CPU দখল করে আর কোনো program-কে চলতে দিত না। এই বিপদ ঠেকাতে CPU-তে দুটি অবস্থা (mode) আছে:

- **User mode:** সাধারণ program চলে। সে সরাসরি hardware ছুঁতে পারে না, অন্যের memory পড়তে পারে না, কিছু বিশেষ নির্দেশনা চালাতে পারে না।

- **Kernel mode:** শুধু kernel চলে। সে সবকিছু করতে পারে — hardware নিয়ন্ত্রণ, যেকোনো memory পড়া, যেকোনো device চালানো।

এই দুই mode-এর মাঝখানের সীমানা CPU hardware দিয়ে রক্ষা করা হয় — শুধু নিয়মের উপর নয়, chip-এর উপর। User mode-এর program kernel mode-এর নির্দেশনা চালাতে চাইলে CPU তা প্রত্যাখ্যান করে। এই সীমানা পেরোনোর একমাত্র বৈধ পথ system call।

৪.২ Kernel Architecture: Monolithic বনাম Microkernel

Kernel সাজানোর দুটি প্রধান ধারা আছে। **Monolithic kernel**-এ প্রায় সব সেবা — process, memory, file system, device driver — একটিই বড় kernel-এর ভেতরে থাকে। Linux এই ধারার। সুবিধা: গতি বেশি, কারণ সেবাগুলো একে অপরকে সরাসরি ডাকতে পারে। অসুবিধা: একটি driver-এ ভুল হলে পুরো kernel আক্রান্ত হতে পারে। **Microkernel**-এ kernel খুব ছোট রাখা হয়, বাকি সেবা user space-এ আলাদা program হিসেবে চলে। সুবিধা: একটি সেবা ভেঙে গেলে kernel বাঁচে। অসুবিধা: সেবাগুলোর মধ্যে যোগাযোগ বেশি লাগে বলে কিছুটা ধীর। বাস্তবে Linux monolithic হলেও driver-কে module হিসেবে load/unload করা যায় — মাঝামাঝি নমনীয়তা।

Linux kernel-এর প্রধান উপাদান: process scheduler, memory manager, virtual file system (VFS), network stack, device driver স্তর। প্রতিটি উপাদান একটি দায়িত্ব পালন করে, আর system call-ই হলো তাদের কাছে পৌঁছানোর দরজা।

User Space বনাম Kernel Space

User Space (সীমিত ক্ষমতা): Application — ব্রাউজার, editor, database, আপনার program; Library — glibc ইত্যাদি

— — — সীমানা: System Call Interface — — —

Kernel Space (পূর্ণ ক্ষমতা): System call handler → Process / Memory / File system / Network / Device driver

↓

Hardware (CPU, RAM, Disk, Network card)

৪.৩ System Call: অনুরোধের ভাষা

Program যখন ফাইল পড়তে চায়, নেটওয়ার্কে পাঠাতে চায়, নতুন process তৈরি করতে চায় — তখন সে system call করে। উদাহরণ: Linux-এ ফাইল পড়ার system call-গুলো:

System Call	কাজ	উদাহরণ
`open()`	ফাইল খোলা	`open("note.txt")`
`read()`	ফাইল থেকে তথ্য পড়া	`read(fd, buffer, 100)`
`write()`	ফাইলে লেখা	`write(fd, "hello", 5)`
`close()`	ফাইল বন্ধ করা	`close(fd)`
`fork()`	নতুন process তৈরি	Chapter 05-এ বিস্তারিত
`execve()`	নতুন program চালানো	shell-এ command চালানো

সাধারণত program সরাসরি system call করে না — সে **library** (যেমন C library, glibc) ব্যবহার করে, আর library system call করে। আপনার Python-এর `open()` আসলে glibc-র `open()`-কে ডাকে, glibc kernel-এর `open` system call করে। এই স্তরগুলো program-কে সহজ রাখে।

System call-এর ধাপ: (১) program library-কে ডাকে; (২) library system call number ও argument সাজিয়ে একটি বিশেষ CPU নির্দেশনা চালায় (x86-64-এ `syscall`); (৩) CPU user mode থেকে kernel mode-এ যায়; (৪) kernel অনুরোধ যাচাই করে (permission আছে কি না, ফাইল আছে কি না), কাজটি করে; (৫) ফলাফল নিয়ে CPU আবার user mode-এ ফেরে। এই যাওয়া-আসার নাম **mode switch** — এতে কিছু সময় লাগে, তাই program অকারণে বারবার system call করলে ধীর হয়।

৪.৪ Interrupt ও Context Switching

Interrupt হলো hardware বা software-এর পাঠানো সংকেত — "এখনই মনোযোগ দাও!" যেমন: কীবোর্ডে key চাপা, disk থেকে তথ্য আসা, নেটওয়ার্ক packet আসা, বা timer-এর সময় শেষ হওয়া। CPU যা করছিল তা থামিয়ে interrupt handler চালায়, তারপর আবার আগের কাজে ফেরে। Interrupt ছাড়া CPU-কে বারবার জিজ্ঞেস করতে হতো "কিছু এসেছে? কিছু এসেছে?" — যাকে বলে polling, অত্যন্ত অপচয়।

Context switching হলো এক process থেকে আরেক process-এ যাওয়া। CPU একটি process-এর অবস্থা (register-এর মান, কোথায় ছিল) সংরক্ষণ করে, আরেকটি process-এর অবস্থা load করে, তারপর সেটি চালায়। এই বদল এত দ্রুত ঘটে (মিলিসেকেন্ডের ভগ্নাংশ) যে মনে হয় সব process একসঙ্গে চলছে। তবে context switch-এরও খরচ আছে — বেশি process বেশি বদল মানে বেশি overhead। তাই হাজার হাজার process চালানো সবসময় ভালো নয়।

একটি ফাইল পড়ার যাত্রা (System Call)

আপনার program: open("note.txt") → C library (glibc) → syscall নির্দেশনা → CPU: User mode → Kernel mode → Kernel: permission যাচাই → File system: disk-এ ফাইল খোঁজা → Device driver: disk-কে পড়তে বলা → তথ্য RAM-এ আসা → ফলাফল (file descriptor) নিয়ে CPU: Kernel mode → User mode → program তথ্য পেল

৪.৫ Device Driver: অনুবাদক

পৃথিবীতে হাজার রকমের hardware — ভিন্ন ভিন্ন disk, network card, প্রিন্টার। Kernel প্রতিটির ভাষা আলাদা করে জানে না। **Device driver** হলো সেই অনুবাদক program, যা একটি নির্দিষ্ট hardware-কে কীভাবে চালাতে হয় তা জানে, আর kernel-কে একটি সাধারণ উপায় দেয়। নতুন hardware লাগালে তার driver install করতে হয় — Ubuntu-তে বেশিরভাগ driver kernel-এর সঙ্গেই আসে, তাই সাধারণত আলাদা install লাগে না। Driver-এ ভুল থাকলে hardware কাজ করে না বা সিস্টেম অস্থিতিশীল হয় — তাই অচেনা উৎস থেকে driver install করা বিপজ্জনক।

REAL WORLD: সিলেটের একটি ছোট company-র সার্ভারে নতুন network card লাগানোর পর নেটওয়ার্ক কাজ করছিল না। কারণ card-এর driver kernel version-এর সঙ্গে মিলছিল না। সমাধান ছিল driver update করা, card বদলানো নয়। DevOps-এ hardware বদলের আগে driver ও kernel compatibility যাচাই করা একটি নিয়মিত কাজ।

৪.৫ক Linux Kernel: সংস্করণ ও আপডেট কেন গুরুত্বপূর্ণ

Linux kernel-এর সংস্করণ দেখতে `uname -r` চালান — যেমন 6.8.0-31। প্রথম সংখ্যাটি বড় সংস্করণ, পরেরগুলো ছোট পরিবর্তন ও নিরাপত্তা সমাধান। Ubuntu 24.04 LTS-এ kernel নিয়মিত update পায়, আর LTS মানে দীর্ঘদিন নিরাপত্তা update পাওয়া যাবে — ২০২৯ সালের এপ্রিল পর্যন্ত। Kernel update-কে অবহেলা করবেন না, কারণ kernel-এর ত্রুটি পুরো সিস্টেমকে আক্রান্ত করতে পারে। তবে kernel update-এর পর অনেক সময় restart লাগে, কারণ চলমান kernel বদলানো যায় না — নতুন kernel পরের boot-এ কার্যকর হয়। Production server-এ তাই kernel update পরিকল্পনা করে, maintenance window-তে করা হয়, হঠাৎ নয়। আরেকটি বিষয়: kernel-এর সঙ্গে driver-এর সামঞ্জস্য। পুরনো hardware-এর driver নতুন kernel-এ নাও থাকতে পারে, আবার নতুন hardware-এর জন্য নতুন kernel লাগতে পারে। এই কারণে LTS release বাছা হয় স্থিতির জন্য — নতুনত্বের চেয়ে নির্ভরযোগ্যতা বড়।

৪.৬ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Kernel Space / User Space	kernel-এর পূর্ণ ক্ষমতার জগৎ / সাধারণ program-এর সীমিত জগৎ	৪
System Call	program-এর kernel-এর কাছে সেবা চাওয়ার উপায়	৪
User Mode / Kernel Mode	CPU-র সীমিত অবস্থা / পূর্ণ ক্ষমতার অবস্থা	৪
Mode Switch	user mode থেকে kernel mode-এ যাওয়া-আসা	৪
Interrupt	hardware/software-এর জরুরি সংকেত	৪
Context Switching	এক process থেকে আরেক process-এ CPU-র বদল	৪
Device Driver	নির্দিষ্ট hardware চালানোর অনুবাদক software	৪
Monolithic Kernel	সব সেবা একটিই বড় kernel-এর ভেতরে থাকা ধারা (Linux)	৪

৪.৭ হাতে-কলমে অনুশীলন: System Call দেখা

Linux-এ `strace` দিয়ে দেখা যায় একটি program কোন system call করছে। Ubuntu-তে `strace` না থাকলে `sudo apt install strace` দিয়ে install করুন (Chapter 14-এ APT বিস্তারিত)।

```
echo "hello" > test.txt
strace -e trace=openat,read,write cat test.txt
```

উদাহরণ আউটপুট (illustrative):

```
$ strace -e trace=openat,read,write cat test.txt
openat(AT_FDCWD, "test.txt", O_RDONLY) = 3
read(3, "hello\n", 131072)              = 6
write(1, "hello\n", 6)                  = 6
hello
+++ exited with 0 +++
```

ব্যাখ্যা: `cat` প্রথমে `openat` system call দিয়ে ফাইলটি খুলল (ফলাফল: file descriptor 3), তারপর `read` দিয়ে ৬ byte পড়ল, তারপর `write` দিয়ে পর্দায় (file descriptor 1 = stdout) লিখল। এই তিনটি system call ছাড়া `cat` কিছুই করতে পারত না। আরও দেখুন: `uname -r` দিয়ে kernel version জানুন।

```
uname -r
ls /lib/modules/
```

৪.৮ Common Mistakes ও Troubleshooting

- **ভুল:** "System call আর function call একই" — না। Function call একই program-এর ভেতরে হয়; system call mode বদলে kernel-এ যায়, অনেক বেশি খরচের।
- **ভুল:** "Root user হলেই kernel mode" — না। Root-ও user space-এ চলে; root-এর ক্ষমতা বেশি, কিন্তু সে-ও hardware সরাসরি ছুঁতে পারে না, system call-ই ব্যবহার করে।
- **সমস্যা:** `Permission denied` — প্রায়ই system call-এ kernel permission যাচাই করে প্রত্যাখ্যান করেছে। ফাইলের permission দেখুন (Chapter 11)।
- **সমস্যা:** নতুন hardware কাজ করছে না — `dmesg | tail -20` দিয়ে দেখুন kernel driver সম্পর্কে কী বলছে (Chapter 18)।

৪.৯ Security Considerations

Kernel space-এর ভুল সবচেয়ে বিপজ্জনক — একটি kernel bug পুরো সিস্টেমকে আক্রান্ত করতে পারে, কারণ kernel-এর ক্ষমতা অসীম। তাই kernel update-কে হালকাভাবে নেবেন না, আর অচেনা kernel module load করবেন না। একইভাবে system call-ই নিরাপত্তার দরজা: kernel প্রতিটি system call-এ permission যাচাই করে। এই যাচাই না থাকলে যেকোনো program যেকোনো ফাইল পড়তে পারত।

৪.১০ Best Practices

- Program লেখার সময় অকারণে বারবার ছোট ছোট read/write system call করবেন না — তথ্য একসঙ্গে পড়ুন (buffering)।
- Troubleshooting-এ `strace` ব্যবহার করুন: program কোথায় আটকাচ্ছে, কোন system call ব্যর্থ হচ্ছে, তা দেখা যায়।
- Kernel ও driver update করার আগে backup নিন (Chapter 20)।

CHECK YOUR UNDERSTANDING:

1. System call কী? A. দুটি program-এর মধ্যে কথা বলা B. Program-এর kernel-এর কাছে সেবা চাওয়ার উপায় C. একটি file open করা D. CPU-র গতি বাড়ানো
2. User space-এর program সরাসরি hardware ছুঁতে পারে না কেন? A. Hardware নেই বলে B. নিরাপত্তা ও স্থিতির জন্য CPU mode-এর সীমানা আছে বলে C. Program জানে না বলে D. Linux নিষেধ করে বলে
3. Linux kernel কোন ধারার? A. Microkernel B. Monolithic kernel C. Hybrid শুধু D. কোনো kernel নেই
4. Interrupt কী? A. Program-এর ভুল B. Hardware/software-এর জরুরি সংকেত C. Disk পূর্ণ হওয়া D. নেটওয়ার্ক বন্ধ হওয়া

5. Device driver-এর কাজ কী? A. ফাইল মুছে ফেলা B. নির্দিষ্ট hardware চালানোর অনুবাদক হিসেবে কাজ করা C. CPU দ্রুত করা D. Password মনে রাখা

Knowledge Check — জ্ঞান যাচাই (উত্তর Answer Guide-এ)

1. System call কী? A) টেলিফোন কল B) Program-এর kernel-এর কাছে সেবা চাওয়ার পথ C) একটি ভাইরাস D) একটি network port
2. User space ও kernel space-এর সীমানা কেন দরকার? A) গতি বাড়াতে B) ভুল program যাতে পুরো সিস্টেম নষ্ট করতে না পারে, সে জন্য C) ফাইল লুকাতে D) RAM বাঁচাতে
3. x86-64-এ system call চালানোর নির্দেশনা কোনটি? A) syscall B) print C) jump D) fork
4. Interrupt কী? A) প্রোগ্রামের ভুল B) Hardware-এর জরুরি সংকেত C) ফাইল মুছে ফেলা D) লগ লেখা
5. Device driver-এর কাজ কী? A) ফাইল তৈরি করা B) Hardware-এর ভাষা kernel-কে বোঝানো C) ইন্টারনেট চালানো D) পাসওয়ার্ড রাখা

৪.১১ Practical Assignment

Assignment 4.1: `strace` দিয়ে `ls` ও `cat` command-এর system call দেখুন। প্রতিটি থেকে তিনটি system call বাছুন, লিখুন সেটি কী কাজ করছে, আর সেটি না থাকলে command-টি কী করতে পারত না। সঙ্গে `uname -r` output লিখে আপনার kernel version নোট করুন।

৪.১২ Chapter Summary

Kernel space ও user space-এর সীমানা CPU hardware দিয়ে রক্ষা করা হয়। Program hardware ছুঁতে চাইলে system call করে kernel-এর কাছে অনুরোধ জানায়; kernel যাচাই করে কাজটি করে। Interrupt hardware-কে জরুরি সংকেত পাঠাতে দেয়, context switching অনেক process-কে এক CPU ভাগ করে নিতে দেয়, আর device driver প্রতিটি hardware-এর ভাষা kernel-কে বোঝায়। Linux একটি monolithic kernel, যার সেবাগুলো system call interface দিয়ে পাওয়া যায়।

Revision Checklist

- ☐ User space ও kernel space-এর পার্থক্য ব্যাখ্যা করতে পারি।
- ☐ System call-এর ধাপগুলো বলতে পারি।
- ☐ User mode ও kernel mode-এর পার্থক্য বলতে পারি।
- ☐ Interrupt ও context switching ব্যাখ্যা করতে পারি।
- ☐ ফাইল পড়ার যাত্রাটি ধাপে ধাপে বলতে পারি।

অধ্যায় ৫ • Part I — Understanding Computer Systems

Chapter 05: Processes, Threads and Memory Management — Process, Thread ও মেমোরি ব্যবস্থাপনা

শুরুতে: একটি বাস্তব দৃশ্য

রাজশাহীর একটি university lab-এ শিক্ষার্থী তানিয়া একটি Python program চালাল যা একটি বড় ফাইল প্রক্রিয়া করে। Program-টি চলতে চলতে হঠাৎ বন্ধ হয়ে গেল — `MemoryError`। আবার আরেকদিন একই program চলল কিন্তু অত্যন্ত ধীর। তৃতীয় দিন দেখা গেল একটি program বন্ধ করার পরও সেটি process list-এ থেকে গেছে। এই তিনটি ঘটনার পেছনে একই বিষয়: process কীভাবে জন্মায়, কীভাবে memory ব্যবহার করে, কীভাবে শেষ হয় — তা না জানা। এই অধ্যায়ে আমরা process-এর জীবনচক্র, thread-এর কাজ, scheduling, memory layout, stack/heap ও virtual memory-র paging — DevOps-এর প্রতিদিনের কাজের ভিত্তি — শিখব।

> **CONCEPT:** Program হলো রেসিপি, process হলো রান্না। একই রেসিপি দিয়ে দুজন রাঁধুনি দুটি আলাদা রান্না করতে পারে — একই program থেকে দুটি আলাদা process তৈরি হতে পারে, প্রতিটির নিজের memory ও অবস্থা আছে।

Learning Objectives — শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন:

- Program ও process-এর পার্থক্য ব্যাখ্যা করতে।
- Process lifecycle (new → ready → running → waiting → terminated) বর্ণনা করতে।
- Thread কী, process থেকে কীভাবে আলাদা, তা ব্যাখ্যা করতে।
- Scheduling ও context switching-এর ভূমিকা বলতে।
- Process memory layout (text, data, heap, stack) আঁকতে ও ব্যাখ্যা করতে।
- Stack ও heap-এর পার্থক্য ও ব্যবহার বলতে।
- Virtual memory ও paging কীভাবে কাজ করে, তা ব্যাখ্যা করতে।
- Zombie ও orphan process চিহ্নিত করতে ও তাদের মোকাবিলা করতে।

Prerequisites — পূর্বজ্ঞান

- Chapter 03–04: OS, kernel, system call, interrupt, context switching।
- Chapter 02: RAM, virtual memory, cache-এর ধারণা।
- Terminal-এ `ps` চালানোর প্রাথমিক অভিজ্ঞতা (Chapter 03-এর অনুশীলন)।

৫.১ Program বনাম Process

Disk-এ থাকা `firefox` ফাইলটি একটি program — নিষ্ক্রিয় নির্দেশনা। যখন OS তাকে RAM-এ তুলে চালাতে শুরু করে, তখন সে একটি process — সক্রিয় সত্তা, যার আছে: নিজের memory, নিজের CPU অবস্থা (কোন নির্দেশনায় আছে), খোলা ফাইলের তালিকা, আর একটি পরিচয় নম্বর **PID (Process ID)**। একই program থেকে একাধিক process তৈরি

হতে পারে — আপনি ব্রাউজারের দুটি window খুললে দুটি process (বা একাধিক) তৈরি হয়। একটি process শেষ হলে তার memory OS ফিরিয়ে নেয়।

প্রতিটি process-এর একটি **parent** থাকে — যে process তাকে তৈরি করেছে। সব process-এর আদি parent হলো PID 1 (`systemd`, Ubuntu-তে `/sbin/init`)। `pstree` command দিয়ে এই পারিবারিক গাছ দেখা যায়।

৫.২ Process Lifecycle ও State

একটি process জীবনে কয়েকটি অবস্থার মধ্য দিয়ে যায়:

1. **New:** তৈরি হচ্ছে, OS তার জন্য জায়গা বরাদ্দ করছে।
2. **Ready:** চলার জন্য প্রস্তুত, CPU-র অপেক্ষায়।
3. **Running:** CPU-তে চলছে।
4. **Waiting (Blocked):** কোনো কিছুর অপেক্ষায় — disk থেকে তথ্য আসা, ব্যবহারকারীর input, নেটওয়ার্কের উত্তর।
5. **Terminated:** কাজ শেষ, OS তার সম্পদ ফিরিয়ে নিচ্ছে।

একটি process বারবার Ready ↔ Running ↔ Waiting ঘোরে। সে CPU পেলে Running, disk-এর অপেক্ষায় গেলে Waiting, আবার তথ্য এলে Ready। এই ঘোরার পরিচালক **scheduler** — kernel-এর অংশ, যা ঠিক করে পরের CPU সময় কে পাবে। Linux-এর scheduler ন্যায্য (fair) হওয়ার চেষ্টা করে: কেউ যাতে বেশি সময় CPU দখল করে অন্যদের আটকে না রাখে, আবার জরুরি কাজ যাতে দেরি না হয়।

Process State Diagram

New → Ready → Running → Terminated

↑ ↓ ↺

Running → Waiting (I/O-র অপেক্ষা) → Ready (I/O শেষ) → Running

Scheduler ঠিক করে Ready অবস্থার মধ্যে কে পরের Running হবে।

৫.৩ Thread: এক process-এর ভেতরে অনেক কাজ

একটি process-এর ভেতরে একাধিক **thread** থাকতে পারে — একই memory ভাগ করে নেওয়া ছোট ছোট কাজের ধারা। উপমা: একটি রান্নাঘরে (process) তিনজন রাঁধুনি (thread) একই উপকরণ ও একই রান্নাঘর ভাগ করে কাজ করছে — একজন সবজি কাটছে, একজন মসলা মেশাচ্ছে, একজন আগুন দেখছে। তারা একে অপরের কাজে হাত দিতে পারে, কারণ memory ভাগ করা।

Process বনাম thread-এর পার্থক্য:

বিষয়	Process	Thread
Memory	নিজের আলাদা memory	একই process-এর memory ভাগ করে
তৈরি করতে খরচ	বেশি (নতুন memory, নতুন অবস্থা)	কম
একটি ভেঙে গেলে	অন্য process বাঁচে	পুরো process ভেঙে যেতে পারে

যোগাযোগ	কঠিন (IPC লাগে)	সহজ (একই memory)
উদাহরণ	Firefox, VS Code	ব্রাউজারের ভেতরের tab, গানের player-এর ভেতরের কাজ

Thread-এর ঝুঁকি: একই memory ভাগ করে নেওয়ায় দুটি thread একই তথ্য একসঙ্গে বদলাতে গেলে তথ্য নষ্ট হতে পারে — একে বলে **race condition**। তাই thread ব্যবহারে সতর্ক সমন্বয় (synchronization) লাগে। DevOps-এ web server-গুলো প্রায়ই thread ব্যবহার করে একসঙ্গে অনেক request সামলাতে — যেমন একটি server ১০০টি thread দিয়ে ১০০টি ব্যবহারকারীর অনুরোধ একসঙ্গে প্রক্রিয়া করে।

৫.৪ Scheduling ও Context Switching — আবার, গভীরে

Chapter 04-এ context switching-এর পরিচয় হয়েছে। এখানে তার খরচ বুঝুন: প্রতিটি switch-এ CPU-কে register সংরক্ষণ করতে হয়, cache-এর তথ্য বদলাতে হয়, memory mapping বদলাতে হয়। Switch বেশি হলে CPU আসল কাজের চেয়ে বদলাতেই বেশি সময় দেয় — একে বলে **thrashing**-এর মতো অবস্থা নয়, তবে overhead বাড়ে। ভালো scheduler কম switch-এ বেশি কাজ করার চেষ্টা করে, আবার কোনো process-কে বেশিক্ষণ অপেক্ষা করায় না।

Linux-এ প্রতিটি process-এর একটি **priority** (nice value) থাকে। Nice value কম মানে অগ্রাধিকার বেশি। `nice` ও `renice` দিয়ে priority বদলানো যায় (Chapter 13-এ হাতে-কলমে)। সাধারণ user নিজের process-এর priority কমাতে পারে, বাড়াতে root লাগে — কারণ সবাই নিজের কাজকে জরুরি বললে scheduling ভেঙে যাবে।

৫.৫ Process Memory Layout: ভেতরে কী কী থাকে

একটি process-এর virtual memory কয়েকটি অঞ্চলে ভাগ করা:

- **Text (Code):** program-এর নির্দেশনা — machine code। সাধারণত শুধু পড়া যায়, বদলানো যায় না।
- **Data:** global variable-এর মান।
- **Heap:** program চলার সময় চেয়ে নেওয়া memory (dynamic allocation — `malloc`, Python-এ list তৈরি)। Heap উপরের দিকে বাড়ে।
- **Stack:** function call-এর তথ্য — কোন function কাকে ডেকেছে, local variable, ফিরে যাওয়ার ঠিকানা। Stack নিচের দিকে বাড়ে।
- **Shared libraries:** অন্য library-র code (যেমন glibc), যা অনেক process ভাগ করে।

Process Memory Layout (সরলীকৃত)

নিচের ঠিকানা (Low) → উপরের ঠিকানা (High)

Text (code) | Data (global) | Heap (↑ বাড়ে — dynamic memory) | ... খালি জায়গা ... | Stack (↓ বাড়ে — function call) | Shared libraries

Heap ও stack একে অপরের দিকে বাড়ে; দুটি মিলে গেলে memory শেষ — stack overflow বা out-of-memory।

৫.৬ Stack বনাম Heap — কোনটি কখন?

Stack: প্রতিটি function call-এ একটি frame তৈরি হয় — local variable, parameter, ফেরার ঠিকানা। Function শেষ হলে frame মুছে যায়। Stack দ্রুত, স্বয়ংক্রিয়, কিন্তু আকার সীমিত। Recursive function অসীমভাবে নিজেকে ডাকলে

stack ভরে যায় — **stack overflow**। **Heap:** program নিজে চেয়ে নেয়, নিজে ছাড়ে (বা garbage collector ছাড়ে)। Heap বড়, নমনীয়, কিন্তু ব্যবস্থাপনা ধীর ও ভুলপ্রবণ — চেয়ে নিয়ে না ছাড়লে **memory leak** হয়, memory ক্রমে বাড়তে থাকে। তানিয়ার `MemoryError`-এর পেছনে প্রায়ই heap-এর অতিরিক্ত ব্যবহার বা leak থাকে।

> **REAL WORLD:** ঢাকার একটি e-commerce সাইটের server প্রতি কয়েকদিনে ধীর হয়ে যেত, restart করলে ঠিক হতো। দেখা গেল একটি service-এ memory leak — প্রতিটি request-এ কিছু memory heap-এ থেকে যাচ্ছে, ছাড়া হচ্ছে না। Restart সাময়িক সমাধান, আসল সমাধান leak খুঁজে বের করা। Chapter 17-তে `free` ও `top` দিয়ে এই বৃদ্ধি ধরা শেখা হবে।

৫.৭ Virtual Memory ও Paging

Chapter 02-এর ধারণা এখন পূর্ণ করুন। প্রতিটি process ভাবে তার কাছে অনেক বড়, একটানা memory আছে। বাস্তবে OS তার memory-কে **page**-এ ভাগ করে (সাধারণত 4 KB) আর একটি **page table** দিয়ে virtual address-কে physical address-এ মেলায়। CPU-র ভেতরের **MMU (Memory Management Unit)** এই অনুবাদ করে।

সুবিধাগুলো: (১) প্রতিটি process আলাদা — একটি অন্যের memory পড়তে পারে না; (২) RAM-এ জায়গা না থাকলে কিছু page disk-এ (swap) রাখা যায়; (৩) একই library-র page অনেক process ভাগ করে নিতে পারে — RAM বাঁচে। অসুবিধা: page RAM-এ না থাকলে (page fault) disk থেকে আনতে হয় — ধীর। Page fault বেশি হলে সিস্টেম ধীর হয়; RAM কম থাকলে page fault বাড়ে।

৫.৮ Zombie ও Orphan Process

- **Zombie process:** process শেষ হয়েছে, কিন্তু তার parent এখনো তার শেষ অবস্থা (exit status) পড়েনি। সে process table-এ মৃতদেহ হিসেবে থেকে যায় — memory প্রায় খায় না, কিন্তু table-এর জায়গা দখল করে। Parent ঠিকমতো `wait()` করলে zombie মুছে যায়। Parent নিজে মারা গেলে zombie-কে PID 1 দত্তক নিয়ে মুছে ফেলে।
- **Orphan process:** parent মারা গেছে, কিন্তু child বেঁচে আছে। তাকে PID 1 (systemd) দত্তক নেয়, সে চলতে থাকে। Orphan নিজে ক্ষতিকর নয় — অনেক daemon ইচ্ছে করে orphan হয়ে background-এ চলে।

`ps aux`-এ zombie-র STAT কলামে `Z` দেখা যায়। অনেক zombie জমা মানে parent program-এ bug — সে child-দের খোঁজ নিচ্ছে না।

৫.৯ পরিভাষা — Terminology

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Process / PID	চলমান program / তার পরিচয় নম্বর	৫
Thread	একই process-এর ভেতরে memory ভাগ করে চলা কাজের ধারা	৫
Scheduler / Priority	CPU সময় ভাগ করার ব্যবস্থা / process-এর অগ্রাধিকার	৫
Stack / Heap	function call-এর স্মৃতি / চলার সময় চেয়ে নেওয়া স্মৃতি	৫
Virtual Memory / Page	প্রতিটি process-কে বড় memory-র ভান করানো / memory-র 4 KB টুকরা	৫
Page Fault / MMU	প্রয়োজনীয় page RAM-এ না পাওয়া / virtual থেকে physical অনুবাদকারী hardware	৫

Zombie / Orphan	শেষ হয়েও parent-এর অপেক্ষায় থাকা process / parent মারা যাওয়া child process	৫
Race Condition	দুটি thread একই তথ্য একসঙ্গে বদলাতে গিয়ে তথ্য নষ্ট হওয়া	৫
Memory Leak	চেয়ে নেওয়া memory না ছাড়ায় memory ক্রমে বাড়া	৫

৫.১০ হাতে-কলমে অনুশীলন: Process দেখা ও বোঝা

```
ps aux | head -15
pstree -p | head -20
top -b -n 1 | head -20
```

উদাহরণ আউটপুট (illustrative):

```
$ ps aux | head -5
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  23712  3120 ?        Ss   10:00   0:01 /sbin/init
root         850  0.0  0.2  15000  5000 ?        Ss   10:00   0:00 /usr/sbin/sshd
nahid       1200  0.5  1.2  500000 90000 ?       Sl   10:05   0:10 firefox

$ pstree -p | head -5
systemd(1)──sshd(850)──sshd(1200)──bash(1201)──pstree(1300)
```

ব্যাখ্যা: `ps aux`-এ `%CPU` ও `%MEM` বলছে প্রতিটি process কত সম্পদ খাচ্ছে; `VSZ` হলো virtual memory আকার, `RSS` হলো আসলে RAM-এ কতটুকু আছে। `STAT`-এ `S` মানে sleeping (অপেক্ষায়), `R` মানে running। `pstree` দেখাচ্ছে `systemd(1)` থেকে `sshd`, তারপর `bash` — parent-child সম্পর্ক। একটি process বন্ধ করতে `kill PID` ব্যবহার হয় — বিস্তারিত Chapter 13-এ। এখনই অচেনা process kill করবেন না।

> **WARNING:** `kill -9` দিয়ে process বন্ধ করা শেষ উপায়। এটি process-কে পরিষ্কার করার সুযোগ দেয় না — খোলা ফাইল অসম্পূর্ণ থেকে যেতে পারে, তথ্য হারাতে পারে। প্রথমে সাধারণ `kill PID` (SIGTERM) ব্যবহার করুন; কাজ না হলে কারণ খুঁজুন, তারপর `kill -9` ভাবুন। বিস্তারিত Chapter 13-এ।

৫.১১ Common Mistakes ও Troubleshooting

- **ভুল:** "VSZ বড় মানেই RAM শেষ" — না। VSZ-এ shared library ও virtual mapping থাকে; আসল RAM ব্যবহার দেখতে RSS দেখুন।
- **ভুল:** "বেশি thread মানেই দ্রুত" — না। Thread বেশি হলে context switch ও race condition বাড়ে; CPU core-এর চেয়ে অনেক বেশি সক্রিয় thread লাভের চেয়ে ক্ষতি বেশি।
- **ভুল:** Zombie দেখে আতঙ্ক — একটি-দুটি zombie স্বাভাবিক; শত শত zombie মানে parent-এ bug।
- **সমস্যা:** Program ধীর — `top`-এ দেখুন সেটি CPU খাচ্ছে (গণনা বেশি) নাকি waiting অবস্থায় আছে (I/O-র অপেক্ষা)। দুটির সমাধান আলাদা।
- **সমস্যা:** `MemoryError` — heap ব্যবহার দেখুন; বড় ফাইল একসঙ্গে memory-তে না তুলে টুকরো টুকরো করে প্রক্রিয়া করুন।

৫.১২ Security Considerations

প্রতিটি process নিজের user হিসেবে চলে — একটি process অন্য user-এর process-এর memory পড়তে পারে না। এই সীমানা ভেঙে গেলে (যেমন একটি service root হিসেবে চলছে অথচ তার root দরকার নেই) একটি আক্রমণ পুরো সিস্টেমে ছড়াতে পারে। তাই service-গুলোকে সবচেয়ে কম privilege-এ চালানো হয় (least privilege) — Chapter 12 ও 19-এ বিস্তারিত। একইভাবে process-এর memory-তে password বা token রাখা ঝুঁকিপূর্ণ — memory dump থেকে তা বের করা যায়।

৫.১৩ Best Practices

- Process-এর resource ব্যবহার নিয়মিত দেখুন — হঠাৎ বৃদ্ধি leak বা আক্রমণের লক্ষণ হতে পারে।
- Background-এ চালানো program-এর log রাখুন, যাতে পরে বোঝা যায় সেটি কী করছিল।
- একটি process-কে প্রয়োজনের বেশি memory বা CPU দেবেন না; limit নির্ধারণ করুন (systemd-এ সম্ভব, Chapter 15)।

CHECK YOUR UNDERSTANDING:

1. Program ও process-এর পার্থক্য কী? A. কোনো পার্থক্য নেই B. Program নিষ্ক্রিয় নির্দেশনা, process চলমান program C. Process disk-এ থাকে, program RAM-এ D. Program বড়, process ছোট
2. Thread কী? A. একটি আলাদা program B. একই process-এর ভেতরে memory ভাগ করে চলা কাজের ধারা C. একটি ফাইল D. একটি hardware
3. Stack overflow কখন হয়? A. Disk পূর্ণ হলে B. Recursive function অসীমভাবে ডাকলে stack ভরে গেলে C. RAM নষ্ট হলে D. নেটওয়ার্ক বন্ধ হলে
4. Zombie process কী? A. ভাইরাস B. শেষ হয়েও parent-এর অপেক্ষায় process table-এ থাকা process C. চলমান process D. Orphan process
5. Virtual memory-র page আকার সাধারণত কত? A. 1 byte B. 4 KB C. 1 MB D. 1 GB

Knowledge Check — জ্ঞান যাচাই (উত্তর Answer Guide-এ)

1. Program ও process-এর পার্থক্য কী? A) দুটি একই জিনিস B) Program ডিস্কের নিষ্ক্রিয় ফাইল, process RAM-এ চলমান অবস্থা C) Process ডিস্কে থাকে D) Program RAM-এ থাকে
2. Zombie process কী? A) CPU 100% খাওয়া process B) মৃত কিন্তু parent-এর স্বীকৃতির অপেক্ষায় থাকা process C) ঘুমন্ত process D) নতুন process
3. Stack-এ সাধারণত কী থাকে? A) বড় ডেটা B) Function call ও local variable C) ফাইল D) Kernel
4. Virtual memory-র সুবিধা কী? A) RAM বেড়ে যায় B) প্রতিটি process আলাদা ঠিকানা-জগৎ পায়, সুরক্ষা বাড়ে C) ডিস্ক দ্রুত হয় D) CPU বেড়ে যায়
5. Context switching-এ কী ঘটে? A) কম্পিউটার বন্ধ হয় B) CPU এক process থেকে আরেক process-এ যায় ও অবস্থা সংরক্ষণ করে C) ফাইল মুছে যায় D) RAM খালি হয়

৫.১৪ Practical Assignment

Assignment 5.1: আপনার সিস্টেমে `ps aux --sort=-%mem | head -10` চালিয়ে সবচেয়ে বেশি memory ব্যবহারকারী ৫টি process চিহ্নিত করুন। প্রতিটির জন্য লিখুন: নাম, PID, %MEM, RSS, আর এটি কী কাজ করছে বলে মনে করেন। এরপর `pstree -p` দিয়ে দেখুন তাদের parent কে। একটি সংক্ষিপ্ত সিদ্ধান্ত লিখুন: কোনো process কি অস্বাভাবিক memory খাচ্ছে?

৫.১৫ Chapter Summary

Program নিষ্ক্রিয় নির্দেশনা; OS তাকে RAM-এ তুলে চালালে সে process হয় — নিজের PID, memory ও অবস্থাসহ। Process new, ready, running, waiting ও terminated অবস্থার মধ্য দিয়ে যায়, scheduler তাদের মধ্যে CPU ভাগ করে। Thread একই process-এর ভেতরে memory ভাগ করে সমান্তরালে কাজ করে, তবে race condition-এর ঝুঁকি থাকে। Process memory-তে text, data, heap ও stack থাকে; heap চলার সময় চেয়ে নেওয়া হয়, stack function call সামলায়। Virtual memory page-এ ভাগ করে প্রতিটি process-কে আলাদা ও নিরাপদ জগৎ দেয়। Zombie হলো শেষ হয়েও parent-এর অপেক্ষায় থাকা process, orphan হলো parent হারানো child।

Revision Checklist

- ☐ Program ও process-এর পার্থক্য ব্যাখ্যা করতে পারি।
- ☐ Process lifecycle-এর পাঁচটি অবস্থা বলতে পারি।
- ☐ Thread ও process-এর পার্থক্য বলতে পারি।
- ☐ Stack ও heap-এর পার্থক্য ও ঝুঁকি (overflow, leak) বলতে পারি।
- ☐ Virtual memory ও paging ব্যাখ্যা করতে পারি।
- ☐ Zombie ও orphan চিহ্নিত করতে পারি।

PART II • DEVOPS ENGINEERING MASTERY

Linux Foundations

লিনাক্সের ভিত্তি — ইতিহাস, ইনস্টলেশন, ফাইলসিস্টেম, টার্মিনাল ও অপরিহার্য কমান্ড।

Chapter 06 — Introduction to Linux and Its Ecosystem
লিনাক্স পরিচিতি ও এর ইকোসিস্টেম

Part II — Linux Foundations | Ubuntu Server 24.04 LTS | আনুমানিক পড়ার সময়: ৫০ মিনিট

শুরুতে: একটা বাস্তব দৃশ্য

ঢাকার একজন জুনিয়র ডেভেলপার, ধরুন নাম রাফি, একটি ছোট সফটওয়্যার কোম্পানিতে কাজ শুরু করলেন। প্রথম দিনই টিম লিড বললেন, “আমাদের অ্যাপটি AWS-এর একটা Ubuntu সার্ভারে চলে। SSH দিয়ে ঢুকে log দেখো, সার্ভিসটা restart করো।” রাফি Windows-এ ভালো কোড লিখতে পারেন, কিন্তু terminal-এর কালো স্ক্রিন দেখে তিনি থমকে গেলেন। কোথায় ফাইল আছে, কোন কমান্ডে সার্ভিস চলে, ভুল করলে কী হবে — কিছুই জানা নেই। এই অধ্যায়টি সেই শূন্যস্থান পূরণের শুরু। Linux শুধু আরেকটি operating system নয়; এটি আধুনিক সার্ভার, ক্লাউড ও DevOps জগতের মূল ভাষা। এই অধ্যায় শেষে আপনি বুঝবেন Linux কী, কেন এটি সার্ভারে প্রায় সর্বত্র ব্যবহৃত হয়, kernel আর distribution-এর পার্থক্য কী, এবং কোন distribution কখন বেছে নিতে হয়।

> **REAL WORLD:** বিশ্বের বেশিরভাগ ওয়েব সার্ভার, Android ফোনের ভিত্তি, সুপারকম্পিউটার, রাউটার এবং ক্লাউড প্ল্যাটফর্ম Linux-এর ওপর চলে। একজন DevOps ইঞ্জিনিয়ারের দৈনন্দিন কাজ — deployment, monitoring, troubleshooting — প্রায় সবই Linux পরিবেশে হয়। তাই Linux না জানলে DevOps শেখা অসম্পূর্ণ থেকে যায়।

Learning Objectives — শেখার লক্ষ্য

- Linux, kernel, GNU ও distribution-এর সংজ্ঞা নিজের ভাষায় ব্যাখ্যা করতে পারবেন।
- Linux-এর ইতিহাসের প্রধান মাইলফলকগুলো সময়ক্রমে বলতে পারবেন।
- Open-source লাইসেন্স ও free software ধারণার পার্থক্য ব্যাখ্যা করতে পারবেন।
- Ubuntu, Debian, Fedora ও RHEL-এর মধ্যে পার্থক্য ও ব্যবহারের ক্ষেত্র চিহ্নিত করতে পারবেন।
- কোন কাজের জন্য কোন distribution উপযুক্ত, তা যুক্তিসহ নির্বাচন করতে পারবেন।
- Linux সার্ভার, ক্লাউড ও DevOps-এ কেন প্রাধান্য পায়, তার পাঁচটি কারণ বলতে পারবেন।

Prerequisites — পূর্বজ্ঞান

এই অধ্যায়ের জন্য Chapter 03 ও 04-এর ধারণা কাজে লাগবে: operating system কী করে, kernel কী, user space ও kernel space-এর পার্থক্য। প্রোগ্রামিংয়ের সাধারণ ধারণা থাকলেই চলবে; Linux-এর আগের অভিজ্ঞতা দরকার নেই।

Linux কী? প্রথম নীতি থেকে বোঝা

সহজ উপমা দিয়ে শুরু করা যাক। একটি বাড়িকে operating system ভাবুন: kernel হলো বাড়ির ভিত্তি, বিদ্যুৎ ও পানির লাইন — যা ছাড়া কিছুই চলে না। আর distribution হলো পুরো সাজানো বাড়ি: ভিত্তির ওপর দেয়াল, দরজা, আসবাব, রান্নাঘর — অর্থাৎ ব্যবহারযোগ্য সম্পূর্ণ ব্যবস্থা। Linux বলতে কঠোর অর্থে শুধু kernel-কে বোঝায় — সেই মূল প্রোগ্রামটি যা hardware-এর সঙ্গে কথা বলে, CPU, memory ও device পরিচালনা করে। কিন্তু দৈনন্দিন ভাষায় আমরা “Linux” বলতে

পুরো operating system-কেই বুঝি, যেখানে kernel-এর সঙ্গে GNU tools, shell, package manager ও অন্যান্য সফটওয়্যার যুক্ত থাকে। এই পার্থক্যটি বোঝা গুরুত্বপূর্ণ, কারণ চাকরির ইন্টারভিউতে এ প্রশ্নটি বারবার আসে।

Linux-এর জন্ম ১৯৯১ সালে। ফিনল্যান্ডের হেলসিংকি বিশ্ববিদ্যালয়ের ছাত্র Linus Torvalds নিজের প্রয়োজনে একটি ছোট kernel লিখতে শুরু করেন। তিনি এটি ইন্টারনেটে প্রকাশ করেন এবং সারা বিশ্বের প্রোগ্রামাররা এতে অবদান রাখতে শুরু করেন। একই সময়ে Richard Stallman-এর নেতৃত্বে GNU Project একটি সম্পূর্ণ free operating system তৈরির চেষ্টা করছিল, কিন্তু তাদের kernel (Hurd) তখনও প্রস্তুত ছিল না। Linux kernel আর GNU-এর tools একত্রিত হয়ে একটি সম্পূর্ণ, ব্যবহারযোগ্য operating system তৈরি হলো। তাই অনেকে একে GNU/Linux বলেন — এটি শুধু নামের বিতর্ক নয়, দুটি আলাদা প্রকল্পের অবদানের স্বীকৃতি।

ইতিহাসের মাইলফলক

সাল	ঘটনা	গুরুত্ব
১৯৮৩	GNU Project শুরু	Free software আন্দোলন ও GNU tools-এর ভিত্তি
১৯৯১	Linus Torvalds Linux kernel 0.01 প্রকাশ	আধুনিক Linux-এর জন্ম
১৯৯২	Linux GPL লাইসেন্সের আওতায় আসে	যে কেউ ব্যবহার, পরিবর্তন ও বিতরণ করতে পারে
১৯৯৩	Debian ও Slackware প্রকাশ	প্রথম দিকের distribution
২০০৪	Ubuntu প্রথম প্রকাশ	ব্যবহারবান্ধব desktop ও সার্ভার Linux জনপ্রিয় হয়
২০০৮	Android (Linux kernel-ভিত্তিক) প্রকাশ	মোবাইলে Linux-এর বিস্তার
২০১৩–বর্তমান	Docker, Kubernetes ও ক্লাউড যুগ	সার্ভার ও DevOps-এ Linux-এর আধিপত্য

এই সময়রেখা থেকে একটি শিক্ষা স্পষ্ট: Linux কোনো একক কোম্পানির পণ্য নয়। এটি হাজার হাজার অবদানকারীর সম্মিলিত কাজ, যা একটি উন্মুক্ত লাইসেন্সের অধীনে পরিচালিত হয়।

Open Source ও Free Software: দুটি কাছাকাছি কিন্তু আলাদা ধারণা

অনেকেই “free” শব্দটি শুনে ভাবেন Linux বিনামূল্যে পাওয়া যায় বলে এটি free। আসলে এখানে free মানে “স্বাধীনতা”, শুধু “বিনামূল্য” নয়। Free Software Foundation-এর সংজ্ঞায় চারটি স্বাধীনতা আছে: প্রোগ্রাম চালানোর, source code দেখার ও বদলানোর, কপি বিতরণের এবং পরিবর্তিত সংস্করণ বিতরণের স্বাধীনতা। Open Source Initiative একই ধরনের ব্যবহারিক ধারণা দেয়, তবে জোর দেয় source code খোলা থাকার ওপর। Linux kernel GNU General Public License (GPL) v2-এর অধীনে প্রকাশিত। GPL-এর মূল শর্ত: আপনি kernel পরিবর্তন করে বিতরণ করলে পরিবর্তিত source code-ও খোলা রাখতে হবে। এই শর্তই নিশ্চিত করে যে Linux কারও একার সম্পত্তি হয়ে যায় না। একজন শিক্ষার্থী হিসেবে এর ব্যবহারিক অর্থ হলো — আপনি বিনামূল্যে Ubuntu ডাউনলোড করতে পারেন, VirtualBox-এ চালাতে পারেন, ভেঙে আবার বানাতে পারেন, কোনো লাইসেন্স ফি ছাড়াই।

Kernel বনাম Distribution: ভেতরের স্থাপত্য

কেন একটাই kernel থেকে শত শত আলাদা “Linux” তৈরি হয়? কারণ kernel একা ব্যবহারযোগ্য operating system নয়। একটি distribution হলো kernel + GNU core utilities + shell + package manager + installer + ডিফল্ট কনফিগারেশন + সাপোর্ট ব্যবস্থার সমন্বয়। উদাহরণ: Ubuntu ও Debian একই পরিবারের, কিন্তু Ubuntu-তে নতুন প্যাকেজ,

নিয়মিত release ও বাণিজ্যিক সাপোর্ট (Canonical) যুক্ত হয়েছে। Fedora হলো Red Hat-এর পরীক্ষামূলক ক্ষেত্র, যেখানে নতুন প্রযুক্তি আগে আসে; আর RHEL (Red Hat Enterprise Linux) হলো স্থিতিশীল, দীর্ঘমেয়াদি সাপোর্টসহ বাণিজ্যিক সংস্করণ।

চিত্র ৬.১ — Linux Distribution-এর স্তরবিন্যাস (নিচ থেকে উপরে)

Hardware → Linux Kernel (CPU, memory, device পরিচালনা) → GNU Core Utilities + System Libraries (glibc) → Shell (Bash) ও System Services (systemd) → Package Manager (APT / DNF) → Applications (Nginx, PostgreSQL, Python) → User / Administrator

নিচের স্তর বদলালে উপরের স্তরও প্রভাবিত হয়; কিন্তু application স্তর একই রেখে distribution বদলানো যায়। এজন্যই একই Nginx Ubuntu ও RHEL — দুটোতেই চলে, যদিও install কমান্ড আলাদা।

চিত্র ৬.২ — প্রধান Linux পরিবার

Debian পরিবার: Debian → Ubuntu → Linux Mint | Ubuntu Server 24.04 LTS (এই বইয়ের মূল পরিবেশ) | RHEL পরিবার: RHEL → Fedora → CentOS Stream / Rocky Linux / AlmaLinux | স্বতন্ত্র: Arch Linux (rolling release), openSUSE, Alpine (হালকা, container-এ জনপ্রিয়)

পরিবার বোঝা জরুরি, কারণ একই পরিবারের distribution-গুলোতে package manager ও ফাইলের অবস্থান প্রায় একই থাকে।

প্রধান Distribution-গুলোর তুলনা

Distribution	Package Manager	Release ধরন	সেরা ব্যবহার	এই বইয়ে ভূমিকা
Ubuntu Server 24.04 LTS	APT (dpkg)	LTS: ৫ বছর সাপোর্ট	সার্ভার, ক্লাউড, DevOps শেখা	প্রধান পরিবেশ
Debian	APT (dpkg)	স্থিতিশীল, ধীর release	স্থিতিশীল সার্ভার	Ubuntu-এর ভিত্তি
Fedora	DNF (RPM)	৬ মাসে নতুন release	নতুন প্রযুক্তি পরীক্ষা	তুলনার জন্য
RHEL / Rocky / AlmaLinux	DNF (RPM)	দীর্ঘমেয়াদি enterprise সাপোর্ট	বড় প্রতিষ্ঠানের সার্ভার	তুলনার জন্য
Arch Linux	pacman	Rolling release	অভিজ্ঞ ব্যবহারকারী, শেখার গভীরতা	শুধু পরিচিতি

LTS মানে Long Term Support। Ubuntu 24.04 LTS এপ্রিল ২০২৪-এ প্রকাশিত এবং এপ্রিল ২০২৯ পর্যন্ত সমর্থিত। সার্ভারের জন্য LTS বেছে নেওয়া হয়, কারণ সার্ভারে ঘন ঘন বড় পরিবর্তন ঝুঁকিপূর্ণ। Desktop-এ নতুন ফিচার চাইলে interim release চলতে পারে, কিন্তু production সার্ভারে স্থিতিশীলতাই প্রথম।

কেন সার্ভার ও DevOps-এ Linux?

প্রথম কারণ, খরচ ও লাইসেন্স: কোনো per-server লাইসেন্স ফি নেই, ফলে ছোট কোম্পানি ও শিক্ষার্থী একই পরিবেশে শিখতে পারে। দ্বিতীয়, স্থিতিশীলতা ও কর্মক্ষমতা: Linux সার্ভার মাসের পর মাস restart ছাড়া চলতে পারে এবং কম resource-এ বেশি কাজ করে। তৃতীয়, নিরাপত্তা মডেল: permission, user ও process আলাদা রাখার নকশা শুরু থেকেই আছে। চতুর্থ, automation: প্রায় সবকিছুই command line ও script দিয়ে করা যায় — এটাই DevOps-এর প্রাণ। পঞ্চম, ecosystem: Nginx, Docker, Kubernetes, PostgreSQL, Python — DevOps-এর প্রায় সব গুরুত্বপূর্ণ টুল Linux-কে প্রথম শ্রেণির নাগরিক হিসেবে ধরে। Windows Server-ও বৈধ পছন্দ, বিশেষ করে .NET পরিবেশে, কিন্তু বাংলাদেশের ছোট সফটওয়্যার কোম্পানি ও ক্লাউডে Linux-এর চাহিদা স্পষ্টভাবে বেশি।

> **CONCEPT:** Distribution বদলালে Linux বদলায় না — kernel, shell ও Unix দর্শন একই থাকে। তাই Ubuntu শিখলে Debian-এ কাজ করা সহজ হবে; শুধু package manager ও কিছু পথের নাম বদলাবে। মূল দক্ষতা হস্তান্তরযোগ্য (transferable)।

বাস্তব উদাহরণ

চট্টগ্রামের একটি ছোট e-commerce স্টার্টআপের কথা ভাবুন। তাদের ওয়েবসাইট Ubuntu সার্ভারে Nginx + PostgreSQL দিয়ে চলে, কোড GitHub থেকে deploy হয়, backup রাতে rsync দিয়ে নেওয়া হয়। এই পুরো ব্যবস্থা একজন জুনিয়র অ্যাডমিন Linux জ্ঞান দিয়েই পরিচালনা করেন — কোনো দামি proprietary সফটওয়্যার ছাড়া। আরেকটি উদাহরণ: বিশ্ববিদ্যালয়ের ল্যাবে পুরোনো কম্পিউটারগুলোতে হালকা Linux (যেমন Lubuntu বা Debian) বসিয়ে সেগুলো আবার ব্যবহারযোগ্য করা যায়, যেখানে Windows 11 চলত না।

প্রায়োগিক কমান্ড — আপনার সিস্টেম চেনা

এই অধ্যায়ে এখনও install করা হয়নি — Chapter 07-এ install শেখানো হবে। তবে আপনার কম্পিউটারে ইতিমধ্যে Linux থাকলে বা Live USB থেকে চালালে নিচের কমান্ডগুলো চেষ্টা করুন। প্রতিটি কমান্ডের কাজ ব্যাখ্যাসহ দেওয়া হলো।

```
uname -a
# kernel-এর নাম, সংস্করণ ও architecture দেখায়। -a মানে সব তথ্য।

lsb_release -a
# distribution-এর নাম ও সংস্করণ দেখায় (Ubuntu-তে কাজ করে)।

cat /etc/os-release
# distribution শনাক্ত করার নির্ভরযোগ্য ফাইল।

hostnamectl
# hostname, kernel ও operating system-এর সংক্ষিপ্ত বিবরণ দেখায়।
```

উদাহরণ আউটপুট (illustrative):

```
Ubuntu 24.04.1 LTS
Linux ubuntu-server 6.8.0-31-generic #31-Ubuntu SMP x86_64 GNU/Linux
# ব্যাখ্যা: প্রথম লাইন distribution, দ্বিতীয় লাইনে kernel সংস্করণ 6.8.0
# ও architecture x86_64 দেখা যাচ্ছে। আপনার সংস্করণ ভিন্ন হতে পারে।
```

Terminology — পরিভাষা

Term	Bangla ব্যাখ্যা
Kernel	Operating system-এর মূল অংশ; hardware পরিচালনা করে
Distribution	Kernel + GNU tools + package manager + installer-এর সম্পূর্ণ ব্যবস্থা
GNU	Free operating system তৈরির প্রকল্প; core utilities-এর উৎস
GPL	GNU General Public License — পরিবর্তিত code খোলা রাখার শর্তসহ লাইসেন্স
LTS	Long Term Support — দীর্ঘমেয়াদি সমর্থনসহ স্থিতিশীল release
Package manager	সফটওয়্যার ইনস্টল, আপডেট ও মুছে ফেলার টুল (যেমন APT, DNF)
Repository	অনুমোদিত প্যাকেজের ভান্ডার, যেখান থেকে package manager ডাউনলোড করে

Hands-on Exercise ৬.১ — নিজের পরিবেশ শনাক্ত করা

LAB: উদ্দেশ্য — কোনো Linux সিস্টেমে distribution ও kernel শনাক্ত করা। ধাপ: (১) terminal খুলুন; (২) `uname -a` চালান ও kernel সংস্করণ লিখে রাখুন; (৩) `cat /etc/os-release` চালান ও NAME, VERSION_ID লিখুন; (৪) `hostnamectl` চালিয়ে machine ID-এর সঙ্গে তুলনা করুন; (৫) আপনার নোটে লিখুন — এটি কোন পরিবারের distribution (Debian না RHEL) এবং কেন তা মনে হলো। যাচাই: তিনটি কমান্ডের তথ্য পরস্পরের সঙ্গে মিলতে হবে।

Common Mistakes & Troubleshooting

- **ভুল:** “Linux” আর “Ubuntu” এক মনে করা। **সংশোধন:** Ubuntu একটি distribution; Linux হলো kernel।
- **ভুল:** Rolling release দিয়ে production সার্ভার চালানো। **সংশোধন:** সার্ভারে LTS ব্যবহার করুন।
- **ভুল:** `lsb_release` না পেলে হতাশ হওয়া। **সমাধান:** `cat /etc/os-release` প্রায় সব distribution-এ কাজ করে।
- **ভুল:** অপরিচিত সাইট থেকে .deb ফাইল ডাউনলোড করে install করা। **সংশোধন:** প্রথমে অফিসিয়াল repository ব্যবহার করুন (Chapter 14)।

Security Considerations

Linux “নিরাপদ” বলে অন্ধভাবে বিশ্বাস করবেন না। নিরাপত্তা আসে সঠিক কনফিগারেশন থেকে: নিয়মিত update, অপ্রয়োজনীয় service বন্ধ রাখা, শক্তিশালী পাসওয়ার্ড ও SSH key ব্যবহার। শুধু Linux ব্যবহার করলেই সার্ভার নিরাপদ হয় না — Chapter 19-এ বিস্তারিত শেখানো হবে।

Best Practices

- শেখার জন্য LTS সংস্করণ ব্যবহার করুন; পরীক্ষামূলক কাজ আলাদা VM-এ করুন।
- কোনো distribution বেছে নেওয়ার আগে তার support সময়কাল ও কমিউনিটি দেখুন।
- অফিসিয়াল ডকুমেন্টেশন (ubuntu.com, kernel.org) পড়ার অভ্যাস করুন।

- নিজের শেখা নোটে distribution, kernel ও package manager একসঙ্গে লিখে রাখুন।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING:

1. Linux বলতে কঠোর অর্থে কী বোঝায়? A) Ubuntu operating system B) Linux kernel C) GNU tools D) Bash shell

2. GNU/Linux নামে GNU উল্লেখের মূল কারণ কী? A) GNU kernel ব্যবহার B) GNU tools ছাড়া সম্পূর্ণ OS হয় না C) GNU কোম্পানি D) GNU একটি distribution

3. Ubuntu 24.04 LTS কতদিন সমর্থিত? A) ১ বছর B) ৩ বছর C) ৫ বছর D) ১০ বছর

4. Fedora কোন পরিবারের distribution? A) Debian B) RHEL C) Arch D) SUSE

5. Production সার্ভারে LTS বেছে নেওয়ার প্রধান কারণ কী? A) নতুন ফিচার B) স্থিতিশীলতা ও দীর্ঘ সাপোর্ট C) কম দাম D) ছোট আকার

Practical Assignment

এক পৃষ্ঠার তুলনামূলক নোট তৈরি করুন: Ubuntu Server, Debian ও Rocky Linux — প্রতিটির package manager, release ধরন, support সময়কাল ও একটি উপযুক্ত ব্যবহার লিখুন। শেষে লিখুন, আপনি নিজের শেখার সার্ভারের জন্য কোনটি বেছে নেবেন এবং কেন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

Linux হলো একটি kernel, যার সঙ্গে GNU tools ও অন্যান্য সফটওয়্যার যুক্ত হয়ে distribution তৈরি হয়। ১৯৯১ সালে শুরু হওয়া এই প্রকল্প open-source লাইসেন্সের কারণে সারা বিশ্বে ছড়িয়ে পড়েছে। Debian ও RHEL — দুটি প্রধান পরিবার — থেকে Ubuntu, Fedoraসহ নানা distribution তৈরি হয়েছে। সার্ভার ও DevOps-এ Linux প্রাধান্য পায় স্থিতিশীলতা, কম খরচ, automation ও বিশাল ecosystem-এর কারণে। এই বইয়ের মূল পরিবেশ Ubuntu Server 24.04 LTS।

Revision Checklist

- ☐ kernel বনাম distribution ব্যাখ্যা করতে পারি
- ☐ GNU/Linux নামের কারণ বলতে পারি
- ☐ Ubuntu, Debian, Fedora, RHEL-এর পার্থক্য জানি
- ☐ LTS-এর অর্থ ও গুরুত্ব বুঝি
- ☐ `uname -a` ও `cat /etc/os-release` চালাতে পারি

Chapter 07 — Linux Installation, Distributions and Boot Process
লিনাক্স ইনস্টলেশন, ডিস্ট্রিবিউশন ও বুট প্রক্রিয়া

Part II — Linux Foundations | Ubuntu Server 24.04 LTS | আনুমানিক পড়ার সময়: ৬০ মিনিট

শুরুতে: কেন ইনস্টল শেখা জরুরি?

একজন শিক্ষার্থী প্রথমবার সার্ভার ইনস্টল করতে গিয়ে প্রায়ই দুটি ভয়ে আটকে যান: “ভুল করলে কি আমার কম্পিউটারের সব ফাইল মুছে যাবে?” এবং “পার্টিশন কী জিনিস?” এই ভয় যৌক্তিক। কিন্তু সমাধান সহজ: আসল কম্পিউটার স্পর্শ না করে VirtualBox-এর ভেতরে একটি ভার্চুয়াল মেশিনে (VM) ইনস্টল শেখা। VM ভাঙলেও আপনার আসল কম্পিউটারের কিছু হয় না; VM মুছে আবার বানানো যায়। এই অধ্যায়ে আপনি VirtualBox-এ Ubuntu Server 24.04 LTS ইনস্টল করবেন, partitioning বুঝবেন, এবং কম্পিউটার চালু হওয়ার মুহূর্ত থেকে login prompt আসা পর্যন্ত ভেতরে কী ঘটে, তা ধাপে ধাপে শিখবেন।

Learning Objectives — শেখার লক্ষ্য

- VirtualBox-এ ২ vCPU, ৪ GB RAM ও ২৫ GB disk-এর VM তৈরি করতে পারবেন।
- Ubuntu Server 24.04 LTS ISO ডাউনলোড করে VM-এ ইনস্টল করতে পারবেন।
- Partition, filesystem ও mount point-এর সম্পর্ক ব্যাখ্যা করতে পারবেন।
- BIOS/UEFI, GRUB, kernel ও systemd-এর ভূমিকা ক্রমানুসারে বলতে পারবেন।
- Boot ব্যর্থ হলে কোন ধাপে সমস্যা, তা শনাক্ত করতে পারবেন।
- প্রথম boot-এর পর প্রাথমিক কনফিগারেশন (update, hostname, SSH) সম্পন্ন করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 02 (storage, RAM), Chapter 06 (distribution, LTS) ও Chapter 03 (OS-এর ভূমিকা) জানা থাকলে এই অধ্যায় সহজ হবে। Windows বা macOS-এ ফাইল ডাউনলোড ও সফটওয়্যার ইনস্টল করার অভিজ্ঞতা যথেষ্ট।

কোন পরিবেশে শিখবেন? তিনটি পথ

প্রথম পথ, **VirtualBox VM (সুপারিশকৃত)**: আপনার বর্তমান কম্পিউটারের ভেতরে একটি আলাদা “কম্পিউটার” চালানো। সুবিধা — বিনামূল্যে, নিরাপদ, snapshot নিয়ে ভুল থেকে ফিরে আসা যায়। অসুবিধা — কিছুটা ধীর হতে পারে। দ্বিতীয় পথ, **সরাসরি পুরোনো কম্পিউটারে install**: শেখার জন্য ভালো, তবে ভুল করলে ডেটা হারানোর ঝুঁকি। তৃতীয় পথ, **Cloud VPS**: DigitalOcean, Hetzner বা AWS-এ ভাড়া করা ভার্চুয়াল সার্ভার। সুবিধা — আসল সার্ভারের অভিজ্ঞতা; অসুবিধা — মাসিক খরচ, ভুল কনফিগারেশনে বিল বা নিরাপত্তা ঝুঁকি। এই বইয়ের নিয়ম: মৌলিক শেখার জন্য কখনও paid cloud বাধ্যতামূলক নয়। প্রথমে VirtualBox-এ শিখুন; আত্মবিশ্বাস বাড়লে, প্রয়োজনে VPS চেষ্টা করুন।

> **WARNING:** Cloud VPS ব্যবহার করলে খরচের সতর্কতা: বেশিরভাগ VPS ঘণ্টা/মাস হিসেবে চার্জ করে, বন্ধ না করলে বিল চলতে থাকে। SSH পোর্ট খোলা রেখে দুর্বল পাসওয়ার্ড ব্যবহার করবেন না — বটগুলো কয়েক মিনিটের মধ্যে আক্রমণ শুরু করে। VPS শুধু তখনই নিন, যখন আপনি Chapter 19-এর SSH নিরাপত্তা বুঝেছেন।

Partitioning: ডিস্ক ভাগ করার ধারণা

উপমা: একটি বড় গুদামকে আলমারি দিয়ে ভাগ করার মতোই partitioning। একটি ফিজিক্যাল ডিস্ককে একাধিক যৌক্তিক ভাগে (partition) ভাগ করা হয়, যাতে প্রতিটি ভাগে আলাদা filesystem বসানো যায়। Ubuntu Server ইনস্টলে সাধারণত অন্তত দুটি partition লাগে: EFI System Partition (UEFI boot-এর জন্য, প্রায় ৫১২ MB, FAT32) এবং root partition (`/`) যেখানে পুরো system থাকে। আলাদা `/home` partition থাকলে ব্যবহারকারীর ফাইল system পুনরায় ইনস্টল করলেও টিকে থাকে; আলাদা swap partition বা swap file RAM পূর্ণ হলে অস্থায়ী জায়গা দেয়। নতুনদের জন্য Ubuntu-এর guided "Use entire disk" বিকল্প নিরাপদ — VM-এ এটি ব্যবহার করুন। আসল কম্পিউটারে dual-boot করতে চাইলে আগে সম্পূর্ণ backup নিন।

Partition table-এর দুটি ধরন জানুন: পুরোনো **MBR** (সর্বোচ্চ ৪টি primary partition, ২ TB সীমা) ও আধুনিক **GPT** (অনেক partition, বড় ডিস্ক সমর্থন, UEFI-এর সঙ্গে ব্যবহৃত হয়)। Ubuntu 24.04 ইনস্টলার GPT ব্যবহার করে।

BIOS/UEFI: কম্পিউটার জেগে ওঠার প্রথম মুহূর্ত

পাওয়ার বাটন চাপলে প্রথমে চলে motherboard-এর firmware — পুরোনো **BIOS** অথবা আধুনিক **UEFI**। এর কাজ: hardware পরীক্ষা করা, কোন ডিস্ক থেকে boot করতে হবে তা ঠিক করা, এবং bootloader-কে নিয়ন্ত্রণ দেওয়া। UEFI-এর সুবিধা: বড় ডিস্ক সমর্থন, Secure Boot, গ্রাফিক্যাল সেটিংস। VirtualBox-এ ডিফল্টভাবে EFI সক্রিয় করা যায়; এই বইয়ের উদাহরণে UEFI ধরা হয়েছে।

Boot Sequence: ধাপে ধাপে

চিত্র ৭.১ — Linux Boot Sequence (ক্রম)

Power On → UEFI/BIOS (hardware চেক, boot device নির্বাচন) → GRUB Bootloader (kernel নির্বাচন, কার্নেল ও initramfs লোড) → Linux Kernel (hardware সক্রিয়, root filesystem mount) → systemd (PID 1, প্রথম process) → Services চালু (network, SSH, log) → Login Prompt

কোনো ধাপে আটকে গেলে সমস্যা সেই ধাপের: GRUB না এলে bootloader, kernel panic হলে kernel/initramfs, login না এলে systemd/service সমস্যা।

চিত্র ৭.২ — GRUB, Kernel ও systemd-এর সম্পর্ক

GRUB: "কোন kernel দিয়ে চালু করব?" সিদ্ধান্ত নেয় ও kernel-কে মেমরিতে তোলে | Kernel: hardware চালু করে, তারপর প্রথম user-space প্রোগ্রাম হিসেবে systemd চালায় | systemd (PID 1): সব service-এর "মা" process — network, SSH, mount, log সব এর অধীনে চালু হয় | systemd বন্ধ হলে system বন্ধ হয়।

GRUB (Grand Unified Bootloader) হলো একটি ছোট প্রোগ্রাম, যা একাধিক kernel বা operating system থাকলে কোনটি চালু করবেন, তা বেছে নিতে দেয়। **initramfs** হলো একটি অস্থায়ী root filesystem, যা আসল root filesystem mount করার আগে দরকারি driver লোড করে। Kernel চালু হলে সে **systemd**-কে PID 1 হিসেবে চালায় — অর্থাৎ system-এর প্রথম process। systemd তারপর `default.target` অনুযায়ী প্রয়োজনীয় service চালু করে: network, disk mount, SSH, log। Ubuntu Server-এ গ্রাফিক্যাল interface থাকে না; login prompt আসে text console-এ।

Boot-এর সময় কী কী দেখা যায়?

সাধারণ boot-এ GRUB menu কয়েক সেকেন্ড দেখা যায়, তারপর kernel বার্তা দ্রুত স্ক্রল হয়। `dmesg` কমান্ডে পরে সেই বার্তাগুলো দেখা যায় (Chapter 18)। Boot ধীর হলে `systemd-analyze blame` দিয়ে দেখা যায় কোন service কত সময় নিয়েছে।

VirtualBox-এ ইনস্টল: ধাপে ধাপে

প্রয়োজন: VirtualBox (virtualbox.org থেকে), Ubuntu Server 24.04 LTS ISO (ubuntu.com/download/server থেকে — ফাইলের নামের মতো `ubuntu-24.04-live-server-amd64.iso` হতে পারে, সংস্করণ যাচাই করে নিন)। আপনার host কম্পিউটারে অন্তত ৮ GB RAM ও ৩০ GB ফাঁকা ডিস্ক থাকা ভালো।

1. VirtualBox খুলে **New** চাপুন; নাম দিন `ubuntu-server-2404`, Type: Linux, Version: Ubuntu (64-bit)।
2. Memory: **৪০৯৬ MB (4 GB)** দিন। কম RAM দিলে ইনস্টলার ধীর হবে।
3. Processor: **২টি vCPU** দিন।
4. Hard disk: **Create a virtual hard disk now**, VDI, Dynamically allocated, **২৫ GB**।
5. VM-এর Settings → Storage → Optical drive-এ Ubuntu ISO যুক্ত করুন; Network-এ Bridged অথবা NAT রাখুন (শুরুতে NAT নিরাপদ)।
6. VM Start করুন, ভাষা English বেছে নিন, “Try or Install Ubuntu Server” নির্বাচন করুন।
7. Installer-এ keyboard, network, storage-এ **Use an entire disk** বেছে নিন (VM-এ এটি নিরাপদ, কারণ এটি ভার্চুয়াল ডিস্ক)।
8. Profile-এ আপনার নাম, server name (যেমন `ubuntu-server`), username ও শক্তিশালী পাসওয়ার্ড দিন। **Install OpenSSH server**-এ টিক দিন।
9. Install শেষ হলে Reboot করুন; ISO সরিয়ে নিন (VirtualBox নিজে থেকেই সরাতে বলবে)। Login prompt-এ username ও পাসওয়ার্ড দিন।

> **PRO TIP:** ইনস্টলের পরপরই VirtualBox-এ একটি Snapshot নিন (Machine → Take Snapshot, নাম: `fresh-install`)। পরে কোনো পরীক্ষায় system ভেঙে গেলে snapshot-এ ফিরে যেতে পারবেন — নতুন করে ইনস্টল করতে হবে না।

প্রথম boot-এর পর প্রাথমিক কনফিগারেশন

```
sudo apt update && sudo apt upgrade -y
# package তালিকা হালনাগাদ করে সব প্যাকেজ আপডেট করে। sudo মানে
# administrator অধিকার; -y মানে প্রতিটি প্রশ্নে হ্যাঁ ধরে নেওয়া।
```

```
hostnamectl
# hostname ও OS তথ্য দেখায়।

ip a
# network interface ও IP ঠিকানা দেখায়। -a মানে সব interface।

systemctl status ssh
# SSH service চলছে কি না দেখায়।

sudo reboot
# সিস্টেম পুনরায় চালু করে; আপডেটের পর প্রয়োজনে চালান।
```

উদাহরণ আউটপুট (illustrative):

```
$ ip a
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
# ব্যাখ্যা: enp0s3 interface-টি UP, IP 10.0.2.15 (NAT নেটওয়ার্কে
# VirtualBox-এর দেওয়া ঠিকানা)। আপনার IP ভিন্ন হবে।
```

Terminology — পরিভাষা

Term	Bangla ব্যাখ্যা
Virtual Machine (VM)	সফটওয়্যার দিয়ে তৈরি ভার্চুয়াল কম্পিউটার
ISO image	ইনস্টলেশন ডিস্কের সম্পূর্ণ কপি (ফাইল)
Partition	ডিস্কের যৌক্তিক ভাগ
UEFI / BIOS	কম্পিউটার চালুর প্রথম firmware
GRUB	Bootloader — কোন kernel চালু হবে তা ঠিক করে
initramfs	Boot-এর সময় ব্যবহৃত অস্থায়ী filesystem
systemd	PID 1 — প্রথম process ও service manager
Snapshot	VM-এর একটি সময়ের সম্পূর্ণ অবস্থার কপি

Hands-on Exercise ৭.১ — VM তৈরি ও প্রথম boot যাচাই

LAB: ধাপ: (১) উপরের ধাপে VM তৈরি করুন (2 vCPU, 4 GB RAM, 25 GB); (২) Ubuntu Server 24.04 ইনস্টল করুন; (৩) login করে `hostnamectl`, `ip a` ও `systemctl status ssh` চালান; (৪) `systemd-analyze` চালিয়ে boot সময় দেখুন; (৫) snapshot নিন। যাচাই: login prompt আসে, SSH service active দেখায়, boot সময় ১ মিনিটের কম (VM-এ কিছুটা বেশি হতে পারে)। ব্যর্থ হলে: ISO যুক্ত আছে কি না, RAM ৪ GB কি না — পরীক্ষা করুন।

Common Mistakes & Troubleshooting

- **"No bootable medium":** ISO Storage-এ যুক্ত হয়নি বা boot order-এ optical drive নেই। Settings → Storage পরীক্ষা করুন।
- **ইনস্টলার আটকে যায়:** RAM কম দেওয়া হয়েছে। ৪ GB দিন; host-এ RAM ফাঁকা আছে কি না দেখুন।
- **GRUB menu-তে আটকে থাকা:** Enter চাপুন; না হলে kernel নির্বাচন ভুল হতে পারে — Chapter 18-এর boot troubleshooting দেখুন।
- **IP পাওয়া যাচ্ছে না:** Network NAT আছে কি না, `ip a`-তে interface UP কি না দেখুন।
- **আসল ডিস্কে "Use entire disk":** VM ছাড়া কখনও যাচাই না করে এটি চাপবেন না — আসল ডেটা মুছে যাবে।

Cloud VPS পথ: কখন ও কীভাবে (ঐচ্ছিক)

যদি আপনি আসল সার্ভারের অভিজ্ঞতা চান, একটি ছোট VPS নিতে পারেন — তবে শুধু শেখার জন্য এটি বাধ্যতামূলক নয়। সাধারণ ধাপ: একটি পরিচিত provider-এ অ্যাকাউন্ট খুলুন, সবচেয়ে ছোট Ubuntu 24.04 LTS plan বেছে নিন, SSH key দিয়ে সার্ভার তৈরি করুন (পাসওয়ার্ড দিয়ে login বন্ধ রাখুন), প্রথম login-এই `sudo apt update && sudo apt upgrade -y` চালান এবং UFW দিয়ে শুধু SSH (port 22) খোলা রাখুন। খরচের নিয়ম মনে রাখুন: VPS বন্ধ করলেও অনেক provider চার্জ নেয় যতক্ষণ instance মুছে না ফেলা হয়; তাই শেখা শেষ হলে instance destroy করুন এবং billing dashboard-এ নিশ্চিত করুন কোনো চার্জ চলছে না। বাংলাদেশ থেকে VPS নেওয়ার সময় ডেটা সেন্টারের অবস্থান (যেমন সিঙ্গাপুর) latency-তে প্রভাব ফেলে — ঢাকা থেকে সিঙ্গাপুর সাধারণত ইউরোপের চেয়ে দ্রুত সাড়া দেয়। VPS-এ কখনও root পাসওয়ার্ড দিয়ে সরাসরি login চালু রাখবেন না; Chapter 19-এর key-based SSH ও firewall না শিখে VPS-কে ইন্টারনেটে উন্মুক্ত রাখা ঝুঁকিপূর্ণ। সন্দেহ থাকলে VirtualBox-এই থাকুন — সেখানকার দক্ষতা VPS-এ সরাসরি কাজে লাগবে, কারণ দুই জায়গাতেই একই Ubuntu, একই systemd, একই কমান্ড।

Boot সমস্যা দেখা: systemd-analyze

Boot ধীর মনে হলে `systemd-analyze` মোট boot সময় দেখায় এবং `systemd-analyze blame` প্রতিটি service কত সময় নিয়েছে, তা সাজিয়ে দেখায়। কোনো service বারবার ব্যর্থ হলে `systemctl status <service>` ও `journalctl -b` (বর্তমান boot-এর log) দিয়ে কারণ খুঁজুন। GRUB menu-তে আটকে গেলে সেখান থেকে recovery mode বেছে নেওয়া যায়, যেখানে root shell পাওয়া যায় — তবে recovery mode-এ ভুল কমান্ড system আরও ভাঙতে পারে, তাই প্রথমে snapshot থেকে ফেরার কথা ভাবুন।

Security Considerations

ইনস্টলের সময়ই নিরাপত্তা শুরু হয়: অনুমানযোগ্য username (`admin`) ও দুর্বল পাসওয়ার্ড ব্যবহার করবেন না। OpenSSH ইনস্টল করলে Chapter 19 না পড়ে ইন্টারনেটে পোর্ট খুলবেন না। VM-এর snapshot-এ পাসওয়ার্ডের তথ্য থাকতে পারে — snapshot ফাইল অন্যের সঙ্গে শেয়ার করবেন না।

Best Practices

- শেখার সব পরীক্ষা VM snapshot-এর সুরক্ষায় করুন।
- Server name অর্থবহ রাখুন (যেমন `dev-server-01`)।
- ইনস্টলের পর প্রথম কাজ: `apt update && apt upgrade`।
- ইনস্টল নোট লিখে রাখুন: ISO সংস্করণ, VM সেটিংস, username (পাসওয়ার্ড নয়)।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING:

1. Boot sequence-এ GRUB-এর পর কী আসে? A) systemd B) Linux kernel C) Login prompt D) SSH service

2. UEFI-এর সুবিধা কোনটি? A) শুধু ছোট ডিস্ক B) বড় ডিস্ক ও Secure Boot সমর্থন C) দ্রুত RAM D) বেশি CPU

3. systemd-এর PID কত? A) 0 B) 1 C) 100 D) 1000

4. VM-এ শেখার সবচেয়ে বড় সুবিধা কী? A) দ্রুত ইন্টারনেট B) ভুল হলে snapshot-এ ফেরা যায় C) বিনামূল্যে cloud D) বেশি storage

5. EFI System Partition সাধারণত কোন filesystem ব্যবহার করে? A) ext4 B) FAT32 C) NTFS D) swap

Practical Assignment

VirtualBox-এ দ্বিতীয় একটি VM তৈরি করুন, এবার ২ GB RAM দিয়ে, এবং লক্ষ্য করুন ইনস্টল কতটা ধীর হয়। একটি সংক্ষিপ্ত নোট লিখুন: RAM কম হলে কোন ধাপে প্রভাব পড়ে? এবং production সার্ভারে RAM নির্বাচনের সময় কী বিবেচনা করবেন? সমাধান Answer Guide-এ দেখুন।

Chapter Summary

শেখার নিরাপদ পথ হলো VirtualBox VM (২ vCPU, ৪ GB RAM, ২৫ GB disk)। Ubuntu Server 24.04 LTS ইনস্টলে partitioning, UEFI, GRUB, kernel ও systemd — এই ক্রমটি বোঝা জরুরি। GRUB kernel বেছে নেয়, kernel hardware চালু করে, systemd PID 1 হিসেবে সব service চালু করে। প্রথম boot-এর পর update, hostname ও network যাচাই করতে হয়। Cloud VPS একটি বিকল্প, তবে খরচ ও নিরাপত্তা সতর্কতা ছাড়া নয়।

Revision Checklist

- ☐ VM তৈরি ও Ubuntu Server ইনস্টল করতে পারি
- ☐ Partition ও GPT/MBR ব্যাখ্যা করতে পারি
- ☐ Boot sequence ক্রমানুসারে বলতে পারি
- ☐ GRUB, kernel ও systemd-এর ভূমিকা জানি
- ☐ Snapshot নিতে পারি

Chapter 08 — Linux Filesystem Hierarchy
লিনাক্স ফাইলসিস্টেম হায়ারার্কি

Part II — Linux Foundations | Ubuntu Server 24.04 LTS | আনুমানিক পড়ার সময়: ৫৫ মিনিট

শুরুতে: হারিয়ে যাওয়া ফাইল

ধরুন আপনি একটি Django প্রজেক্ট deploy করেছেন, কিন্তু log ফাইল কোথায়, কনফিগারেশন কোথায়, ডেটাবেস ফাইল কোথায় — খুঁজে পাচ্ছেন না। Windows-এ আপনি জানতেন `C:\Program Files` বা `D:\` কোথায় কী থাকে। Linux-এ কোনো drive letter নেই; সবকিছু একটি গাছের (tree) মতো একটিমাত্র root (`/`) থেকে শুরু হয়। এই গাছের প্রতিটি ডালের নির্দিষ্ট কাজ আছে, এবং সেই নিয়ম জানলে আপনি যেকোনো Linux সার্ভারে — Ubuntu হোক বা RHEL — ফাইল খুঁজে পাবেন। এই অধ্যায়ে আপনি সেই মানচিত্রটি শিখবেন।

Learning Objectives — শেখার লক্ষ্য

- Filesystem hierarchy-এর মূল নীতি ("everything is a file") ব্যাখ্যা করতে পারবেন।
- `/etc, /home, /var, /tmp, /usr, /opt, /boot, /dev, /proc, /sys`-এর কাজ বলতে পারবেন।
- Absolute ও relative path-এর পার্থক্য ও ব্যবহার দেখাতে পারবেন।
- File type (regular, directory, symlink, device) চিহ্নিত করতে পারবেন।
- Mount point-এর ধারণা ব্যাখ্যা করতে পারবেন।
- Symbolic ও hard link-এর পার্থক্য ও ব্যবহার দেখাতে পারবেন।
- `ls, find, du` দিয়ে filesystem অনুসন্ধান করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 07-এর partitioning ও mount ধারণা এবং terminal-এ `cd, ls`-এর প্রাথমিক ধারণা থাকলে ভালো হয়। না থাকলে Chapter 09 পড়ে ফিরে আসুন — তবু এই অধ্যায়ের তত্ত্ব বোঝা যাবে।

Filesystem কী? প্রথম নীতি

Filesystem হলো ডিস্কের ওপর ডেটা সাজানোর নিয়ম। কাঁচা ডিস্ক শুধু block-এর সমষ্টি — কোথায় কোন ফাইল শুরু, কোথায় শেষ, তার নাম কী — এসব filesystem ঠিক করে। Linux-এ ব্যবহৃত প্রধান filesystem **ext4** (Ubuntu-এর ডিফল্ট), এছাড়া **XFS** (RHEL-এ সাধারণ) ও **Btrfs** (snapshot সুবিধাসহ) আছে। Windows-এর NTFS বা FAT32-এর সঙ্গে তুলনা করলে ext4-এর বৈশিষ্ট্য: journaling (হঠাৎ বিদ্যুৎ গেলে ডেটা পুনরুদ্ধারে সাহায্য করে), permission সমর্থন, বড় ফাইল ও ডিস্ক সমর্থন।

Linux-এর বিখ্যাত নীতি: **"Everything is a file"** । শুধু আপনার document নয় — hardware device (`/dev/sda`), চলমান process-এর তথ্য (`/proc/1234`), এমনকি kernel-এর সেটিংস (`/sys`) — সবই ফাইল হিসেবে দেখা যায়। এর মানে, ফাইল পড়ার একই টুল দিয়ে আপনি device ও system-এর অবস্থাও দেখতে পারেন। এটি শেখা সহজ করে: একটি মাত্র মডেল — ফাইল — দিয়ে পুরো system বোঝা যায়।

Filesystem Hierarchy Standard (FHS)

সব Linux distribution একই মূল নিয়ম মেনে চলে, যাকে বলে Filesystem Hierarchy Standard (FHS)। এর ফলে Ubuntu শেখা ব্যক্তি RHEL-এ গিয়ে `/etc/nginx/nginx.conf` খুঁজতে জানেন। নিচের চিত্রটি মূল মানচিত্র।

চিত্র ৮.১ — Root থেকে প্রধান ডিরেক্টরি (গাছ)

/ (root)
├── bin, sbin → অপরিহার্য কমান্ড (আধুনিক Ubuntu-তে /usr/bin-এর symlink)
├── boot → kernel ও GRUB ফাইল
├── etc → সিস্টেম কনফিগারেশন
├── home → ব্যবহারকারীর ফাইল (/home/rafi, /home/nahid)
├── var → পরিবর্তনশীল ডেটা (log, cache, spool)
├── usr → ব্যবহারকারীর প্রোগ্রাম ও লাইব্রেরি
├── opt → অতিরিক্ত/তৃতীয় পক্ষের সফটওয়্যার
├── tmp → অস্থায়ী ফাইল
├── dev → device ফাইল
├── proc → চলমান process ও kernel তথ্য (ভার্চুয়াল)
├── sys → kernel ও hardware তথ্য (ভার্চুয়াল)
├── mnt, media → অস্থায়ী mount point
└── root → root ব্যবহারকারীর home

চিত্র ৮.২ — কোথায় কী খুঁজবেন (সিদ্ধান্ত প্রবাহ)

কনফিগারেশন বদলাতে চান? → /etc | নিজের ফাইল? → /home/username | সমস্যার log? → /var/log | ডিস্ক ভরেছে? → /var ও /tmp পরীক্ষা করুন | প্রোগ্রাম ইনস্টল হয়েছে কোথায়? → /usr/bin, /opt | Boot সমস্যা? → /boot ও GRUB | Hardware তথ্য? → /proc, /sys

প্রধান ডিরেক্টরি বিস্তারিত

ডিরেক্টরি	কাজ	উদাহরণ
`/`	Root — গাছের শীর্ষ	সবকিছুর শুরু
`/etc`	System কনফিগারেশন (editable text)	`/etc/passwd, /etc/nginx/nginx.conf, /etc/ssh/sshd_config`
`/home`	সাধারণ ব্যবহারকারীর home	`/home/nahid/projects/`
`/var`	পরিবর্তনশীল ডেটা: log, cache, mail	`/var/log/syslog, /var/www/html`
`/tmp`	অস্থায়ী ফাইল; reboot-এ মুছে যেতে পারে	ইনস্টলারের অস্থায়ী ফাইল
`/usr`	বেশিরভাগ প্রোগ্রাম, লাইব্রেরি, ডকুমেন্টেশন	`/usr/bin/python3, /usr/share/doc/`
`/opt`	তৃতীয় পক্ষের বড় সফটওয়্যার	`/opt/google/chrome` (থাকলে)

`/boot`	Kernel, initramfs, GRUB	`/boot/vmlinuz-6.8.0-31-generic`
`/dev`	Device ফাইল	`/dev/sda, /dev/null, /dev/tty`
`/proc`	Process ও kernel তথ্য (RAM-ভিত্তিক, ভার্চুয়াল)	`/proc/cpuinfo, /proc/meminfo`
`/sys`	Kernel ও hardware তথ্য (sysfs)	`/sys/class/net/`
`/root`	root ব্যবহারকারীর home (সাধারণ user-এর নয়)	`/root/.ssh/`

মনে রাখার কৌশল: **/etc** = “edit to configure” (আসলে নামটি এসেছে etcetera থেকে, কিন্তু এই মনে রাখার কৌশল কাজ করে); **/var** = variable, অর্থাৎ যা বদলায় — log বাড়ে, cache জমে; **/usr** = user programs, যদিও নামটি Unix System Resources থেকে এসেছে।

/proc ও /sys: ডিস্কে নেই, তবু ফাইল

`/proc/cpuinfo` পড়লে CPU-এর তথ্য পাওয়া যায়, কিন্তু ডিস্কে এই নামে কোনো ফাইল নেই। kernel চাহিদা অনুযায়ী তথ্য তৈরি করে দেয়। `cat /proc/meminfo` দিয়ে RAM-এর অবস্থা, `cat /proc/loadavg` দিয়ে load average দেখা যায়। একইভাবে `/sys/class/net/enp0s3/address`-এ network card-এর MAC address থাকে। এগুলো পড়া নিরাপদ; লেখা (যেখানে অনুমোদিত) kernel-এর আচরণ বদলাতে পারে — সতর্ক থাকুন।

Absolute বনাম Relative Path

Absolute path `/` থেকে শুরু হয়: `/home/nahid/projects/bookora` — যেখান থেকেই চালান, একই জায়গায় যায়। Relative path বর্তমান ডিরেক্টরির সাপেক্ষে: আপনি `/home/nahid`-এ থাকলে `projects/bookora` একই জায়গা বোঝায়। বিশেষ চিহ্ন: `.` = বর্তমান ডিরেক্টরি, `..` = parent ডিরেক্টরি, `~` = আপনার home, `-` = আগের ডিরেক্টরি (cd-তে)। Script-এ সবসময় absolute path ব্যবহার করুন — relative path script কোথা থেকে চালানো হচ্ছে, তার ওপর নির্ভর করে ভুল করতে পারে।

কেন /bin ও /sbin এখন /usr-এর দিকে যায়?

আধুনিক Ubuntu-তে `/bin` আসলে `/usr/bin`-এর symlink, একইভাবে `/sbin` হলো `/usr/sbin`-এর symlink। এর কারণ ঐতিহাসিক: একসময় ছোট ডিস্কে boot-এর জন্য প্রয়োজনীয় কমান্ড আলাদা রাখা হতো, যাতে root filesystem ছোট থাকে। আজ ডিস্ক বড় এবং initramfs boot-এর প্রাথমিক কাজ করে, তাই এই ভাগ আর দরকার নেই। এই পরিবর্তনকে বলে usrmerge। ব্যবহারকারী হিসেবে এর অর্থ সহজ: `/bin/ls` ও `/usr/bin/ls` একই ফাইল। কিন্তু পুরোনো টিউটোরিয়ালে `/bin` দেখলে বিভ্রান্ত হবেন না — দুটি পথই কাজ করে। একইভাবে `/lib` এখন `/usr/lib`-এর symlink। FHS-এর মূল নীতি বদলায়নি; শুধু বাস্তবায়ন সরল হয়েছে। কোনো distribution-এ এই symlink না থাকলে সেটি পুরোনো নকশা অনুসরণ করছে — কমান্ডের কাজ একই থাকে।

/etc-তে কী ধরনের ফাইল থাকে, কী থাকে না?

/etc-তে শুধু text কনফিগারেশন থাকে — binary প্রোগ্রাম বা বড় ডেটা নয়। যেমন `/etc/passwd` ব্যবহারকারীর তালিকা, `/etc/hosts` hostname-এর স্থানীয় ম্যাপিং, `/etc/apt/sources.list.d/` package repository-এর তালিকা। কনফিগারেশন বদলাতে হলে এখানেই বদলাতে হয়, এবং বদলানোর পর সংশ্লিষ্ট service restart করতে হয় (যেমন Nginx-

এর কনফিগারেশন বদলে `sudo systemctl reload nginx`)। এই নিয়মটি মনে রাখলে troubleshooting অনেক সহজ হয়: কোনো service ঠিকমতো চলছে না? প্রথমে তার /etc-এর কনফিগারেশন ও /var/log-এর log — দুটোই দেখুন।

File Type ও Link

`ls -l`-এর প্রথম অক্ষর file type বলে: `-` regular file, `d` directory, `l` symbolic link, `c` character device, `b` block device, `p` pipe, `s` socket। **Symbolic link (symlink)** হলো shortcut: এটি আরেকটি পথের দিকে নির্দেশ করে; মূল ফাইল মুছলে symlink ভেঙে যায় (dangling link)। **Hard link** হলো একই inode-এর আরেকটি নাম — মূল ফাইল মুছলেও ডেটা থাকে, যতক্ষণ একটি link বেঁচে থাকে। Hard link ডিরেক্টরি বা অন্য filesystem-এ করা যায় না; symlink যায়। বাস্তবে কনফিগারেশনে symlink বেশি ব্যবহৃত হয় — যেমন `/etc/nginx/sites-enabled/`-এ `sites-available`-এর symlink।

Mount Point

Mount মানে একটি filesystem-কে গাছের কোনো ডালে যুক্ত করা। উদাহরণ: USB ড্রাইভ `/media/usb`-এ mount করলে সেই ডালের নিচে USB-এর ফাইল দেখা যায়। `/etc/fstab` ফাইলে boot-এর সময় কোন filesystem কোথায় mount হবে, তা লেখা থাকে (বিস্তারিত Chapter 16)। `df -h` দিয়ে দেখা যায় কোন mount point কতটা পূর্ণ, `mount | grep /dev` দিয়ে বর্তমান mount তালিকা দেখা যায়।

প্রায়োগিক কমান্ড

```
pwd
# বর্তমান ডিরেক্টরির absolute path দেখায়।

ls -l /
# root-এর সবকিছু দীর্ঘ তালিকায় দেখায়; -l মানে long format।

ls -ld /etc /home /var
# -d মানে ডিরেক্টরির ভেতর না দেখিয়ে ডিরেক্টরিটিকে দেখানো।

cat /proc/cpuinfo | head -20
# CPU তথ্যের প্রথম ২০ লাইন দেখায়।

ls -l /bin
# /bin একটি symlink কি না দেখুন — Ubuntu 24.04-এ এটি /usr/bin-এর দিকে যায়।

file /etc/passwd
# ফাইলের ধরন জানায়।

stat /etc/hostname
# ফাইলের আকার, inode ও সময়ের বিস্তারিত তথ্য দেয়।
```

উদাহরণ আউটপুট (illustrative):

```
$ ls -ld /etc /home /var
drwxr-xr-x  98 root root 4096 Nov  8 10:00 /etc
drwxr-xr-x   3 root root 4096 Nov  8 09:00 /home
drwxr-xr-x  12 root root 4096 Nov  8 09:00 /var
```

```
# ব্যাখ্যা: d মানে directory, rwxr-xr-x মানে permission (Chapter 11),
# root মানে মালিক root। আকার ও তারিখ আপনার সিস্টেমে ভিন্ন হবে।
```

Terminology — পরিভাষা

Term	Bangla ব্যাখ্যা
Filesystem	ডিস্কে ডেটা সাজানোর নিয়ম (যেমন ext4)
FHS	Filesystem Hierarchy Standard — ডিরেক্টরির মান
Root directory (/)	গাছের শীর্ষ; root ব্যবহারকারীর home (/root) থেকে আলাদা
Mount point	যে ডিরেক্টরিতে একটি filesystem যুক্ত হয়
Inode	ফাইলের metadata (মালিক, permission, অবস্থান) রাখার কাঠামো
Symlink / Hard link	একই ফাইলের দিকে নির্দেশকারী shortcut / বিকল্প নাম
Virtual filesystem	ডিস্কে নয়, kernel-এর তৈরি filesystem (/proc, /sys)

Hands-on Exercise ৮.১ — Filesystem অনুসন্ধান

LAB: ধাপ: (১) `ls /` চালিয়ে প্রধান ডিরেক্টরিগুলো লিখুন; (২) `ls /etc | head -20` ও `ls /var/log | head -20` চালান; (৩) `cat /proc/meminfo | head -5` ও `cat /proc/cpuinfo | grep "model name" | head -1` চালান; (৪) `ls -l /bin` দিয়ে symlink যাচাই করুন; (৫) `mkdir -p ~/lab8/test && ln -s ~/lab8/test ~/lab8/link-test && ls -l ~/lab8/` দিয়ে symlink তৈরি করে দেখুন। যাচাই: symlink-এর তীর (`->`) দেখা যায়। Cleanup: `rm ~/lab8/link-test && rmdir ~/lab8/test ~/lab8` ।

Common Mistakes & Troubleshooting

- **ভুল:** `/root` আর `/` গুলিয়ে ফেলা। **সংশোধন:** `/` হলো গাছের শীর্ষ, `/root` হলো root user-এর home।
- **ভুল:** `/tmp`-এ গুরুত্বপূর্ণ ফাইল রাখা। **সংশোধন:** reboot-এ মুছে যেতে পারে; গুরুত্বপূর্ণ ফাইল `/home`-এ রাখুন।
- **ভুল:** `/proc`-এর ফাইল edit করার চেষ্টা। **সংশোধন:** এগুলো ভার্চুয়াল; প্রয়োজনীয় সেটিংস `/etc/sysctl.conf` বা `/sys`-এর নির্দিষ্ট পথে বদলান।
- **ভুল:** Relative path দিয়ে script-এ ফাইল খোঁজা। **সংশোধন:** Script-এ absolute path ব্যবহার করুন।

Security Considerations

`/etc/shadow` (পাসওয়ার্ডের hash) শুধু root পড়তে পারে — এটি স্বাভাবিক। `/tmp` সবার লেখার যোগ্য, তাই সেখানে সংবেদনশীল ফাইল রাখবেন না। অপরিচিত symlink অনুসরণ করে ফাইল মুছবেন না — symlink কোথায় নির্দেশ করছে, তা `ls -l` দিয়ে যাচাই করুন।

Best Practices

- কনফিগারেশন বদলানোর আগে `/etc`-এর ফাইলের backup কপি নিন (`sudo cp file file.bak`)।
- নিজের কাজ `/home/username`-এর ভেতরে রাখুন; system ডিরেক্টরিতে নয়।
- Log দেখতে `/var/log`, কনফিগারেশন দেখতে `/etc` — এই অভ্যাস গড়ে তুলুন।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING:

1. Ubuntu-এর ডিফল্ট filesystem কোনটি? A) NTFS B) ext4 C) FAT32 D) XFS

2. SSH-এর কনফিগারেশন ফাইল সাধারণত কোথায় থাকে? A) /home B) /etc/ssh C) /var/log D) /tmp

3. /proc কী ধরনের filesystem? A) ডিস্কভিত্তিক B) ভার্চুয়াল, kernel-এর তৈরি C) নেটওয়ার্ক D) USB

4. Hard link সম্পর্কে কোনটি সত্য? A) অন্য filesystem-এ করা যায় B) মূল ফাইল মুছলেও ডেটা থাকে C) ডিরেক্টরিতে করা যায় D) শুধু shortcut

5. `ls -l`-এ প্রথম অক্ষর `d` মানে কী? A) device B) directory C) deleted D) disk

Practical Assignment

আপনার VM-এ `tree` না থাকলে `ls -R /etc | head -60` চালিয়ে একটি মানচিত্র আঁকুন: /etc-এর নিচে কোন কোন সাবডিরেক্টরি আছে এবং প্রতিটি কীসের জন্য (অন্তত ৫টি)। এরপর /var/log-এর ৩টি ফাইলের নাম ও কাজ লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

Linux-এ সবকিছু একটি গাছের মতো root (`/`) থেকে শুরু হয়। /etc-তে কনফিগারেশন, /home-তে ব্যবহারকারীর ফাইল, /var-তে log ও পরিবর্তনশীল ডেটা, /usr-তে প্রোগ্রাম, /boot-তে kernel, /dev-তে device, /proc ও /sys-তে kernel তথ্য থাকে। Absolute path সবসময় / থেকে শুরু হয়; relative path বর্তমান ডিরেক্টরির সাপেক্ষে। Symlink হলো shortcut, hard link হলো একই inode-এর আরেক নাম। এই মানচিত্র জানলে যেকোনো Linux সার্ভারে ফাইল খুঁজে পাওয়া যায়।

Revision Checklist

- ☐ ১০টি প্রধান ডিরেক্টরির কাজ বলতে পারি
- ☐ Absolute ও relative path-এর পার্থক্য জানি
- ☐ /proc ও /sys ভার্চুয়াল কেন, বুঝি
- ☐ Symlink ও hard link-এর পার্থক্য জানি
- ☐ `ls -l /` ও `cat /proc/cpuinfo` চালাতে পারি

Chapter 09 — Terminal, Shell and Command-Line Fundamentals
টার্মিনাল, শেল ও কমান্ড-লাইন মৌলিক বিষয়

Part II — Linux Foundations | Ubuntu Server 24.04 LTS | আনুমানিক পড়ার সময়: ৬০ মিনিট

শুরুতে: কালো স্ক্রিনের ভয়

অনেক শিক্ষার্থী terminal দেখে ভয় পান, কারণ এখানে বোতাম নেই, মাউস কাজ করে না, ভুল করলে কী হবে বোঝা যায় না। কিন্তু একবার বুঝে গেলে terminal-ই সবচেয়ে দ্রুত ও শক্তিশালী উপায়। একটি উদাহরণ: GUI-তে ১০০টি ফাইলের নাম বদলাতে এক ঘণ্টা লাগতে পারে; terminal-এ একটি কমান্ডে কয়েক সেকেন্ডে হয়। সার্ভারে তো GUI থাকেই না — SSH দিয়ে terminal-ই একমাত্র পথ। এই অধ্যায়ে আপনি terminal ও shell-এর পার্থক্য, Bash-এর মূল নিয়ম, pipe ও redirection, environment variable এবং নিরাপদে কমান্ড চালানোর অভ্যাস শিখবেন।

Learning Objectives — শেখার লক্ষ্য

- Terminal, shell ও Bash-এর পার্থক্য ব্যাখ্যা করতে পারবেন।
- Prompt-এর অংশগুলো পড়ে বর্তমান user, host ও ডিরেক্টরি বলতে পারবেন।
- কমান্ডের গঠন (command, option, argument) বিশ্লেষণ করতে পারবেন।
- Absolute/relative path, environment variable ও quoting সঠিকভাবে ব্যবহার করতে পারবেন।
- Pipe (`|`) ও redirection (`>`, `>>`, `<`) দিয়ে ডেটা প্রবাহ নিয়ন্ত্রণ করতে পারবেন।
- Command chaining (`&&`, `;`, `||`), history ও help system ব্যবহার করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 08-এর path ধারণা ও Chapter 07-এর VM-এ login করার অভিজ্ঞতা দরকার। প্রোগ্রামিংয়ে variable-এর ধারণা থাকলে environment variable সহজ হবে।

Terminal বনাম Shell: প্রথম নীতি

উপমা: terminal হলো টেলিফোন সেট, shell হলো ওপাশের ব্যক্তি যিনি আপনার কথা বুঝে কাজ করেন। **Terminal** (বা terminal emulator) হলো একটি প্রোগ্রাম যা আপনার কীবোর্ড ইনপুট নেয় ও স্ক্রিনে আউটপুট দেখায় — যেমন GNOME Terminal, Windows Terminal, বা SSH client। **Shell** হলো command interpreter — আপনার টাইপ করা কমান্ড পড়ে, বুঝে, প্রোগ্রাম চালায়। Ubuntu-এর ডিফল্ট shell **Bash** (Bourne Again Shell)। অন্য shell-ও আছে: `sh` (সহজ, POSIX), `zsh` (আধুনিক, জনপ্রিয়), `fish` (ব্যবহারবান্ধব)। এই বইয়ে Bash ব্যবহার করা হয়েছে, কারণ এটি সার্ভারে সর্বত্র পাওয়া যায়।

Shell কীভাবে কাজ করে? আপনি `ls -l` লিখে Enter চাপলে shell প্রথমে শব্দগুলো ভাগ করে: কমান্ড `ls`, option `-l`। তারপর PATH variable-এ খোঁজে `ls` প্রোগ্রামটি কোথায় (`/usr/bin/ls`)। তারপর সেটি একটি নতুন process হিসেবে চালায়, আউটপুট terminal-এ দেখায়, এবং exit status ফেরত দেয় (০ মানে সফল)। এই exit status-ই `&&` ও `||`-এর ভিত্তি।

Prompt Anatomy — প্রম্পট পড়া

চিত্র ৯.১ — Bash Prompt-এর অংশ

nahid@ubuntu-server:~$ | nahid = username | @ = বিভাজক | ubuntu-server = hostname | : = বিভাজক | ~ = বর্তমান ডিরেক্টরি (home) | $ = সাধারণ user (root হলে # হয়)

Prompt দেখেই বুঝুন আপনি কে, কোথায় আছেন, root কি না। # দেখলে অতিরিক্ত সতর্ক হোন — root হিসেবে ভুল কমান্ডের ক্ষতি বেশি।

চিত্র ৯.২ — কমান্ড কীভাবে চলে (প্রবাহ)

আপনি টাইপ করেন → Shell লাইনটি ভাগ করে (parsing) → Variable ও wildcard প্রসারিত করে (expansion) → প্রোগ্রাম খুঁজে চালায় (execution) → আউটপুট stdout/stderr-এ যায় → Exit status (0 = সফল) ফেরত আসে

এই প্রবাহ বুঝলে quoting ও pipe-এর আচরণ আর রহস্য থাকে না।

Command Structure — কমান্ডের গঠন

```
ls -l /home/nahid
# ls = command, -l = option (long format), /home/nahid = argument (কোথায়)

ls -lh
# -l সঙ্গে -h (human-readable আকার) — একসঙ্গে -lh লেখা যায়।

ls --help
# দীর্ঘ option: --help মানে সাহায্য দেখানো।

cp -r ~/projects/bookora ~/backup/
# -r মানে recursive (ডিরেক্টরিসহ কপি); দুটি argument: উৎস ও গন্তব্য।
```

Option-এর দুটি রূপ: সংক্ষিপ্ত (`-l`, একটি ড্যাশ) ও দীর্ঘ (`--all`, দুটি ড্যাশ)। `man ls` (manual) দিয়ে সম্পূর্ণ বিবরণ, `ls --help` দিয়ে সংক্ষিপ্ত বিবরণ পাওয়া যায়। কোনো কমান্ডের কাজ না জানলে প্রথমে `--help` দেখুন, ইন্টারনেটে যাওয়ার আগে।

Paths, Navigation ও Environment Variables

```
pwd
# বর্তমান ডিরেক্টরি দেখায়।

cd /var/log
# absolute path-এ যাওয়া।

cd ~/projects
# home-এর নিচে projects-এ যাওয়া।

cd ..
# parent ডিরেক্টরিতে যাওয়া।
```

```
cd -
# আগের ডিরেক্টরিতে ফেরা।

echo $HOME
# $HOME environment variable-এর মান দেখায়।

echo $USER
# বর্তমান username দেখায়।

echo $PATH
# কোলন দিয়ে আলাদা করা ডিরেক্টরি তালিকা — shell এখানে কমান্ড খোঁজে।

export MY_PROJECT=/home/nahid/projects/bookora
# নিজের variable তৈরি করা (শুধু এই shell session-এ থাকে)।

env | sort
# সব environment variable সাজিয়ে দেখায়।
```

Environment variable হলো shell-এর "সেটিংস" — যেমন `PATH` বলে দেয় কমান্ড কোথায় খুঁজতে হবে, `HOME` বলে দেয় আপনার home কোথায়। স্থায়ীভাবে variable রাখতে চাইলে `~/.bashrc` ফাইলে `export` লাইন যুক্ত করতে হয়, তারপর `source ~/.bashrc` চালাতে হয়।

Quoting ও Escaping — কখন কী ব্যবহার করবেন

Shell-এ space মানে শব্দের বিভাজন। ফাইলের নামে space থাকলে (`my file.txt`) shell তাকে দুটি argument ভাববে। সমাধান: quote বা escape। **Single quote** (`'...'`) ভেতরের সবকিছু আক্ষরিক ধরে — variable-ও প্রসারিত হয় না। **Double quote** (`"..."`) variable প্রসারিত করে, কিন্তু space ও বিশেষ অক্ষর রক্ষা করে। **Backslash** (`\`) পরের অক্ষরটিকে আক্ষরিক করে।

```
echo $HOME
# /home/nahid দেখায়।

echo '$HOME'
# $HOME আক্ষরিক দেখায় — single quote variable প্রসারিত করে না।

echo "$HOME"
# /home/nahid দেখায় — double quote variable প্রসারিত করে।

touch "my file.txt"
# space-সহ ফাইলের নাম নিরাপদে তৈরি করে।

echo "Price: \$50"
# \$ মানে $ আক্ষরিক — variable নয়।
```

Pipes ও Redirection — ডেটার প্রবাহ

প্রতিটি কমান্ডের তিনটি "দরজা" আছে: **stdin** (০, ইনপুট), **stdout** (১, সাধারণ আউটপুট), **stderr** (২, ভুলের বার্তা)। Pipe (`|`) একটি কমান্ডের stdout-কে আরেকটির stdin-এ পাঠায়। Redirection ফাইলের সঙ্গে যুক্ত করে।

```
ls /etc | head -10
# /etc-এর তালিকার প্রথম ১০ লাইন দেখায়।
```

```
cat /var/log/syslog | grep -i error | head -5
# log থেকে error-যুক্ত লাইন খুঁজে প্রথম ৫টি দেখায়।

echo "Hello Linux" > ~/hello.txt
# > মানে ফাইলে লেখা (আগের内容 মুছে নতুন লেখে — সতর্ক!)

echo "Second line" >> ~/hello.txt
# >> মানে ফাইলের শেষে যোগ করা (আগের内容 থাকে)।

ls /nonexistent 2> ~/error.txt
# 2> মানে stderr ফাইলে পাঠানো; স্ক্রিনে ভুল দেখাবে না।

cat < ~/hello.txt
# < মানে ফাইল থেকে ইনপুট নেওয়া।
```

WARNING: `>` দিয়ে ভুল ফাইলে লিখলে আগের内容 মুছে যায়। গুরুত্বপূর্ণ ফাইলে লেখার আগে `>>` না `>` — দুবার ভাবুন। কনফিগারেশন ফাইলে `>` চালানোর আগে backup নিন।

Command Chaining, History ও Help

```
sudo apt update && sudo apt upgrade -y
# && মানে প্রথমটি সফল হলে দ্বিতীয়টি চালাও।

mkdir ~/test || echo "already exists"
# || মানে প্রথমটি ব্যর্থ হলে দ্বিতীয়টি চালাও।

cd /var/log; ls -l
# ; মানে একের পর এক চালাও, সফল-ব্যর্থ যাই হোক।

history
# আগের কমান্ডগুলোর তালিকা দেখায়।

!42
# history-র ৪২ নম্বর কমান্ড আবার চালায় (সতর্ক: কী কমান্ড, দেখে নিন)।

Ctrl+R
# history-তে reverse search — টাইপ করলেই পুরোনো কমান্ড খুঁজে পাওয়া যায়।

man grep
# grep-এর manual দেখায়; q চাপলে বের হওয়া যায়।

type ls
# ls আসলে কী — alias, builtin, না /usr/bin/ls — তা বলে।

which python3
# python3 প্রোগ্রামটি কোথায় আছে, তা দেখায়।
```

দরকারি shortcut: `Tab` = auto-complete (অসম্পূর্ণ কমান্ড/ফাইল নাম পূরণ করে), `↑/↓` = আগের/পরের কমান্ড, `Ctrl+C` = চলমান কমান্ড থামানো, `Ctrl+L` = স্ক্রিন পরিষ্কার, `Ctrl+A / Ctrl+E` = লাইনের শুরু/শেষে যাওয়া।

Builtin কমান্ড বনাম বাইরের প্রোগ্রাম

সব কমান্ড ডিস্কের প্রোগ্রাম নয়। `cd, pwd, echo, export`-এর মতো কিছু কমান্ড shell-এর ভেতরেই তৈরি (builtin) — এগুলো shell নিজেই চালায়, নতুন process তৈরি করে না। আর `ls, grep, tar` হলো ডিস্কের আলাদা প্রোগ্রাম (`/usr/bin/ls`), যেগুলো shell খুঁজে চালায়। পার্থক্যটি কেন জরুরি? `cd` যদি বাইরের প্রোগ্রাম হতো, তবে সেটি নিজের process-এর ডিরেক্টরি বদলে শেষ হয়ে যেত — আপনার shell-এর ডিরেক্টরি বদলাত না। তাই `cd` builtin হতেই হয়। `type cd` চালালে "shell builtin" দেখাবে, আর `type ls` চালালে `/usr/bin/ls` বা alias দেখাবে। এই যাচাই অভ্যাসটি troubleshooting-এ কাজে লাগে: কোনো কমান্ড অদ্ভুত আচরণ করলে প্রথমে দেখুন সেটি alias কি না (`alias` কমান্ডে তালিকা পাওয়া যায়)।

Terminology — পরিভাষা

Term	Bangla ব্যাখ্যা
Terminal / Terminal emulator	কীবোর্ড ইনপুট ও স্ক্রিন আউটপুট দেখানোর প্রোগ্রাম
Shell (Bash)	কমান্ড পড়ে চালানো interpreter
Prompt	user, host ও ডিরেক্টরি দেখানো চিহ্ন (`$` বা `#`)
Option / Flag	কমান্ডের আচরণ বদলানো চিহ্ন (`-l`, `--help`)
Argument	কমান্ড যার ওপর কাজ করে (ফাইল, ডিরেক্টরি)
Environment variable	Shell-এর সেটিংস মান (যেমন PATH, HOME)
Pipe (`|`)	এক কমান্ডের আউটপুট আরেক কমান্ডের ইনপুটে পাঠানো
Redirection	আউটপুট/ইনপুট ফাইলের দিকে ঘোরানো (`>`, `>>`, `<`)
Exit status	কমান্ডের ফলাফল সংখ্যা; ০ মানে সফল

Hands-on Exercise ৯.১ — Pipe ও Redirection দিয়ে log খোঁজা

> **LAB:** ধাপ: (১) `echo "Linux শেখা" > ~/lab9.txt && echo "দ্বিতীয় লাইন" >> ~/lab9.txt && cat ~/lab9.txt` চালান; (২) `ls /etc | grep -i ssh` চালিয়ে SSH-সংক্রান্ত ফাইল খুঁজুন; (৩) `cat /etc/passwd | cut -d: -f1 | sort | head -10` চালিয়ে প্রথম ১০টি username দেখুন; (৪) `echo $PATH | tr ':' '\n'` চালিয়ে PATH-এর প্রতিটি ডিরেক্টরি আলাদা লাইনে দেখুন। যাচাই: প্রতিটি কমান্ডের আউটপুট ব্যাখ্যা করে নোটে লিখুন। Cleanup: `rm ~/lab9.txt`।

Common Mistakes & Troubleshooting

- **ভুল:** `cd /home/nahid` আর `cd ~/nahid` গুলিয়ে ফেলা। **সংশোধন:** `~` নিজেই home, তাই `~/nahid` মানে `/home/nahid/nahid`।
- **ভুল:** Single quote-এ variable লিখে মান না পাওয়া। **সংশোধন:** Variable চাইলে double quote ব্যবহার করুন।

- **ভুল:** `rm -rf` টাইপ করে Enter চাপা। **সংশোধন:** Chapter 10-এর নিরাপদ পদ্ধতি দেখুন; প্রথমে `ls` দিয়ে যাচাই করুন।
- **ভুল:** কমান্ড না পেলে "command not found" দেখে ঘাবড়ানো। **সমাধান:** `which` ও `type` দিয়ে দেখুন; হয়তো প্যাকেজ ইনস্টল হয়নি (Chapter 14)।
- **ভুল:** `man` থেকে বের হতে না পারা। **সমাধান:** `q` চাপুন।

Security Considerations

ইন্টারনেট থেকে কপি করা কমান্ড, বিশেষ করে `curl ... | sudo bash` ধরনের কমান্ড, না বুঝে চালাবেন না — এটি যেকোনো কোড root হিসেবে চালাতে পারে। `history`-তে পাসওয়ার্ড টাইপ করবেন না; পাসওয়ার্ড history ফাইলে (`~/.bash_history`) থেকে যায়। Root prompt (`#`) দেখলে প্রতিটি কমান্ড দুবার পড়ুন।

Best Practices

- প্রতিদিন Tab-completion ও `Ctrl+R` ব্যবহারের অভ্যাস করুন — টাইপিং ভুল কমবে।
- দীর্ঘ কমান্ড চালানোর আগে `echo` দিয়ে variable-এর মান যাচাই করুন।
- নিজের দরকারি কমান্ড নোটে রাখুন; শুধু history-র ওপর নির্ভর করবেন না।
- Script-এ `set -e` ব্যবহারের অভ্যাস পরে শিখুন (এই বইয়ের পরের খণ্ডে)।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING:

1. Terminal ও shell-এর পার্থক্য কী? A) একই জিনিস B) Terminal ইনপুট/আউটপুট দেখায়, shell কমান্ড চালায় C) Shell শুধু GUI D) Terminal শুধু root-এর জন্য

2. Prompt-এ `#` মানে কী? A) সাধারণ user B) root user C) ভুল D) home ডিরেক্টরি

3. `echo '$HOME'`-এর আউটপুট কী? A) /home/username B) $HOME C) ভুল বার্তা D) খালি লাইন

4. `cmd1 && cmd2`-এ cmd2 কখন চলে? A) সবসময় B) cmd1 সফল হলে C) cmd1 ব্যর্থ হলে D) কখনও না

5. `ls /etc | head -5`-এ pipe কী করে? A) ফাইলে লেখে B) ls-এর আউটপুট head-এর ইনপুটে পাঠায় C) ফাইল মুছে দেয় D) ডিরেক্টরি বদলায়

Practical Assignment

একটি "cheat sheet" তৈরি করুন: অন্তত ১৫টি কমান্ড — navigation, help, pipe, redirection, history — প্রতিটির সঙ্গে এক লাইনে কাজ ও একটি উদাহরণ। এরপর `cat /etc/passwd | cut -d: -f1 | sort` চালিয়ে আপনার সিস্টেমের সব username-এর তালিকা তৈরি করুন ও সংরক্ষণ করুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

Terminal হলো ইনপুট/আউটপুটের জানালা, shell (Bash) হলো কমান্ড বোঝা ও চালানোর প্রোগ্রাম। Prompt থেকে user, host, ডিরেক্টরি ও root কি না — বোঝা যায়। কমান্ড = command + option + argument। Environment variable (PATH, HOME) shell-এর সেটিংস। Single quote আক্ষরিক, double quote variable প্রসারিত করে। Pipe একটি কমান্ডের আউটপুট আরেকটিতে পাঠায়; `>` ফাইলে লেখে (মুছে), `>>` যোগ করে। `&&`, `||`, history ও `man` দৈনন্দিন কাজের অপরিহার্য টুল।

Revision Checklist

- ☐ Terminal বনাম shell ব্যাখ্যা করতে পারি
- ☐ Prompt পড়তে পারি
- ☐ Single বনাম double quote-এর পার্থক্য জানি
- ☐ Pipe ও redirection ব্যবহার করতে পারি
- ☐ `man`, `history`, `Ctrl+R` ব্যবহার করতে পারি

Chapter 10 — Essential Linux Commands and Text Processing
অপরিহার্য লিনাক্স কমান্ড ও টেক্সট প্রসেসিং

Part II — Linux Foundations | Ubuntu Server 24.04 LTS | আনুমানিক পড়ার সময়: ৭০ মিনিট

শুরুতে: একজন অ্যাডমিনের সকাল

সকাল ৯টায় একজন জুনিয়র সিস্টেম অ্যাডমিন দেখলেন ওয়েবসাইট ধীর। তিনি terminal খুলে কয়েক মিনিটে কাজটি সারলেন: `df -h` দিয়ে দেখলেন ডিস্ক প্রায় পূর্ণ, `du -sh /var/*` দিয়ে দেখলেন log ফোল্ডারটিই বড়, `grep -i error /var/log/nginx/error.log | tail -20` দিয়ে শেষ ভুলগুলো দেখলেন, `tar` দিয়ে পুরোনো log সংরক্ষণ করলেন। কোনো GUI টুল খোলা হয়নি, কোনো সার্ভার restart করা হয়নি। এই অধ্যায়ের কমান্ডগুলোই সেই দক্ষতার ভিত্তি। আপনি এখানে ফাইল পরিচালনা, খোঁজা, টেক্সট প্রক্রিয়াকরণ ও সংরক্ষণ (archive/compression) শিখবেন — এমনভাবে, যাতে প্রতিটি কমান্ডের কাজ ও ঝুঁকি বুঝে ব্যবহার করতে পারেন।

Learning Objectives — শেখার লক্ষ্য

- `pwd, ls, cd, mkdir, touch, cp, mv, rm, ln` নিরাপদে ব্যবহার করতে পারবেন।
- `cat, less, head, tail` দিয়ে ফাইল পড়তে ও log অনুসরণ করতে পারবেন।
- `find, locate, grep` দিয়ে ফাইল ও টেক্সট খুঁজতে পারবেন।
- `sort, uniq, wc, cut, tr, sed, awk, xargs` দিয়ে টেক্সট প্রক্রিয়া করতে পারবেন।
- `tar, gzip` দিয়ে archive ও compression করতে পারবেন।
- একাধিক কমান্ড pipe দিয়ে যুক্ত করে বাস্তব সমস্যা সমাধান করতে পারবেন।
- ধ্বংসাত্মক কমান্ডের ঝুঁকি চিনে নিরাপদ বিকল্প ব্যবহার করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

Chapter 08 (filesystem, path) ও Chapter 09 (shell, pipe, redirection, quoting) আবশ্যক। এই অধ্যায়ের সব অনুশীলন VM-এ, নিজের home ডিরেক্টরির ভেতরে করুন — system ডিরেক্টরিতে নয়।

ফাইল ও ডিরেক্টরি পরিচালনা: মূল কমান্ড

এই কমান্ডগুলো GNU coreutils-এর অংশ — প্রায় সব Linux distribution-এ একইভাবে কাজ করে। প্রতিটি কমান্ডের আগে ভাবুন: এটি কী বদলাবে? ভুল হলে ফিরে আসা যাবে কি?

```
pwd; ls -la
# বর্তমান ডিরেক্টরি ও লুকানো ফাইলসহ (-a) দীর্ঘ তালিকা (-l) দেখায়।

mkdir -p ~/projects/bookora/src
# -p মানে প্রয়োজনে parent ডিরেক্টরিও তৈরি করা; না থাকলে ভুল হয় না।

touch ~/projects/bookora/README.md
# খালি ফাইল তৈরি করে; থাকলে শুধু সময় (timestamp) হালনাগাদ করে।
```

```
cp ~/projects/bookora/README.md ~/projects/bookora/README.md.bak
# ফাইল কপি করে; মূল ফাইল অপরিবর্তিত থাকে।

cp -r ~/projects/bookora ~/backup-bookora
# -r মানে ডিরেক্টরিসহ recursive কপি।

mv ~/projects/bookora/README.md ~/projects/bookora/README.txt
# mv মানে সরানো বা নাম বদলানো — একই filesystem-এ দ্রুত হয়।

ln -s ~/projects/bookora ~/bookora-link
# symbolic link তৈরি করে (Chapter 08 দেখুন)।

ls -lh ~/projects/
# -h মানে আকার মানুষের পড়ার মতো (KB, MB) দেখানো।
```

WARNING — rm ব্যবহারের আগে পড়ুন: Linux-এ `rm` দিয়ে মুছে ফেলা ফাইল সাধারণত Recycle Bin-এ যায় না — সরাসরি মুছে যায়। `rm -rf` বিশেষভাবে বিপজ্জনক: `-r` মানে recursive, `-f` মানে জিজ্ঞেস না করে মুছে ফেলা। ভুল পথ দিলে পুরো ডিরেক্টরি মুছে যেতে পারে। নিরাপদ অভ্যাস: (১) প্রথমে `ls <পথ>` দিয়ে যাচাই করুন; (২) `rm -i` (interactive — প্রতিটি ফাইলে জিজ্ঞেস করে) ব্যবহার করুন; (৩) গুরুত্বপূর্ণ ফাইলের backup নিন; (৪) root হিসেবে `rm -rf /` বা `rm -rf ~` কখনও চালাবেন না। শেখার সময় শুধু নিজের তৈরি test ফোল্ডারে `rm` অনুশীলন করুন।

```
rm -i ~/test-file.txt
# -i মানে মুছে ফেলার আগে জিজ্ঞেস করবে — নিরাপদ।

rmdir ~/empty-dir
# শুধু খালি ডিরেক্টরি মোছা যায়; ভরা ডিরেক্টরি মোছা যায় না।

rm -r ~/lab10-old
# ডিরেক্টরিসহ মোছা — শুধু নিজের test ফোল্ডারে, যাচাই করে চালান।
```

ফাইল পড়া: cat, less, head, tail

```
cat /etc/hostname
# ছোট ফাইলের পুরো内容 দেখায়।

less /var/log/syslog
# বড় ফাইল পৃষ্ঠা ধরে দেখায়; / দিয়ে খোঁজা যায়, q দিয়ে বের হওয়া যায়।

head -20 /var/log/syslog
# প্রথম ২০ লাইন দেখায়।

tail -20 /var/log/syslog
# শেষ ২০ লাইন দেখায়।

tail -f /var/log/syslog
# -f মানে follow — নতুন লাইন এলেই দেখায়; থামাতে Ctrl+C।
# log সরাসরি দেখার জন্য এটি অ্যাডমিনের প্রিয় কমান্ড।

wc -l /etc/passwd
# -l মানে লাইন সংখ্যা গোনা।
```

```
cat -n ~/notes.txt
# -n মানে লাইন নম্বরসহ দেখানো।
```

কখন কোনটি? ছোট ফাইলে `cat`, বড় ফাইলে `less`, শুরুর অংশে `head`, শেষের অংশে `tail`, live log-এ `tail -f`। হাজার লাইনের log `cat` দিয়ে দেখলে স্ক্রিন ভরে যাবে — সেটি ভুল টুল নির্বাচন।

খোঁজা: find, locate, grep

চিত্র ১০.১ — খোঁজার তিনটি টুল: কখন কোনটি

ফাইলের নাম দিয়ে খুঁজবেন? → find (live, ধীর কিন্তু নির্ভুল) বা locate (দ্রুত, তবে database হালনাগাদ থাকতে হয়) | ফাইলের ভেতরের লেখা খুঁজবেন? → grep | বড় গাছে নির্দিষ্ট ধরনের ফাইল? → find -name "*.log" -mtime -7 (গত ৭ দিনে বদলানো .log ফাইল)

```
find ~/projects -name "*.py"
# ~/projects-এর নিচে সব .py ফাইল খোঁজে।

find /var/log -name "*.log" -mtime -7
# গত ৭ দিনে বদলানো .log ফাইল খোঁজে। -mtime -7 মানে ৭ দিনের কম পুরোনো।

find ~/projects -type f -size +1M
# ১ MB-এর বড় regular ফাইল (-type f) খোঁজে।

grep -i "error" /var/log/syslog
# -i মানে case-insensitive; error/Error/ERROR সব মিলবে।

grep -rn "TODO" ~/projects/
# -r মানে recursive, -n মানে লাইন নম্বরসহ দেখানো।

grep -c "Failed" /var/log/auth.log
# -c মানে মিলের সংখ্যা গোনা (ব্যর্থ login গুনতে কাজে লাগে)।

ps aux | grep -i nginx
# চলমান process-এর মধ্যে nginx খোঁজা — pipe-এর উদাহরণ।
```

`locate` একটি database (`updatedb` দিয়ে তৈরি) থেকে খোঁজে, তাই দ্রুত; তবে নতুন ফাইল সঙ্গে সঙ্গে পাওয়া যায় না। Ubuntu Server-এ `locate` ডিফল্টভাবে ইনস্টল নাও থাকতে পারে — তখন `find` ব্যবহার করুন।

টেক্সট প্রসেসিং: sort, uniq, wc, cut, tr

এই টুলগুলো একসঙ্গে pipe দিয়ে ব্যবহার করলেই শক্তিশালী হয়। প্রতিটি টুল একটি ছোট কাজ করে — Unix দর্শনের মূল কথা এটাই।

```
sort ~/names.txt
# লাইনগুলো বর্ণানুক্রমে সাজায়।

sort -n ~/numbers.txt
# -n মানে সংখ্যা হিসেবে সাজানো (১০-কে ২-এর পরে নয়, সঠিক ক্রমে)।
```

```
sort ~/names.txt | uniq -c
# uniq -c মানে প্রতিটি আলাদা লাইন কতবার এসেছে, তা গোনা।
# মনে রাখবেন: uniq কাজ করে শুধু পাশাপাশি একই লাইনে — তাই আগে sort করা জরুরি।

wc -l -w -c ~/notes.txt
# -l লাইন, -w শব্দ, -c byte গোনে।

cut -d: -f1 /etc/passwd
# -d: মানে : দিয়ে ভাগ করা, -f1 মানে প্রথম ক্ষেত্র (username) নেওয়া।

cut -d: -f1,3 /etc/passwd
# প্রথম ও তৃতীয় ক্ষেত্র (username ও UID) নেওয়া।

echo "Hello World" | tr 'a-z' 'A-Z'
# tr মানে translate — ছোট হাতের অক্ষর বড় হাতের করা।

cat /var/log/auth.log | grep "Failed" | wc -l
# ব্যর্থ login-এর লাইন সংখ্যা গোনা — বাস্তব নিরাপত্তা পরীক্ষা।
```

sed ও awk: সংক্ষিপ্ত পরিচিতি

`sed` (stream editor) টেক্সট বদলানোর জন্য, `awk` কলামভিত্তিক ডেটা প্রক্রিয়ার জন্য ব্যবহৃত হয়। দুটোই গভীর টুল; এখানে শুধু নিরাপদ, সাধারণ ব্যবহার শেখানো হলো — বিস্তারিত শেখা পরের ধাপে।

```
sed 's/old/new/' ~/file.txt
# প্রথম old-কে new দিয়ে বদলে দেখায় (ফাইল বদলায় না, স্ক্রিনে দেখায়)।

sed -i 's/old/new/g' ~/file.txt
# -i মানে ফাইলেই বদলানো; g মানে লাইনের সবগুলো বদলানো।
# সতর্ক: -i চালানোর আগে ফাইলের backup নিন (cp file file.bak)।

sed -n '1,5p' ~/file.txt
# ১ থেকে ৫ নম্বর লাইন শুধু দেখায়।

awk -F: '{print $1}' /etc/passwd
# -F: মানে : দিয়ে ভাগ করা; $1 মানে প্রথম ক্ষেত্র ছাপানো।

awk '{print $1, $3}' ~/data.txt
# space দিয়ে ভাগ করা প্রথম ও তৃতীয় কলাম ছাপানো।

df -h | awk '{print $1, $5}'
# ডিস্ক ব্যবহারের শুধু filesystem ও ব্যবহারের শতাংশ দেখানো।
```

> **CONCEPT:** `sed`-এর `s/old/new/` হলো substitute কমান্ড। `awk`-এ `$0` মানে পুরো লাইন, `$1` প্রথম ক্ষেত্র, `NF` ক্ষেত্রের সংখ্যা। দুটো টুলই শেখার জন্য একটি ছোট test ফাইল দিয়ে অনুশীলন করুন — আসল log বা কনফিগারেশনে সরাসরি `-i` চালাবেন না।

xargs ও তার/কমপ্রেশন

```
find ~/projects -name "*.tmp" | xargs rm -i
# find-এর ফলাফল xargs-এর মাধ্যমে rm-এর argument হিসেবে পাঠানো।
```

```
# -i থাকায় প্রতিটি ফাইলে জিজ্ঞেস করবে — নিরাপদ।

echo ~/projects/*.txt | xargs wc -l
# একাধিক ফাইলের লাইন একসঙ্গে গোনা।

tar -czf ~/backup-2026-11-08.tar.gz ~/projects/bookora
# -c মানে তৈরি করা, -z মানে gzip দিয়ে সংকোচন, -f মানে ফাইলের নাম।
# ফলাফল: একটি .tar.gz ফাইল।

tar -tzf ~/backup-2026-11-08.tar.gz
# -t মানে ভেতরে কী আছে, না খুলে দেখা।

tar -xzf ~/backup-2026-11-08.tar.gz -C ~/restore/
# -x মানে খোলা, -C মানে নির্দিষ্ট ডিরেক্টরিতে খোলা।

gzip ~/large.log
# ফাইল সংকুচিত করে .gz বানায় (মূল ফাইল সরিয়ে নেয়)।

gunzip ~/large.log.gz
# সংকোচন খোলে।
```

চিত্র ১০.২ — Pipe দিয়ে বাস্তব সমস্যা সমাধান (উদাহরণ)

সমস্যা: কোন IP থেকে সবচেয়ে বেশি ব্যর্থ SSH login আসছে? → grep "Failed" /var/log/auth.log → awk দিয়ে IP ক্ষেত্র নেওয়া → sort → uniq -c → sort -nr (সংখ্যা অনুযায়ী উল্টো ক্রম) → head -5 | ফলাফল: সন্দেহজনক IP-এর তালিকা, যা দিয়ে UFW/firewall সিদ্ধান্ত নেওয়া যায় (Chapter 19)।

Terminology — পরিভাষা

Term	Bangla ব্যাখ্যা
coreutils	GNU-এর মৌলিক কমান্ডের সমষ্টি (ls, cp, rm ইত্যাদি)
Recursive (-r)	ডিরেক্টরির ভেতরের সবকিছুসহ কাজ করা
Regular expression (regex)	টেক্সট খোঁজার নকশা (যেমন `^error`)
Archive (tar)	একাধিক ফাইল একত্রিত করা (সংকোচন ছাড়াও হতে পারে)
Compression (gzip)	ফাইলের আকার ছোট করা
stdin / stdout / stderr	ইনপুট, সাধারণ আউটপুট, ভুলের বার্তার প্রবাহ
Timestamp	ফাইল তৈরি/বদলানোর সময়ের তথ্য

Hands-on Exercise ১০.১ — Log বিশ্লেষণ ল্যাব

LAB — ল্যাব ১০.১: পরিস্থিতি: আপনি জুনিয়র অ্যাডমিন; /var/log-এ একটি test log বিশ্লেষণ করতে হবে। ধাপ: (১) `mkdir -p ~/lab10 && printf 'ERROR disk full\nINFO started\nERROR timeout\nINFO done\nERROR disk full\n' > ~/lab10/app.log` দিয়ে test log তৈরি করুন; (২) `grep -c ERROR ~/lab10/app.log` দিয়ে ERROR গুনুন; (৩) `sort ~/lab10/app.log | uniq -c` চালান; (৪) `cut -d' ' -f1 ~/lab10/app.log | sort | uniq -c` দিয়ে level অনুযায়ী গুনুন; (৫) `tar -czf ~/lab10-backup.tar.gz ~/lab10 && tar -tzf ~/lab10-backup.tar.gz` দিয়ে backup যাচাই করুন। যাচাই: ERROR সংখ্যা ৩, backup তালিকায় app.log দেখা যায়। Cleanup: `rm -r ~/lab10 ~/lab10-backup.tar.gz` (শুধু নিজের তৈরি ফাইল, যাচাই করে)।

বাস্তব উদাহরণ

রাজশাহীর একটি হোস্টিং কোম্পানির সাপোর্ট ইঞ্জিনিয়ার প্রতিদিন সকালে চালান: `df -h | grep -v tmpfs` (ডিস্ক), `grep -i "error" /var/log/nginx/error.log | tail -20` (ওয়েব সার্ভারের ভুল), `wc -l /var/log/auth.log` (login চেষ্টা)। এই তিনটি কমান্ডেই সার্ভারের স্বাস্থ্যের প্রাথমিক ছবি পাওয়া যায়। আরেকটি উদাহরণ: বিশ্ববিদ্যালয়ের প্রজেক্টে ২০০টি CSV ফাইলের প্রথম কলাম একত্রিত করতে `cut` ও `sort | uniq` ব্যবহার করা হয় — Excel খুলে হাতে করা অসম্ভব কাজ কয়েক সেকেন্ডে হয়।

Common Mistakes & Troubleshooting

- **ভুল:** `grep error` লিখে `Error` না পাওয়া। **সমাধান:** `grep -i error` ব্যবহার করুন।
- **ভুল:** `uniq` চালিয়ে ডুপ্লিকেট না কমা। **সমাধান:** আগে `sort` করুন — uniq শুধু পাশাপাশি একই লাইন মেলায়।
- **ভুল:** `sed -i` দিয়ে কনফিগারেশন নষ্ট করা। **সমাধান:** আগে backup, প্রথমে `-i` ছাড়া পরীক্ষা করুন।
- **ভুল:** `tar -xzf` ভুল ডিরেক্টরিতে চালিয়ে ফাইল ছড়িয়ে ফেলা। **সমাধান:** আগে `tar -tzf` দিয়ে ভেতরে কী আছে দেখুন, `-C` দিয়ে গন্তব্য ঠিক করুন।
- **ভুল:** `rm *.txt` চালানোর আগে কী মুছবে না দেখা। **সমাধান:** আগে `ls *.txt` দিয়ে তালিকা দেখুন।
- **ভুল:** `find / -name ...` চালিয়ে permission denied বার্তায় ভয় পাওয়া। **সমাধান:** এটি স্বাভাবিক; `2>/dev/null` দিয়ে ভুলের বার্তা লুকানো যায়।

Security Considerations

`grep` দিয়ে log-এ পাসওয়ার্ড বা token খুঁজে পেলে সেগুলো শেয়ার করবেন না, স্ক্রিনশটে ঢেকে দিন। `history`-তে সংবেদনশীল তথ্যসহ কমান্ড থাকলে তা মুছে ফেলুন। অপরিচিত ফাইল `tar` দিয়ে খোলার আগে `-t` দিয়ে তালিকা দেখুন — ক্ষতিকর archive-এ absolute path থাকতে পারে। `xargs rm` চালানোর আগে `xargs echo rm` দিয়ে দেখুন কী মুছবে।

Best Practices

- ধ্বংসাত্মক কমান্ডের আগে `ls` বা `echo` দিয়ে যাচাই করুন; `-i` ব্যবহার করুন।
- বড় ফাইলে `cat` নয়, `less` বা `head/tail` ব্যবহার করুন।
- বারবার ব্যবহৃত pipe-এর ক্রম নোটে লিখে রাখুন (যেমন log বিশ্লেষণের ক্রম)।

- কনফিগারেশন বদলানোর আগে `cp file file.bak` দিয়ে backup নিন।
- নিয়মিত অনুশীলন করুন: প্রতিদিন একটি নতুন option শিখুন (`man` পড়ে)।

Knowledge Check — যাচাই করুন

CHECK YOUR UNDERSTANDING:

1. `rm -rf`-এর `-f` মানে কী? A) ফাইল খোঁজা B) জিজ্ঞেস না করে মুছে ফেলা C) ফাইল খুঁজে বের করা D) ফরম্যাট করা

2. Live log অনুসরণ করতে কোন কমান্ড? A) cat B) head C) tail -f D) less -f

3. `uniq -c`-এর আগে কেন sort করা জরুরি? A) গতি বাড়াতে B) uniq শুধু পাশাপাশি একই লাইন মেলায় C) ফাইল ছোট করতে D) রঙ দেখাতে

4. `cut -d: -f1 /etc/passwd` কী দেয়? A) পুরো লাইন B) username C) UID D) home ডিরেক্টরি

5. `tar -tzf backup.tar.gz` কী করে? A) ফাইল খোলে B) ভেতরের তালিকা না খুলে দেখায় C) ফাইল মুছে দেয় D) সংকোচন করে

Practical Assignment

আপনার VM-এ একটি test ডিরেক্টরি তৈরি করে ১০টি test ফাইল বানান (নামে তারিখসহ, যেমন `log-2026-11-01.txt`)। তারপর (ক) `find` দিয়ে সব .txt ফাইল খুঁজুন; (খ) `grep` দিয়ে একটি শব্দ খুঁজুন; (গ) `sort | uniq -c` দিয়ে একটি কলাম বিশ্লেষণ করুন; (ঘ) `tar -czf` দিয়ে backup নিন ও `tar -tzf` দিয়ে যাচাই করুন। প্রতিটি ধাপের কমান্ড ও আউটপুট নোটে লিখুন। সমাধান Answer Guide-এ দেখুন।

Chapter Summary

ফাইল পরিচালনার মূল কমান্ড — `ls, cp, mv, mkdir, touch, ln` — ও নিরাপদ `rm` ব্যবহার শেখা হয়েছে। ফাইল পড়তে `cat, less, head, tail` (live log-এ `tail -f`), খুঁজতে `find, grep`, টেক্সট প্রক্রিয়ায় `sort, uniq, wc, cut, tr, sed, awk, xargs` ব্যবহৃত হয়। `tar` ও `gzip` দিয়ে backup ও সংরক্ষণ করা যায়। আসল দক্ষতা হলো pipe দিয়ে এই ছোট টুলগুলো যুক্ত করে বাস্তব সমস্যা সমাধান করা — এবং ধ্বংসাত্মক কমান্ডের আগে যাচাই করা।

Revision Checklist

- ☐ নিরাপদে cp, mv, rm ব্যবহার করতে পারি
- ☐ tail -f দিয়ে log অনুসরণ করতে পারি
- ☐ find ও grep দিয়ে খুঁজতে পারি
- ☐ sort | uniq -c দিয়ে গুনতে পারি
- ☐ tar দিয়ে backup নিতে ও যাচাই করতে পারি

PART III • DEVOPS ENGINEERING MASTERY

Linux Administration

সিস্টেম প্রশাসন — ব্যবহারকারী, অনুমতি, প্রসেস, প্যাকেজ ও systemd।

Chapter 11: Users, Groups and Permission Management — ব্যবহারকারী, গ্রুপ ও অনুমতি ব্যবস্থাপনা

ভূমিকা: একই সার্ভারে অনেক মানুষ, তবু প্রত্যেকের নিজস্ব সীমানা

ধরুন, ঢাকার একটি ছোট সফটওয়্যার কোম্পানিতে একটি Ubuntu সার্ভার আছে। সেই সার্ভারে একজন ডেভেলপার কোড deploy করে, একজন ডিজাইনার ছবি আপলোড করে, আর একজন হিসাবরক্ষক বিলের ফাইল রাখে। যদি সবাই সব ফাইল পড়তে, বদলাতে ও মুছতে পারত, তাহলে একজনের ভুলে আরেকজনের কাজ নষ্ট হয়ে যেত। আবার একজন ইন্টার্ন যদি ভুল করে সিস্টেমের গুরুত্বপূর্ণ ফাইল মুছে ফেলে, তাহলে পুরো সার্ভার বন্ধ হয়ে যেতে পারে। এই সমস্যার সমাধানই হলো Linux-এর identity ও permission ব্যবস্থা। Linux প্রত্যেক ব্যবহারকারীকে আলাদা পরিচয় দেয়, প্রত্যেক ফাইলকে একজন মালিক দেয়, আর প্রত্যেক ফাইলে লিখে রাখে — কে পড়তে পারবে, কে বদলাতে পারবে, আর কে চালাতে পারবে। এই অধ্যায়ে আমরা সেই ব্যবস্থার ভেতরের যুক্তি বুঝব, যাতে আপনি শুধু কমান্ড মুখস্থ না করে, কেন একটি অনুমতি দেওয়া নিরাপদ আর কেন আরেকটি বিপজ্জনক, তা নিজে বিচার করতে পারেন।

Learning Objectives — শেখার লক্ষ্য

- Linux-এর identity model ব্যাখ্যা করতে পারবেন: user, group, UID ও GID কী এবং কেন দরকার।
- `useradd`, `usermod`, `userdel`, `groupadd` ও `passwd` দিয়ে ব্যবহারকারী ও গ্রুপ তৈরি ও পরিচালনা করতে পারবেন।
- `/etc/passwd`, `/etc/shadow` ও `/etc/group` ফাইলের গঠন পড়ে বুঝতে পারবেন।
- read, write ও execute অনুমতির অর্থ ব্যাখ্যা করতে পারবেন এবং `chmod` ও `chown` সঠিকভাবে ব্যবহার করতে পারবেন।
- `umask` ও special permission (setuid, setgid, sticky bit) এর কাজ বুঝতে পারবেন।
- একটি shared project directory নিরাপদভাবে সাজাতে পারবেন — `chmod 777` ব্যবহার না করে।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৮ (Filesystem Hierarchy) ও অধ্যায় ৯–১০ (Terminal, Shell ও Essential Commands) জানা থাকতে হবে। বিশেষ করে `ls -l`, `cd`, `mkdir` ও `sudo` সম্পর্কে ধারণা থাকা দরকার। Root ও সাধারণ ব্যবহারকারীর পার্থক্য সম্পর্কে প্রাথমিক ধারণা থাকলেই চলবে; বিস্তারিত এই অধ্যায়েই ব্যাখ্যা করা হয়েছে।

প্রথম নীতি থেকে: পরিচয় কেন দরকার?

কম্পিউটার নিজে থেকে জানে না কে ভালো আর কে খারাপ। সে শুধু জানে, কোনো process কোনো ব্যবহারকারীর নামে চলছে। Linux-এ প্রতিটি process একজন ব্যবহারকারীর পরিচয়ে চলে, প্রতিটি ফাইলের একজন মালিক থাকে। যখন কোনো process একটি ফাইল খুলতে চায়, kernel তিনটি প্রশ্ন করে: তুমি কে? এই ফাইলের মালিক কে? তোমাকে কী করার অনুমতি দেওয়া হয়েছে? এই তিন প্রশ্নের উত্তর মিলিয়ে kernel সিদ্ধান্ত নেয় — অনুমতি দেবে, নাকি `Permission denied` বলবে। এটাই discretionary access control-এর মূল কথা। এর ফলে একজন ব্যবহারকারীর ভুল বা ক্ষতিকর কাজ পুরো সিস্টেমে ছড়িয়ে পড়তে পারে না। একজন ছাত্রের হোম ডিরেক্টরির ফাইল আরেকজন ছাত্র পড়তে পারে না, কারণ মালিকানা ও অনুমতি সেই সীমানা টেনে দেয়।

Linux Identity Model: User, Group, UID ও GID

Linux মানুষকে নামে চেনে না, সংখ্যায় চেনে। প্রত্যেক ব্যবহারকারীর একটি User ID (UID) থাকে, প্রত্যেক গ্রুপের একটি Group ID (GID) থাকে। নাম শুধু মানুষের সুবিধার জন্য; kernel সংখ্যাটাই দেখে। Ubuntu-তে সাধারণত UID 1000 থেকে সাধারণ ব্যবহারকারী শুরু হয়, UID 0 হলো root — সর্বময় ক্ষমতার অধিকারী। UID 1–999 সাধারণত system user-দের জন্য সংরক্ষিত থাকে, যেমন `www-data` বা `mysql` — এরা মানুষ নয়, এরা service-এর পরিচয়। Group একই ধরনের কাজ করা ব্যবহারকারীদের একত্র করে। যেমন `developers` গ্রুপে থাকা সবাই একটি project directory-তে কাজ করতে পারবে, কিন্তু `accounts` গ্রুপের কেউ পারবে না। এতে প্রত্যেক ব্যক্তিকে আলাদা করে অনুমতি না দিয়ে গ্রুপকে একবার অনুমতি দিলেই হয়।

চিত্র ১১.১ — Identity যাচাইয়ের প্রবাহ

ব্যবহারকারী login করে → PAM ও /etc/shadow পাসওয়ার্ড যাচাই করে → shell একটি UID/GID নিয়ে শুরু হয় → প্রতিটি process সেই UID বহন করে → ফাইল খোলার সময় kernel মালিকানা ও permission পরীক্ষা করে → অনুমতি দিলে কাজ হয়, না দিলে Permission denied।

পরিচয়	উদাহরণ	অর্থ
UID 0	root	সবকিছু করতে পারে; দৈনন্দিন কাজে ব্যবহার করা উচিত নয়
UID 1000	nahid	প্রথম সাধারণ ব্যবহারকারী
UID 1001	tasnim	দ্বিতীয় সাধারণ ব্যবহারকারী
GID 1000	developers	ডেভেলপারদের গ্রুপ
System UID	www-data (33)	ওয়েব সার্ভার service-এর পরিচয়, মানুষ নয়

/etc/passwd, /etc/shadow ও /etc/group — পরিচয়ের খাতা

ব্যবহারকারীর তথ্য তিনটি ফাইলে থাকে। `/etc/passwd` সবার পড়ার মতো তথ্য রাখে — নাম, UID, GID, home directory, shell — কিন্তু পাসওয়ার্ড রাখে না। পাসওয়ার্ডের hash থাকে `/etc/shadow`-এ, যা শুধু root পড়তে পারে। `/etc/group` গ্রুপের নাম, GID ও সদস্যদের তালিকা রাখে। একটি লাইন দেখলেই বোঝা যায় কাঠামোটি: `nahid:x:1000:1000:Nahid Hasan:/home/nahid:/bin/bash` — এখানে `x` মানে পাসওয়ার্ড shadow ফাইলে আছে। এই বিভাজনই নিরাপত্তা: কেউ passwd পড়ে পাসওয়ার্ডের hash পায় না। মনে রাখবেন, এই ফাইলগুলো হাতে সরাসরি সম্পাদনা করা বিপজ্জনক; সবসময় `useradd`, `usermod` বা `vipw`-এর মতো টুল ব্যবহার করবেন।

চিত্র ১১.২ — Permission-এর তিনটি শ্রেণি

শ্রেণি	চিহ্ন	যাদের ক্ষেত্রে প্রযোজ্য
Owner (u)	প্রথম তিনটি অক্ষর	ফাইলের মালিক
Group (g)	মাঝের তিনটি অক্ষর	মালিকের গ্রুপের সদস্যরা
Others (o)	শেষ তিনটি অক্ষর	বাকি সবাই

উদাহরণ: `-rwxr-xr--` মানে owner পড়তে, লিখতে ও চালাতে পারে; group পড়তে ও চালাতে পারে; others শুধু পড়তে পারে। ডিরেক্টরিতে execute মানে ভেতরে ঢোকার অনুমতি, আর read মানে তালিকা দেখার অনুমতি — দুটি আলাদা জিনিস।

পরিভাষা

Term	বাংলা ব্যাখ্যা
UID / GID	ব্যবহারকারী ও গ্রুপের সংখ্যাগত পরিচয়; kernel এটাই দেখে
root	UID 0; সব অনুমতির অধিকারী superuser
Home directory	ব্যবহারকারীর ব্যক্তিগত ডিরেক্টরি, সাধারণত /home/নাম
Permission bits	r (read), w (write), x (execute) — নয়টি বিটের সমষ্টি
umask	নতুন ফাইল ও ডিরেক্টরির অনুমতি থেকে যা বাদ যাবে, তার mask
setuid / setgid	ফাইল চালানোর সময় মালিক বা গ্রুপের পরিচয়ে চলার বিশেষ বিট
Sticky bit	shared ডিরেক্টরিতে নিজের ফাইল ছাড়া অন্যের ফাইল মোছা যায় না
Primary group	ব্যবহারকারীর মূল গ্রুপ; নতুন ফাইল সাধারণত এই গ্রুপ পায়

বাস্তব উদাহরণ: বিশ্ববিদ্যালয়ের ল্যাব সার্ভার

চট্টগ্রাম বিশ্ববিদ্যালয়ের একটি ল্যাবে ৩০ জন ছাত্র একই সার্ভার ব্যবহার করে। প্রত্যেকের home directory `/home/student01` থেকে `/home/student30`। শিক্ষক চান, ছাত্ররা একে অপরের assignment দেখতে না পারুক, কিন্তু `/srv/lab-shared` ডিরেক্টরিতে সবাই পড়তে পারুক। সমাধান: home directory-র অনুমতি `700` (শুধু মালিক), shared ডিরেক্টরি `755` (সবাই পড়তে পারে, শুধু মালিক লিখতে পারে)। এতে কারও ব্যক্তিগত কাজে অন্যের হাত পড়ে না, আবার সাধারণ উপকরণ সবাই পায়। এটাই permission-এর দৈনন্দিন ব্যবহার।

ব্যবহারিক কমান্ড

```
id nahid
# ব্যবহারকারীর UID, GID ও সদস্য গ্রুপগুলো দেখায়।

sudo useradd -m -s /bin/bash -c "Tasnim Rahman" tasnim
# -m home directory তৈরি করে, -s shell ঠিক করে, -c পূর্ণ নাম লেখে।
```

```
# useradd নিজে থেকে পাসওয়ার্ড দেয় না — পরের কমান্ডে দিতে হয়।

sudo passwd tasnim
# tasnim-এর পাসওয়ার্ড সেট করে। পাসওয়ার্ড টাইপের সময় কিছু দেখায় না — এটাই স্বাভাবিক।

sudo usermod -aG developers tasnim
# -aG মানে append to group: আগের গ্রুপ না মুছে developers গ্রুপে যোগ করে।
# -G একা ব্যবহার করলে আগের সব supplementary গ্রুপ মুছে যায় — এই ভুলটি খুব সাধারণ।

sudo groupadd developers
sudo userdel -r tasnim
# -r দিলে home directory ও mail spool-সহ মুছে যায়। সাবধান: মুছে ফেলা ফাইল ফেরত আসে না।

ls -l /home/nahid/report.txt
# -rw-r--r-- মানে owner rw, group r, others r।

sudo chmod 640 /home/nahid/report.txt
sudo chown nahid:developers /srv/project
# প্রথমটি অনুমতি বদলায়, দ্বিতীয়টি মালিক ও গ্রুপ বদলায়।

umask
# বর্তমান umask দেখায়; সাধারণত 0002 বা 0022। নতুন ফাইলের অনুমতি = 666 - umask।
```

WARNING: কখনো সমস্যা সমাধানের শর্টকাট হিসেবে `chmod 777` ব্যবহার করবেন না। এতে পৃথিবীর যে কেউ ফাইল বদলাতে ও মুছতে পারে। সমস্যা হলে মালিকানা বা গ্রুপ ঠিক করুন — অনুমতি সবার জন্য খুলে দেবেন না। `chmod -R 777 /` বা `chmod -R 777 /etc` চালালে সিস্টেম কার্যত ধ্বংস হয়ে যায়।

CONCEPT: সংখ্যায় permission মনে রাখার উপায়: r=4, w=2, x=1। 7 মানে 4+2+1 (rwx), 6 মানে 4+2 (rw-), 5 মানে 4+1 (r-x), 0 মানে কোনো অনুমতি নেই। `755` তাই মানে owner rwx, group r-x, others r-x।

হাতে-কলমে অনুশীলন: Project গ্রুপ ও Shared Directory

লক্ষ্য: `developers` গ্রুপের দুইজন ব্যবহারকারী একটি shared ডিরেক্টরিতে একসঙ্গে কাজ করবে, বাইরের কেউ লিখতে পারবে না।

1. `sudo groupadd developers` দিয়ে গ্রুপ তৈরি করুন।
2. `sudo useradd -m -s /bin/bash rafi` ও `sudo useradd -m -s /bin/bash nabila` দিয়ে দুজন ব্যবহারকারী তৈরি করুন।
3. দুজনকে গ্রুপে যোগ করুন: `sudo usermod -aG developers rafi` এবং `sudo usermod -aG developers nabila`।
4. `sudo mkdir -p /srv/project` ও `sudo chown root:developers /srv/project` চালান।
5. `sudo chmod 2770 /srv/project` চালান — 2 হলো setgid বিট, যাতে নতুন ফাইলও developers গ্রুপ পায়।
6. যাচাই করুন: `ls -ld /srv/project` ও `id rafi`।

উদাহরণ আউটপুট (illustrative):

```
drwxrws--- 2 root developers 4096 Nov  8 10:00 /srv/project
uid=1001(rafi) gid=1001(rafi) groups=1001(rafi),1002(developers)
```

`drwxrws---`-এর `s` দেখাচ্ছে setgid চালু আছে, আর `---` দেখাচ্ছে others-এর কোনো অনুমতি নেই। গ্রুপ সদস্য হিসেবে পরিবর্তন কার্যকর হতে হলে rafi ও nabila-কে নতুন করে login করতে হবে — পুরোনো shell-এ গ্রুপ সদস্যপদ আপডেট হয় না।

একটি প্রশ্ন প্রায়ই ওঠে: home directory-র ডিফল্ট অনুমতি কেন `755` নয়, `750` বা `700` হওয়া উচিত? Ubuntu-তে নতুন ব্যবহারকারীর home directory সাধারণত `755` হয়, অর্থাৎ অন্যরা ভেতরের ফাইলের নাম দেখতে পারে, যদিও ফাইল পড়তে পারে কি না তা ফাইলের নিজের অনুমতির ওপর নির্ভর করে। সংবেদনশীল কাজের ক্ষেত্রে `chmod 700 /home/ব্যবহারকারী` নিরাপদ, কারণ তখন অন্য কেউ ডিরেক্টরিতে ঢুকতেই পারে না। তবে মনে রাখবেন, ওয়েব সার্ভার যদি ব্যবহারকারীর home directory থেকে ফাইল serve করে, তাহলে সেখানে execute অনুমতি দরকার হতে পারে — তাই অনুমতি বদলানোর আগে সেবাটি কীভাবে ফাইল পড়ে, তা যাচাই করুন।

সাধারণ ভুল ও সমস্যা সমাধান

- **ভুল:** `usermod -G developers rafi` (-a ছাড়া) চালিয়ে আগের সব গ্রুপ হারানো। **সমাধান:** `id rafi` দিয়ে পরীক্ষা করুন, প্রয়োজনে `usermod -aG` দিয়ে আবার যোগ করুন।
- **ভুল:** `useradd`-এর পর পাসওয়ার্ড না দেওয়া, ফলে login করা যায় না। **সমাধান:** `sudo passwd ব্যবহারকারী` চালান।
- **ভুল:** ডিরেক্টরিতে read থাকলেও execute না থাকায় ভেতরে ঢোকা যায় না। **সমাধান:** ডিরেক্টরিতে সাধারণত `r-x` একসঙ্গে দিতে হয়।
- **ভুল:** home directory-র মালিকানা root হয়ে যাওয়া। **সমাধান:** `sudo chown -R ব্যবহারকারী:ব্যবহারকারী /home/ব্যবহারকারী`। `-R` recursive — ভুল পথে চালালে বিপদ, তাই পথ দুবার পরীক্ষা করুন।

TROUBLESHOOTING: `Permission denied` পেলে প্রথমে `ls -l` দিয়ে মালিক ও অনুমতি দেখুন, তারপর `id` দিয়ে নিজের UID ও গ্রুপ দেখুন। তিনটি শ্রেণির মধ্যে কোনটিতে আপনি পড়েন, সেটা মিলিয়ে দেখুন। অনুমতি খুলে দেওয়ার আগে প্রশ্ন করুন — আমি কি owner, group সদস্য, নাকি others?

নিরাপত্তা বিবেচনা

Root অ্যাকাউন্টে সরাসরি login বন্ধ রাখুন, দৈনন্দিন কাজ সাধারণ ব্যবহারকারী দিয়ে করুন, আর প্রয়োজনে `sudo` ব্যবহার করুন। অপ্রয়োজনীয় ব্যবহারকারী তৈরি করবেন না, আর চাকরি ছেড়ে যাওয়া কারও অ্যাকাউন্ট দ্রুত lock করুন (`sudo usermod -L ব্যবহারকারী`) বা মুছে ফেলুন। পাসওয়ার্ড কখনো স্ক্রিপ্টে বা চ্যাটে লিখবেন না। Shared অ্যাকাউন্ট ব্যবহার করবেন না — কে কী করেছে, তা জানতে প্রত্যেকের আলাদা অ্যাকাউন্ট থাকা দরকার।

শিল্পের সেরা চর্চা (Best Practices)

- Least privilege: প্রত্যেককে শুধু সেইটুকু অনুমতি দিন, যা তার কাজের জন্য দরকার।
- গ্রুপ-ভিত্তিক অনুমতি ব্যবহার করুন; ব্যক্তিগত অনুমতি ছড়িয়ে দেবেন না।
- নতুন ফাইলের অনুমতি umask দিয়ে নিয়ন্ত্রণ করুন (সাধারণত 0022)।
- নিয়মিত `sudo awk -F: '$3 >= 1000 {print $1, $3}' /etc/passwd` দিয়ে সাধারণ ব্যবহারকারীদের তালিকা পরীক্ষা করুন।

- Permission পরিবর্তনের আগে `ls -l` দিয়ে বর্তমান অবস্থা লিখে রাখুন, যাতে ভুল হলে ফেরত নেওয়া যায়।

Knowledge-Check Questions — ৫টি MCQ (উত্তর দেওয়া হয়নি)

1. Ubuntu-তে root ব্যবহারকারীর UID কত? A) 1 B) 0 C) 1000 D) 999
2. `-rwxr-xr--` অনুমতিতে group কী করতে পারে? A) পড়া, লেখা, চালানো B) শুধু পড়া C) পড়া ও চালানো D) কিছুই না
3. `usermod -aG developers rafi`-এ `-a` ফ্ল্যাগের কাজ কী? A) অ্যাকাউন্ট মুছে ফেলা B) আগের গ্রুপ না মুছে যোগ করা C) পাসওয়ার্ড বদলানো D) home directory তৈরি করা
4. নতুন ফাইলের অনুমতি নির্ধারণ করে কোনটি? A) chmod B) chown C) umask D) passwd
5. একটি ডিরেক্টরিতে ঢুকতে কোন অনুমতি দরকার? A) শুধু read B) শুধু write C) execute D) কোনোটিই না

Practical Assignment

একটি কাল্পনিক দলের জন্য `webteam` গ্রুপ তৈরি করুন, তিনজন ব্যবহারকারী যোগ করুন, `/srv/website` ডিরেক্টরি তৈরি করে গ্রুপকে লেখার অনুমতি দিন, আর others-কে শুধু পড়ার অনুমতি দিন। প্রতিটি ধাপের কমান্ড ও `ls -ld` আউটপুট একটি ফাইলে লিখে রাখুন।

সারসংক্ষেপ

Linux-এ নিরাপত্তা শুরু হয় পরিচয় থেকে। UID/GID মানুষ ও গ্রুপকে চেনে, /etc/passwd ও /etc/shadow সেই পরিচয় সংরক্ষণ করে, আর নয়টি permission bit ঠিক করে কে কী করতে পারে। গ্রুপ ব্যবহার করলে অনুমতি পরিচালনা সহজ হয়, setgid shared কাজে সাহায্য করে, আর umask নতুন ফাইলের ডিফল্ট অনুমতি নিয়ন্ত্রণ করে। সবচেয়ে গুরুত্বপূর্ণ শিক্ষা: অনুমতি খুলে দেওয়া সমাধান নয় — সঠিক মালিকানা ও গ্রুপই সমাধান।

Revision Checklist

- ☐ UID, GID, root ও system user-এর পার্থক্য বলতে পারি
- ☐ /etc/passwd, /etc/shadow ও /etc/group-এর একটি লাইন ব্যাখ্যা করতে পারি
- ☐ useradd, usermod -aG, passwd ও userdel ব্যবহার করতে পারি
- ☐ rwx ও 755/640-এর মতো সংখ্যা অনুবাদ করতে পারি
- ☐ setgid ও sticky bit-এর ব্যবহার জানি এবং chmod 777 কেন বিপজ্জনক তা বলতে পারি

Chapter 12: File Ownership, ACLs and Sudo — ফাইল মালিকানা, ACL ও Sudo

ভূমিকা: যখন নয়টি বিট যথেষ্ট নয়

অধ্যায় ১১-এ আমরা শিখেছি owner, group ও others — এই তিন শ্রেণির অনুমতি। কিন্তু বাস্তব জীবনে প্রায়ই এমন হয়: একটি project directory-তে `developers` গ্রুপের সবাই লিখতে পারবে, কিন্তু `tasnim` নামের একজন পরীক্ষক শুধু পড়তে পারবে, যদিও সে developers গ্রুপে নেই। আবার একজন জুনিয়র অ্যাডমিনকে শুধু ওয়েব সার্ভার restart করার অনুমতি দিতে হবে, পুরো root ক্ষমতা নয়। তিন শ্রেণির মডেল দিয়ে এই সূক্ষ্ম নিয়ন্ত্রণ করা যায় না। এখানেই আসে Access Control Lists (ACL) ও sudo। ACL একটি ফাইলে নির্দিষ্ট ব্যক্তি বা গ্রুপের জন্য আলাদা নিয়ম লিখতে দেয়, আর sudo নির্দিষ্ট কমান্ডের জন্য সীমিত root ক্ষমতা দিতে দেয়। এই অধ্যায়ে আমরা শিখব কীভাবে মালিকানা উত্তরাধিকার কাজ করে, কখন ACL দরকার, আর কীভাবে sudo নিরাপদে সাজাতে হয় — যাতে সুবিধা বাড়ে, কিন্তু নিরাপত্তা কমে না।

Learning Objectives — শেখার লক্ষ্য

- ফাইল তৈরির সময় মালিকানা ও গ্রুপ কীভাবে নির্ধারিত হয় তা ব্যাখ্যা করতে পারবেন।
- `getfacl` ও `setfacl` দিয়ে নির্দিষ্ট ব্যবহারকারী ও গ্রুপের জন্য ACL সেট করতে পারবেন।
- Default ACL দিয়ে নতুন ফাইলের অনুমতি নিয়ন্ত্রণ করতে পারবেন।
- sudo কীভাবে কাজ করে এবং `/etc/sudoers` কেন সরাসরি সম্পাদনা করা উচিত নয় তা বুঝতে পারবেন।
- `visudo` দিয়ে নিরাপদ sudo নিয়ম লিখতে পারবেন।
- Least privilege নীতি অনুযায়ী shared directory ও sudo সাজাতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ১১-এর permission মডেল, UID/GID ও `chmod` / `chown` জানা থাকতে হবে। `ls -l` আউটপুট পড়তে পারা এবং টেক্সট এডিটরে ফাইল সম্পাদনার অভিজ্ঞতা দরকার।

প্রথম নীতি থেকে: মালিকানা কীভাবে তৈরি হয়?

Linux-এ একটি ফাইল তৈরি হলে তার মালিক হয় সেই ব্যবহারকারী, যে process-টি ফাইলটি তৈরি করেছে। গ্রুপ হয় তৈরি করা process-এর primary group — যদি না ডিরেক্টরিতে setgid বিট থাকে, তখন ডিরেক্টরির গ্রুপই নতুন ফাইলের গ্রুপ হয়। এটাই ownership inheritance। ধরুন, `/srv/project`-এ setgid চালু আছে এবং গ্রুপ `developers`। rafi সেখানে ফাইল তৈরি করলে ফাইলের মালিক rafi, কিন্তু গ্রুপ developers হবে — rafi-র নিজের primary group নয়। এতে দলের সবাই ফাইলটি গ্রুপ অনুমতি অনুযায়ী ব্যবহার করতে পারে। এই ছোট নিয়মটিই দলগত কাজের ভিত্তি। অন্যদিকে `cp` করলে নতুন ফাইল তৈরি হয়, তাই মালিকানা বদলে যায়; কিন্তু `mv` একই filesystem-এ করলে মালিকানা অপরিবর্তিত থাকে, কারণ ফাইলটি নতুন করে তৈরি হয় না, শুধু নাম বদলায়। এই পার্থক্য না জানলে backup বা deploy-এর সময় অনুমতি নষ্ট হয়ে যেতে পারে।

ACL: তিন শ্রেণির বাইরে সূক্ষ্ম নিয়ন্ত্রণ

Access Control List হলো একটি ফাইল বা ডিরেক্টরির সঙ্গে যুক্ত অতিরিক্ত নিয়মের তালিকা। সাধারণ permission বলে — owner কী পারে, group কী পারে, others কী পারে। ACL বলে — এই নির্দিষ্ট ব্যক্তি কী পারে, এই নির্দিষ্ট গ্রুপ কী পারে। উদাহরণ: `/srv/reports`-এর মালিক `accounts` গ্রুপ, কিন্তু auditor `tasnim` শুধু পড়তে পারবে। `setfacl -m u:tasnim:r-- /srv/reports` দিলে tasnim পড়ার অনুমতি পায়, অথচ সে accounts গ্রুপে যোগ হয় না। `ls -l`-এ ফাইলের অনুমতির শেষে `+` চিহ্ন দেখা গেলে বুঝবেন ACL আছে, যেমন `-rw-r--r--+`। ACL-এর একটি গুরুত্বপূর্ণ ধারণা হলো mask: এটি ACL-এর মাধ্যমে দেওয়া সর্বোচ্চ কার্যকর অনুমতি সীমাবদ্ধ করে। `getfacl` আউটপুটে `mask::rwx` দেখে বোঝা যায়। Default ACL ডিরেক্টরিতে সেট করলে তার ভেতরে তৈরি নতুন ফাইল ও সাবডিরেক্টরি সেই ACL উত্তরাধিকার সূত্রে পায়।

চিত্র ১২.১ — সাধারণ Permission বনাম ACL

ব্যবস্থা	কার জন্য নিয়ম লেখা যায়	উদাহরণ
সাধারণ permission	একজন owner, একটি group, others	640: owner rw, group r, others —
ACL	যেকোনো নির্দিষ্ট user বা group	u:tasnim:r--, g:auditors:r--
Default ACL	ডিরেক্টরির ভেতরের নতুন ফাইল	d:u:tasnim:r--

মনে রাখবেন: ACL সাধারণ permission-কে বাতিল করে না, তার ওপরে অতিরিক্ত স্তর যোগ করে। Kernel প্রথমে owner, তারপর named user ACL, তারপর group ACL, তারপর others — এই ক্রমে পরীক্ষা করে।

চিত্র ১২.২ — Sudo-এর ভেতরের প্রবাহ

ব্যবহারকারী `sudo apt update` চালায় → sudo `/etc/sudoers` ও `/etc/sudoers.d/` পড়ে → ব্যবহারকারী এই কমান্ডের অনুমতি পেয়েছে কি না যাচাই করে → পাসওয়ার্ড চায় (নিয়ম অনুযায়ী) → অনুমতি থাকলে root হিসেবে কমান্ড চালায় → ঘটনাটি log-এ লেখে (`/var/log/auth.log`) → কাজ শেষ হলে আবার সাধারণ ব্যবহারকারীতে ফিরে আসে।

পরিভাষা

Term	বাংলা ব্যাখ্যা
ACL	Access Control List; নির্দিষ্ট user/group-এর জন্য অতিরিক্ত অনুমতির তালিকা
Mask	ACL-এর মাধ্যমে দেওয়া অনুমতির সর্বোচ্চ কার্যকর সীমা
Default ACL	ডিরেক্টরির নতুন সন্তানদের জন্য উত্তরাধিকারসূত্রে পাওয়া ACL
sudo	Superuser Do; সীমিত root ক্ষমতায় কমান্ড চালানোর টুল
sudoers	কে sudo দিয়ে কী করতে পারে, তার নিয়মের ফাইল
visudo	sudoers নিরাপদে সম্পাদনার টুল; ভুল syntax আটকে দেয়

Least privilege	কাজের জন্য যতটুকু দরকার, ঠিক ততটুকু ক্ষমতা দেওয়ার নীতি
NOPASSWD	পাসওয়ার্ড ছাড়া sudo চালানোর নিয়ম — শুধু বিশেষ ক্ষেত্রে, সতর্কতার সঙ্গে

বাস্তব উদাহরণ: একটি অফিসের Shared Folder

সিলেটের একটি এজেন্সিতে `/srv/clients` ডিরেক্টরিতে ক্লায়েন্টের ফাইল থাকে। ডিজাইনাররা লিখতে পারে, হিসাব বিভাগ শুধু পড়তে পারে, আর বাইরের একজন নিরীক্ষক শুধু একটি সাবডিরেক্টরি পড়তে পারে। সাধারণ group দিয়ে এটি সাজানো কঠিন, কারণ নিরীক্ষককে পুরো গ্রুপে যোগ করলে সে সবকিছু দেখতে পেত। সমাধান: মূল ডিরেক্টরির গ্রুপ `designers`, ACL দিয়ে `g:accounts:r-x`, আর সাবডিরেক্টরিতে `u:auditor:r-x`। এতে প্রত্যেকের সীমানা স্পষ্ট থাকে, আর কাউকে অপ্রয়োজনীয় গ্রুপে যোগ করতে হয় না।

ব্যবহারিক কমান্ড

```
getfacl /srv/reports
# ফাইলের সাধারণ অনুমতি ও সব ACL নিয়ম দেখায়।

sudo setfacl -m u:tasnim:r-- /srv/reports
# tasnim-কে শুধু পড়ার অনুমতি দেয়।

sudo setfacl -m g:auditors:r-x /srv/reports
# auditors গ্রুপকে পড়া ও ভেতরে ঢোকার অনুমতি দেয়।

sudo setfacl -m d:u:tasnim:r-- /srv/project
# default ACL: /srv/project-এ তৈরি নতুন ফাইল tasnim পড়তে পারবে।

sudo setfacl -x u:tasnim /srv/reports
# একটি ACL নিয়ম মুছে ফেলে।

sudo setfacl -b /srv/reports
# সব ACL মুছে ফেলে, শুধু সাধারণ permission রাখে।

sudo visudo
# /etc/sudoers নিরাপদে সম্পাদনা করে; ভুল syntax থাকলে সংরক্ষণ করতে দেয় না।

sudo -l
# আমি sudo দিয়ে কী কী করতে পারি, তার তালিকা দেখায়।

sudo -u www-data whoami
# www-data ব্যবহারকারী হিসেবে একটি কমান্ড চালায় — www-data কে তা পরীক্ষা করতে কাজে লাগে।
```

WARNING: `/etc/sudoers` সরাসরি `nano` বা অন্য এডিটরে খুলে সম্পাদনা করবেন না। একটি ছোট syntax ভুল পুরো sudo অচল করে দিতে পারে, তখন আর কোনো প্রশাসনিক কাজ করা যাবে না। সবসময় `sudo visudo` ব্যবহার করুন, আর বড় নিয়ম `/etc/sudoers.d/`-এ আলাদা ফাইলে রাখুন। কোনো ব্যবহারকারীকে `ALL=(ALL) NOPASSWD: ALL` দেওয়া মানে তাকে কার্যত root বানিয়ে দেওয়া — শুধু বিশ্বস্ত প্রশাসকের জন্য, কখনো সাধারণ ডেভেলপার বা ইন্টার্নের জন্য নয়।

> **CONCEPT:** sudo root পাসওয়ার্ড চায় না — এটি ব্যবহারকারীর নিজের পাসওয়ার্ড চায়। এর অর্থ, root পাসওয়ার্ড কাউকে না জানিয়ে প্রত্যেককে তার নিজের পাসওয়ার্ড দিয়ে সীমিত ক্ষমতা দেওয়া যায়। আর প্রতিটি sudo ব্যবহার log হয়, তাই পরে কে কী করেছে, তা যাচাই করা যায়।

হাতে-কলমে অনুশীলন: ACL ও সীমিত Sudo নিয়ম

লক্ষ্য: `/srv/shared`-এ developers লিখতে পারবে, tasnim শুধু পড়তে পারবে, আর `junior` নামের ব্যবহারকারী শুধু `systemctl restart nginx` চালাতে পারবে।

1. `sudo mkdir -p /srv/shared` ও `sudo chown root:developers /srv/shared` চালান।
2. `sudo chmod 2770 /srv/shared` দিয়ে setgid চালু করুন।
3. `sudo setfacl -m u:tasnim:r-x /srv/shared` ও `sudo setfacl -m d:u:tasnim:r-- /srv/shared` চালান।
4. `getfacl /srv/shared` দিয়ে যাচাই করুন; `ls -ld /srv/shared`-এ `+` চিহ্ন দেখুন।
5. `sudo visudo -f /etc/sudoers.d/junior-nginx` খুলে এই লাইনটি যোগ করুন: `junior ALL=(root) /usr/bin/systemctl restart nginx`। সংরক্ষণের আগে visudo syntax যাচাই করবে।
6. `sudo -l -U junior` দিয়ে যাচাই করুন যে junior শুধু এই কমান্ডটিই পায়।

উদাহরণ আউটপুট (illustrative):

```
$ getfacl /srv/shared
# file: /srv/shared
# owner: root
# group: developers
user::rwx
user:tasnim:r-x
group::rwx
mask::rwx
other::---
default:user:tasnim:r--

$ sudo -l -U junior
User junior may run the following commands on ubuntu-server:
    (root) /usr/bin/systemctl restart nginx
```

লক্ষ্য করুন, junior অন্য কোনো systemctl কমান্ড — যেমন stop বা status — চালাতে পারবে না, শুধু restart nginx পারবে। এটাই least privilege-এর বাস্তব রূপ।

> **PRO TIP:** sudo নিয়মে সবসময় কমান্ডের সম্পূর্ণ পথ লিখুন (`/usr/bin/systemctl`, শুধু `systemctl` নয়)। আপেক্ষিক নাম ব্যবহার করলে PATH বদলে ভিন্ন প্রোগ্রাম চালানোর ঝুঁকি থাকে। `which systemctl` দিয়ে সঠিক পথ জেনে নিন।

একটি বিষয় প্রায়ই ভুল বোঝা হয়: ACL থাকলেই সাধারণ permission অপ্রাসঙ্গিক হয়ে যায় না। ধরুন, একটি ফাইলের others অনুমতি `r--` কিন্তু একজন নির্দিষ্ট ব্যবহারকারীর ACL `---`। সেই ব্যবহারকারী others শ্রেণিতে পড়লেও named user ACL আগে পরীক্ষা করা হয়, তাই সে ফাইলটি পড়তে পারবে না। আবার mask যদি `r--` হয়, তাহলে আপনি ACL-এ

`rwx` দিলেও কার্যকর অনুমতি শুধু পড়া পর্যন্ত সীমাবদ্ধ থাকবে। তাই `getfacl` আউটপুটে সবসময় তিনটি জিনিস একসঙ্গে দেখুন: user ও group নিয়ম, mask, এবং others। নতুনদের জন্য পরামর্শ: প্রথমে ছোট একটি পরীক্ষামূলক ডিরেক্টরিতে ACL চর্চা করুন, তারপর production ডিরেক্টরিতে প্রয়োগ করুন। আর sudo নিয়ম লেখার সময় মনে রাখবেন, একটি নিয়মে `/usr/bin/vim` দেওয়া মানে ব্যবহারকারী vim-এর ভেতর থেকে `:shell` চালিয়ে root পেতে পারে — তাই এডিটর বা shell-এর sudo অনুমতি কখনো দেবেন না।

সাধারণ ভুল ও সমস্যা সমাধান

- **ভুল:** ACL দিয়েও ফাইল খুলতে না পারা — কারণ parent ডিরেক্টরিতে execute অনুমতি নেই। **সমাধান:** পুরো পথের প্রতিটি ডিরেক্টরিতে `x` আছে কি না পরীক্ষা করুন।
- **ভুল:** `setfacl`-এর পর `ls -l`-এ অনুমতি বদলে গেছে মনে হওয়া। আসলে group অনুমতির জায়গায় mask দেখায়। **সমাধান:** আসল অবস্থা জানতে `getfacl` ব্যবহার করুন।
- **ভুল:** visudo ছাড়া sudoers সম্পাদনা করে sudo অচল করে ফেলা। **সমাধান:** Ubuntu-তে recovery mode বা live USB থেকে ফাইলটি ঠিক করতে হয় — তাই আগেই সতর্ক থাকুন।
- **ভুল:** `sudo` দিয়ে তৈরি ফাইলের মালিক root হয়ে যাওয়া, ফলে সাধারণ ব্যবহারকারী আর লিখতে পারে না। **সমাধান:** `sudo chown ব্যবহারকারী:গ্রুপ ফাইল` দিয়ে মালিকানা ফিরিয়ে দিন।

নিরাপত্তা বিবেচনা

ACL যত সূক্ষ্ম, তত বেশি নিয়ম — আর তত বেশি ভুলের সুযোগ। নিয়মিত `getfacl` দিয়ে গুরুত্বপূর্ণ ডিরেক্টরি পরীক্ষা করুন। sudo নিয়মে কখনো shell (`/bin/bash`) বা এডিটর (`/usr/bin/vim`)-এর অনুমতি দেবেন না, কারণ সেখান থেকে root shell পাওয়া যায়। `NOPASSWD` শুধু automated service অ্যাকাউন্টের নির্দিষ্ট কমান্ডের জন্য ব্যবহার করুন, মানুষের অ্যাকাউন্টে নয়। sudo log নিয়মিত দেখুন: `sudo grep sudo /var/log/auth.log | tail -20`।

শিল্পের সেরা চর্চা (Best Practices)

- প্রথমে group দিয়ে সমস্যা সমাধানের চেষ্টা করুন; group দিয়ে না হলে তবেই ACL ব্যবহার করুন।
- ACL নিয়মের কারণ একটি README বা মন্তব্যে লিখে রাখুন, যাতে ছয় মাস পরেও বোঝা যায়।
- sudo নিয়ম `/etc/sudoers.d/`-এ ছোট ছোট ফাইলে ভাগ করে রাখুন, যাতে পরিবর্তন পর্যালোচনা সহজ হয়।
- নতুন sudo নিয়ম দেওয়ার পর `sudo -l -U ব্যবহারকারী` দিয়ে যাচাই করুন।
- Shared ডিরেক্টরিতে setgid + default ACL একসঙ্গে ব্যবহার করুন, যাতে নতুন ফাইলও সঠিক গ্রুপ ও অনুমতি পায়।

Knowledge-Check Questions — ৫টি MCQ (উত্তর দেওয়া হয়নি)

1. `ls -l`-এ `-rw-r--r--+`-এর শেষে `+` চিহ্নের অর্থ কী? A) ফাইলটি executable B) ACL আছে C) ফাইলটি লুকানো D) sticky bit চালু
2. Default ACL-এর কাজ কী? A) পুরোনো ফাইল মুছে ফেলা B) নতুন ফাইলে ACL উত্তরাধিকার দেওয়া C) পাসওয়ার্ড বদলানো D) গ্রুপ মুছে ফেলা
3. `/etc/sudoers` সম্পাদনার নিরাপদ উপায় কোনটি? A) সরাসরি nano B) visudo C) echo দিয়ে লেখা D) chmod 777 করে সম্পাদনা
4. sudo সাধারণত কার পাসওয়ার্ড চায়? A) root-এর B) ব্যবহারকারীর নিজের C) কোনো পাসওয়ার্ড চায় না D) গ্রুপের

5. `setfacl -m u:tasnim:r-- file` কী করে? A) tasnim-কে ফাইলের মালিক বানায় B) tasnim-কে শুধু পড়ার অনুমতি দেয় C) tasnim-এর অ্যাকাউন্ট মুছে ফেলে D) ফাইলটি মুছে ফেলে

Practical Assignment

`/srv/audit` নামে একটি ডিরেক্টরি তৈরি করুন। `finance` গ্রুপ লিখতে পারবে, `auditor` নামের একজন ব্যবহারকারী (কোনো গ্রুপে না থাকা) শুধু পড়তে পারবে, others কিছুই করতে পারবে না। Default ACL এমনভাবে সাজান যাতে নতুন ফাইলেও auditor পড়তে পারে। `getfacl` আউটপুট সংরক্ষণ করুন।

সারসংক্ষেপ

সাধারণ permission তিন শ্রেণির জন্য যথেষ্ট, কিন্তু বাস্তব দলগত কাজে প্রায়ই আরও সূক্ষ্ম নিয়ন্ত্রণ দরকার হয়। ACL সেই সূক্ষ্মতা দেয় — নির্দিষ্ট ব্যক্তি বা গ্রুপের জন্য আলাদা নিয়ম, আর default ACL নতুন ফাইলে সেই নিয়ম বয়ে নিয়ে যায়। sudo একই নীতি ক্ষমতার ক্ষেত্রে প্রয়োগ করে: পুরো root নয়, শুধু নির্দিষ্ট কমান্ড, শুধু নির্দিষ্ট ব্যক্তি, আর প্রতিটি ব্যবহার log-এ লেখা থাকে। মালিকানা, ACL ও sudo — তিনটি মিলিয়ে একটি সার্ভারে সীমানা স্পষ্ট থাকে, আর কাজও থেমে থাকে না।

Revision Checklist

- ☐ ফাইল তৈরির সময় মালিকানা ও গ্রুপ কীভাবে নির্ধারিত হয় তা বলতে পারি
- ☐ getfacl ও setfacl দিয়ে ACL পড়তে ও সেট করতে পারি
- ☐ mask ও default ACL-এর অর্থ বুঝি
- ☐ visudo দিয়ে নিরাপদ sudo নিয়ম লিখতে পারি
- ☐ least privilege অনুযায়ী sudo নিয়ম যাচাই করতে পারি (sudo -l)

Chapter 13: Process and Job Management — প্রসেস ও জব ব্যবস্থাপনা

ভূমিকা: সার্ভার ধীর কেন? উত্তর লুকিয়ে আছে প্রসেসে

একদিন সকালে আপনার Ubuntu সার্ভারে ওয়েবসাইট খুলতে দেরি হচ্ছে। কেউ বলছে ইন্টারনেট ধীর, কেউ বলছে সার্ভার নষ্ট। কিন্তু আসল কারণ জানতে হলে দেখতে হবে — এই মুহূর্তে সার্ভারে কী কী process চলছে, কোনটি CPU খাচ্ছে, কোনটি মেমরি ভরে ফেলেছে, আর কোনটি আটকে আছে। Process হলো চলমান প্রোগ্রাম। আপনি যখন `ls` চালান, তখন একটি process তৈরি হয়, কাজ শেষ হলে মারা যায়। ওয়েব সার্ভার, ডেটাবেস, SSH — সবই process। এই অধ্যায়ে আমরা শিখব কীভাবে process দেখতে হয়, কীভাবে থামাতে হয়, কীভাবে background-এ চালাতে হয়, আর কীভাবে একটি বিপজ্জনক process-কে নিরাপদে নিয়ন্ত্রণ করতে হয় — যাতে ভুল process মেরে সার্ভার বন্ধ করে না ফেলেন।

Learning Objectives — শেখার লক্ষ্য

- Program ও process-এর পার্থক্য এবং process lifecycle ব্যাখ্যা করতে পারবেন।
- `ps`, `top` ও `htop` দিয়ে চলমান process পরীক্ষা করতে পারবেন।
- Signal-এর ধারণা বুঝে `kill`, `pkill` ও `pgrep` নিরাপদে ব্যবহার করতে পারবেন।
- Foreground ও background job, `jobs`, `bg`, `fg` ও `nohup` ব্যবহার করতে পারবেন।
- `nice` ও `renice` দিয়ে process priority নিয়ন্ত্রণ করতে পারবেন।
- Process tree দেখে একটি সমস্যার মূল process খুঁজে বের করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৪–৫ (Kernel, Process ও Memory Management) ও অধ্যায় ৯ (Terminal ও Shell) জানা থাকতে হবে। PID, parent process ও signal সম্পর্কে প্রাথমিক ধারণা এই অধ্যায়ে আবার সংক্ষেপে ব্যাখ্যা করা হয়েছে।

প্রথম নীতি থেকে: Program বনাম Process

Program হলো ডিস্কে থাকা নির্দেশের স্থির তালিকা — যেমন `/usr/bin/python3` ফাইলটি। Process হলো সেই program যখন CPU-তে চলছে — তার নিজস্ব মেমরি, নিজস্ব PID (Process ID), নিজস্ব অবস্থা নিয়ে। একই program থেকে অনেক process তৈরি হতে পারে: পাঁচজন ব্যবহারকারী একই ওয়েব সার্ভার চালালে পাঁচটি process হয়। প্রতিটি process-এর একটি parent থাকে; সব process-এর আদি parent হলো PID 1, Ubuntu-তে systemd। Process জন্ম নেয় fork() ও exec() system call-এর মাধ্যমে: প্রথমে parent নিজের একটি কপি তৈরি করে (fork), তারপর সেই কপি নতুন program load করে (exec)। Process মারা গেলে parent-কে জানাতে হয়; parent যদি সেই সংবাদ না নেয়, process zombie হয়ে পড়ে থাকে — মৃত, কিন্তু তালিকা থেকে মুছে যায়নি। আর parent যদি সন্তানের আগেই মারা যায়, সন্তান orphan হয়, তখন systemd তাকে দত্তক নেয়।

Scheduling ও Context Switching

CPU একসঙ্গে একটি কাজই করে (প্রতিটি core-এ একটি), কিন্তু শত শত process চলছে মনে হয় — কারণ scheduler খুব দ্রুত process বদলায়। এই বদলকে context switching বলে: CPU একটি process-এর registers ও অবস্থা সংরক্ষণ করে,

আরেকটি process-এর অবস্থা load করে। প্রতিটি process-কে একটি ছোট সময়ের টুকরো (timeslice) দেওয়া হয়। Priority নির্ধারণ করে কে আগে সময় পাবে। Linux-এ nice value -20 (সবচেয়ে বেশি অগ্রাধিকার) থেকে 19 (সবচেয়ে কম) পর্যন্ত হয়। সাধারণ ব্যবহারকারী শুধু nice value বাড়াতে পারে (অগ্রাধিকার কমাতে পারে), কমাতে পারে শুধু root। এটি ইচ্ছাকৃত: কেউ যেন নিজের process-কে সবার ওপরে তুলে সিস্টেম দখল করতে না পারে।

চিত্র ১৩.১ — Process Lifecycle

New (তৈরি হচ্ছে) → Ready (CPU-এর অপেক্ষায়) → Running (CPU-তে চলছে) → Waiting (I/O বা event-এর অপেক্ষায়) → Running-এ ফেরত → Terminated (শেষ)। একটি process বারবার Ready ও Running-এর মধ্যে ঘোরে, যতক্ষণ না সে শেষ হয় বা থামানো হয়। Zombie হলো Terminated-এর পরে parent-এর স্বীকৃতির অপেক্ষায় থাকা অবস্থা।

অবস্থা (ps-এ চিহ্ন)	অর্থ
R (running)	CPU-তে চলছে বা চলার জন্য প্রস্তুত
S (sleeping)	কোনো event বা I/O-এর অপেক্ষায় ঘুমিয়ে আছে — স্বাভাবিক
D (uninterruptible sleep)	ডিস্ক I/O-এর অপেক্ষায়; এই অবস্থায় kill করলেও মরে না
Z (zombie)	মৃত, কিন্তু parent এখনো status নেয়নি
T (stopped)	থামানো হয়েছে (যেমন Ctrl+Z)

চিত্র ১৩.২ — Process Tree (pstree-এর ধারণা)

systemd (PID 1) → sshd → bash → python3 app.py; systemd → nginx → nginx worker process-গুলো; systemd → cron → backup script। একটি service-এর সমস্যা হলে শুধু worker নয়, মূল parent process-টি খুঁজে বের করা দরকার। `pstree -p` বা `ps --forest` এই গাছটি দেখায়।

পরিভাষা

Term	বাংলা ব্যাখ্যা
PID / PPID	Process ID ও Parent Process ID; প্রতিটি process-এর পরিচয়
Signal	Kernel বা অন্য process-এর পাঠানো বার্তা; যেমন TERM, KILL, HUP
Foreground / Background	Terminal দখল করে চলা কাজ বনাম পেছনে চলা কাজ
Job	Shell-এর দৃষ্টিতে একটি background বা suspended কাজ
Nice value	Process-এর অগ্রাধিকার; -20 সর্বোচ্চ, 19 সর্বনিম্ন
Zombie / Orphan	মৃত কিন্তু তালিকায় থাকা process / parent-হারা process
Load average	CPU-এর জন্য অপেক্ষমাণ process-এর গড় সংখ্যা
	পেছনে চলা service process, যেমন sshd বা nginx

Daemon	

বাস্তব উদাহরণ: রাতের Backup সার্ভারকে ধীর করে দিল

খুলনার একটি ছোট কোম্পানিতে রাত ২টায় backup script চলত, কিন্তু সকালে ওয়েবসাইট ধীর থাকত। পরীক্ষা করে দেখা গেল, backup-এর `tar` process CPU-এর বড় অংশ দখল করে রাখছে। সমাধান ছিল backup-কে কম অগ্রাধিকারে চালানো: `nice -n 10 tar -czf backup.tar.gz /srv/data`। এতে backup ধীরে চলে, কিন্তু ওয়েবসাইটের কাজে বাধা দেয় না। আবার একটি ভুল লুপে আটকে যাওয়া Python script CPU 100% খাচ্ছিল — `top` দিয়ে PID বের করে `kill PID` দিয়ে থামানো হলো। প্রথমে `kill -9` নয়, সাধারণ `kill` (SIGTERM) — যাতে process নিজে পরিষ্কার হয়ে বন্ধ হতে পারে।

ব্যবহারিক কমান্ড

```
ps aux
# সব ব্যবহারকারীর সব process দেখায়: USER, PID, %CPU, %MEM, COMMAND।

ps aux --sort=-%cpu | head -10
# CPU সবচেয়ে বেশি খাওয়া ১০টি process দেখায়।

top
# লাইভ আপডেট হওয়া process তালিকা; q চেপে বের হবেন।
# top-এর ভেতরে M চাপলে মেমরি অনুযায়ী, P চাপলে CPU অনুযায়ী সাজে।

htop
# top-এর রঙিন ও সহজ সংস্করণ; Ubuntu 24.04-এ আলাদাভাবে ইনস্টল করতে হয় (sudo apt install htop)।
# F9 দিয়ে signal পাঠানো যায়, তীরচিহ্ন দিয়ে process বেছে নেওয়া যায়।

pgrep -a python3
# python3 নামের process-গুলোর PID ও কমান্ড দেখায়।

pstree -p
# process tree দেখায়; কোন process কার সন্তান, তা বোঝা যায়।

kill 1234
# PID 1234-কে SIGTERM (15) পাঠায় — ভদ্রভাবে বন্ধ হতে বলে।

kill -9 1234
# SIGKILL (9): জোর করে মেরে ফেলে, পরিষ্কার হওয়ার সুযোগ দেয় না। শেষ উপায় হিসেবে ব্যবহার করুন।

pkill -f "python3 app.py"
# নাম বা কমান্ড লাইন মিলিয়ে process মারে; -f ছাড়া শুধু নাম মেলে। সাবধান: ভুল pattern সব মিলিয়ে ফেলতে পারে।

sleep 300 &
# sleep কমান্ড background-এ চলে; shell সঙ্গে সঙ্গে prompt ফেরত দেয়।

jobs
# বর্তমান shell-এর background ও suspended job-গুলো দেখায়।

fg %1
# ১ নম্বর job-কে foreground-এ আনে।

bg %1
# থামানো job-কে background-এ চালায়।
```

```
nohup python3 app.py > app.log 2>&1 &
# terminal বন্ধ করলেও process চলতে থাকে; আউটপুট app.log-এ যায়।

nice -n 10 tar -czf backup.tar.gz /srv/data
# কম অগ্রাধিকারে backup চালায়।

sudo renice -n 5 -p 1234
# চলমান process 1234-এর nice value বদলায়; কমাতে root দরকার।
```

WARNING: `kill -9` প্রথমে ব্যবহার করবেন না। এটি process-কে ফাইল সংরক্ষণ বা সংযোগ বন্ধ করার সুযোগ দেয় না — ডেটাবেস বা ফাইল নষ্ট হতে পারে। প্রথমে সাধারণ `kill` (SIGTERM) দিন, কয়েক সেকেন্ড অপেক্ষা করুন, তারপরও না মরলে `kill -9` বিবেচনা করুন। `pkill` ব্যবহারের আগে `pgrep -a` দিয়ে দেখুন কোন কোন PID মিলছে — ভুল pattern-এ গুরুত্বপূর্ণ service মারা যেতে পারে। PID 1 (systemd) কখনো kill করবেন না।

CONCEPT: Signal হলো process-কে পাঠানো একটি সংক্ষিপ্ত বার্তা। SIGTERM (15) মানে "দয়া করে বন্ধ হও", SIGKILL (9) মানে "এখনই মরো", SIGHUP (1) মানে "terminal বন্ধ হয়েছে — অনেক daemon এতে configuration আবার পড়ে", SIGSTOP (19) মানে "থামো", SIGCONT (18) মানে "আবার চলো"। Ctrl+C আসলে SIGINT (2) পাঠায়, Ctrl+Z পাঠায় SIGTSTP (20)।

হাতে-কলমে অনুশীলন: Background Job ও CPU-খেকো Process থামানো

লক্ষ্য: একটি দীর্ঘ কাজ background-এ চালানো, তারপর একটি ইচ্ছাকৃত CPU-খেকো process খুঁজে থামানো।

1. `sleep 600 &` চালান, তারপর `jobs` দিয়ে দেখুন।
2. `fg %1` দিয়ে foreground-এ আনুন, তারপর Ctrl+Z চেপে থামান। `bg %1` দিয়ে আবার background-এ চালান।
3. একটি পরীক্ষামূলক CPU লোড তৈরি করুন: `yes > /dev/null &` — এটি একটি CPU core পুরো ব্যবহার করবে। PID-টি লিখে রাখুন (`echo $!`)।
4. `top` বা `ps aux --sort=-%cpu | head -5` দিয়ে দেখুন `yes` সবার ওপরে আছে।
5. `kill PID` দিয়ে থামান, তারপর `ps -p PID` দিয়ে যাচাই করুন যে process নেই।
6. `sleep` job-টিও `kill %1` দিয়ে থামান।

উদাহরণ আউটপুট (illustrative):

```
$ ps aux --sort=-%cpu | head -3
USER   PID %CPU %MEM COMMAND
nahid 4521 98.5  0.0 yes
nahid 4510  0.0  0.0 sleep 600

$ kill 4521
$ ps -p 4521
PID TTY TIME CMD
# কোনো লাইন নেই — process শেষ হয়েছে।
```

পরীক্ষা শেষে অবশ্যই `yes` process-টি মেরে ফেলুন, নাহলে সেটি CPU খেতে থাকবে। Lab-এর শেষে `jobs` দিয়ে দেখুন কোনো job পড়ে আছে কি না।

একটি প্রায়-অদেখা বিষয় হলো process-এর environment ও working directory। একটি process যখন fork হয়, তখন সে parent-এর environment variable ও বর্তমান ডিরেক্টরি উত্তরাধিকার সূত্রে পায়। তাই cron বা systemd থেকে চালানো স্ক্রিপ্টে `python3 app.py` আপেক্ষিক পথে কাজ নাও করতে পারে, কারণ working directory ভিন্ন হতে পারে। সবসময় স্ক্রিপ্টে সম্পূর্ণ পথ ব্যবহার করুন, অথবা স্ক্রিপ্টের শুরুতে `cd /srv/myapp` লিখুন। আরেকটি বিষয়: `ps aux`-এর `TIME` কলাম দেখায় process মোট কত CPU সময় ব্যবহার করেছে, কতক্ষণ ধরে চলছে তা নয়। একটি process এক ঘণ্টা ধরে চললেও TIME কম হতে পারে, যদি সে বেশিরভাগ সময় ঘুমিয়ে থাকে।

সাধারণ ভুল ও সমস্যা সমাধান

- **ভুল:** `kill` করেও process মরছে না। **কারণ:** process D অবস্থায় (ডিস্ক I/O-এর অপেক্ষায়) থাকতে পারে, অথবা PID ভুল। **সমাধান:** `ps -o pid,stat,cmd -p PID` দিয়ে অবস্থা দেখুন; D অবস্থায় অপেক্ষা করা ছাড়া উপায় নেই।
- **ভুল:** Terminal বন্ধ করলে background job মরে যাওয়া। **সমাধান:** `nohup` বা `disown` ব্যবহার করুন, অথবা দীর্ঘ কাজের জন্য systemd service বা `tmux` ব্যবহার করুন।
- **ভুল:** `killall python3` চালিয়ে সব Python process মেরে ফেলা। **সমাধান:** আগে `pgrep -a python3` দিয়ে তালিকা দেখুন, তারপর নির্দিষ্ট PID-তে `kill` করুন।
- **ভুল:** Zombie process দেখে আতঙ্কিত হওয়া। **সমাধান:** Zombie নিজে ক্ষতি করে না; parent process-কে ঠিক করতে হয়। Parent-কে restart করলে zombie মুছে যায়।

TROUBLESHOOTING: সার্ভার ধীর হলে এই ক্রমে পরীক্ষা করুন: `uptime` (load average) → `top` (কে CPU খাচ্ছে) → `free -h` (মেমরি) → `ps aux --sort=-%mem` (কে মেমরি খাচ্ছে) → `journalctl -xe` (কোনো service বারবার restart হচ্ছে কি না)। প্রথমে কারণ খুঁজুন, তারপর process মারুন — উল্টোটা নয়।

নিরাপত্তা বিবেচনা

অন্য ব্যবহারকারীর process আপনি kill করতে পারবেন না — শুধু নিজের process, যদি না আপনি root হন। এটি নিরাপত্তার অংশ। অপরিচিত process দেখলে সঙ্গে সঙ্গে kill করবেন না; `ps -ef` ও `cat /proc/PID/cmdline` দিয়ে সেটি কী, তা আগে বুঝুন। কোনো process যদি সন্দেহজনকভাবে নেটওয়ার্ক ব্যবহার করে, তাহলে `ss -tulpn` দিয়ে তার সংযোগ পরীক্ষা করুন। Production সার্ভারে পরীক্ষামূলক `yes` বা stress টুল চালাবেন না।

শিল্পের সেরা চর্চা (Best Practices)

- দীর্ঘস্থায়ী কাজের জন্য background job নয়, systemd service বা cron ব্যবহার করুন।
- প্রথমে SIGTERM, তারপর প্রয়োজনে SIGKILL — এই ক্রম মেনে চলুন।
- Process মারার আগে PID ও কমান্ড দুবার যাচাই করুন।
- নিয়মিত `top` বা `htop` দিয়ে সার্ভারের স্বাভাবিক অবস্থা চিনে রাখুন, যাতে অস্বাভাবিক অবস্থা চোখে পড়ে।
- Resource-heavy কাজ `nice` দিয়ে কম অগ্রাধিকারে চালান।

Knowledge-Check Questions — ৫টি MCQ (উত্তর দেওয়া হয়নি)

1. Ubuntu-তে PID 1 কোন process? A) bash B) systemd C) sshd D) cron
2. সাধারণ `kill PID` কোন signal পাঠায়? A) SIGKILL (9) B) SIGTERM (15) C) SIGHUP (1) D) SIGINT (2)
3. Zombie process কী? A) CPU 100% খাওয়া process B) মৃত কিন্তু parent-এর স্বীকৃতির অপেক্ষায় থাকা process C) ঘুমন্ত process D) root-এর process
4. Terminal বন্ধ করলেও process চালু রাখতে কোনটি ব্যবহার করবেন? A) fg B) nohup C) kill D) nice
5. nice value -20 মানে কী? A) সবচেয়ে কম অগ্রাধিকার B) সবচেয়ে বেশি অগ্রাধিকার C) process থামানো D) process মুছে ফেলা

Practical Assignment

একটি Python স্ক্রিপ্ট লিখুন যা 60 সেকেন্ড ধরে চলে এবং প্রতি 10 সেকেন্ডে একটি লাইন log ফাইলে লেখে। সেটিকে `nohup` দিয়ে background-এ চালান, terminal বন্ধ করে আবার login করে দেখুন সেটি এখনো চলছে কি না। তারপর `ps` দিয়ে PID বের করে `kill` দিয়ে থামান এবং log ফাইল পরীক্ষা করুন।

সারসংক্ষেপ

Program স্থির, process চলমান। প্রতিটি process-এর PID, parent ও অবস্থা থাকে, আর scheduler তাদের মধ্যে CPU ভাগ করে দেয়। `ps`, `top` ও `htop` দিয়ে process দেখা যায়, signal দিয়ে নিয়ন্ত্রণ করা যায়, আর `jobs`, `bg`, `fg` ও `nohup` দিয়ে shell job পরিচালনা করা যায়। ভালো অ্যাডমিন প্রথমে কারণ খুঁজে বের করে, তারপর সঠিক process-কে সঠিক signal দেয় — আন্দাজে `kill -9` চালায় না।

Revision Checklist

- ☐ Program ও process-এর পার্থক্য এবং lifecycle বলতে পারি
- ☐ ps, top, htop ও pstree দিয়ে process পরীক্ষা করতে পারি
- ☐ SIGTERM, SIGKILL, SIGHUP ও SIGINT-এর পার্থক্য জানি
- ☐ jobs, bg, fg ও nohup ব্যবহার করতে পারি
- ☐ nice ও renice দিয়ে priority নিয়ন্ত্রণ করতে পারি

Chapter 14: Package Management and Software Installation — প্যাকেজ ব্যবস্থাপনা ও সফটওয়্যার ইনস্টলেশন

ভূমিকা: সফটওয়্যার ইনস্টল করা কেন এত সহজ — আবার কখনো এত কঠিন?

আপনার ফোনে অ্যাপ ইনস্টল করা এক ক্লিকের কাজ। Ubuntu সার্ভারেও `sudo apt install nginx` এক লাইনে একটি পূর্ণ ওয়েব সার্ভার ইনস্টল করে দেয়। কিন্তু এর পেছনে একটি বিশাল ব্যবস্থা কাজ করে: repository, dependency, version, signature যাচাই। আবার কখনো `apt` বলে — dependency পূরণ হচ্ছে না, অথবা package-টি পাওয়া যাচ্ছে না। তখন কী করবেন? ইন্টারনেট থেকে একটি `.deb` ফাইল নামিয়ে ইনস্টল করা কি নিরাপদ? এই অধ্যায়ে আমরা শিখব প্যাকেজ ব্যবস্থা কীভাবে কাজ করে, কেন dependency দরকার, কীভাবে নিরাপদে সফটওয়্যার ইনস্টল, আপডেট ও মুছে ফেলতে হয়, আর কীভাবে ভাঙা প্যাকেজ ঠিক করতে হয় — যাতে আপনি সার্ভারে যা ইনস্টল করছেন, তার উৎস ও ঝুঁকি সম্পর্কে নিশ্চিত থাকতে পারেন।

Learning Objectives — শেখার লক্ষ্য

- Package, repository ও dependency-র ধারণা ব্যাখ্যা করতে পারবেন।
- `apt` ও `dpkg`-এর পার্থক্য ও ব্যবহার জানতে পারবেন।
- সফটওয়্যার ইনস্টল, আপডেট, আপগ্রেড ও নিরাপদে মুছে ফেলতে পারবেন।
- Repository যোগ করা ও যাচাই করার নিরাপদ পদ্ধতি বুঝতে পারবেন।
- Package-এর তথ্য, ফাইল তালিকা ও version পরীক্ষা করতে পারবেন।
- সাধারণ package ব্যর্থতা নির্ণয় ও মেরামত করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৯–১০ (Terminal ও Essential Commands) ও অধ্যায় ১৩ (Process Management) জানা থাকতে হবে। `sudo` ব্যবহারের অভিজ্ঞতা এবং ফাইল সম্পাদনার ধারণা দরকার।

প্রথম নীতি থেকে: Package কেন দরকার?

একটি প্রোগ্রাম একা চলে না। nginx চালাতে OpenSSL দরকার, Python চালাতে অনেক library দরকার। যদি প্রতিটি সফটওয়্যার নিজের সঙ্গে সবকিছু নিয়ে আসত, তাহলে ডিস্ক ভরে যেত, আর একটি library-তে নিরাপত্তা ত্রুটি থাকলে প্রতিটি প্রোগ্রাম আলাদা করে ঠিক করতে হতো। Package ব্যবস্থা এই সমস্যার সমাধান: প্রতিটি সফটওয়্যার একটি package — যাতে program ফাইল, configuration ফাইল, documentation ও metadata (নাম, version, dependency-র তালিকা) থাকে। Repository হলো package-এর ভান্ডার — Ubuntu-র অফিসিয়াল সার্ভার, যেখানে হাজার হাজার পরীক্ষিত package থাকে। `apt` repository থেকে package নামায়, dependency-র তালিকা দেখে প্রয়োজনীয় package-গুলোও নামায়, সঠিক ক্রমে ইনস্টল করে, আর ডিজিটাল signature যাচাই করে নিশ্চিত হয় যে package-টি আসল Ubuntu থেকে এসেছে, মাঝপথে বদলায়নি।

APT বনাম dpkg: দুই স্তরের ব্যবস্থা

নিচের স্তরে আছে `dpkg` — এটি একটি `.deb` ফাইল ইনস্টল করে, ফাইলের তালিকা রাখে, কিন্তু dependency নিজে থেকে সমাধান করে না। ওপরের স্তরে আছে `apt` (Advanced Package Tool) — এটি repository-র সঙ্গে কথা বলে, dependency সমাধান করে, তারপর dpkg-কে দিয়ে ইনস্টল করায়। তাই সাধারণ কাজে `apt` ব্যবহার করুন; `dpkg` শুধু তখনই ব্যবহার করুন যখন একটি নির্দিষ্ট `.deb` ফাইল হাতে আছে এবং আপনি জানেন সেটি কোথা থেকে এসেছে। `apt update` repository-র তালিকা আপডেট করে (কোনো সফটওয়্যার ইনস্টল করে না), আর `apt upgrade` ইনস্টল করা package-গুলো নতুন version-এ নিয়ে যায়। দুটির পার্থক্য না জানলে অনেকেই ভাবে `apt update` মানেই আপডেট হয়ে গেছে — আসলে তা শুধু তালিকা refresh করে।

চিত্র ১৪.১ — apt install-এর প্রবাহ

ব্যবহারকারী `apt install nginx` চালায় → apt `/etc/apt/sources.list` ও `/etc/apt/sources.list.d/`-এর repository তালিকা দেখে → package ও তার dependency-র তালিকা বের করে → GPG signature যাচাই করে → .deb ফাইল ডাউনলোড করে → dpkg দিয়ে সঠিক ক্রমে ইনস্টল করে → configuration ফাইল সাজায় → প্রয়োজনে service চালু করে।

কমান্ড	কাজ
apt update	Repository-র package তালিকা refresh করে
apt upgrade	ইনস্টল করা package আপগ্রেড করে
apt install নাম	Package ও dependency ইনস্টল করে
apt remove নাম	Package মুছে ফেলে, configuration রাখে
apt purge নাম	Package ও তার configuration মুছে ফেলে
apt autoremove	আর দরকার নেই এমন dependency মুছে ফেলে

চিত্র ১৪.২ — Dependency গাছ (উদাহরণ)

nginx → libpcre3, zlib1g, libssl3, libcrypt1। আপনি শুধু nginx চাইলেও apt এই চারটি library-ও ইনস্টল করে, কারণ nginx ছাড়া এগুলো ছাড়া চলতে পারে না। একটি library আপডেট হলে nginx-কে আবার ইনস্টল করতে হয় না — এটাই shared library-র সুবিধা।

পরিবার	Package ফরম্যাট	টুল	উদাহরণ
Debian / Ubuntu	.deb	apt, dpkg	Ubuntu Server 24.04 LTS
Fedora / RHEL	.rpm	dnf, rpm	Fedora, Rocky Linux, AlmaLinux
Arch	pkg.tar.zst	pacman	Arch Linux

এই বইয়ে Ubuntu-র apt ব্যবস্থাই প্রধান; অন্য পরিবারের নাম শুধু চিনে রাখার জন্য দেওয়া হলো।

পরিভাষা

Term	বাংলা ব্যাখ্যা
Package	সফটওয়্যার, তার ফাইল ও metadata-র একক
Repository	Package-এর বিশ্বস্ত ভান্ডার; apt এখান থেকে নামায়
Dependency	একটি package চালাতে প্রয়োজনীয় অন্য package
apt / dpkg	উচ্চ স্তরের package manager / নিম্ন স্তরের .deb টুল
GPG signature	Package আসল উৎস থেকে এসেছে, তার ডিজিটাল প্রমাণ
PPA	Personal Package Archive; তৃতীয় পক্ষের repository — সতর্কতার সঙ্গে ব্যবহার করতে হয়
Hold	একটি package-কে আপগ্রেড থেকে আটকে রাখা
Orphan package	আর কোনো package যার ওপর নির্ভর করে না

বাস্তব উদাহরণ: ঢাকার স্টার্টআপে নতুন সার্ভার সাজানো

একটি স্টার্টআপ নতুন Ubuntu সার্ভার পেল। প্রথম কাজ: `sudo apt update && sudo apt upgrade -y` দিয়ে সিস্টেম হালনাগাদ করা। তারপর `sudo apt install nginx python3 curl git htop ufw` দিয়ে প্রয়োজনীয় টুল ইনস্টল করা। প্রতিটি package অফিসিয়াল repository থেকে এল, signature যাচাই হলো, dependency নিজে থেকে মিটল। কেউ যদি ইন্টারনেটের অপরিচিত সাইট থেকে `.deb` নামিয়ে ইনস্টল করত, তাহলে সেই ফাইলে ক্ষতিকর কোড থাকতে পারত, আর ভবিষ্যতে আপডেটও পাওয়া যেত না। তাই নিয়ম: প্রথমে অফিসিয়াল repository, তারপর বিশ্বস্ত তৃতীয় পক্ষ, আর সবশেষে হাতে ইনস্টল — তাও যাচাই করে।

ব্যবহারিক কমান্ড

```
sudo apt update
# Repository-র তালিকা refresh করে। কোনো সফটওয়্যার বদলায় না।

sudo apt upgrade
# ইনস্টল করা package-গুলো নতুন version-এ নিয়ে যায়।

sudo apt install nginx curl git htop
# একসঙ্গে একাধিক package ইনস্টল করে।

sudo apt install -y nginx
# -y মানে নিশ্চিতকরণ ছাড়াই ইনস্টল করা — স্ক্রিপ্টে কাজে লাগে, হাতে চালালে সতর্ক থাকুন।

apt search nginx
# nginx সম্পর্কিত package খোঁজে। sudo দরকার নেই।

apt show nginx
# nginx-এর version, বিবরণ ও dependency দেখায়।

apt list --installed | grep nginx
# nginx ইনস্টল আছে কি না পরীক্ষা করে।
```

```
dpkg -l | grep nginx
# dpkg-এর দৃষ্টিতে ইনস্টল করা package-র তালিকা।

dpkg -L nginx
# nginx package কোন কোন ফাইল ইনস্টল করেছে, তার তালিকা দেখায়।

dpkg -S /usr/sbin/nginx
# এই ফাইলটি কোন package-এর, তা বলে দেয়।

sudo apt remove nginx
# package মুছে ফেলে, /etc/nginx-এর configuration রাখে।

sudo apt purge nginx
# package ও configuration দুটোই মুছে ফেলে।

sudo apt autoremove
# আর প্রয়োজন নেই এমন dependency মুছে ফেলে।

sudo apt clean
# ডাউনলোড করা .deb ফাইলের cache মুছে ফেলে, জায়গা খালি করে।

sudo dpkg -i ./package.deb
# হাতে পাওয়া .deb ফাইল ইনস্টল করে; dependency না মিললে ব্যর্থ হতে পারে।

sudo apt -f install
# ভাঙা dependency মেরামতের চেষ্টা করে।
```

WARNING: অপরিচিত ওয়েবসাইট বা ফোরাম থেকে `.deb` ফাইল নামিয়ে ইনস্টল করবেন না, যতক্ষণ না উৎসটি যাচাই করতে পারছেন। একটি ক্ষতিকর package root ক্ষমতায় ইনস্টল হয় — অর্থাৎ সেটি সিস্টেমে যা খুশি করতে পারে। PPA যোগ করার আগে জানুন সেটি কে চালায়, কতদিন ধরে চালায়, আর Ubuntu 24.04 (noble)-এর জন্য package আছে কি না। Production সার্ভারে `apt upgrade -y` অন্ধভাবে চালাবেন না — প্রথমে কী আপগ্রেড হবে, তা `apt list --upgradable` দিয়ে দেখুন।

CONCEPT: `apt update` বনাম `apt upgrade` মনে রাখার সহজ উপায়: update মানে "দোকানের তালিকা দেখে আসা", upgrade মানে "দোকান থেকে নতুন জিনিস কিনে আনা"। তালিকা না দেখে কিনতে গেলে পুরোনো জিনিসই পাবেন।

হাতে-কলমে অনুশীলন: নিরাপদে ইনস্টল, যাচাই ও অপসারণ

লক্ষ্য: `htop` ইনস্টল করে যাচাই করা, তারপর নিরাপদে মুছে ফেলা।

1. `sudo apt update` চালান।
2. `apt show htop` দিয়ে version ও বিবরণ দেখুন।
3. `sudo apt install htop` চালান।
4. `htop --version` ও `dpkg -L htop` দিয়ে যাচাই করুন।
5. `sudo apt remove htop` দিয়ে মুছে ফেলুন, তারপর `apt list --installed | grep htop` দিয়ে নিশ্চিত হোন।

6. `sudo apt autoremove` দিয়ে অপ্রয়োজনীয় dependency পরিষ্কার করুন।

উদাহরণ আউটপুট (illustrative):

```
$ apt show htop
Package: htop
Version: 3.3.0-1build1
Description: interactive processes viewer

$ htop --version
htop 3.3.0

$ dpkg -L htop
/usr/bin/htop
/usr/share/doc/htop
/usr/share/man/man1/htop.1.gz
```

Version সংখ্যা আপনার সিস্টেমে ভিন্ন হতে পারে — এটি illustrative উদাহরণ। আসল version জানতে সবসময় `apt show` ব্যবহার করুন, বইয়ের সংখ্যা মুখস্থ করবেন না।

> **PRO TIP:** কোনো package ইনস্টল করার আগে `apt show` ও `apt list --upgradable` দেখে নিন। আর একটি সার্ভারে কী ইনস্টল আছে, তার তালিকা সংরক্ষণ করুন: `dpkg --get-selections > packages-list.txt`। নতুন সার্ভারে একই পরিবেশ তৈরি করতে এই ফাইলটি কাজে লাগে।

আরেকটি গুরুত্বপূর্ণ পার্থক্য হলো `apt` ও `apt-get`-এর মধ্যে। Ubuntu 24.04-এ `apt` হলো মানুষের জন্য তৈরি সহজ ইন্টারফেস — রঙিন আউটপুট ও progress bar দেয়। `apt-get` পুরোনো ও স্ক্রিপ্টের জন্য স্থিতিশীল ইন্টারফেস। হাতে কাজ করলে `apt` ব্যবহার করুন, স্ক্রিপ্টে `apt-get` বেশি নিরাপদ। আরেকটি বিষয়: `apt upgrade` কখনো ইনস্টল করা package মুছে ফেলে না, কিন্তু `apt full-upgrade` প্রয়োজনে পুরোনো package মুছে নতুন dependency আনতে পারে — তাই production সার্ভারে `full-upgrade` চালানোর আগে কী মুছবে তা `--dry-run` দিয়ে দেখে নিন। উদাহরণ: `sudo apt full-upgrade --dry-run`। নিয়মিত `apt list --upgradable` দেখার অভ্যাস করলে কোনো আপডেটে kernel বা nginx বদলাবে কি না, তা আগেই জানা যায়, এবং maintenance window পরিকল্পনা করা সহজ হয়।

সাধারণ ভুল ও সমস্যা সমাধান

- **ভুল:** `E: Could not get lock /var/lib/dpkg/lock-frontend` — আরেকটি apt process চলছে। **সমাধান:** অপেক্ষা করুন বা চলমান process শেষ করুন; lock ফাইল জোর করে মুছবেন না।
- **ভুল:** `apt update` না করে `apt install` চালিয়ে পুরোনো version পাওয়া। **সমাধান:** সবসময় আগে `apt update` চালান।
- **ভুল:** `dpkg -i`-এর পর dependency error। **সমাধান:** `sudo apt -f install` চালিয়ে dependency মেটান।
- **ভুল:** ভুল Ubuntu version-এর PPA যোগ করা (যেমন jammy-র PPA noble-তে)। **সমাধান:** `/etc/apt/sources.list.d/`-এর ফাইলটি পরীক্ষা করুন, প্রয়োজনে ফাইলটি মুছে `sudo apt update` চালান।
- **ভুল:** `apt upgrade` চলার সময় terminal বন্ধ করা। **সমাধান:** `sudo dpkg --configure -a` দিয়ে অসম্পূর্ণ ইনস্টলেশন শেষ করুন।

নিরাপত্তা বিবেচনা

নিয়মিত নিরাপত্তা আপডেট ইনস্টল করা সবচেয়ে গুরুত্বপূর্ণ সার্ভার নিরাপত্তা চর্চা। Ubuntu-তে `sudo apt update && sudo apt upgrade` নিয়মিত চালান, আর নিরাপত্তা আপডেটের জন্য `unattended-upgrades` বিবেচনা করুন (অধ্যায় ১৯-এ বিস্তারিত)। শুধু বিশ্বস্ত repository ব্যবহার করুন, GPG key যাচাই না করে কোনো repository যোগ করবেন না। ইনস্টল করা package-র সংখ্যা কম রাখুন — প্রতিটি package একটি সম্ভাব্য আক্রমণের পথ।

শিল্পের সেরা চর্চা (Best Practices)

- Production সার্ভারে পরিবর্তনের আগে `apt list --upgradable` দেখে কী বদলাবে, তা জেনে নিন।
- গুরুত্বপূর্ণ সার্ভারে প্রথমে test পরিবেশে আপগ্রেড পরীক্ষা করুন।
- `remove` বনাম `purge`-এর পার্থক্য বুঝে ব্যবহার করুন — configuration রাখতে চাইলে remove, সম্পূর্ণ মুছতে চাইলে purge।
- নিয়মিত `sudo apt autoremove && sudo apt clean` দিয়ে অপ্রয়োজনীয় package ও cache পরিষ্কার করুন।
- কোনো তৃতীয় পক্ষের repository যোগ করলে তার নাম, URL ও যোগ করার তারিখ লিখে রাখুন।

Knowledge-Check Questions — ৫টি MCQ (উত্তর দেওয়া হয়নি)

1. `apt update` কী করে? A) Package আপগ্রেড করে B) Repository-র তালিকা refresh করে C) Package মুছে ফেলে D) সার্ভার restart করে
2. dpkg-এর কাজ কী? A) Repository থেকে dependency সমাধান করা B) একটি .deb ফাইল ইনস্টল করা C) ইন্টারনেট থেকে package খোঁজা D) পাসওয়ার্ড বদলানো
3. কোন কমান্ডে package ও তার configuration দুটোই মুছে যায়? A) apt remove B) apt purge C) apt clean D) apt autoremove
4. কোন ফাইলটি দেখায় একটি package কোন ফাইল ইনস্টল করেছে? A) dpkg -L B) apt search C) dpkg -S D) apt show
5. ভাঙা dependency মেরামতের চেষ্টায় কোন কমান্ড ব্যবহার করবেন? A) apt -f install B) apt clean C) dpkg -l D) apt search

Practical Assignment

একটি নতুন Ubuntu 24.04 VM-এ `nginx`, `curl` ও `htop` ইনস্টল করুন। প্রতিটির version `apt show` দিয়ে লিখে রাখুন, `dpkg -L` দিয়ে একটি package-এর ফাইল তালিকা দেখুন, তারপর `nginx` remove করে আবার purge-এর পার্থক্য পরীক্ষা করুন (`/etc/nginx` থাকে কি না)।

সারসংক্ষেপ

Package ব্যবস্থা সফটওয়্যার ইনস্টলেশনকে নিরাপদ ও পুনরাবৃত্তিযোগ্য করে। apt repository থেকে package নামায়, dependency সমাধান করে, signature যাচাই করে, আর dpkg দিয়ে ইনস্টল করায়। update মানে তালিকা refresh, upgrade মানে নতুন version। নিরাপদ অভ্যাস: অফিসিয়াল repository প্রথমে, অপরিচিত .deb নয়, নিয়মিত আপডেট, আর পরিবর্তনের আগে কী বদলাবে তা জেনে নেওয়া।

Revision Checklist

- ☐ apt ও dpkg-এর পার্থক্য বলতে পারি
- ☐ update, upgrade, install, remove, purge ও autoremove ব্যবহার করতে পারি
- ☐ apt show, dpkg -L ও dpkg -S দিয়ে package পরীক্ষা করতে পারি
- ☐ dependency কী এবং কেন দরকার তা বুঝি
- ☐ ভাঙা package মেরামতের ধাপগুলো জানি

Chapter 15: systemd, Services and Boot Management — systemd, সার্ভিস ও বুট ব্যবস্থাপনা

ভূমিকা: সার্ভার চালু হওয়ার পর কে সবকিছু চালায়?

আপনি সার্ভারের power বোতাম চাপলেন। কয়েক সেকেন্ডে login prompt এল। কিন্তু এর মধ্যে কী ঘটল? কে নেটওয়ার্ক চালু করল, কে ওয়েব সার্ভার চালু করল, কে log লেখা শুরু করল? উত্তর: systemd। আধুনিক Ubuntu-তে systemd হলো PID 1 — প্রথম process, আর সব service-এর পরিচালক। আগে SysV init নামের পুরোনো ব্যবস্থা ছিল, যেখানে service-গুলো একের পর এক ক্রমান্বয়ে চালু হতো। systemd service-গুলোকে parallel-এ চালায়, dependency বোঝে, ব্যর্থ service-কে আবার চালু করে, আর সব log এক জায়গায় জমা করে। এই অধ্যায়ে আমরা বুট প্রক্রিয়া থেকে শুরু করে নিজের একটি service তৈরি করা পর্যন্ত শিখব — যাতে একটি service বন্ধ হলে আপনি আন্দাজে restart না দিয়ে কারণ খুঁজে ঠিক করতে পারেন।

Learning Objectives — শেখার লক্ষ্য

- Linux boot sequence ব্যাখ্যা করতে পারবেন: firmware থেকে systemd পর্যন্ত।
- systemd-এর architecture বুঝতে পারবেন: unit, service, target ও dependency।
- `systemctl` দিয়ে service start, stop, restart, enable ও disable করতে পারবেন।
- `journalctl` দিয়ে service-এর log পড়ে সমস্যা নির্ণয় করতে পারবেন।
- একটি সহজ custom systemd service তৈরি করতে পারবেন।
- ব্যর্থ service নির্ণয়ের ধাপগুলো অনুসরণ করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৭ (Boot Process), অধ্যায় ১৩ (Process Management) ও অধ্যায় ১৮-এর log ধারণা সহায়ক হবে। Terminal-এ টেক্সট ফাইল সম্পাদনা ও `sudo` ব্যবহার জানা থাকতে হবে।

প্রথম নীতি থেকে: Boot কীভাবে হয়?

Boot হলো কম্পিউটারের নিজেকে জাগিয়ে তোলার প্রক্রিয়া। প্রথমে firmware (UEFI বা BIOS) হার্ডওয়্যার পরীক্ষা করে, তারপর bootloader (Ubuntu-তে GRUB) kernel-কে মেমরিতে load করে। Kernel হার্ডওয়্যার চিনে, driver load করে, root filesystem mount করে, তারপর PID 1 হিসেবে systemd-কে চালায়। systemd তারপর target অনুযায়ী service-গুলো চালু করে: নেটওয়ার্ক, log, SSH, ওয়েব সার্ভার। একটি service আরেকটির ওপর নির্ভর করতে পারে — যেমন ওয়েব সার্ভার নেটওয়ার্ক চালু হওয়ার পরেই চালু হওয়া উচিত। systemd এই dependency graph বুঝে সঠিক ক্রমে, যতটা সম্ভব parallel-এ, service চালু করে। এতে boot দ্রুত হয়, আর কোনো service ব্যর্থ হলে systemd জানে সেটি কার ওপর প্রভাব ফেলবে।

systemd Architecture: Unit, Service ও Target

systemd-এ সবকিছুই একটি unit। Service unit (`.service`) একটি process চালায়, যেমন `nginx.service`। Socket unit (`.socket`) নেটওয়ার্ক সংযোগের জন্য অপেক্ষা করে, timer unit (`.timer`) cron-এর মতো সময়মতো কাজ চালায়,

mount unit filesystem mount করে। Target unit (`.target`) একটি অবস্থা বোঝায় — যেমন `multi-user.target` মানে টেক্সট login-সহ পূর্ণ সার্ভার, `graphical.target` মানে ডেস্কটপসহ অবস্থা। প্রতিটি unit ফাইলে তিনটি অংশ থাকে: `[Unit]` (বিবরণ ও dependency), `[Service]` (কী চালাবে, কীভাবে restart করবে) ও `[Install]` (কোন target-এ চালু হবে)। `systemctl enable nginx` মানে boot-এর সময় nginx চালু হবে; `systemctl start nginx` মানে এখনই চালু করো। Enable ও start আলাদা — একটি ভবিষ্যতের জন্য, আরেকটি বর্তমানের জন্য।

চিত্র ১৫.১ — Boot Sequence (Ubuntu 24.04)

UEFI/BIOS → GRUB bootloader → Linux kernel → systemd (PID 1) → basic.target → multi-user.target → নেটওয়ার্ক, SSH, nginx, cron-সহ service-গুলো → login prompt। GRUB-এ kernel parameter বদলানো যায়, কিন্তু সতর্কতার সঙ্গে — ভুল parameter-এ সিস্টেম boot নাও হতে পারে।

ধাপ	কী হয়	সমস্যা হলে কোথায় দেখবেন
Firmware	হার্ডওয়্যার পরীক্ষা, boot ডিভাইস খোঁজা	BIOS/UEFI সেটিংস
GRUB	Kernel বেছে load করা	GRUB menu, /boot/grub/grub.cfg
Kernel	Driver load, root filesystem mount	dmesg
systemd	Service চালু, target-এ পৌঁছানো	journalctl -b, systemctl status

চিত্র ১৫.২ — একটি Service Unit ফাইলের গঠন

```
[Unit]
Description=My simple web app
After=network.target

[Service]
Type=simple
User=www-data
ExecStart=/usr/bin/python3 /srv/myapp/app.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

`After=network.target` মানে নেটওয়ার্কের পরে চালু হবে। `Restart=on-failure` মানে crash করলে systemd নিজে আবার চালু করবে। `WantedBy=multi-user.target` মানে `enable` করলে boot-এর সময় চালু হবে।

পরিভাষা

Term	বাংলা ব্যাখ্যা
systemd	PID 1 init system ও service manager; Ubuntu-র মূল পরিচালক

Unit	systemd-এর পরিচালনার একক; service, socket, timer, target ইত্যাদি
Target	সিস্টেমের একটি অবস্থা, যেমন multi-user.target
systemctl	systemd নিয়ন্ত্রণের প্রধান কমান্ড
journalctl	systemd journal থেকে log পড়ার টুল
Daemon	পেছনে চলা service process
GRUB	Bootloader; kernel বেছে load করে
Socket activation	সংযোগ এলে তবেই service চালু হওয়ার ব্যবস্থা

বাস্তব উদাহরণ: ওয়েবসাইট Down — কারণ খুঁজে বের করা

রাজশাহীর একটি কোম্পানির ওয়েবসাইট হঠাৎ খুলছে না। জুনিয়র অ্যাডমিন প্রথমে `systemctl status nginx` চালাল — দেখা গেল service failed, আর log-এ লেখা `bind() to 0.0.0.0:80 failed: address already in use`। অর্থাৎ 80 নম্বর পোর্ট আরেকটি process দখল করে আছে। `ss -tulpn | grep :80` দিয়ে দেখা গেল একটি পুরোনো Apache process চলছে। Apache থামিয়ে nginx restart করতেই সাইট ফিরে এল। এখানে শিক্ষা: service restart করার আগে `status` ও `journalctl` দিয়ে কারণ পড়ুন। অন্ধভাবে restart করলে সমস্যা সাময়িকভাবে লুকায়, মূল কারণ থেকে যায়।

ব্যবহারিক কমান্ড

```
systemctl status nginx
# nginx service-এর অবস্থা, PID, সাম্প্রতিক log ও enabled কি না দেখায়।

sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx
# এখনই চালু, থামানো বা আবার চালু করা।

sudo systemctl reload nginx
# process না মেরে configuration আবার পড়ে — সম্ভব হলে restart-এর চেয়ে নিরাপদ।

sudo systemctl enable nginx
sudo systemctl disable nginx
# boot-এর সময় চালু হবে কি না, তা ঠিক করে।

systemctl is-enabled nginx
systemctl is-active nginx
# enabled/active অবস্থা সংক্ষেপে জানায়।

systemctl list-units --type=service --state=failed
# ব্যর্থ service-গুলোর তালিকা দেখায়।

sudo systemctl daemon-reload
# unit ফাইল বদলানোর পর systemd-কে আবার পড়তে বলে। নতুন service তৈরি করলে এটি চালাতেই হবে।

journalctl -u nginx
# nginx service-এর সব log দেখায়।
```

```
journalctl -u nginx --since "2026-11-08 09:00" --until "2026-11-08 10:00"
# নির্দিষ্ট সময়ের log দেখায়।

journalctl -u nginx -f
# লাইভ log অনুসরণ করে; Ctrl+C দিয়ে বের হবেন।

journalctl -b -p err
# বর্তমান boot-এর error স্তরের log দেখায়।

journalctl --disk-usage
# journal কত জায়গা নিচ্ছে, তা দেখায়।

systemctl cat nginx
# nginx unit ফাইলের সম্পূর্ণ বিষয় দেখায়।
```

WARNING: Unit ফাইল সম্পাদনার আগে তার একটি কপি সংরক্ষণ করুন। `Restart=always` ভুলভাবে ব্যবহার করলে একটি ভুল configuration-সহ service বারবার crash করে restart হতে থাকবে — log ভরে যাবে। নতুন service প্রথমে `start` করে পরীক্ষা করুন, ঠিক থাকলে তবেই `enable` করুন। `systemctl stop ssh` চালালে দূরবর্তী সার্ভার থেকে আপনার সংযোগ বিচ্ছিন্ন হতে পারে — remote সার্ভারে SSH service থামানোর আগে সতর্ক থাকুন।

CONCEPT: `reload` বনাম `restart`: reload মানে একই process configuration আবার পড়ে — সংযোগ বিচ্ছিন্ন হয় না। restart মানে পুরোনো process মেরে নতুন process চালু করা — সাময়িকভাবে service বন্ধ থাকে। Configuration বদলালে প্রথমে reload চেষ্টা করুন; service-টি reload সমর্থন না করলে তবেই restart করুন।

হাতে-কলমে অনুশীলন: নিজের একটি Service তৈরি করা

লক্ষ্য: একটি সহজ Python স্ক্রিপ্টকে systemd service হিসেবে চালানো, যাতে boot-এর সময় সেটি নিজে চালু হয়।

1. একটি সহজ স্ক্রিপ্ট তৈরি করুন: `sudo mkdir -p /srv/myapp`, তারপর `/srv/myapp/app.py` ফাইলে একটি ছোট Python প্রোগ্রাম লিখুন যা প্রতি 10 সেকেন্ডে একটি লাইন print করে।
2. Unit ফাইল তৈরি করুন: `sudo nano /etc/systemd/system/myapp.service` — উপরের চিত্র ১৫.২-এর বিষয়টি ব্যবহার করুন, `ExecStart`-এ আপনার স্ক্রিপ্টের সঠিক পথ দিন।
3. `sudo systemctl daemon-reload` চালান।
4. `sudo systemctl start myapp` ও `systemctl status myapp` দিয়ে পরীক্ষা করুন।
5. `journalctl -u myapp -f` দিয়ে দেখুন স্ক্রিপ্টের আউটপুট আসছে কি না।
6. ঠিক থাকলে `sudo systemctl enable myapp` চালান, তারপর VM restart করে `systemctl is-active myapp` দিয়ে যাচাই করুন।

উদাহরণ আউটপুট (illustrative):

```
$ systemctl status myapp
● myapp.service - My simple web app
     Loaded: loaded (/etc/systemd/system/myapp.service; enabled)
     Active: active (running) since Sat 2026-11-08 10:15:00 +06; 2min ago
```

```
   Main PID: 1234 (python3)
      Tasks: 1
     Memory: 12.0M
        CPU: 0.120s

$ journalctl -u myapp
Nov 08 10:15:00 ubuntu-server python3[1234]: My app is running
```

যদি status-এ `failed` দেখায়, তাহলে `journalctl -u myapp`-এর শেষ লাইনগুলো পড়ুন — সেখানে কারণ লেখা থাকে: ভুল পথ, অনুমতি নেই, অথবা Python error। Unit ফাইল বদলালে আবার `daemon-reload` ও `restart` করতে ভুলবেন না।

> **PRO TIP:** Service-এর সমস্যা নির্ণয়ের ক্রম মনে রাখুন: `systemctl status নাম` → `journalctl -u নাম` → configuration ফাইল পরীক্ষা (`nginx -t`-এর মতো) → dependency ও পোর্ট পরীক্ষা (`ss -tulpn`) → তারপর restart। এই ক্রমে কাজ করলে আপনি কারণ না জেনে restart করার ভুল করবেন না।

একটি সাধারণ বিভ্রান্তি হলো `systemctl restart` ও `reboot`-এর মধ্যে। Restart শুধু একটি service-কে আবার চালু করে, reboot পুরো সিস্টেমকে বন্ধ করে আবার চালু করে। সার্ভারের সমস্যায় প্রথমে service restart করুন, পুরো reboot শেষ উপায় হিসেবে রাখুন — কারণ reboot-এ সব service একসঙ্গে বন্ধ হয়, আর boot-এর সময় কোনো service ব্যর্থ হলে সমস্যা আরও জটিল হতে পারে। আরেকটি দরকারি টুল হলো `systemd-analyze blame`, যা দেখায় boot-এর সময় কোন service কতক্ষণ নিয়েছে। যদি boot ধীর হয়, এই কমান্ড দিয়ে ধীর service খুঁজে বের করুন। `systemd-analyze critical-chain` দিয়ে দেখতে পারবেন কোন dependency chain boot-কে আটকে রেখেছে। এই দুটি কমান্ড boot সমস্যা নির্ণয়ে প্রথম ধাপ হিসেবে ব্যবহার করুন।

সাধারণ ভুল ও সমস্যা সমাধান

- **ভুল:** Unit ফাইল বদলানোর পর `daemon-reload` না করা, ফলে পুরোনো সংস্করণই চলছে। **সমাধান:** প্রতিবার unit ফাইল বদলালে `sudo systemctl daemon-reload` চালান।
- **ভুল:** `enable` না করে ভাবা যে boot-এর সময় service চালু হবে। **সমাধান:** `systemctl is-enabled নাম` দিয়ে যাচাই করুন।
- **ভুল:** `ExecStart`-এ আপেক্ষিক পথ বা shell feature (যেমন `>`, `|`) ব্যবহার করা। systemd সরাসরি কমান্ড চালায়, shell নয়। **সমাধান:** সম্পূর্ণ পথ ব্যবহার করুন, অথবা shell দরকার হলে `ExecStart=/bin/bash -c '...'` লিখুন।
- **ভুল:** Service failed হলে বারবার restart করা, কিন্তু log না পড়া। **সমাধান:** `journalctl -u নাম --no-pager | tail -50` দিয়ে শেষ 50 লাইন পড়ুন।
- **ভুল:** Journal-এর জায়গা শেষ হয়ে যাওয়া। **সমাধান:** `journalctl --disk-usage` দেখুন, প্রয়োজনে `sudo journalctl --vacuum-size=100M` দিয়ে পুরোনো log কমান (সতর্কতার সঙ্গে)।

নিরাপত্তা বিবেচনা

Service যত কম ক্ষমতায় চলে, তত নিরাপদ। Unit ফাইলে `User=` ও `Group=` দিয়ে নির্দিষ্ট কম-ক্ষমতার ব্যবহারকারী ঠিক করুন — root দিয়ে service চালাবেন না, যদি না একান্ত দরকার হয়। অপ্রয়োজনীয় service disable করুন: `systemctl list-unit-files --state=enabled` দিয়ে দেখুন boot-এর সময় কী কী চালু হয়। প্রতিটি service একটি সম্ভাব্য আক্রমণের পথ, বিশেষ করে নেটওয়ার্কে শোনা service। `ss -tulpn` দিয়ে নিয়মিত পরীক্ষা করুন কোন পোর্ট খোলা আছে।

শিল্পের সেরা চর্চা (Best Practices)

- নিজের service-র জন্য `/etc/systemd/system/` -এ unit ফাইল রাখুন; distribution-এর ফাইল `/lib/systemd/system/` -এ হাত দেবেন না।
- Unit ফাইলে `Description`, `After` ও `Restart` স্পষ্টভাবে লিখুন।
- Service-এর log-এ অর্থবহ বার্তা লিখুন, যাতে ভবিষ্যতে সমস্যা নির্ণয় সহজ হয়।
- গুরুত্বপূর্ণ service-র পরিবর্তনের আগে `systemctl cat নাম` দিয়ে বর্তমান ফাইল সংরক্ষণ করুন।
- Boot সমস্যার ক্ষেত্রে `journalctl -b -1` দিয়ে আগের boot-এর log দেখুন।

Knowledge-Check Questions — ৫টি MCQ (উত্তর দেওয়া হয়নি)

1. Ubuntu 24.04-এ PID 1 কোনটি? A) GRUB B) kernel C) systemd D) bash
2. `systemctl enable nginx` কী করে? A) nginx এখনই চালু করে B) boot-এর সময় nginx চালু হওয়ার ব্যবস্থা করে C) nginx মুছে ফেলে D) nginx-এর log দেখায়
3. Unit ফাইল বদলানোর পর কোন কমান্ড চালাতে হয়? A) systemctl start B) systemctl daemon-reload C) systemctl stop D) journalctl -f
4. একটি service-এর log দেখতে কোন কমান্ড ব্যবহার করবেন? A) ps aux B) journalctl -u নাম C) ls -l D) apt show
5. `reload` ও `restart` -এর মধ্যে পার্থক্য কী? A) কোনো পার্থক্য নেই B) reload process না মেরে configuration আবার পড়ে, restart নতুন process চালু করে C) reload service মুছে ফেলে D) restart শুধু log দেখায়

Practical Assignment

একটি কাল্পনিক কোম্পানির জন্য একটি `backup.service` তৈরি করুন, যা `/srv/data` ডিরেক্টরিকে `/backup` -এ `tar` দিয়ে সংরক্ষণ করে। একটি timer unit (`.timer`) দিয়ে সেটিকে প্রতিদিন রাত ২টায় চালানোর ব্যবস্থা করুন। `systemctl list-timers` দিয়ে যাচাই করুন এবং service-টি হাতে চালিয়ে log পরীক্ষা করুন।

সারসংক্ষেপ

systemd হলো Ubuntu সার্ভারের হৃদয়: firmware থেকে kernel, kernel থেকে systemd, systemd থেকে সব service। Unit ফাইল বলে কী চালাতে হবে, কীভাবে চালাতে হবে, আর কখন চালাতে হবে। `systemctl` দিয়ে service নিয়ন্ত্রণ করা যায়, `journalctl` দিয়ে তার log পড়া যায়, আর নিজের service তৈরি করে যেকোনো স্ক্রিপ্টকে নির্ভরযোগ্য service-এ রূপান্তর করা যায়। ভালো অ্যাডমিন service restart করার আগে কারণ পড়ে — কারণ systemd log-এই লেখা থাকে।

Revision Checklist

- ☐ Boot sequence-এর ধাপগুলো (firmware → GRUB → kernel → systemd) বলতে পারি
- ☐ systemctl দিয়ে start, stop, restart, reload, enable ও disable করতে পারি
- ☐ journalctl দিয়ে service-এর log পড়তে পারি
- ☐ একটি সহজ custom service unit ফাইল লিখতে পারি
- ☐ ব্যর্থ service নির্ণয়ের ধাপগুলো অনুসরণ করতে পারি

PART IV • DEVOPS ENGINEERING MASTERY

Operating & Troubleshooting Linux

সার্ভার চালানো ও সমস্যা সমাধান — স্টোরেজ, মনিটরিং, লগ, নিরাপত্তা, ব্যাকআপ।

Chapter 16: Disk, Filesystem and Storage Management — ডিস্ক, ফাইলসিস্টেম ও স্টোরেজ ব্যবস্থাপনা

শুরুর কথা: সার্ভার যখন হঠাৎ থেমে যায়

ঢাকার একটি ছোট সফটওয়্যার কোম্পানির ওয়েবসাইট এক সকালে হঠাৎ বন্ধ হয়ে গেল। অ্যাপ্লিকেশন চলছে, ডেটাবেস চলছে, কিন্তু নতুন কোনো ব্যবহারকারী নিবন্ধন করতে পারছে না, ছবি আপলোড হচ্ছে না, লগ লেখা যাচ্ছে না। জুনিয়র অ্যাডমিন প্রথমে ভাবলেন সার্ভার ডাউন। আসলে সার্ভার চলছিল — ডিস্ক পূর্ণ হয়ে গিয়েছিল। একটি লগ ফাইল কয়েক সপ্তাহ ধরে বড় হতে হতে পুরো ডিস্ক দখল করে নিয়েছিল। এই একটি ঘটনা বোঝায় কেন স্টোরেজ ব্যবস্থাপনা শুধু “জায়গা দেখা” নয়, এটি সার্ভার চালু রাখার মৌলিক দক্ষতা। ডিস্ক পূর্ণ হলে সিস্টেম ধীর হয়, সার্ভিস বন্ধ হয়, ডেটাবেস লেখা বন্ধ করে দেয়, এমনকি লগইন করাও কঠিন হয়ে যায়। এই অধ্যায়ে আপনি শিখবেন ডিস্ক কীভাবে কাজ করে, Linux কীভাবে ফাইলসিস্টেম সাজায়, কোথায় জায়গা খরচ হচ্ছে তা কীভাবে খুঁজে বের করতে হয় এবং সমস্যা হওয়ার আগেই কীভাবে তা প্রতিরোধ করতে হয়।

Learning Objectives — শেখার উদ্দেশ্য

- Block device, partition, filesystem ও mount-এর পার্থক্য ব্যাখ্যা করতে পারবেন।
- `lsblk`, `df`, `du`, `mount` ও `find` দিয়ে ডিস্ক ব্যবহার বিশ্লেষণ করতে পারবেন।
- `/etc/fstab` পড়ে ও নিরাপদে সম্পাদনা করে স্থায়ী mount বুঝতে পারবেন।
- inode, block ও filesystem-এর ভেতরের গঠন ব্যাখ্যা করতে পারবেন।
- “Disk full” সমস্যা ধাপে ধাপে নির্ণয় ও সমাধান করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৮-এর filesystem hierarchy, অধ্যায় ৯-এর টার্মিনাল ব্যবহার এবং অধ্যায় ১০-এর মৌলিক কমান্ড জানা থাকতে হবে। `sudo` কী এবং কেন সতর্কভাবে ব্যবহার করতে হয়, তা অধ্যায় ১২ থেকে জানা ধরে নেওয়া হয়েছে।

প্রথম নীতি থেকে: স্টোরেজ আসলে কী?

কম্পিউটারের মেমোরি দুই ধরনের। RAM হলো অস্থায়ী কাজের টেবিল — বিদ্যুৎ গেলে সব মুছে যায়। আর ডিস্ক হলো স্থায়ী গুদাম — বিদ্যুৎ গেলেও তথ্য থেকে যায়। কিন্তু ডিস্ক সরাসরি “ফাইল” বোঝে না। ডিস্ক বোঝে শুধু block — নির্দিষ্ট আকারের ছোট ছোট টুকরো, সাধারণত ৪ কিলোবাইট। এর ওপরে filesystem নামের একটি নিয়ম-কাঠামো বসে, যা ঠিক করে কোন block-এ কোন ফাইলের কোন অংশ থাকবে, ফাইলের নাম কী, মালিক কে, অনুমতি কী। একে ভাবা যায় লাইব্রেরির মতো: ডিস্ক হলো ভবন, block হলো তাকের খোপ, আর filesystem হলো ক্যাটালগ ও সাজানোর নিয়ম। ক্যাটালগ ছাড়া বই খুঁজে পাওয়া যেমন অসম্ভব, filesystem ছাড়া block থেকে ফাইল খুঁজে পাওয়াও তেমনি অসম্ভব।

এই স্তরগুলো আলাদা রাখা জরুরি। Hardware স্তরে থাকে physical disk — HDD, SSD বা NVMe। তার ওপরে partition — ডিস্ককে ভাগ করা। প্রতিটি partition-এ একটি filesystem তৈরি করা হয়, যেমন ext4 বা XFS। তারপর সেই filesystem-কে ডিরেক্টরি গাছের কোনো এক জায়গায় mount করা হয়। Linux-এ সবকিছু একটি গাছ — `/` থেকে শুরু। আলাদা ডিস্কও

সেই গাছেরই একটি শাখা হয়ে যায়। তাই `/home` আলাদা ডিস্কে থাকলেও ব্যবহারকারীর কাছে তা একই গাছের অংশ মনে হয়।

ভেতরের গঠন: partition, filesystem, inode ও mount

Partition হলো ডিস্কের যৌক্তিক ভাগ। পুরোনো MBR পদ্ধতিতে সর্বোচ্চ চারটি primary partition করা যেত, আর আধুনিক GPT পদ্ধতিতে অনেক বেশি partition সম্ভব এবং ২ টেরাবাইটের বড় ডিস্কও সমর্থিত। Ubuntu Server 24.04 সাধারণত GPT ব্যবহার করে। Partition তৈরি করা আর তাতে filesystem বানানো এক জিনিস নয় — partition হলো জমির প্লট, filesystem হলো সেই প্লটে বাড়ি বানানো।

ext4 filesystem-এ প্রতিটি ফাইলের জন্য একটি inode থাকে। inode হলো ফাইলের পরিচয়পত্র — এতে থাকে ফাইলের আকার, মালিক, অনুমতি, সময় এবং কোন block-গুলোতে তথ্য আছে তার তালিকা। ফাইলের নাম inode-তে থাকে না; নাম থাকে ডিরেক্টরি entry-তে, যা নামকে inode নম্বরের সঙ্গে যুক্ত করে। এ কারণেই একই ফাইলের দুটি নাম (hard link) সম্ভব — দুটি নাম একই inode-কে নির্দেশ করে। আর একটি গুরুত্বপূর্ণ সত্য: inode-র সংখ্যা সীমিত। লক্ষ লক্ষ ছোট ফাইল তৈরি হলে ডিস্কে জায়গা থাকলেও inode শেষ হয়ে যেতে পারে, তখন নতুন ফাইল তৈরি বন্ধ হয়ে যায়। `df -i` দিয়ে inode ব্যবহার দেখা যায় — এটি অনেক নতুন অ্যাডমিনের অজানা একটি পরীক্ষা।

Mount হলো filesystem-কে গাছের সঙ্গে যুক্ত করা। `mount /dev/sdb1 /mnt/data` মানে sdb1 partition-এর filesystem-কে `/mnt/data` ডিরেক্টরিতে বসানো। বুটের সময় কোন filesystem কোথায় mount হবে, তা লেখা থাকে `/etc/fstab`-এ। এখানে ভুল করলে সিস্টেম বুট হতে না-ও পারে, তাই fstab সম্পাদনার আগে ব্যাকআপ নেওয়া এবং `mount -a` দিয়ে পরীক্ষা করা বাধ্যতামূলক অভ্যাস।

চিত্র ১৬.১ — স্টোরেজের স্তরসমূহ

স্তর	কী করে	উদাহরণ
Physical disk	তথ্য স্থায়ীভাবে ধরে রাখে	SSD / NVMe / HDD
Partition	ডিস্ককে যৌক্তিক ভাগে ভাগ করে	/dev/sda1, /dev/sda2 (GPT)
Filesystem	Block-কে ফাইলে সাজায়	ext4, XFS
Mount point	Filesystem-কে ডিরেক্টরি গাছে যুক্ত করে	/, /home, /mnt/data
File / Directory	ব্যবহারকারী যা দেখে ও ব্যবহার করে	/home/nahid/report.txt

চিত্র ১৬.২ — inode কীভাবে ফাইলকে block-এর সঙ্গে যুক্ত করে

ডিরেক্টরি entry: `report.txt → inode 1048576` → inode-তে আছে: আকার, মালিক, অনুমতি, সময় + block তালিকা [block 1201, 1202, 1203 ...] → block-গুলোতে আসল তথ্য। নাম মুছে গেলে ডিরেক্টরি entry মুছে যায়; শেষ hard link মুছে গেলে inode ও block মুক্ত হয়। তাই `rm` করলেও কোনো প্রোগ্রাম ফাইলটি খোলা রাখলে ডিস্কের জায়গা তখনই ফেরত আসে না — প্রোগ্রাম বন্ধ হলে আসে।

পরিভাষা

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Block device	যে ডিভাইস block হিসেবে তথ্য পড়ে ও লেখে, যেমন ডিস্ক	১৬
Partition	ডিস্কের যৌক্তিক ভাগ	১৬
Filesystem	Block-কে ফাইলে সাজানোর নিয়ম ও গঠন	১৬
inode	ফাইলের metadata ধারণকারী গঠন; নাম এতে থাকে না	১৬
Mount point	যে ডিরেক্টরিতে একটি filesystem যুক্ত হয়	১৬
fstab	বুটের সময় mount-এর তালিকা: /etc/fstab	১৬
ext4	Linux-এর বহুল ব্যবহৃত journaling filesystem	১৬
Swap	RAM পূর্ণ হলে ব্যবহৃত ডিস্কের অস্থায়ী জায়গা	১৬, ১৭

বাস্তব উদাহরণ

একটি বিশ্ববিদ্যালয়ের ল্যাব সার্ভারে `/var/log`-এ একটি ত্রুটিপূর্ণ সার্ভিস প্রতি সেকেন্ডে একই ভুল বার্তা লিখছিল। তিন দিনে লগ ৪০ গিগাবাইট হয়ে যায়। ওয়েবসাইট বন্ধ হওয়ার পর অ্যাডমিন `df -h` দিয়ে দেখলেন `/` ১০০% পূর্ণ, তারপর `du -sh /var/*` দিয়ে দেখলেন `/var/log`-ই দায়ী। তিনি লগ ঘোরানোর (logrotate) ব্যবস্থা করলেন এবং সার্ভিসের ভুল ঠিক করলেন। শিক্ষা: ডিস্ক পূর্ণ হওয়া সাধারণত হঠাৎ ঘটে না — এটি ধীরে ধীরে বাড়ে, আর নিয়মিত পর্যবেক্ষণই একে আগেভাগে ধরে।

ব্যবহারিক কমান্ড

```
lsblk -f
# সব block device, partition, filesystem ও mount point দেখায়। -f দিয়ে filesystem ধরন ও UUID দেখা যায়।

df -h
# প্রতিটি mounted filesystem-এ কত জায়গা আছে, কত ব্যবহৃত, কত ফাঁকা — মানুষের পড়ার মতো এককে।

df -i
# inode ব্যবহার দেখায়। জায়গা থাকলেও inode শেষ হলে ফাইল তৈরি হবে না।

du -sh /var/* 2>/dev/null | sort -rh | head -20
# /var-এর ভেতরে কোন ডিরেক্টরি কত জায়গা নিয়েছে, বড় থেকে ছোট সাজিয়ে প্রথম ২০টি।

sudo du -sh /home/* 2>/dev/null | sort -rh | head -10
# প্রতিটি ব্যবহারকারীর home কত বড়। অনুমতি না থাকলে sudo লাগে।

lsblk -o NAME,SIZE,TYPE,MOUNTPOINT
# সংক্ষিপ্ত, পরিষ্কার ডিস্ক-তালিকা।

sudo blkid
# প্রতিটি partition-এর UUID ও filesystem ধরন দেখায়; fstab-এ UUID ব্যবহারের জন্য দরকারি।

mount | grep -E ' / | /home'
```

```
# বর্তমানে কী কী mount আছে তা দেখায়।

find /var/log -type f -size +100M -exec ls -lh {} \;
# /var/log-এ ১০০ মেগাবাইটের বড় ফাইল খুঁজে বের করে।
```

`df` আর `du`-র পার্থক্য মনে রাখা জরুরি। `df` filesystem-কে জিজ্ঞেস করে — "তোমার মোট কত, কত ফাঁকা?" `du` ডিরেক্টরি ঘুরে ফাইলের আকার যোগ করে। মুছে ফেলা কিন্তু খোলা থাকা ফাইল থাকলে দুটির ফল মেলে না — `df` বলে জায়গা নেই, `du` বলে জায়গা আছে। তখন দায়ী প্রোগ্রামটি খুঁজে বন্ধ করতে হয়, অথবা সার্ভিস restart করতে হয়।

> **WARNING:** ডিস্ক খালি করতে কখনোই অন্ধভাবে `rm -rf /var/log/*` বা `rm -rf /`-জাতীয় কমান্ড চালাবেন না। চলমান সার্ভিসের লগ মুছে ফেললে সমস্যার প্রমাণ নষ্ট হয়, আর ভুল পথ মুছলে সিস্টেম অচল হয়ে যেতে পারে। আগে কোন ফাইল বড় তা নিশ্চিত করুন, সার্ভিস বন্ধ বা truncate করার নিরাপদ পথ বেছে নিন।

> **PRO TIP:** চলমান লগ ফাইল খালি করার নিরাপদ উপায় হলো `sudo truncate -s 0 /var/log/বড়-ফাইল.log` — এতে ফাইলটি খালি হয় কিন্তু inode একই থাকে, তাই লেখা প্রোগ্রামটি ভাঙে না। তবে মূল কারণ ঠিক না করলে ফাইল আবার বড় হবে।

হাতে-কলমে অনুশীলন: ডিস্ক পূর্ণ হওয়ার তদন্ত

নিজের Ubuntu 24.04 ভার্চুয়াল মেশিনে এই অনুশীলন করুন। কোনো ফাইল মুছবেন না — শুধু পর্যবেক্ষণ করবেন।

1. `lsblk -f` চালিয়ে আপনার ডিস্ক, partition ও mount point লিখে রাখুন।
2. `df -h` ও `df -i` চালিয়ে কোন filesystem কত শতাংশ পূর্ণ এবং inode কত ব্যবহৃত তা নোট করুন।
3. `du -sh /* 2>/dev/null | sort -rh | head -10` দিয়ে সবচেয়ে বড় ১০টি শীর্ষ ডিরেক্টরি খুঁজুন।
4. সবচেয়ে বড় ডিরেক্টরির ভেতরে আরও এক ধাপ গভীরে যান: `du -sh /var/* 2>/dev/null | sort -rh | head -10`।
5. `find /var/log -type f -size +10M -exec ls -lh {} \;` দিয়ে বড় লগ ফাইল আছে কি না দেখুন।
6. ফলাফল একটি নোটে লিখুন: কোন filesystem সবচেয়ে পূর্ণ, কোন ডিরেক্টরি সবচেয়ে বড়, এবং ৮০% ছাড়ালে আপনি কী করতেন।

উদাহরণ আউটপুট (illustrative):

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        20G  8.5G   11G  44% /
tmpfs           1.9G     0  1.9G   0% /dev/shm

12G     /usr
4.1G    /var
2.3G    /home
1.2G    /var/log
```

ব্যাখ্যা: `/` ৪৪% পূর্ণ — এখনো নিরাপদ, কিন্তু `/var/log` ১.২ গিগাবাইট হলে তা নজরে রাখা উচিত। ৮০% ছাড়ালে সতর্ক হোন, ৯০% ছাড়ালে জরুরি ব্যবস্থা নিন।

fstab: স্থায়ী mount যেভাবে কাজ করে

`/etc/fstab`-এর প্রতিটি লাইনে ছয়টি ক্ষেত্র থাকে: কোন ডিভাইস (সাধারণত UUID), কোথায় mount হবে, কোন filesystem, কী options, dump ও fsck সংক্রান্ত সংখ্যা। UUID ব্যবহার করা হয় কারণ `/dev/sda` নামটি ডিস্ক যোগ-বিয়োগে বদলে যেতে পারে, কিন্তু UUID বদলায় না। নতুন mount যোগ করার নিরাপদ ক্রম: (১) fstab-এর ব্যাকআপ নিন — `sudo cp /etc/fstab /etc/fstab.bak`; (২) লাইন যোগ করুন; (৩) `sudo mount -a` দিয়ে পরীক্ষা করুন — কোনো ভুল থাকলে এখানেই ধরা পড়বে, পরের বুটে নয়; (৪) তবেই রিবুট করুন। `mount -a` ছাড়া রিবুট করা নবীন অ্যাডমিনের সবচেয়ে সাধারণ ও ব্যয়বহুল ভুলগুলোর একটি।

```
sudo cp /etc/fstab /etc/fstab.bak
cat /etc/fstab
sudo blkid
sudo mount -a
#  কোনো আউটপুট না আসা মানে কোনো ত্রুটি নেই  —  এটাই সাফল্যের চিহ্ন।
```

সাধারণ ভুল ও সমস্যা সমাধান

- **“No space left on device” কিন্তু df-তে জায়গা দেখা যাচ্ছে:** inode শেষ হয়েছে কি না `df -i` দিয়ে দেখুন, অথবা মুছে ফেলা খোলা ফাইল আছে কি না দেখুন।
- **ফাইল মুছেও জায়গা ফেরত আসছে না:** কোনো সার্ভিস ফাইলটি খোলা রেখেছে। সংশ্লিষ্ট সার্ভিস restart করুন, অথবা `truncate -s 0` ব্যবহার করুন।
- **ভুল fstab-এর কারণে বুট আটকে গেছে:** recovery mode বা live USB থেকে বুট করে fstab.bak দিয়ে ফিরিয়ে আনুন।
- **অন্য ডিস্ক mount হচ্ছে না:** `lsblk -f` দিয়ে filesystem আছে কি না দেখুন; নতুন ডিস্কে filesystem তৈরি না থাকলে mount হবে না। নতুন ডিস্কে `mkfs` চালানোর আগে তিনবার নিশ্চিত হোন — এটি ডিস্কের সব তথ্য মুছে দেয়।
- **USB বা বাহ্যিক ডিস্ক হঠাৎ অদৃশ্য:** `dmesg | tail -20` দিয়ে kernel কী বলছে দেখুন (অধ্যায় ১৮-এ বিস্তারিত)।

TROUBLESHOOTING: ডিস্ক পূর্ণ সমস্যার ধাপ: ১) `df -h` — কোন filesystem পূর্ণ; ২) `du` দিয়ে বড় ডিরেক্টরি খুঁজুন; ৩) বড় ফাইল চিহ্নিত করুন; ৪) ফাইলটি কী, কে লিখছে, কেন বড় হচ্ছে তা বুঝুন; ৫) নিরাপদে জায়গা করুন ও মূল কারণ ঠিক করুন। শুধু মুছে ফেলা সমাধান নয় — কারণ ঠিক না করলে সমস্যা ফিরে আসবে।

নিরাপত্তা বিবেচনা

স্টোরেজ নিরাপত্তার অংশ। `/tmp` ও `/var/tmp` সবার লেখার জায়গা — সেখানে সংবেদনশীল ফাইল রাখবেন না। ব্যাকআপ ডিস্ক mount করে রাখলে ransomware বা ভুল কমান্ড ব্যাকআপও মুছে দিতে পারে, তাই গুরুত্বপূর্ণ ব্যাকআপ আলাদা রাখুন (অধ্যায় ২০)। ডিস্ক ফেলে দেওয়ার আগে তথ্য মুছে ফেলার বিষয়টি মনে রাখুন — সাধারণ delete-এ তথ্য পুনরুদ্ধার করা যায়।

শিল্পের সেরা চর্চা

- নিয়মিত `df -h` ও `df -i` পর্যবেক্ষণ করুন; ৮০%-এ সতর্কতা, ৯০%-এ জরুরি পদক্ষেপ।
- logrotate সক্রিয় রাখুন যাতে লগ স্বয়ংক্রিয়ভাবে ঘোরে ও পুরোনো লগ মুছে যায়।
- fstab-এ ডিভাইসের নাম নয়, UUID ব্যবহার করুন।

- গুরুত্বপূর্ণ ডেটার জন্য আলাদা partition বা ডিস্ক ব্যবহার করুন, যাতে লগ পূর্ণ হলেও `/home` বা ডেটা আক্রান্ত না হয়।
- কোনো ফাইল মুছের আগে তা কী, তা নিশ্চিত করুন; সন্দেহ হলে আগে ব্যাকআপ নিন।

Knowledge Check — জ্ঞান যাচাই (উত্তর অধ্যায়ের শেষে নয়, উত্তর-নির্দেশিকায়)

1. inode সম্পর্কে কোনটি সঠিক? A) inode-তে ফাইলের নাম থাকে B) inode-তে ফাইলের metadata ও block তালিকা থাকে C) inode শুধু ডিরেক্টরির জন্য থাকে D) inode মুছলে ফাইলের নাম মুছে যায় কিন্তু তথ্য থাকে
2. `df -h` কী দেখায়? A) ডিরেক্টরির আকার B) প্রতিটি mounted filesystem-এর মোট, ব্যবহৃত ও ফাঁকা জায়গা C) RAM ব্যবহার D) CPU লোড
3. ডিস্কে জায়গা থাকলেও নতুন ফাইল তৈরি না হওয়ার একটি সম্ভাব্য কারণ কী? A) RAM কম B) inode শেষ হয়ে যাওয়া C) CPU ব্যস্ত D) নেটওয়ার্ক বন্ধ
4. fstab সম্পাদনার পর রিবুটের আগে কোন পরীক্ষা করা উচিত? A) `reboot` B) `mount -a` C) `df -h` D) `lsblk`
5. চলমান লগ ফাইল নিরাপদে খালি করার উপায় কোনটি? A) `rm` দিয়ে মুছে ফেলা B) `truncate -s 0` C) `chmod 000` D) ডিস্ক format করা

Practical Assignment — ব্যবহারিক কাজ

আপনার ভার্চুয়াল মেশিনের একটি “ডিস্ক স্বাস্থ্য প্রতিবেদন” তৈরি করুন: `lsblk -f`, `df -h`, `df -i` ও শীর্ষ ১০ বড় ডিরেক্টরির তালিকা সংগ্রহ করুন। প্রতিটি filesystem-এর ব্যবহার শতাংশ লিখুন এবং ৮০% ছাড়ানো filesystem থাকলে তার কারণ ও করণীয় লিখুন। কোনো ফাইল মুছবেন না।

সারসংক্ষেপ

ডিস্ক শুধু block-এর সমষ্টি; filesystem সেই block-কে ফাইলে সাজায়, inode metadata ধারণ করে, আর mount সেই filesystem-কে একটিমাত্র ডিরেক্টরি গাছে যুক্ত করে। `df` filesystem-স্তরের, `du` ফাইল-স্তরের ছবি দেয়; দুটির অমিল নিজেই একটি রোগনির্ণয়ের সূত্র। fstab-এ UUID ব্যবহার ও `mount -a` দিয়ে পরীক্ষা নিরাপদ প্রশাসনের অভ্যাস।

Revision Checklist — পুনরালোচনার তালিকা

- ☐ block, partition, filesystem, mount-এর পার্থক্য বলতে পারি
- ☐ inode কী ও কেন তা শেষ হতে পারে, ব্যাখ্যা করতে পারি
- ☐ `df -h`, `df -i`, `du -sh` দিয়ে তদন্ত করতে পারি
- ☐ fstab-এর ব্যাকআপ নিয়ে নিরাপদে পরীক্ষা করতে পারি
- ☐ ডিস্ক পূর্ণ সমস্যার ধাপে ধাপে কারণ খুঁজতে পারি

Chapter 17: CPU, RAM and System Resource Monitoring — CPU, RAM ও সিস্টেম রিসোর্স পর্যবেক্ষণ

শুরুর কথা: “সার্ভার ধীর” — কিন্তু কেন?

একজন ব্যবহারকারী অভিযোগ করলেন, “ওয়েবসাইট ধীর।” জুনিয়র অ্যাডমিন সঙ্গে সঙ্গে RAM বাড়ানোর প্রস্তাব দিলেন। কিন্তু পরীক্ষা করে দেখা গেল RAM প্রায় ফাঁকা, CPU-ও শান্ত — সমস্যা ছিল ডিস্ক I/O-তে: একটি ব্যাকআপ প্রক্রিয়া ডিস্ককে পুরোপুরি ব্যস্ত রেখেছিল। ভুল নির্ণয়ে টাকা ও সময় দুটোই নষ্ট হতো। “ধীর” কোনো রোগনির্ণয় নয়, এটি একটি উপসর্গ। পেশাদার অ্যাডমিন প্রথমে জিজ্ঞেস করেন: CPU ব্যস্ত, নাকি RAM চাপে, নাকি ডিস্ক অপেক্ষা করছে, নাকি নেটওয়ার্ক? এই অধ্যায়ে আপনি শিখবেন প্রতিটি রিসোর্স কীভাবে মাপতে হয়, সংখ্যাগুলোর আসল অর্থ কী, এবং কোন সংখ্যা দেখে কোন সিদ্ধান্ত নিতে হয়।

Learning Objectives — শেখার উদ্দেশ্য

- Load average, CPU ব্যবহার, memory চাপ ও I/O অপেক্ষার অর্থ ব্যাখ্যা করতে পারবেন।
- `uptime`, `free`, `vmstat`, `top`, `htop` ও `iostat` দিয়ে সিস্টেম পর্যবেক্ষণ করতে পারবেন।
- CPU saturation, memory pressure ও swap ব্যবহারের মধ্যে পার্থক্য করতে পারবেন।
- কোন প্রক্রিয়া রিসোর্স খাচ্ছে তা শনাক্ত করে নিরাপদ পদক্ষেপ নিতে পারবেন।
- একটি সুশৃঙ্খল রিসোর্স তদন্ত পদ্ধতি অনুসরণ করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ২-এর CPU ও RAM গঠন, অধ্যায় ৫-এর প্রক্রিয়া ও scheduling এবং অধ্যায় ১৩-এর প্রক্রিয়া ব্যবস্থাপনা জানা থাকতে হবে। `top` ও `ps`-এর মৌলিক ব্যবহার অধ্যায় ১৩ থেকে পরিচিত ধরে নেওয়া হয়েছে।

প্রথম নীতি থেকে: রিসোর্স মানে কী?

কম্পিউটারের চারটি প্রধান রিসোর্স হলো CPU, RAM, ডিস্ক I/O ও নেটওয়ার্ক। প্রতিটি কাজ কোনো না কোনো রিসোর্সের জন্য অপেক্ষা করে। CPU হলো শ্রমিক, RAM হলো কাজের টেবিল, ডিস্ক হলো গুদাম, আর I/O হলো গুদাম থেকে মাল আনা-নেওয়ার রাস্তা। শ্রমিক বসে থাকলে সমস্যা শ্রমিকে নয় — হয়তো মাল আসছে না। তাই একটি সংখ্যা দেখে সিদ্ধান্ত নেওয়া যায় না; চারটি রিসোর্স একসঙ্গে দেখতে হয়। আরেকটি মৌলিক সত্য: Linux ফাঁকা RAM-কে “অপচয়” মনে করে না। অব্যবহৃত RAM দিয়ে সে ডিস্কের তথ্য cache করে রাখে, যাতে পরের বার দ্রুত পাওয়া যায়। তাই `free`-তে “free” কম দেখালেই আতঙ্কিত হবেন না — দেখতে হবে “available” কত, কারণ cache প্রয়োজনে ছেড়ে দেওয়া যায়।

ভেতরের গঠন: load average, CPU সময় ও memory

Load average হলো নির্দিষ্ট সময়ে গড়ে কতগুলো প্রক্রিয়া CPU-র জন্য অপেক্ষা করছিল বা চলছিল তার সংখ্যা — ১, ৫ ও ১৫ মিনিটের গড়। একে CPU কোরের সংখ্যার সঙ্গে তুলনা করতে হয়। ৪ কোরের সার্ভারে load ৪ মানে CPU পুরোপুরি ব্যবহৃত হচ্ছে; load ৮ মানে গড়ে দ্বিগুণ কাজ অপেক্ষা করছে। ১ কোরের সার্ভারে load ৪ মানে গুরুতর চাপ। শুধু load দেখে CPU-ই দায়ী বলা যায় না — ডিস্কের জন্য অপেক্ষমান প্রক্রিয়াও load বাড়ায়।

CPU সময় ভাগ হয়: user (অ্যাপ্লিকেশন), system (kernel), idle (ফাঁকা), iowait (ডিস্কের অপেক্ষা) ও steal (ভার্চুয়াল মেশিনে হোস্ট অন্য অতিথিকে CPU দিয়েছে)। iowait বেশি মানে CPU বসে আছে ডিস্কের আশায় — তখন CPU বাড়িয়ে লাভ নেই, ডিস্কের সমস্যা ঠিক করতে হবে। steal বেশি মানে আপনার VPS-এর প্রতিবেশী CPU নিচ্ছে বা হোস্ট অতিরিক্ত বিক্রি করেছে — এটি ক্লাউডে একটি বাস্তব ও গুরুত্বপূর্ণ সংকেত।

Memory-তে তিনটি অবস্থা মনে রাখুন। Used হলো প্রোগ্রাম ব্যবহার করছে। Buff/cache হলো Linux ডিস্কের তথ্য ধরে রেখেছে — এটি প্রয়োজনে মুক্ত করা যায়। Available হলো নতুন প্রোগ্রামের জন্য আসলে কত পাওয়া যাবে — এটিই দেখার সংখ্যা। Swap হলো RAM পূর্ণ হলে ডিস্কে সরিয়ে রাখা memory। অল্প swap ব্যবহার স্বাভাবিক হতে পারে, কিন্তু ক্রমাগত swap in/out (thrashing) মানে RAM গুরুতর কম এবং সিস্টেম ডিস্কে বারবার তথ্য আনা-নেওয়া করছে — তখন সিস্টেম অত্যন্ত ধীর হয়।

চিত্র ১৭.১ — চার রিসোর্স ও উপসর্গের মানচিত্র

উপসর্গ	সম্ভাব্য কারণ	প্রথম পরীক্ষা
সবকিছু ধীর, load বেশি	CPU saturation	`top` — কোন প্রক্রিয়া %CPU বেশি নিচ্ছে
নতুন প্রোগ্রাম চালু হচ্ছে না	Memory pressure / OOM	`free -h` — available কত; `dmesg` -এ OOM
CPU ফাঁকা, তবুও ধীর	ডিস্ক I/O অপেক্ষা	`iostat -x 1`, `vmstat` -এ wa
মাঝে মাঝে হঠাৎ ধীর (VPS)	CPU steal	`top` -এ %st, `vmstat` -এ st

চিত্র ১৭.২ — RAM-এর ভাগ: used, cache ও available

মোট RAM ৮ GB ধরা যাক: প্রোগ্রাম ব্যবহার করছে ৩ GB (used) + cache ৪ GB + সত্যিকারের ফাঁকা ১ GB = মোট ৮ GB। নতুন প্রোগ্রামের জন্য available ≈ ফাঁকা ১ GB + ছেড়ে দেওয়া যায় এমন cache ≈ ৫ GB। তাই `free` -তে “free ১ GB” দেখে ভয় পাবেন না — “available ৫ GB” দেখুন। Cache বেশি থাকা ভালো লক্ষণ, খারাপ নয়।

পরিভাষা

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Load average	১/৫/১৫ মিনিটে গড়ে CPU-র জন্য অপেক্ষমান বা চলমান প্রক্রিয়ার সংখ্যা	১৭
CPU saturation	CPU চাহিদা ক্ষমতার চেয়ে বেশি হওয়া	১৭
iowait	CPU ডিস্ক I/O-র জন্য অপেক্ষা করছে এমন সময়ের হার	১৭
CPU steal	ভার্চুয়াল মেশিনে হোস্ট CPU অন্য অতিথিকে দেওয়ায় হারানো সময়	১৭
Memory pressure	RAM চাহিদা সরবরাহের কাছাকাছি বা বেশি হওয়া	১৭
Swap / Thrashing	RAM-এর তথ্য ডিস্কে সরানো; বারবার আনা-নেওয়ায় চরম ধীরগতি	১৭
Cache	দ্রুত পাওয়ার জন্য RAM-এ ধরে রাখা ডিস্কের তথ্য; প্রয়োজনে মুক্ত হয়	১৭

OOM killer	RAM ফুরিয়ে গেলে kernel যে প্রক্রিয়াকে বন্ধ করে দেয়	১৭

বাস্তব উদাহরণ

চট্টগ্রামের একটি ই-কমার্স সাইটে দুপুরে অর্ডার বাড়লেই সাইট ধীর হয়ে যেত। অ্যাডমিন `top` দেখে ভাবলেন CPU কম। আসলে `vmstat`-এ দেখা গেল swap ক্রমাগত ব্যবহৃত হচ্ছে — ডেটাবেসের cache RAM-এ ধরছিল না। সমাধান ছিল RAM বাড়ানো নয়, প্রথমে ডেটাবেসের অপ্রয়োজনীয় বড় query ঠিক করা ও cache সেটিং সমন্বয় করা; পরে RAM বাড়ানো। শিক্ষা: রিসোর্স বাড়ানোর আগে জিজ্ঞেস করুন — চাহিদা কি আসলেই বেশি, নাকি কোথাও অপচয় হচ্ছে?

ব্যবহারিক কমান্ড

```
uptime
# load average (১/৫/১৫ মিনিট) ও কতক্ষণ ধরে সিস্টেম চলছে তা দেখায়।

free -h
# RAM ও swap: total, used, free, shared, buff/cache, available। available-ই আসল সংখ্যা।

vmstat 1 5
# ১ সেকেন্ড পরপর ৫ বার: CPU (us, sy, id, wa, st), memory, swap in/out (si/so), I/O।

top
# চলমান প্রক্রিয়া, CPU ও memory ব্যবহার; q দিয়ে বের হোন। M চাপলে memory অনুযায়ী সাজে।

htop
# top-এর রঙিন, সহজ সংস্করণ; না থাকলে: sudo apt install htop

ps aux --sort=-%cpu | head -10
# সবচেয়ে বেশি CPU ব্যবহারকারী ১০টি প্রক্রিয়া।

ps aux --sort=-%mem | head -10
# সবচেয়ে বেশি memory ব্যবহারকারী ১০টি প্রক্রিয়া।

iostat -x 1 3
# প্রতিটি ডিস্কের ব্যবহার, অপেক্ষা ও ব্যস্ততা। sysstat প্যাকেজের অংশ; না থাকলে: sudo apt install sysstat

sudo dmesg | grep -i -E 'oom|out of memory' | tail -5
# kernel RAM ফুরিয়ে কোনো প্রক্রিয়া বন্ধ করেছে কি না তা দেখায়।
```

`top`-এর উপরের অংশে %Cpu(s) লাইনে us, sy, id, wa, st দেখুন। wa বেশি হলে ডিস্ক, st বেশি হলে VPS হোস্ট, us বেশি হলে অ্যাপ্লিকেশন — প্রতিটির সমাধান আলাদা। `htop` না থাকলে `top`-ই যথেষ্ট; টুলের চেয়ে ব্যাখ্যা বোঝা গুরুত্বপূর্ণ।

> **CONCEPT:** Load average-কে কোর সংখ্যা দিয়ে ভাগ করুন। ৪ কোরে load ৪ = ১০০% ব্যস্ত, কিন্তু স্বাভাবিক। ১ কোরে load ৪ = ৪০০% চাপ। সংখ্যাটি একা কিছু বলে না — প্রসঙ্গই অর্থ দেয়।

> **WARNING:** বেশি CPU বা memory খাচ্ছে দেখেই কোনো প্রক্রিয়া `kill -9` দিয়ে বন্ধ করবেন না। প্রথমে জানুন সেটি কী, কেন চলছে, বন্ধ করলে কী ভাঙবে। ডেটাবেস বা গুরুত্বপূর্ণ সার্ভিস জোর করে বন্ধ করলে তথ্য নষ্ট হতে পারে। আগে `systemctl status` ও লগ দেখুন (অধ্যায় ১৮)।

হাতে-কলমে অনুশীলন: ধীর সিস্টেমের তদন্ত

1. `uptime` ও `nproc` চালান। load average-কে কোর সংখ্যার সঙ্গে তুলনা করে লিখুন সিস্টেম চাপে আছে কি না।
2. `free -h` চালিয়ে total, used, buff/cache ও available লিখুন। available কম হলে কেন, ভাবুন।
3. `vmstat 1 5` চালিয়ে us, sy, id, wa, st কলামগুলো পড়ুন। wa বেশি কি না নোট করুন।
4. `ps aux --sort=-%cpu | head -10` ও `ps aux --sort=-%mem | head -10` দিয়ে শীর্ষ প্রক্রিয়াগুলো চিহ্নিত করুন — সেগুলো কী, জানেন কি?
5. `top` খুলে ৩০ সেকেন্ড দেখুন, তারপর `q` দিয়ে বের হোন। সবচেয়ে বেশি CPU নেওয়া প্রক্রিয়ার নাম ও %CPU লিখুন।
6. একটি এক লাইনের সিদ্ধান্ত লিখুন: “আমার সিস্টেম এখন CPU / RAM / I/O — কোনটিতে চাপে, নাকি কোনোটিতেই নয়, এবং তার প্রমাণ কী।”

উদাহরণ আউটপুট (illustrative):

```
$ uptime
 10:15:00 up 3 days,  2 users,  load average: 0.45, 0.60, 0.55

$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.8Gi       2.1Gi       1.2Gi       0.3Gi       4.5Gi       5.0Gi
Swap:          2.0Gi       0.0Gi       2.0Gi
```

ব্যাখ্যা: ৪ কোরের মেশিনে load ০.৪৫ মানে CPU প্রায় ফাঁকা। available ৫.০ GB মানে RAM-এ প্রচুর জায়গা আছে, যদিও free মাত্র ১.২ GB — বাকিটা cache, যা ভালো লক্ষণ। Swap ০ মানে কোনো memory চাপ নেই।

সাধারণ ভুল ও সমস্যা সমাধান

- **free কম দেখে আতঙ্ক:** available দেখুন, free নয়। cache বেশি থাকা স্বাভাবিক ও উপকারী।
- **load বেশি মানেই CPU কম — এই ধারণা:** `vmstat`-এ wa দেখুন; ডিস্কের অপেক্ষাও load বাড়ায়।
- **এক মুহূর্তের সংখ্যা দেখে সিদ্ধান্ত:** `vmstat 1 5` বা `top`-এ কিছুক্ষণ দেখুন; এক সেকেন্ডের spike স্বাভাবিক হতে পারে।
- **OOM killer-এ প্রক্রিয়া মারা যাওয়া:** `dmesg`-এ “Out of memory” খুঁজুন; কোন প্রক্রিয়া বন্ধ হয়েছে ও কেন RAM ফুরিয়েছিল তা বুঝুন, শুধু restart করলেই হবে না।
- **htop/iostat নেই:** Ubuntu 24.04-এ `sudo apt install htop sysstat` দিয়ে ইনস্টল করুন; তবে `top` ও `vmstat` প্রায় সবসময় থাকে।

নিরাপত্তা বিবেচনা

রিসোর্স পর্যবেক্ষণ নিরাপত্তারও অংশ। হঠাৎ CPU ১০০% ব্যবহার করা অজানা প্রক্রিয়া cryptomining ম্যালওয়্যারের লক্ষণ হতে পারে। অজানা প্রক্রিয়া দেখলে তার নাম, পথ (`ps -o pid,cmd -p PID`) ও কে চালাচ্ছে তা যাচাই করুন। পর্যবেক্ষণের তথ্য (যেমন `top` স্ক্রিনশট) শেয়ার করার সময় হোস্টনেম, IP বা ব্যবহারকারীর নাম অপ্রয়োজনে প্রকাশ করবেন না।

শিল্পের সেরা চর্চা

- একটি সংখ্যা নয়, CPU + RAM + I/O + load — চারটি একসঙ্গে দেখুন।
- স্বাভাবিক অবস্থার একটি baseline লিখে রাখুন (সকালে load কত, available কত) — অস্বাভাবিক ধরতে baseline-ই ভরসা।
- সমস্যার সময় স্ক্রিনশট নয়, কমান্ডের আউটপুট ও সময় সংরক্ষণ করুন।
- রিসোর্স বাড়ানোর আগে অপচয় খুঁজুন: অপ্রয়োজনীয় সার্ভিস, ভুল query, অসীম লুপ।
- নিয়মিত পর্যবেক্ষণকে অভ্যাস করুন; সমস্যা হলে তবেই দেখা — এই অভ্যাস দেরি করায়।

Knowledge Check — জ্ঞান যাচাই

1. ৪ কোরের সার্ভারে load average ৪.০ মানে কী? A) সিস্টেম অচল B) CPU গড়ে পুরোপুরি ব্যস্ত, কিন্তু অতিরিক্ত চাপ নেই C) RAM শেষ D) ডিস্ক পূর্ণ
2. `free -h`-তে নতুন প্রোগ্রামের জন্য আসলে কত RAM পাওয়া যাবে তা কোন কলাম বলে? A) free B) used C) available D) shared
3. iowait বেশি হলে সবচেয়ে যুক্তিসঙ্গত ব্যাখ্যা কোনটি? A) CPU দুর্বল B) CPU ডিস্ক I/O-র অপেক্ষায় বসে আছে C) RAM শেষ D) নেটওয়ার্ক বন্ধ
4. Swap ক্রমাগত in/out হওয়া (thrashing) কী নির্দেশ করে? A) CPU দ্রুত B) RAM গুরুতর চাপে, ডিস্কে বারবার আনা-নেওয়া হচ্ছে C) ডিস্ক ফাঁকা D) cache ভালো কাজ করছে
5. VPS-এ CPU steal বেশি দেখা গেলে কী বোঝায়? A) নিজের প্রোগ্রামে ভুল B) হোস্ট CPU অন্য অতিথিকে দিয়েছে C) RAM নষ্ট D) ডিস্ক নষ্ট

Practical Assignment — ব্যবহারিক কাজ

তিন দিন ধরে দিনে দুইবার (সকাল ও সন্ধ্যা) `uptime`, `free -h` ও `vmstat 1 3`-এর ফল একটি ফাইলে লিখে রাখুন। শেষে একটি ছোট প্রতিবেদন লিখুন: আপনার মেশিনের স্বাভাবিক load ও available RAM কত, কখন সবচেয়ে বেশি চাপ দেখা গেল এবং তার সম্ভাব্য কারণ কী। কোনো সেটিং বদলাবেন না — শুধু পর্যবেক্ষণ করুন।

সারসংক্ষেপ

“ধীর” একটি উপসর্গ, রোগনির্ণয় নয়। Load average-কে কোর সংখ্যার সঙ্গে, RAM-কে available দিয়ে, CPU সময়কে us/sy/wa/st ভাগে ভাগ করে পড়তে হয়। Cache বেশি থাকা ভালো, swap ক্রমাগত ব্যবহার খারাপ। সিদ্ধান্ত নেওয়ার আগে চারটি রিসোর্স একসঙ্গে দেখুন এবং স্বাভাবিক অবস্থার baseline-এর সঙ্গে তুলনা করুন।

Revision Checklist — পুনরালোচনার তালিকা

- ☐ load average-কে কোর সংখ্যার সঙ্গে তুলনা করে ব্যাখ্যা করতে পারি
- ☐ free, available ও cache-এর পার্থক্য বলতে পারি
- ☐ `vmstat`-এ wa ও st পড়তে পারি
- ☐ শীর্ষ CPU/memory প্রক্রিয়া খুঁজে বের করতে পারি
- ☐ OOM-এর লক্ষণ `dmesg`-এ খুঁজতে পারি

Chapter 18: Logs, Diagnostics and Troubleshooting — লগ, রোগনির্ণয় ও সমস্যা সমাধান

শুরুর কথা: সার্ভার কথা বলে — লগ তার ভাষা

একটি সার্ভিস হঠাৎ বন্ধ হয়ে গেল। অ্যাডমিন restart করলেন — আবার বন্ধ হলো। তিনি আবার restart করলেন — আবার বন্ধ। তৃতীয়বারে কেউ জিজ্ঞেস করল, "লগে কী লেখা আছে?" লগে লেখা ছিল: configuration ফাইলে একটি ভুল পথ, তাই সার্ভিস চালু হয়েই বন্ধ হয়ে যাচ্ছিল। Restart সমস্যা ঢাকছিল, লগ সমস্যা দেখাচ্ছিল। পেশাদার সমস্যা সমাধানের মূলমন্ত্র: অনুমান নয়, প্রমাণ। আর Linux-এ প্রমাণের প্রধান উৎস হলো লগ। এই অধ্যায়ে আপনি শিখবেন Linux কোথায় কী লেখে, কীভাবে পড়তে হয়, ভুল বার্তা কীভাবে বুঝতে হয় এবং একটি সুশৃঙ্খল পদ্ধতিতে যেকোনো সমস্যার কারণ খুঁজে বের করতে হয়।

Learning Objectives — শেখার উদ্দেশ্য

- systemd journal ও `/var/log`-এর গঠন ব্যাখ্যা করতে পারবেন।
- `journalctl` দিয়ে সময়, সার্ভিস ও গুরুত্ব অনুযায়ী লগ খুঁজতে পারবেন।
- `dmesg` দিয়ে kernel বার্তা পড়তে পারবেন।
- ভুল বার্তা থেকে কারণ আলাদা করে সেবা-ব্যর্থতা, ডিস্ক পূর্ণ ও উচ্চ CPU/memory সমস্যা তদন্ত করতে পারবেন।
- একটি লিখিত troubleshooting workflow অনুসরণ করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ১৫-এর systemd ও সার্ভিস, অধ্যায় ১৬-এর ডিস্ক এবং অধ্যায় ১৭-এর রিসোর্স পর্যবেক্ষণ জানা থাকতে হবে। `grep`, `tail` ও `less`-এর ব্যবহার অধ্যায় ১০ থেকে পরিচিত ধরে নেওয়া হয়েছে।

প্রথম নীতি থেকে: লগ কেন থাকে?

কম্পিউটার ভুলে যায় না — সে লিখে রাখে। প্রতিটি গুরুত্বপূর্ণ ঘটনা — সার্ভিস চালু হওয়া, লগইন চেষ্টা, ভুল, সতর্কতা — কোথাও না কোথাও লেখা হয়। লগ হলো সিস্টেমের ডায়েরি। ডায়েরি ছাড়া আপনি জানবেন না কখন কী হয়েছিল, কী বদলেছিল, কে লগইন করেছিল। সমস্যা সমাধানে সবচেয়ে মূল্যবান প্রশ্ন হলো: "শেষবার যখন ঠিক ছিল, তখন থেকে কী বদলেছে?" লগ সেই প্রশ্নের উত্তর দেয় — সময়সহ।

Linux-এ দুটি প্রধান লগ ব্যবস্থা আছে। পুরোনো ব্যবস্থায় বিভিন্ন সার্ভিস `/var/log`-এর ভেতরে আলাদা ফাইলে লেখে — যেমন `auth.log` (লগইন), `syslog` (সাধারণ সিস্টেম বার্তা)। আধুনিক systemd ব্যবস্থায় journal নামের একটি কেন্দ্রীয় সংগ্রহ আছে, যা `journalctl` দিয়ে পড়া যায়। Ubuntu 24.04-এ দুটোই আছে — systemd সার্ভিসগুলো journal-এ লেখে, আর অনেক তথ্য `/var/log`-এও পাওয়া যায়। কোনটি কোথায়, তা জানা থাকলে তদন্ত দ্রুত হয়।

ভেতরের গঠন: journal, syslog ও kernel বার্তা

systemd journal প্রতিটি বার্তার সঙ্গে সময়, সার্ভিসের নাম, প্রক্রিয়ার ID ও গুরুত্ব (priority) সংরক্ষণ করে। গুরুত্বের স্তরগুলো হলো: emerg, alert, crit, err, warning, notice, info, debug। `journalctl -p err` শুধু ভুল ও তার চেয়ে গুরুতর বার্তা দেখায় — সমস্যার সময় এটিই প্রথমে দেখা উচিত। `journalctl -u nginx.service` শুধু একটি সার্ভিসের

লগ দেখায়, `journalctl --since "1 hour ago"` গত এক ঘণ্টার লগ দেখায়। সময় দিয়ে ছাঁকা সবচেয়ে শক্তিশালী কৌশল — সমস্যা কখন শুরু হয়েছিল তা জানলে তার আশেপাশের লগই যথেষ্ট।

Kernel-এর নিজস্ব বার্তা থাকে ring buffer-এ, যা `dmesg` দিয়ে পড়া যায়। হার্ডওয়্যার শনাক্ত হওয়া, ডিস্কের ভুল, RAM-এর সমস্যা, OOM killer-এর কাজ — এসব এখানে লেখা হয়। `dmesg`-এর সময় মানুষের পড়ার মতো নয় বলে `dmesg -T` ব্যবহার করুন। আরেকটি গুরুত্বপূর্ণ ফাইল `/var/log/auth.log` — কে কখন SSH-এ লগইন করল, কে ব্যর্থ হলো, কে `sudo` ব্যবহার করল — সব এখানে থাকে। নিরাপত্তা তদন্তে এটি প্রথম দেখার ফাইল।

চিত্র ১৮.১ — Linux লগ ব্যবস্থার মানচিত্র

উৎস	কোথায় পড়বেন	কী পাবেন
systemd সার্ভিস	`journalctl -u নাম.service`	সার্ভিসের চালু, বন্ধ, ভুল
সাধারণ সিস্টেম বার্তা	`/var/log/syslog`	বিভিন্ন সার্ভিসের বার্তা
লগইন ও sudo	`/var/log/auth.log`	কে লগইন করল, কে ব্যর্থ হলো
Kernel / হার্ডওয়্যার	`dmesg -T`	ডিস্ক, RAM, ডিভাইসের ভুল
ওয়েব সার্ভার (nginx)	`/var/log/nginx/error.log`	ওয়েব সার্ভারের ভুল

চিত্র ১৮.২ — সুশৃঙ্খল troubleshooting workflow

১) সমস্যাটি ঠিক কী, কখন শুরু হলো, কাকে প্রভাবিত করছে — লিখুন → ২) সাম্প্রতিক পরিবর্তন কী হয়েছে — জিজ্ঞেস করুন → ৩) লগ দেখুন (`journalctl -p err`, সংশ্লিষ্ট সার্ভিস, `dmesg`) → ৪) রিসোর্স দেখুন (`df -h`, `free -h`, `top`) → ৫) একটি অনুমান করুন → ৬) সবচেয়ে ছোট ও নিরাপদ পরীক্ষা করুন → ৭) ফল লিখুন; ঠিক না হলে পরের অনুমানে যান → ৮) সমাধান ও কারণ নথিভুক্ত করুন। একসঙ্গে অনেক কিছু বদলাবেন না — তাহলে কোনটি কাজ করল তা জানা যাবে না।

পরিভাষা

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Journal	systemd-এর কেন্দ্রীয় লগ সংগ্রহ	১৮
journalctl	journal পড়া ও ছাঁকার কমান্ড	১৮
Syslog	সাধারণ সিস্টেম লগ ব্যবস্থা ও /var/log/syslog ফাইল	১৮
dmesg	kernel ring buffer-এর বার্তা দেখার কমান্ড	১৮
Log level / Priority	বার্তার গুরুত্ব: err, warning, info, debug ইত্যাদি	১৮
Log rotation	পুরোনো লগ সংরক্ষণ ও নতুন লগ শুরু করা (logrotate)	১৮
Root cause	সমস্যার মূল কারণ; উপসর্গ নয়	১৮

Reproduce	সমস্যাটি আবার ঘটিয়ে দেখা, যাতে কারণ নিশ্চিত করা যায়	১৮

বাস্তব উদাহরণ

রাজশাহীর একটি কলেজের ওয়েবসাইটে “502 Bad Gateway” দেখা দিল। অ্যাডমিন প্রথমে nginx restart করলেন — কাজ হলো না। তারপর `journalctl -u nginx.service` দেখে বুঝলেন nginx চলছে, কিন্তু `journalctl -u php8.3-fpm.service`-এ দেখা গেল PHP-FPM বন্ধ হয়ে আছে — কারণ তার configuration-এ ভুল ছিল। আসল সার্ভিসটি ঠিক করতেই সাইট ফিরে এলো। শিক্ষা: ভুল বার্তাটি কোন সার্ভিসের, তা না দেখে শুধু সামনের সার্ভিস restart করা সময় নষ্ট করে।

ব্যবহারিক কমান্ড

```
sudo journalctl -p err -b
# এই বুট থেকে শুধু ভুল (err) ও তার চেয়ে গুরুতর বার্তা। সমস্যার সময় প্রথম কমান্ড।

sudo journalctl -u ssh.service --since "1 hour ago"
# গত এক ঘণ্টায় SSH সার্ভিসের লগ।

sudo journalctl -u nginx.service -n 50 --no-pager
# nginx-এর শেষ ৫০ লাইন, pager ছাড়া।

sudo journalctl -f
# লাইভ লগ দেখা; Ctrl+C দিয়ে বের হোন। সার্ভিস restart করে দেখার সময় উপকারী।

sudo tail -n 100 /var/log/syslog
# সাধারণ সিস্টেম লগের শেষ ১০০ লাইন।

sudo tail -n 100 /var/log/auth.log
# লগইন ও sudo সংক্রান্ত শেষ ১০০ লাইন।

sudo grep -i "failed\|error" /var/log/auth.log | tail -20
# auth লগে ব্যর্থ লগইন বা ভুল খোঁজা।

sudo dmesg -T | tail -30
# kernel বার্তার শেষ ৩০ লাইন, মানুষের পড়ার মতো সময়সহ।

sudo systemctl status nginx.service
# সার্ভিসের অবস্থা ও শেষ কয়েক লাইন লগ একসঙ্গে দেখায়।
```

`journalctl`-এ `-b` মানে বর্তমান বুট, `-n` মানে শেষ কয়টি লাইন, `--since` মানে কখন থেকে। `grep`-এর সঙ্গে মিলিয়ে ব্যবহার করলে হাজার লাইনের ভেতর থেকে প্রয়োজনীয় লাইনটি পাওয়া যায়। লগ পড়ার সময় প্রথমে সময় দেখুন — সমস্যা শুরুর সময়ের আশেপাশে কী লেখা আছে, সেটিই সূত্র।

REAL WORLD: একটি ভুল বার্তা প্রায়ই তিনটি অংশে ভাগ করা যায়: কী ঘটেছে, কোথায় ঘটেছে, কেন ঘটেছে। যেমন “Failed to start nginx: bind() to 0.0.0.0:80 failed: Address already in use” — কী: nginx চালু হয়নি; কোথায়: পোর্ট ৮০; কেন: অন্য প্রোগ্রাম ইতিমধ্যে পোর্ট ৮০ ব্যবহার করছে। বার্তাটি পুরো পড়ুন; শুধু প্রথম শব্দ দেখে সিদ্ধান্ত নেবেন না।

TROUBLESHOOTING: সার্ভিস বন্ধ হলে ক্রম: `systemctl status নাম` → `journalctl -u নাম -n 50` → configuration ফাইল পরীক্ষা (`nginx -t`, `sshd -t` ইত্যাদি) → `df -h` ও `free -h` দিয়ে রিসোর্স দেখা → `dmesg`-এ kernel ভুল আছে কি না দেখা।

হাতে-কলমে অনুশীলন: একটি ব্যর্থ সার্ভিসের তদন্ত

নিরাপদ অনুশীলন: একটি পরীক্ষামূলক সার্ভিসের ভুল খুঁজে বের করা। সিস্টেমের কোনো গুরুত্বপূর্ণ সার্ভিস বন্ধ করবেন না।

1. `sudo systemctl status ssh.service` চালিয়ে SSH সার্ভিস চলছে কি না ও শেষ লগ লাইনগুলো পড়ুন।
2. `sudo journalctl -u ssh.service -n 30 --no-pager` চালিয়ে শেষ ৩০ লাইন পড়ুন।
3. `sudo journalctl -p err -b --no-pager | head -20` দিয়ে এই বুটের ভুলগুলো দেখুন — কোনো ভুল আছে কি?
4. `sudo dmesg -T | tail -20` দিয়ে kernel বার্তায় কোনো সতর্কতা আছে কি না দেখুন।
5. `sudo tail -n 30 /var/log/auth.log` দিয়ে সাম্প্রতিক লগইন চেষ্টাগুলো দেখুন।
6. একটি সংক্ষিপ্ত তদন্ত নোট লিখুন: কোন সার্ভিস পরীক্ষা করলেন, কী পেলেন, কোনো ভুল পেলে তার সম্ভাব্য কারণ কী।

উদাহরণ আউটপুট (illustrative):

```
$ sudo systemctl status ssh.service
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled)
     Active: active (running) since Sat 2026-11-07 09:00:00 +06; 1 day ago
   Main PID: 812 (sshd)
```

ব্যাখ্যা: `Active: active (running)` মানে সার্ভিস চলছে। `enabled` মানে বুটের সময় স্বয়ংক্রিয়ভাবে চালু হবে। যদি `failed` দেখাত, তাহলে পরের ধাপে `journalctl -u ssh.service` দেখতে হতো।

সাধারণ সমস্যার তদন্ত-ছক

- **সার্ভিস চালু হচ্ছে না:** `systemctl status` ও `journalctl -u` দেখুন; configuration পরীক্ষার কমান্ড চালান; পোর্ট ইতিমধ্যে ব্যবহৃত হচ্ছে কি না `ss -tlnp` দিয়ে দেখুন।
- **ডিস্ক পূর্ণ:** অধ্যায় ১৬-এর পদ্ধতি — `df -h`, তারপর `du` দিয়ে বড় ডিরেক্টরি, তারপর বড় ফাইল। লগ ঘোরানো হচ্ছে কি না দেখুন।
- **উচ্চ CPU:** অধ্যায় ১৭-এর পদ্ধতি — `top` দিয়ে কোন প্রক্রিয়া, তারপর তার লগ দেখুন; লগে বারবার একই ভুল লেখা হচ্ছে কি না দেখুন (ভুলের লুপ CPU খায়)।
- **SSH লগইন হচ্ছে না:** `auth.log` দেখুন — ভুল পাসওয়ার্ড, ভুল key, নাকি সার্ভিস বন্ধ? `sshd -t` দিয়ে configuration পরীক্ষা করুন।
- **বুট হচ্ছে না:** GRUB-এ recovery mode, তারপর `journalctl -b -1` দিয়ে আগের বুটের লগ দেখুন।

সাধারণ ভুল ও সমস্যা সমাধান

- **শুধু restart করে সমস্যা "ঠিক" করা:** restart উপসর্গ ঢাকে, কারণ ঠিক করে না। লগ না দেখে restart করলে সমস্যা ফিরে আসবে।

- **লগ না পড়ে ইন্টারনেটে খোঁজা:** ভুল বার্তাটি না পড়ে সাধারণভাবে খুঁজলে ভুল সমাধান পাবেন। প্রথমে বার্তাটি পুরো পড়ুন।
- **একসঙ্গে অনেক কিছু বদলানো:** কোন পরিবর্তনে কাজ হলো তা জানা যায় না। একবারে একটি পরিবর্তন করুন, ফল লিখুন।
- **লগ মুছে ফেলা:** তদন্তের আগে লগ মুছবেন না — প্রমাণ নষ্ট হয়। জায়গা দরকার হলে পুরোনো লগ archive করুন।
- **সময় অঞ্চল না মেলানো:** লগের সময় ও আপনার ঘড়ির সময় না মিললে ভুল সময়ের লগ দেখবেন। `timedatectl` দিয়ে সময় যাচাই করুন।

নিরাপত্তা বিবেচনা

লগে সংবেদনশীল তথ্য থাকতে পারে — IP ঠিকানা, ব্যবহারকারীর নাম, ভুল করে লেখা পাসওয়ার্ড। লগ শেয়ার করার আগে সংবেদনশীল অংশ মুছে ফেলুন। `auth.log` নিয়মিত দেখুন: একই IP থেকে বারবার ব্যর্থ লগইন brute-force চেষ্টার লক্ষণ (অধ্যায় ১৯)। লগের অনুমতি যেন এমন না হয় যে যেকেউ পড়তে পারে — `/var/log`-এর ফাইলগুলোর অনুমতি অকারণে বদলাবেন না।

শিল্পের সেরা চর্চা

- সমস্যা, সময়, লক্ষণ, পরীক্ষা ও ফল — সব লিখে রাখুন। লিখিত তদন্তই পেশাদারিত্ব।
- “কী বদলেছে?” — এই প্রশ্নটি প্রথমে করুন। বেশিরভাগ সমস্যার পেছনে সাম্প্রতিক পরিবর্তন থাকে।
- অনুমান করুন, তারপর সবচেয়ে ছোট পরীক্ষা দিয়ে যাচাই করুন।
- সমাধানের পর মূল কারণ (root cause) লিখুন, যাতে একই সমস্যা আবার হলে দ্রুত ধরা যায়।
- গুরুত্বপূর্ণ সার্ভিসের লগ নিয়মিত দেখার অভ্যাস করুন, শুধু সমস্যা হলে নয়।

Knowledge Check — জ্ঞান যাচাই

1. একটি সার্ভিসের শুধু ভুল বার্তা দেখতে কোন কমান্ডটি সঠিক? A) `journalctl -u nginx.service -p err` B) `journalctl --all` C) `tail /var/log/syslog` D) `dmesg -T`
2. Kernel বার্তা (হার্ডওয়্যার, OOM) কোথায় দেখা যায়? A) /var/log/auth.log B) `dmesg` C) /etc/fstab D) `top`
3. SSH-এ বারবার ব্যর্থ লগইন চেষ্টা কোথায় দেখা যাবে? A) /var/log/auth.log B) /var/log/nginx/error.log C) `df -h` D) /etc/passwd
4. Troubleshooting-এ একসঙ্গে অনেক কিছু বদলানো কেন ভুল? A) সময় বেশি লাগে B) কোন পরিবর্তনে কাজ হলো তা জানা যায় না C) লগ মুছে যায় D) সার্ভিস বন্ধ হয়
5. “Failed to start nginx: Address already in use” বার্তার সবচেয়ে সম্ভাব্য অর্থ কী? A) nginx ইনস্টল নেই B) পোর্ট ৮০ অন্য প্রোগ্রাম ব্যবহার করছে C) RAM শেষ D) ডিস্ক পূর্ণ

Practical Assignment — ব্যবহারিক কাজ

একটি কাল্পনিক ঘটনার তদন্ত প্রতিবেদন লিখুন: “সকাল ৯টায় ওয়েবসাইট বন্ধ, nginx চলছে কিন্তু সাইট আসছে না।” এই অধ্যায়ের workflow অনুযায়ী ধাপে ধাপে লিখুন — আপনি কোন কমান্ড চালাতেন, কোন লগ দেখতেন, তিনটি সম্ভাব্য কারণ কী হতে পারে এবং প্রতিটি কীভাবে যাচাই করতেন। কোনো সার্ভিস বন্ধ করবেন না; এটি একটি লিখিত অনুশীলন।

সারসংক্ষেপ

লগ হলো সিস্টেমের ডায়েরি — সময়, সার্ভিস ও গুরুত্বসহ লেখা প্রমাণ। `journalctl` দিয়ে সার্ভিস ও সময় অনুযায়ী, `dmesg` দিয়ে kernel বার্তা, `auth.log` দিয়ে লগইন তদন্ত করা যায়। ভালো troubleshooting অনুমান দিয়ে নয়, প্রমাণ দিয়ে চলে: সমস্যা লিখুন, পরিবর্তন খুঁজুন, লগ ও রিসোর্স দেখুন, একটি অনুমান পরীক্ষা করুন, ফল লিখুন।

Revision Checklist — পুনরালোচনার তালিকা

- ☐ `journalctl` -এ সার্ভিস, সময় ও গুরুত্ব দিয়ে ছাঁকতে পারি
- ☐ `dmesg` ও `auth.log` কখন দেখতে হয়, জানি
- ☐ ভুল বার্তা তিন ভাগে ভেঙে পড়তে পারি
- ☐ সার্ভিস, ডিস্ক, CPU ও SSH সমস্যার তদন্ত-ক্রম বলতে পারি
- ☐ তদন্তের প্রতিটি ধাপ লিখে রাখার অভ্যাস বুঝি

Chapter 19: Linux Security and System Maintenance — Linux নিরাপত্তা ও সিস্টেম রক্ষণাবেক্ষণ

শুরুর কথা: দরজা খোলা রেখে ঘুমানো যায় না

একটি ছোট কোম্পানি ইন্টারনেটে একটি Ubuntu সার্ভার চালু করল। দুর্বল root পাসওয়ার্ড, পাসওয়ার্ড দিয়ে SSH লগইন চালু, ফায়ারওয়াল বন্ধ। তিন দিনের মধ্যে সার্ভারটি cryptomining-এর কাজে ব্যবহৃত হচ্ছিল — CPU ১০০%, বিল বাড়ছে, কোম্পানির নামে স্প্যাম যাচ্ছে। আক্রমণকারী কোনো জটিল কৌশল ব্যবহার করেনি; সে শুধু খোলা দরজা দিয়ে ঢুকেছিল। নিরাপত্তা কোনো একক পণ্য নয়, এটি অভ্যাসের সমষ্টি — আপডেট, শক্ত প্রমাণীকরণ, ফায়ারওয়াল, ন্যূনতম অনুমতি। এই অধ্যায়ে আপনি শিখবেন একটি ছোট সার্ভারকে বাস্তবসম্মতভাবে নিরাপদ রাখার মৌলিক কাজগুলো, যা প্রতিটি জুনিয়র অ্যাডমিনের জানা উচিত।

Learning Objectives — শেখার উদ্দেশ্য

- নিরাপত্তার মূল নীতিগুলো (least privilege, defence in depth, আপডেট) ব্যাখ্যা করতে পারবেন।
- Ubuntu 24.04-এ আপডেট ও প্যাচিং নিরাপদে করতে পারবেন।
- SSH key-based লগইন সেট আপ করে পাসওয়ার্ড লগইন বন্ধ করতে পারবেন (নিরাপদ ক্রমে)।
- UFW দিয়ে ফায়ারওয়াল কনফিগার করে খোলা পোর্ট পরীক্ষা করতে পারবেন।
- সাধারণ ভুল কনফিগারেশন শনাক্ত করে একটি মৌলিক নিরাপত্তা নিরীক্ষা করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ১১-১২-এর ব্যবহারকারী, অনুমতি ও sudo, অধ্যায় ১৪-এর প্যাকেজ ব্যবস্থাপনা এবং অধ্যায় ১৫-এর সার্ভিস ব্যবস্থাপনা জানা থাকতে হবে। SSH কী তা অধ্যায় ৭ থেকে পরিচিত ধরে নেওয়া হয়েছে।

প্রথম নীতি থেকে: নিরাপত্তা মানে কী?

নিরাপত্তার তিনটি লক্ষ্য হলো গোপনীয়তা (অননুমোদিত কেউ তথ্য দেখতে পারবে না), অখণ্ডতা (অননুমোদিত কেউ তথ্য বদলাতে পারবে না) ও প্রাপ্যতা (অনুমোদিত ব্যবহারকারী প্রয়োজনে সেবা পাবে)। এই তিনটির ভারসাম্য রাখতে হয় — এমন নিরাপত্তা যা নিজের কাজই বন্ধ করে দেয়, তা ভালো নিরাপত্তা নয়। দুটি নীতি সবচেয়ে গুরুত্বপূর্ণ। Least privilege: প্রত্যেককে শুধু ততটুকু অধিকার দিন, যতটুকু তার কাজের জন্য দরকার — তার বেশি নয়। Defence in depth: একটি দেয়াল নয়, একাধিক স্তর — ফায়ারওয়াল, শক্ত লগইন, আপডেট, অনুমতি — যাতে একটি স্তর ভাঙলেও পরেরটি থাকে। আক্রমণকারী সাধারণত সবচেয়ে সহজ পথ খোঁজে; আপনার কাজ হলো সহজ পথগুলো বন্ধ করা।

আরেকটি বাস্তব সত্য: ইন্টারনেটে সংযুক্ত সার্ভারকে স্বয়ংক্রিয় বট প্রতিনিয়ত স্ক্যান করে। SSH পোর্ট ২২ খোলা থাকলে কয়েক মিনিটের মধ্যে লগইন চেষ্টা শুরু হয় — এটি ব্যক্তিগত আক্রমণ নয়, স্বয়ংক্রিয়। তাই “আমার ছোট সার্ভার কেউ খুঁজবে না” — এই ধারণা ভুল। `/var/log/auth.log`-এ ব্যর্থ লগইন চেষ্টা দেখলেই বুঝবেন।

ভেতরের গঠন: SSH, key ও ফায়ারওয়াল কীভাবে কাজ করে

SSH হলো দূর থেকে সার্ভারে নিরাপদে লগইন ও কমান্ড চালানোর প্রোটোকল। পাসওয়ার্ড দিয়ে লগইন করলে আক্রমণকারী বারবার অনুমান করতে পারে। Key-based লগইনে থাকে একজোড়া চাবি: private key আপনার কম্পিউটারে থাকে

(কাউকে দেবেন না), public key সার্ভারে `~/.ssh/authorized_keys`-এ থাকে। লগইনের সময় সার্ভার একটি চ্যালেঞ্জ দেয়, যা শুধু private key দিয়ে সমাধান করা যায় — পাসওয়ার্ড অনুমান করে ঢোকা যায় না। এ কারণে key-based লগইন পাসওয়ার্ডের চেয়ে অনেক শক্তিশালী।

ফায়ারওয়াল হলো দরজার প্রহরী — কোন পোর্টে কে ঢুকতে পারবে, তা ঠিক করে। Ubuntu-তে UFW (Uncomplicated Firewall) হলো `iptables`/`nftables`-এর সহজ মুখ। নীতি: প্রথমে যা দরকার তা খুলুন (যেমন SSH), তারপর ফায়ারওয়াল চালু করুন, তারপর বাকি সব বন্ধ থাকবে। ভুল ক্রমে ফায়ারওয়াল চালু করলে নিজেই সার্ভার থেকে বেরিয়ে যেতে পারেন — তাই SSH নিয়ম যোগ করার পরই `ufw enable` করুন।

আপডেট হলো নিরাপত্তার সবচেয়ে সস্তা ও কার্যকর কাজ। বেশিরভাগ সফল আক্রমণ নতুন কোনো কৌশলে হয় না — পুরোনো, জানা দুর্বলতায় হয়, যার প্যাচ অনেক আগেই বেরিয়েছে। Ubuntu 24.04 LTS-এ নিরাপত্তা আপডেট ২০২৯ সালের এপ্রিল পর্যন্ত পাওয়া যাবে। নিয়মিত `apt update && apt upgrade` করা মানে জানা দরজাগুলো বন্ধ রাখা।

চিত্র ১৯.১ — নিরাপত্তার স্তরসমূহ (Defence in Depth)

স্তর	কী করে	উদাহরণ
নেটওয়ার্ক	অপ্রয়োজনীয় পথ বন্ধ করে	UFW — শুধু ২২, ৮০, ৪৪৩ খোলা
প্রমাণীকরণ	শুধু সঠিক ব্যক্তি ঢুকতে পারে	SSH key, পাসওয়ার্ড লগইন বন্ধ
অধিকার	ঢুকলেও ক্ষতি সীমিত থাকে	sudo, least privilege, ফাইল অনুমতি
আপডেট	জানা দুর্বলতা বন্ধ করে	নিয়মিত apt upgrade
পর্যবেক্ষণ	অস্বাভাবিকতা ধরা পড়ে	auth.log দেখা, লগইন সতর্কতা

চিত্র ১৯.২ — SSH key লগইনের প্রবাহ

১) আপনার কম্পিউটারে key জোড়া তৈরি হয়: private key (গোপন) + public key। ২) public key সার্ভারের `~/.ssh/authorized_keys`-এ রাখা হয়। ৩) লগইনের সময় সার্ভার একটি চ্যালেঞ্জ পাঠায়। ৪) আপনার SSH client private key দিয়ে চ্যালেঞ্জের উত্তর দেয়। ৫) উত্তর মিললে লগইন হয় — পাসওয়ার্ড পাঠানো হয় না, অনুমান করার সুযোগ থাকে না। Private key কখনো সার্ভারে পাঠাবেন না, কাউকে দেবেন না, Git-এ তুলবেন না।

পরিভাষা

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Least privilege	কাজের জন্য যতটুকু দরকার, শুধু ততটুকু অধিকার দেওয়া	১৯
Defence in depth	একাধিক নিরাপত্তা স্তর, যাতে একটি ভাঙলেও অন্যটি থাকে	১৯
SSH key pair	private + public key জোড়া; পাসওয়ার্ডের চেয়ে শক্ত লগইন	১৯
Brute force	বারবার অনুমান করে পাসওয়ার্ড ভাঙার চেষ্টা	১৯

UFW	Ubuntu-র সহজ ফায়ারওয়াল ব্যবস্থাপনা টুল	১৯
Port	নেটওয়ার্ক সেবার দরজা নম্বর, যেমন SSH ২২, HTTP ৮০	১৯
Patching	নিরাপত্তা দুর্বলতা ঠিক করতে আপডেট প্রয়োগ করা	১৯
Root login	root ব্যবহারকারী হিসেবে সরাসরি লগইন; সাধারণত বন্ধ রাখা উচিত	১৯

বাস্তব উদাহরণ

সিলেটের একটি স্টার্টআপ তাদের VPS-এ শুধু পাসওয়ার্ড দিয়ে root লগইন চালু রেখেছিল। `auth.log`-এ দেখা গেল প্রতিদিন হাজার হাজার ব্যর্থ চেষ্টা। তারা key-based লগইন চালু করল, root লগইন বন্ধ করল, UFW-তে শুধু SSH ও HTTPS খোলা রাখল। ব্যর্থ চেষ্টা তখনো আসত, কিন্তু ঢোকার পথ বন্ধ হয়ে গেল। শিক্ষা: আক্রমণ চেষ্টা থামানো যায় না, কিন্তু সফল হওয়ার পথ বন্ধ করা যায় — আর সেটিই আপনার কাজ।

ব্যবহারিক কমান্ড

```
sudo apt update && sudo apt upgrade -y
# প্যাকেজ তালিকা হালনাগাদ করে নিরাপত্তা আপডেটসহ সব আপডেট প্রয়োগ করে।
# গুরুত্বপূর্ণ সার্ভারে -y ছাড়া চালিয়ে কী আপডেট হবে তা আগে দেখে নেওয়া ভালো।

sudo apt autoremove
# আর দরকার নেই এমন পুরোনো প্যাকেজ সরিয়ে দেয়।

ssh-keygen -t ed25519 -C "nahid@example.com"
# নিজের কম্পিউটারে (সার্ভারে নয়) key জোড়া তৈরি করে। পাসফ্রেজ দিলে key আরও নিরাপদ হয়।

ssh-copy-id nahid@SERVER_IP
# public key সার্ভারে পাঠায়। এরপর পাসওয়ার্ড ছাড়া key দিয়ে লগইন পরীক্ষা করুন।

sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status verbose
# প্রথমে SSH খুলে তারপর ফায়ারওয়াল চালু করুন। ভুল ক্রমে নিজেই বাইরে আটকে যেতে পারেন।

sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
# ওয়েব সার্ভারের জন্য HTTP ও HTTPS খোলা।

ss -tlnp
# কোন পোর্টে কোন সার্ভিস শুনছে (listening) তা দেখায়। অচেনা পোর্ট থাকলে তদন্ত করুন।

sudo grep "Failed password" /var/log/auth.log | tail -10
# সাম্প্রতিক ব্যর্থ SSH লগইন চেষ্টা দেখায়।

sudo systemctl status ufw
# ফায়ারওয়াল চলছে কি না তা দেখায়।
```

SSH কনফিগারেশন থাকে `/etc/ssh/sshd_config`-এ। পাসওয়ার্ড লগইন বন্ধ করার সেটিং হলো `PasswordAuthentication no` এবং root লগইন বন্ধ করার সেটিং `PermitRootLogin no`। কিন্তু এগুলো বদলানোর আগে অবশ্যই key দিয়ে লগইন পরীক্ষা করে নিন — নইলে নিজেই সার্ভারে ঢুকতে পারবেন না। পরিবর্তনের পর `sudo sshd`

`-t` দিয়ে configuration পরীক্ষা করুন, তারপর `sudo systemctl restart ssh.service` দিয়ে প্রয়োগ করুন। বর্তমান SSH সংযোগটি বন্ধ করবেন না — নতুন একটি টার্মিনালে লগইন পরীক্ষা করুন, পুরোনো সংযোগটি খোলা রাখুন যাতে ভুল হলে ফিরে যেতে পারেন।

> **WARNING:** পাসওয়ার্ড লগইন বন্ধ করার আগে তিনটি জিনিস নিশ্চিত করুন: (১) key দিয়ে লগইন কাজ করছে, (২) আপনার ব্যবহারকারীর sudo অধিকার আছে, (৩) private key-র ব্যাকআপ নিরাপদ জায়গায় আছে। এই তিনটির একটি না থাকলে সার্ভারে ঢোকা স্থায়ীভাবে বন্ধ হয়ে যেতে পারে। VPS-এর console (ওয়েব কনসোল) থাকলে তা শেষ ভরসা হিসেবে জেনে রাখুন।

> **PRO TIP:** `sudo ufw --dry-run enable` দিয়ে আগে দেখে নিতে পারেন কী নিয়ম প্রয়োগ হবে। আর ফায়ারওয়াল চালু করার সময় VPS-এর ওয়েব কনসোল খোলা রাখুন — ভুল হলে সেখান থেকে ঠিক করা যায়।

হাতে-কলমে অনুশীলন: সার্ভারের নিরাপত্তা নিরীক্ষা

নিজের ভার্চুয়াল মেশিনে (ইন্টারনেটে খোলা সার্ভারে নয়) এই নিরীক্ষা করুন। SSH সেটিং বদলাবেন না — শুধু পরীক্ষা করবেন।

1. `sudo apt update` চালিয়ে দেখুন কোনো আপডেট বাকি আছে কি না। থাকলে নোট করুন, প্রয়োগ করার আগে কী আপডেট হবে তা পড়ুন।
2. `sudo ufw status verbose` চালিয়ে ফায়ারওয়ালের অবস্থা লিখুন। বন্ধ থাকলে কেন, ভাবুন।
3. `ss -tlnp` চালিয়ে কোন পোর্টগুলো খোলা আছে তা লিখুন। প্রতিটি পোর্টের পেছনে কোন সার্ভিস, তা চিহ্নিত করুন।
4. `sudo grep "Failed password" /var/log/auth.log | tail -5` দিয়ে ব্যর্থ লগইন চেষ্টা আছে কি না দেখুন।
5. `grep -E 'PermitRootLogin|PasswordAuthentication' /etc/ssh/sshd_config` দিয়ে বর্তমান SSH সেটিং দেখুন।
6. একটি নিরীক্ষা নোট লিখুন: কোন তিনটি জিনিস ঠিক আছে, কোন তিনটি উন্নত করা দরকার এবং কোনটি প্রথমে করবেন।

উদাহরণ আউটপুট (illustrative):

```
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       Anywhere
80/tcp                     ALLOW       Anywhere
```

ব্যাখ্যা: incoming ডিফল্ট deny মানে যা স্পষ্টভাবে খোলা হয়নি, তা বন্ধ। শুধু ২২ (SSH) ও ৮০ (HTTP) খোলা — এটি একটি ভালো, ন্যূনতম অবস্থা।

সাধারণ ভুল ও সমস্যা সমাধান

- **ফায়ারওয়াল চালু করে নিজেই আটকে যাওয়া:** VPS কনসোল থেকে `ufw disable` বা নিয়ম ঠিক করুন। ভবিষ্যতে SSH নিয়ম আগে যোগ করুন।
- **key দিয়ে লগইন হচ্ছে না:** `~/.ssh` ডিরেক্টরির অনুমতি ৭০০ ও `authorized_keys`-এর অনুমতি ৬০০ কি না দেখুন; ভুল অনুমতিতে SSH key উপেক্ষা করে।
- **আপডেটের পর সার্ভিস বন্ধ:** আপডেটের আগে কী বদলাবে তা পড়ুন; গুরুত্বপূর্ণ সার্ভারে প্রথমে পরীক্ষামূলক মেশিনে আপডেট করুন।
- **সবাইকে sudo দেওয়া:** sudo শুধু যাদের দরকার তাদের দিন; `/etc/sudoers` সরাসরি সম্পাদনা না করে `visudo` ব্যবহার করুন।
- **অপ্রয়োজনীয় সার্ভিস চালু রাখা:** `ss -tlnp` দিয়ে দেখুন; যা দরকার নেই, তা বন্ধ ও disable করুন।

নিরাপত্তা বিবেচনা

এই অধ্যায়ের প্রতিটি কাজই নিরাপত্তা। অতিরিক্ত মনে রাখবেন: পাসওয়ার্ড কখনো স্ক্রিপ্ট, চ্যাট বা Git-এ লিখবেন না; private key কাউকে দেবেন না; সার্ভারের ব্যাকআপেও সংবেদনশীল ফাইলের অনুমতি ঠিক রাখুন; পুরোনো ব্যবহারকারীর অ্যাকাউন্ট মুছে ফেলুন বা disable করুন; নিয়মিত `auth.log` দেখুন। নিরাপত্তা একবারের কাজ নয় — এটি রক্ষণাবেক্ষণের অংশ।

শিল্পের সেরা চর্চা

- নিয়মিত আপডেট করুন; নিরাপত্তা আপডেট দেরি করবেন না।
- SSH-এ key ব্যবহার করুন, পাসওয়ার্ড লগইন বন্ধ করুন, root লগইন বন্ধ করুন।
- ফায়ারওয়ালে ন্যূনতম পোর্ট খুলুন; ডিফল্ট deny রাখুন।
- প্রতিটি ব্যবহারকারীকে শুধু তার দরকারি অধিকার দিন।
- লগ নিয়মিত দেখুন; অস্বাভাবিক লগইন চেষ্টা তদন্ত করুন।
- পরিবর্তনের আগে ব্যাকআপ নিন ও পরীক্ষা করুন; পরিবর্তনের পর লগইন পরীক্ষা করুন।

Knowledge Check — জ্ঞান যাচাই

1. Least privilege নীতির অর্থ কী? A) সবাইকে root দেওয়া B) প্রত্যেককে শুধু তার কাজের জন্য যতটুকু দরকার, ততটুকু অধিকার দেওয়া C) ফায়ারওয়াল বন্ধ রাখা D) আপডেট না করা
2. SSH key লগইনে private key কোথায় থাকে? A) সার্ভারে B) আপনার নিজের কম্পিউটারে, গোপনে C) Git-এ D) /var/log-এ
3. UFW চালু করার আগে প্রথমে কী করা উচিত? A) সব পোর্ট বন্ধ করা B) SSH-এর নিয়ম যোগ করা C) সার্ভার restart করা D) root লগইন চালু করা
4. পাসওয়ার্ড লগইন বন্ধ করার আগে সবচেয়ে গুরুত্বপূর্ণ পরীক্ষা কোনটি? A) ফায়ারওয়াল বন্ধ করা B) key দিয়ে লগইন কাজ করছে কি না পরীক্ষা করা C) আপডেট করা D) লগ মুছে ফেলা
5. `ss -tlnp` কী দেখায়? A) CPU ব্যবহার B) কোন পোর্টে কোন সার্ভিস শুনছে C) ডিস্কের জায়গা D) RAM ব্যবহার

Practical Assignment — ব্যবহারিক কাজ

আপনার ভার্চুয়াল মেশিনের একটি এক পৃষ্ঠার “নিরাপত্তা নিরীক্ষা প্রতিবেদন” লিখুন: আপডেটের অবস্থা, ফায়ারওয়ালের অবস্থা, খোলা পোর্টের তালিকা, SSH সেটিং, sudo ব্যবহারকারীর তালিকা এবং ব্যর্থ লগইন চেষ্টার সারসংক্ষেপ। প্রতিটি বিষয়ে “ঠিক আছে / উন্নত করা দরকার” লিখুন এবং সবচেয়ে জরুরি তিনটি পদক্ষেপের তালিকা দিন। SSH সেটিং বদলাবেন না।

সারসংক্ষেপ

নিরাপত্তা মানে খোলা দরজা বন্ধ করা: নিয়মিত আপডেট, key-based SSH, root ও পাসওয়ার্ড লগইন বন্ধ, UFW-তে ন্যূনতম পোর্ট, least privilege অনুমতি এবং নিয়মিত লগ পর্যবেক্ষণ। একটি স্তর যথেষ্ট নয় — একাধিক স্তর একসঙ্গে কাজ করলে একটি ভাঙলেও সার্ভার সুরক্ষিত থাকে। পরিবর্তনের আগে পরীক্ষা করুন, যাতে নিরাপত্তা বাড়াতে গিয়ে নিজেই বাইরে আটকে না যান।

Revision Checklist — পুনরালোচনার তালিকা

- ☐ least privilege ও defence in depth ব্যাখ্যা করতে পারি
- ☐ SSH key জোড়া কীভাবে কাজ করে, বলতে পারি
- ☐ UFW-তে SSH নিয়ম আগে যোগ করে ফায়ারওয়াল চালু করতে পারি
- ☐ `ss -tlnp` দিয়ে খোলা পোর্ট পরীক্ষা করতে পারি
- ☐ পাসওয়ার্ড লগইন বন্ধ করার নিরাপদ ক্রম বলতে পারি

Chapter 20: Backup, Recovery and Practical Server Administration — ব্যাকআপ, পুনরুদ্ধার ও ব্যবহারিক সার্ভার প্রশাসন

শুরুর কথা: ব্যাকআপের মূল্য বোঝা যায় যেদিন তা দরকার হয়

একটি কোম্পানির সার্ভারের ডিস্ক নষ্ট হলো। তাদের ব্যাকআপ ছিল — কিন্তু কেউ কখনো তা থেকে ফিরিয়ে আনার পরীক্ষা করেনি। ফিরিয়ে আনতে গিয়ে দেখা গেল ব্যাকআপ ফাইলটি নষ্ট, আর গুরুত্বপূর্ণ ডেটাবেসের ব্যাকআপই নেওয়া হয়নি। কোম্পানিটি তিন বছরের গ্রাহক তথ্য হারাল। এই গল্পের শিক্ষা: ব্যাকআপ নেওয়া কাজের অর্ধেক; বাকি অর্ধেক হলো তা যাচাই করা ও ফিরিয়ে আনতে জানা। পরীক্ষা না করা ব্যাকআপ কোনো ব্যাকআপ নয় — এটি একটি আশা মাত্র। এই শেষ অধ্যায়ে আপনি শিখবেন ব্যাকআপের নীতি, `tar` ও `rsync` দিয়ে ব্যাকআপ ও পুনরুদ্ধার, পুনরুদ্ধার পরিকল্পনা এবং পুরো বইয়ের দক্ষতাগুলো একসঙ্গে ব্যবহার করে একটি ছোট সার্ভার পরিচালনা করা।

Learning Objectives — শেখার উদ্দেশ্য

- ব্যাকআপের নীতি (3-2-1, full/incremental, যাচাই) ব্যাখ্যা করতে পারবেন।
- `tar` দিয়ে ব্যাকআপ তৈরি, তালিকা দেখা ও ফিরিয়ে আনতে পারবেন।
- `rsync` দিয়ে নিরাপদে ফাইল সিঙ্ক ও ব্যাকআপ করতে পারবেন।
- একটি পুনরুদ্ধার পরিকল্পনা লিখে তা পরীক্ষা করতে পারবেন।
- দৈনিক/সাপ্তাহিক রক্ষণাবেক্ষণ চেকলিস্ট অনুসরণ করে একটি ছোট সার্ভার পরিচালনা করতে পারবেন।

Prerequisites — পূর্বজ্ঞান

অধ্যায় ৮-এর filesystem, অধ্যায় ১০-এর `tar` ও ফাইল কমান্ড, অধ্যায় ১৬-এর ডিস্ক, অধ্যায় ১৮-এর লগ ও অধ্যায় ১৯-এর নিরাপত্তা — সবগুলো এখানে একসঙ্গে ব্যবহৃত হবে।

প্রথম নীতি থেকে: ব্যাকআপ কেন, কীসের?

তথ্য হারানোর কারণ অনেক: ডিস্ক নষ্ট হওয়া, ভুল করে ফাইল মোছা, ভুল কমান্ড, ransomware, সার্ভার চুরি, আগুন বা বন্যা। ব্যাকআপ মানে তথ্যের আরেকটি কপি, এমন জায়গায় যা মূল তথ্যের সঙ্গে একসঙ্গে নষ্ট হবে না। একই ডিস্কে ব্যাকআপ রাখলে ডিস্ক নষ্ট হলে দুটোই যাবে — তাই ব্যাকআপ আলাদা ডিস্কে, আলাদা সার্ভারে বা আলাদা স্থানে রাখতে হয়। 3-2-1 নীতি মনে রাখুন: তথ্যের ৩টি কপি, ২টি ভিন্ন মাধ্যমে, ১টি ভিন্ন স্থানে। ছোট প্রকল্পে এর সরল রূপ: মূল কপি + বাহ্যিক ডিস্কে কপি + (সম্ভব হলে) দূরবর্তী স্থানে কপি।

দুটি প্রশ্ন ব্যাকআপ পরিকল্পনা ঠিক করে। RPO (Recovery Point Objective): কতটুকু তথ্য হারানো সহ্য করা যায়? দৈনিক ব্যাকআপ মানে এক দিনের তথ্য হারাতে পারে; ঘণ্টায় ব্যাকআপ মানে এক ঘণ্টার। RTO (Recovery Time Objective): কতক্ষণে সেবা ফিরিয়ে আনতে হবে? এক ঘণ্টায় ফিরতে হলে পরিকল্পনা ও পরীক্ষা থাকতে হবে। ছোট কোম্পানির জন্য এই দুটি সংখ্যা লিখে রাখাই পেশাদারিত্বের শুরু — মুখে “ব্যাকআপ আছে” বলা নয়।

ভেতরের গঠন: full, incremental ও যাচাই

Full backup মানে সব তথ্যের সম্পূর্ণ কপি — ফিরিয়ে আনা সহজ, কিন্তু সময় ও জায়গা বেশি লাগে। Incremental backup মানে শেষ ব্যাকআপের পর যা বদলেছে, শুধু তা — দ্রুত ও ছোট, কিন্তু ফিরিয়ে আনতে full + সব incremental লাগে। বাস্তবে দুটি মিলিয়ে ব্যবহার করা হয়: সাপ্তাহিক full, দৈনিক incremental। `tar` একটি ডিরেক্টরিকে একটি archive ফাইলে বাঁধে; `gzip` তা ছোট করে। `rsync` শুধু বদলানো অংশ কপি করে, তাই বারবার ব্যাকআপে দ্রুত। কিন্তু `rsync`-এ একটি বিখ্যাত বিপদ আছে: `--delete` অপশন গন্তব্যে থাকা অতিরিক্ত ফাইল মুছে দেয় — উৎস ও গন্তব্য উল্টে গেলে মূল তথ্যই মুছে যেতে পারে। তাই `--delete` ব্যবহারের আগে `--dry-run` দিয়ে পরীক্ষা করুন।

ব্যাকআপের সবচেয়ে অবহেলিত ধাপ হলো যাচাই (verification)। ব্যাকআপ ফাইল তৈরি হলো মানেই তা ভালো নয় — ফাইলটি খুলে দেখুন, তালিকা দেখুন, একটি পরীক্ষামূলক ডিরেক্টরিতে ফিরিয়ে এনে দেখুন। ডেটাবেসের ব্যাকআপ নিলে শুধু ফাইল কপি যথেষ্ট নয় — চলমান ডেটাবেসের ফাইল সরাসরি কপি করলে তা অসম্পূর্ণ হতে পারে; ডেটাবেসের নিজস্ব dump টুল ব্যবহার করতে হয়। এই বইয়ের পরিসরে মনে রাখুন: ফাইল ব্যাকআপ ও ডেটাবেস ব্যাকআপ আলাদা কাজ।

চিত্র ২০.১ — 3-2-1 ব্যাকআপ নীতি

কপি	কোথায়	কী থেকে রক্ষা করে
১. মূল তথ্য	সার্ভারের ডিস্কে	দৈনন্দিন কাজ
২. ব্যাকআপ কপি	আলাদা বাহ্যিক ডিস্ক / সার্ভারে	ডিস্ক নষ্ট, ভুল করে মোছা
৩. দূরবর্তী কপি	ভিন্ন স্থানে / অন্য সার্ভারে	আগুন, চুরি, পুরো সাইটের ক্ষতি

তিনটি কপির অন্তত একটি যেন মূল সার্ভারের সঙ্গে সবসময় যুক্ত না থাকে — নইলে ransomware বা ভুল কমান্ড সব কপি একসঙ্গে মুছে দিতে পারে।

চিত্র ২০.২ — পুনরুদ্ধারের ক্রম (Recovery Flow)

১) কী হারিয়েছে, কখন হারিয়েছে — নির্ধারণ করুন → ২) সার্ভিস বন্ধ করুন যাতে আংশিক তথ্য লেখা না হয় → ৩) সঠিক ব্যাকআপ বেছে নিন (তারিখ যাচাই করুন) → ৪) প্রথমে পরীক্ষামূলক জায়গায় ফিরিয়ে এনে যাচাই করুন → ৫) মূল জায়গায় ফিরিয়ে আনুন → ৬) সার্ভিস চালু করে পরীক্ষা করুন → ৭) কী ঘটেছিল ও কী শিখলেন, লিখে রাখুন। তাড়াহুড়ো করে সরাসরি মূল জায়গায় ফিরিয়ে আনা বিপজ্জনক — ভুল ব্যাকআপ হলে ভালো তথ্যও নষ্ট হতে পারে।

পরিভাষা

English Term	বাংলা ব্যাখ্যা	অধ্যায়
Backup	তথ্যের অতিরিক্ত কপি, হারানো তথ্য ফিরিয়ে আনার জন্য	২০
Restore / Recovery	ব্যাকআপ থেকে তথ্য বা সেবা ফিরিয়ে আনা	২০
Full backup	সব তথ্যের সম্পূর্ণ কপি	২০
Incremental backup	শেষ ব্যাকআপের পর বদলানো অংশের কপি	২০

RPO	কতটুকু তথ্য হারানো সহ্য করা যায় (সময়ে মাপা)	২০
RTO	কতক্ষণে সেবা ফিরিয়ে আনতে হবে	২০
3-2-1 rule	৩ কপি, ২ ভিন্ন মাধ্যম, ১ ভিন্ন স্থান	২০
rsync	শুধু বদলানো অংশ কপি করে ফাইল সিঙ্ক করার টুল	২০

বাস্তব উদাহরণ

খুলনার একটি ছোট এজেন্সি প্রতিদিন রাতে `rsync` দিয়ে ওয়েবসাইটের ফাইল অন্য সার্ভারে কপি করত। একদিন একজন কর্মী ভুল করে পুরো সাইট মুছে ফেললেন। অ্যাডমিন আতঙ্কিত না হয়ে গত রাতের কপি থেকে ফাইল ফিরিয়ে আনলেন — সাইট দুই ঘণ্টায় ফিরে এলো, শুধু সেদিনের কয়েকটি ছবি হারাল। পরে তারা ব্যাকআপের সময় ও যাচাইয়ের নিয়ম লিখে রাখল। শিক্ষা: ব্যাকআপের মূল্য তার আকারে নয়, ফিরিয়ে আনার গতিতে ও নিশ্চয়তায়।

ব্যবহারিক কমান্ড

```
tar -czf /tmp/home-backup-2026-11-08.tar.gz /home/nahid
# /home/nahid-এর compressed ব্যাকআপ তৈরি করে। -c তৈরি, -z gzip, -f ফাইলের নাম।

tar -tzf /tmp/home-backup-2026-11-08.tar.gz | head -20
# ব্যাকআপ না খুলে ভেতরে কী আছে তা তালিকা দেখে যাচাই করা।

mkdir -p /tmp/restore-test
tar -xzf /tmp/home-backup-2026-11-08.tar.gz -C /tmp/restore-test
# পরীক্ষামূলক ডিরেক্টরিতে ফিরিয়ে এনে যাচাই করা — মূল জায়গায় নয়।

ls -lh /tmp/home-backup-*.tar.gz
# ব্যাকআপ ফাইলের আকার ও তারিখ দেখা।

rsync -av /home/nahid/ /mnt/backup/nahid/
# /home/nahid-এর সব ফাইল ব্যাকআপ জায়গায় সিঙ্ক করে। -a অনুমতি ও সময় ধরে রাখে, -v দেখায় কী কপি হচ্ছে।

rsync -av --dry-run --delete /home/nahid/ /mnt/backup/nahid/
# --delete-এর আগে শুকনো পরীক্ষা: কী মুছবে তা না মুছে দেখায়। নিরাপদ অভ্যাস।

rsync -av /home/nahid/ nahid@backup-server:/backup/nahid/
# SSH দিয়ে দূরবর্তী সার্ভারে ব্যাকআপ পাঠানো (key-based লগইন থাকলে পাসওয়ার্ড লাগে না)।

sha256sum /tmp/home-backup-2026-11-08.tar.gz
# ব্যাকআপ ফাইলের checksum — পরে ফাইলটি নষ্ট হয়েছে কি না যাচাই করা যায়।
```

`tar`-এ পথের শেষে `/` থাকা-না-থাকা ও `rsync`-এ উৎস পথের শেষে `/` থাকা-না-থাকা ফল বদলে দেয়। `rsync /home/nahid/ গন্তব্য` মানে ডিরেক্টরির ভেতরের জিনিস কপি করা; `rsync /home/nahid গন্তব্য` মানে ডিরেক্টরিটিসহ কপি করা। ছোট পার্থক্য, বড় ভুলের কারণ — তাই প্রথমে `--dry-run` দিয়ে পরীক্ষা করুন।

> **WARNING:** `rsync --delete` ও `tar -xzf` মূল জায়গায় চালানোর আগে তিনবার যাচাই করুন — উৎস ও গন্তব্য উল্টে গেলে বা ভুল ডিরেক্টরিতে ফিরিয়ে আনলে মূল তথ্য মুছে যেতে পারে বা উপরে লেখা (overwrite) হতে পারে। প্রথমে পরীক্ষামূলক ডিরেক্টরিতে ফিরিয়ে আনুন, তারপর মূল জায়গায় যান। গুরুত্বপূর্ণ ফিরিয়ে আনার আগে বর্তমান অবস্থারও ব্যাকআপ নিন।

> **PRO TIP:** ব্যাকআপের নামে তারিখ রাখুন — যেমন `home-backup-2026-11-08.tar.gz`। তারিখ ছাড়া নামে কোনটি নতুন, কোনটি পুরোনো, তা বোঝা যায় না; আর পুরোনো ব্যাকআপ নতুনের উপরে লেখা হয়ে গেলে ফিরে যাওয়ার পথ থাকে না।

হাতে-কলমে অনুশীলন: ব্যাকআপ তৈরি ও ফিরিয়ে আনা

নিরাপদ অনুশীলন: নিজের home-এর একটি ছোট পরীক্ষামূলক ডিরেক্টরি নিয়ে কাজ করুন। গুরুত্বপূর্ণ ফাইল নিয়ে পরীক্ষা করবেন না।

1. একটি পরীক্ষামূলক ডিরেক্টরি তৈরি করুন: `mkdir -p ~/backup-lab && echo "hello backup" > ~/backup-lab/note.txt`।
2. ব্যাকআপ তৈরি করুন: `tar -czf /tmp/backup-lab.tar.gz ~/backup-lab`।
3. যাচাই করুন: `tar -tzf /tmp/backup-lab.tar.gz` দিয়ে ভেতরে কী আছে দেখুন।
4. মূল ফাইলটি সাময়িকভাবে সরান (মুছবেন না): `mv ~/backup-lab /tmp/backup-lab-orig`।
5. পরীক্ষামূলক জায়গায় ফিরিয়ে আনুন: `mkdir -p /tmp/restore-test && tar -xzf /tmp/backup-lab.tar.gz -C /tmp/restore-test`।
6. `cat /tmp/restore-test/*/backup-lab/note.txt` বা সঠিক পথে ফাইলটি পড়ে দেখুন — “hello backup” ফিরে এসেছে কি?
7. `rsync -av --dry-run ~/backup-lab/ /tmp/rsync-test/` দিয়ে dry-run পরীক্ষা করুন, তারপর `--dry-run` ছাড়া চালিয়ে দেখুন কী কপি হলো।
8. পরিষ্কার করুন: `mv /tmp/backup-lab-orig ~/backup-lab` দিয়ে মূল ডিরেক্টরি ফিরিয়ে আনুন এবং `/tmp/restore-test` ও `/tmp/rsync-test` মুছে ফেলুন।

উদাহরণ আউটপুট (illustrative):

```
$ tar -tzf /tmp/backup-lab.tar.gz
home/nahid/backup-lab/
home/nahid/backup-lab/note.txt

$ cat /tmp/restore-test/home/nahid/backup-lab/note.txt
hello backup
```

ব্যাখ্যা: ব্যাকআপে ফাইলটি আছে এবং তা থেকে ফিরিয়ে আনা যাচ্ছে — এটিই যাচাই। শুধু tar ফাইল তৈরি হওয়া যাচাই নয়; ফিরিয়ে এনে পড়তে পারাই আসল পরীক্ষা।

সার্ভার রক্ষণাবেক্ষণ চেকলিস্ট

দৈনিক (৫ মিনিট)

- `uptime`, `df -h`, `free -h` দেখুন — load, ডিস্ক ও RAM স্বাভাবিক কি না।
- `sudo systemctl --failed` দিয়ে ব্যর্থ সার্ভিস আছে কি না দেখুন।
- `sudo journalctl -p err -b --no-pager | tail -10` দিয়ে নতুন ভুল দেখুন।

সাপ্তাহিক (৩০ মিনিট)

- `sudo apt update && sudo apt upgrade` — আপডেট পরীক্ষা ও প্রয়োগ করুন।
- ব্যাকআপ যাচাই করুন — একটি ফাইল ফিরিয়ে এনে দেখুন।
- `sudo grep "Failed password" /var/log/auth.log | tail -10` দিয়ে লগইন চেষ্টা দেখুন।
- `du -sh /var/*` দিয়ে ডিস্কের বৃদ্ধি দেখুন।

মাসিক (১ ঘণ্টা)

- অপ্রয়োজনীয় ব্যবহারকারী, প্যাকেজ ও সার্ভিস পর্যালোচনা করুন।
- ফায়ারওয়াল নিয়ম ও SSH সেটিং পরীক্ষা করুন।
- পূর্ণ পুনরুদ্ধার পরীক্ষা করুন — পরীক্ষামূলক মেশিনে ব্যাকআপ থেকে ফিরিয়ে আনুন।
- পরিবর্তন ও ঘটনার নোট হালনাগাদ করুন।

সাধারণ ভুল ও সমস্যা সমাধান

- **ব্যাকআপ নেওয়া হয়, যাচাই করা হয় না:** মাসে অন্তত একবার ফিরিয়ে এনে পরীক্ষা করুন।
- **ব্যাকআপ একই ডিস্কে রাখা:** ডিস্ক নষ্ট হলে মূল ও ব্যাকআপ দুটোই যাবে। আলাদা মাধ্যম ব্যবহার করুন।
- **`rsync --delete` না বুঝে ব্যবহার:** সবসময় প্রথমে `--dry-run` করুন।
- **চলমান ডেটাবেসের ফাইল সরাসরি কপি করা:** ডেটাবেসের dump টুল ব্যবহার করুন; সরাসরি কপি অসম্পূর্ণ হতে পারে।
- **ব্যাকআপে তারিখ না রাখা:** কোনটি কোন দিনের, তা না জানলে ভুল ব্যাকআপ থেকে ফিরিয়ে আনা হতে পারে।
- **পুনরুদ্ধারের সময় তাড়াহুড়ো:** প্রথমে পরীক্ষামূলক জায়গায় ফিরিয়ে আনুন; মূল জায়গায় যাওয়ার আগে বর্তমান অবস্থার ব্যাকআপ নিন।

নিরাপত্তা বিবেচনা

ব্যাকআপে সব সংবেদনশীল তথ্য থাকে — তাই ব্যাকআপের অনুমতি ও সংরক্ষণ মূল সার্ভারের মতোই গুরুত্বপূর্ণ। ব্যাকআপ ডিস্ক বা সার্ভারে শক্ত অনুমতি দিন, প্রয়োজনে encryption ব্যবহার করুন, এবং ব্যাকআপের অ্যাক্সেস শুধু যাদের দরকার তাদের দিন। পুরোনো ব্যাকআপ মুছে ফেলার সময় মনে রাখুন — তাতে পুরোনো পাসওয়ার্ড বা তথ্য থাকতে পারে। ব্যাকআপ সর্বদা যুক্ত রাখবেন না; ransomware যুক্ত ব্যাকআপও মুছে দিতে পারে।

শিল্পের সেরা চর্চা

- 3-2-1 নীতি অনুসরণ করুন; অন্তত একটি কপি ভিন্ন স্থানে রাখুন।

- ব্যাকআপ স্বয়ংক্রিয় করুন, কিন্তু যাচাই হাতে করুন — স্বয়ংক্রিয় ব্যাকআপ নীরবে ব্যর্থ হতে পারে।
- RPO ও RTO লিখে রাখুন; ব্যাকআপের সময়সূচি সেই অনুযায়ী ঠিক করুন।
- পুনরুদ্ধার পরীক্ষা করুন ও তার ফল লিখুন — পরীক্ষা না করা ব্যাকআপ কোনো ব্যাকআপ নয়।
- পরিবর্তনের আগে ব্যাকআপ নিন; বড় পরিবর্তনের পর আবার ব্যাকআপ নিন।

Knowledge Check — জ্ঞান যাচাই

1. 3-2-1 নীতির অর্থ কী? A) ৩ দিনে ২ বার ব্যাকআপ B) ৩ কপি, ২ ভিন্ন মাধ্যম, ১ ভিন্ন স্থান C) ৩টি সার্ভার, ২টি ডিস্ক D) ৩ GB ব্যাকআপ, ২ GB ফাঁকা
2. ব্যাকআপ যাচাইয়ের সবচেয়ে নির্ভরযোগ্য উপায় কোনটি? A) ফাইলের আকার দেখা B) ফিরিয়ে এনে পড়ে দেখা C) ফাইলের নাম দেখা D) তারিখ দেখা
3. `rsync --delete` ব্যবহারের আগে কী করা উচিত? A) সরাসরি চালানো B) `--dry-run` দিয়ে পরীক্ষা করা C) ফায়ারওয়াল বন্ধ করা D) সার্ভিস বন্ধ করা
4. RPO-র অর্থ কী? A) কতক্ষণে সেবা ফিরবে B) কতটুকু তথ্য হারানো সহ্য করা যায় C) ব্যাকআপের আকার D) ডিস্কের গতি
5. চলমান ডেটাবেসের ব্যাকআপ নেওয়ার সঠিক পথ কোনটি? A) ডেটাবেসের ফাইল সরাসরি কপি করা B) ডেটাবেসের নিজস্ব dump টুল ব্যবহার করা C) সার্ভার বন্ধ না করে `rm` করা D) শুধু `tar` দিয়ে ফোল্ডার কপি করা

Practical Assignment — ব্যবহারিক কাজ

আপনার ভার্চুয়াল মেশিনের জন্য এক পৃষ্ঠার “ব্যাকআপ ও পুনরুদ্ধার পরিকল্পনা” লিখুন: কী কী ব্যাকআপ নেবেন (home, /etc-এর গুরুত্বপূর্ণ ফাইল), কত ঘন ঘন নেবেন, কোথায় রাখবেন, কীভাবে যাচাই করবেন, RPO ও RTO কত ধরবেন এবং পুনরুদ্ধারের ধাপগুলো কী হবে। তারপর এই অধ্যায়ের অনুশীলন অনুযায়ী একটি পরীক্ষামূলক ব্যাকআপ ও পুনরুদ্ধার করে ফল লিখুন।

End-to-End Administration Scenario — পুরো বইয়ের সমন্বয়: TechNova সার্ভার

> **REAL WORLD — TechNova:** TechNova একটি কাল্পনিক ছোট সফটওয়্যার কোম্পানি। আপনি তার জুনিয়র Linux সিস্টেম অ্যাডমিন। আপনার হাতে একটি Ubuntu Server 24.04 LTS সার্ভার। এক সোমবার সকালে তিনটি সমস্যা একসঙ্গে এলো: ওয়েবসাইট ধীর, একটি সার্ভিস বন্ধ, আর ডিস্ক ৯২% পূর্ণ। এই পরিস্থিতিতে পুরো বইয়ের দক্ষতা একসঙ্গে কাজে লাগে।

পরিস্থিতি ও তদন্তের ক্রম: প্রথমে আতঙ্কিত না হয়ে তথ্য সংগ্রহ করুন। `uptime` ও `free -h` দিয়ে দেখুন load ও RAM স্বাভাবিক কি না (অধ্যায় ১৭)। `df -h` দিয়ে নিশ্চিত করুন কোন filesystem পূর্ণ, তারপর `du -sh /var/*` দিয়ে দেখুন `/var/log`-ই বড় কি না (অধ্যায় ১৬)। `sudo systemctl --failed` ও `journalctl -u বন্ধ-সার্ভিস -n 50` দিয়ে দেখুন সার্ভিস কেন বন্ধ হলো — হয়তো ডিস্ক পূর্ণ হওয়ায় সে লিখতে পারছিল না (অধ্যায় ১৮)। `sudo grep "Failed password" /var/log/auth.log | tail -10` দিয়ে দেখুন একই সময়ে অস্বাভাবিক লগইন চেষ্টা ছিল কি না (অধ্যায় ১৯)।

সমাধানের ক্রম: (১) বড় লগ ফাইলটি `truncate -s 0` দিয়ে নিরাপদে খালি করুন, যাতে ডিস্কে জায়গা ফেরে; (২) লগ বড় হওয়ার মূল কারণ খুঁজুন — কোন সার্ভিস বারবার একই ভুল লিখছিল, তার configuration ঠিক করুন; (৩) সার্ভিসটি

চালু করুন ও `journalctl -f` দিয়ে দেখুন তা চালু থাকছে কি না; (৪) `top` দিয়ে দেখুন CPU/memory এখন স্বাভাবিক কি না; (৫) `ss -tlnp` ও `ufw status` দিয়ে দেখুন অপ্রয়োজনীয় পোর্ট খোলা নেই তো; (৬) সমস্যা ঠিক হওয়ার পর `tar` দিয়ে গুরুত্বপূর্ণ তথ্যের ব্যাকআপ নিন ও তা যাচাই করুন (এই অধ্যায়)।

প্রতিরোধের ব্যবস্থা: logrotate সক্রিয় আছে কি না যাচাই করুন; ডিস্ক ৮০% ছাড়ালে সতর্ক হওয়ার নিয়ম লিখুন; সাপ্তাহিক ব্যাকআপ যাচাইয়ের সময় ঠিক করুন; ঘটনাটি লিখে রাখুন — কখন শুরু হলো, কারণ কী ছিল, কী করলেন, ভবিষ্যতে কী করবেন। এই লিখিত নোটই আপনাকে পরের বার একই সমস্যায় দ্রুত সিদ্ধান্ত নিতে সাহায্য করবে। একজন ভালো অ্যাডমিন শুধু সমস্যা ঠিক করেন না — তিনি এমন ব্যবস্থা করেন যাতে একই সমস্যা আবার না হয়, আর হলেও দ্রুত ধরা পড়ে।

সারসংক্ষেপ

ব্যাকআপ মানে তথ্যের আরেকটি কপি, এমন জায়গায় যা মূলের সঙ্গে একসঙ্গে নষ্ট হবে না। 3-2-1 নীতি, full ও incremental-এর ভারসাম্য, `tar` ও `rsync`-এর সঠিক ব্যবহার এবং — সবচেয়ে গুরুত্বপূর্ণ — ফিরিয়ে এনে যাচাই করা — এই চারটি মিলে একটি নির্ভরযোগ্য ব্যাকআপ ব্যবস্থা তৈরি হয়। আর একজন প্রশাসকের দৈনন্দিন কাজ হলো ছোট ছোট পরীক্ষার সমষ্টি: load, ডিস্ক, RAM, ব্যর্থ সার্ভিস, ভুল লগ ও লগইন চেষ্টা দেখা — যাতে ছোট সমস্যা বড় হওয়ার আগেই ধরা পড়ে।

Revision Checklist — পুনরালোচনার তালিকা

- ☐ 3-2-1, RPO ও RTO ব্যাখ্যা করতে পারি
- ☐ `tar` দিয়ে ব্যাকআপ তৈরি, তালিকা দেখা ও ফিরিয়ে আনতে পারি
- ☐ `rsync`-এ `--dry-run` দিয়ে পরীক্ষা করতে পারি
- ☐ ব্যাকআপ ফিরিয়ে এনে যাচাই করতে পারি
- ☐ দৈনিক, সাপ্তাহিক ও মাসিক রক্ষণাবেক্ষণ চেকলিস্ট অনুসরণ করতে পারি
- ☐ TechNova পরিস্থিতিতে তদন্তের ক্রম বলতে পারি: রিসোর্স → ডিস্ক → লগ → সার্ভিস → নিরাপত্তা → ব্যাকআপ

পরিশিষ্ট A — Guided Labs (গাইডেড ল্যাব)

এই অংশের ১৫টি ল্যাব বইয়ের ২০টি অধ্যায়ের হাতে-কলমে প্রয়োগ। প্রতিটি ল্যাব Ubuntu Server 24.04 LTS-এ পরীক্ষা করার জন্য তৈরি। কোনো ল্যাবেই ধ্বংসাত্মক কমান্ড অন্ধভাবে চালাতে বলা হয়নি; যেখানে ঝুঁকি আছে, সেখানে সতর্কতা ও নিরাপদ বিকল্প দেওয়া হয়েছে। সব expected output **illustrative (উদাহরণমূলক)** — আপনার সিস্টেমে নাম, সংখ্যা, তারিখ বা UUID ভিন্ন হতে পারে। Lab শুরু করার আগে VirtualBox-এ একটি snapshot নিন, যাতে ভুল হলে ফিরে আসা যায়।

> **সতর্কতা:** Lab-এ `rm`, `mkfs`, `dd`, `chmod 777`, firewall পরিবর্তন — এসব কমান্ড শুধু নিজের VM-এ চালাবেন। হোস্ট কম্পিউটার বা অন্য কারও সার্ভারে নয়। Root হিসেবে কাজ করার আগে কমান্ডটি দুইবার পড়ুন।

> **কীভাবে ল্যাব করবেন:** প্রতিটি ল্যাবের ধাপগুলো ক্রমানুসারে করুন। একটি ধাপের verification না মিললে পরের ধাপে যাবেন না — Troubleshooting অংশ দেখুন। প্রতিটি ল্যাব শেষে Cleanup অংশটি করুন, যাতে পরের ল্যাব পরিষ্কার অবস্থা থেকে শুরু হয়।

Lab 01 — VirtualBox-এ Ubuntu Server 24.04 LTS ইনস্টল ও প্রথম Boot

> **Scenario:** আপনি ঢাকার একটি বিশ্ববিদ্যালয়ের ছাত্র। আপনার ল্যাপটপে Windows আছে, কিন্তু Linux শিখতে একটি নিরাপদ আলাদা পরিবেশ দরকার। হোস্ট OS নষ্ট না করে VirtualBox-এ Ubuntu Server চালাবেন।

শেখার উদ্দেশ্য: VM তৈরি, Ubuntu Server 24.04 LTS ইনস্টল, প্রথম login, নেটওয়ার্ক ও SSH যাচাই করতে পারবেন।

Prerequisites: অধ্যায় ০৭। হোস্টে কমপক্ষে 4 GB RAM, 25 GB ফাঁকা ডিস্ক।

Tools / Environment: VirtualBox 7.x, Ubuntu Server 24.04 LTS ISO (ubuntu.com থেকে), NAT নেটওয়ার্ক।

Initial state: হোস্টে কোনো VM নেই; ISO ডাউনলোড করা আছে।

ধাপসমূহ

1. VirtualBox-এ New VM তৈরি করুন: নাম `ubuntu-lab`, RAM 2048 MB, Disk 20 GB (VDI, dynamically allocated)।
2. Settings → Storage-এ ISO যুক্ত করুন, Network NAT রাখুন, Start করুন।
3. Installer-এ ভাষা English, Keyboard layout, নেটওয়ার্ক DHCP, Storage: Use an entire disk (LVM নয়, প্রথমবারের জন্য সরল রাখুন) নির্বাচন করুন।
4. Profile: আপনার নাম, সার্ভারের নাম `ubuntu-lab`, username (যেমন `nahid`), শক্তিশালী পাসওয়ার্ড দিন। OpenSSH server ইনস্টল করতে টিক দিন।
5. Install শেষে Reboot করুন, ISO detach করুন, username/password দিয়ে login করুন।

```
lsb_release -a
uname -r
```

```
ip a
systemctl status ssh --no-pager
sudo apt update
```

কমান্ড ব্যাখ্যা: `lsb_release -a` — distribution ও version দেখায়; `uname -r` — চলমান kernel version; `ip a` — নেটওয়ার্ক interface ও IP ঠিকানা; `systemctl status ssh` — SSH service চলছে কি না; `sudo apt update` — package তালিকা হালনাগাদ করে, কিছু ইনস্টল করে না।

Expected output (illustrative):

```
Description:    Ubuntu 24.04 LTS
Release:        24.04
Linux 6.8.0-xx-generic
2: enp0s3: inet 10.0.2.15/24
Active: active (running) – OpenBSD Secure Shell server
```

Verification: `lsb_release -a`-তে 24.04 দেখা যায়; `ip a`-তে IP আছে; `ping -c 3 8.8.8.8` সফল হয়।

সাধারণ ব্যর্থতা: ISO boot না হলে Storage-এ ISO ঠিকমতো যুক্ত হয়েছে কি না দেখুন; VT-x/AMD-V BIOS-এ বন্ধ থাকলে VirtualBox ত্রুটি দেবে — BIOS থেকে virtualization চালু করুন।

> **Troubleshooting:** Login-এ ভুল password হলে Caps Lock দেখুন। নেটওয়ার্ক না পেলে VM Settings → Network → NAT ও Cable Connected টিক আছে কি না দেখুন।

Cleanup / Reset: VM বন্ধ করে VirtualBox-এ Snapshot নিন — নাম `fresh-install`। এটি পরের সব ল্যাবের নিরাপদ ফিরতি পয়েন্ট।

Lab 02 — Filesystem Exploration: কোথায় কী থাকে

> **Scenario:** নতুন সার্ভারে login করে আপনি জানতে চান — configuration কোথায়, log কোথায়, ব্যবহারকারীর ফাইল কোথায়। ভুল ডিরেক্টরিতে ফাইল তৈরি করে সিস্টেম নষ্ট করতে চান না।

শেখার উদ্দেশ্য: Filesystem Hierarchy Standard (FHS) অনুযায়ী প্রধান ডিরেক্টরিগুলো চিনতে ও `ls`, `find` দিয়ে অনুসন্ধান করতে পারবেন।

Prerequisites: অধ্যায় ০৮। Lab 01 সম্পন্ন।

Tools / Environment: Ubuntu Server 24.04 LTS VM, সাধারণ user।

Initial state: Fresh install, home ডিরেক্টরি প্রায় খালি।

ধাপসমূহ

1. Root ও home ঘুরে দেখুন।
2. প্রতিটি প্রধান ডিরেক্টরির ভেতরে কী আছে তালিকা করুন।
3. `/etc`-তে SSH ও hostname সংক্রান্ত ফাইল খুঁজুন।

4. `/var/log`-এ সাম্প্রতিক log ফাইলের আকার দেখুন।

```
pwd; ls -la /
ls -ld /etc /home /var /tmp /usr /opt /boot /dev /proc /sys
ls /etc | head -20
ls -lh /var/log/ | head -20
find /etc -maxdepth 1 -name "*ssh*" -o -maxdepth 1 -name "hostname*"
df -h / /home
```

কমান্ড ব্যাখ্যা: `ls -ld` — ডিরেক্টরি নিজের তথ্য দেখায়, ভেতরের ফাইল নয়; `ls -lh` — human-readable আকার; `find` — শর্ত মিলিয়ে ফাইল খোঁজে; `df -h` — মাউন্ট করা filesystem-এর ফাঁকা জায়গা।

Expected output (illustrative): `/etc`-তে `ssh/`, `hostname`, `passwd` দেখা যায়; `/var/log/`-তে `syslog`, `auth.log`, `apt/`; `df -h`-তে `/` প্রায় 20 GB, ব্যবহৃত অল্প।

Verification: `cat /etc/hostname`-তে `ubuntu-lab` মেলে; `cat /proc/cpuinfo | head` CPU তথ্য দেয় — বুঝুন `/proc` আসল ফাইল নয়, kernel-এর live view।

সাধারণ ব্যর্থতা: `/root`-এ ঢুকতে Permission denied — এটি স্বাভাবিক, root-এর home সাধারণ user-এর জন্য বন্ধ। `sudo ls /root` দিয়ে যাচাই করুন।

> **সতর্কতা:** `/etc`, `/boot`, `/proc`, `/sys`-এ কিছু মুছবেন বা সম্পাদনা করবেন না — এগুলো সিস্টেমের প্রাণ। অনুশীলন শুধু `~/lab02` ডিরেক্টরিতে করুন।

Cleanup / Reset: `mkdir -p ~/lab02 && cd ~/lab02`-তে তৈরি করা পরীক্ষামূলক ফাইল মুছে ফেলুন: `rm -rf ~/lab02`।

Lab 03 — Shell, Pipes ও Text Processing: Log থেকে তথ্য বের করা

> **Scenario:** একটি ওয়েব সার্ভারের access log-এ হাজার লাইন আছে। আপনাকে জানতে হবে — কোন IP সবচেয়ে বেশি request করেছে, কয়টি 404 error, কোন URL জনপ্রিয়। GUI নয়, shell দিয়েই উত্তর বের করতে হবে।

শেখার উদ্দেশ্য: pipe, redirection, `grep`, `sort`, `uniq`, `awk`, `cut` দিয়ে বাস্তব log বিশ্লেষণ করতে পারবেন।

Prerequisites: অধ্যায় ০৯–১০।

Tools / Environment: Ubuntu VM, Bash।

Initial state: `~/lab03` খালি।

ধাপসমূহ

1. একটি নমুনা log ফাইল তৈরি করুন।
2. Error লাইন গুনুন, IP অনুযায়ী সাজান, সবচেয়ে বেশি request করা IP বের করুন।
3. ফলাফল একটি ফাইলে সংরক্ষণ করুন।

```
mkdir -p ~/lab03 && cd ~/lab03
cat > access.log << 'EOF'
192.168.1.10 - - [08/Nov/2026] "GET /index.html HTTP/1.1" 200
192.168.1.20 - - [08/Nov/2026] "GET /missing.html HTTP/1.1" 404
192.168.1.10 - - [08/Nov/2026] "GET /about.html HTTP/1.1" 200
192.168.1.20 - - [08/Nov/2026] "GET /missing.html HTTP/1.1" 404
192.168.1.30 - - [08/Nov/2026] "GET /index.html HTTP/1.1" 200
EOF
wc -l access.log
grep "404" access.log
awk '{print $1}' access.log | sort | uniq -c | sort -nr
grep -c " 200 " access.log
awk '{print $7}' access.log | sort | uniq -c | sort -nr > top-urls.txt
cat top-urls.txt
```

কমান্ড ব্যাখ্যা: `awk '{print $1}'` — প্রতি লাইনের প্রথম field (IP) নেয়; `sort | uniq -c` — গুনে সাজায়; `sort -nr` — সংখ্যা অনুযায়ী উল্টো সাজায়; `>` — output ফাইলে পাঠায়, `>>` হলে যোগ করত; heredoc `<< 'EOF'` — বহু লাইন একসাথে ফাইলে লেখে।

Expected output (illustrative): `wc -l` → 5; `grep "404"` → 2 লাইন; IP গণনায় `192.168.1.10` ও `192.168.1.20` প্রত্যেকে 2 বার; `top-urls.txt`-তে `/index.html` 2 বার সবার উপরে।

Verification: `cat top-urls.txt` ও হাতে গোনা সংখ্যা মিলিয়ে দেখুন।

সাধারণ ব্যর্থতা: `uniq`-এর আগে `sort` না করলে গণনা ভুল হবে — `uniq` শুধু পাশাপাশি একই লাইন গোনে। Quoting ভুল হলে `awk` ভিন্ন ফল দেবে; single quote ব্যবহার করুন।

> **Pro Tip:** বড় ফাইলে প্রথমে `head -20` দিয়ে গঠন দেখে নিন, তারপর পুরো ফাইলে pipe চালান। ভুল pipe-এ পুরো ফাইল নষ্ট হওয়ার ঝুঁকি কমে।

Cleanup / Reset: `cd ~ && rm -rf ~/lab03`।

Lab 04 — Users, Groups ও Permissions: একটি টিমের ফোল্ডার তৈরি

> **Scenario:** TechNova-তে দুইজন নতুন intern যোগ দিয়েছেন — `intern1` ও `intern2`। তারা `/srv/technova` শেয়ার্ড ফোল্ডারে কাজ করবেন, কিন্তু একে অপরের ব্যক্তিগত ফাইল দেখতে পারবেন না। আপনাকে user, group ও permission সাজাতে হবে।

শেখার উদ্দেশ্য: user/group তৈরি, `chmod` / `chown`, `umask` বুঝে প্রয়োগ করতে পারবেন।

Prerequisites: অধ্যায় ১১। sudo অধিকার থাকা user।

Tools / Environment: Ubuntu VM, sudo।

Initial state: `intern1`, `intern2` কেউ নেই; `/srv/technova` নেই।

ধাপসমূহ

1. Group ও user তৈরি করুন, শেয়ার্ড ডিরেক্টরি বানান।

2. মালিকানা ও permission দিন, যাচাই করুন।
3. একজন user হিসেবে লেখার পরীক্ষা করুন।

```
sudo groupadd technova
sudo useradd -m -s /bin/bash -G technova intern1
sudo useradd -m -s /bin/bash -G technova intern2
sudo passwd intern1
sudo mkdir -p /srv/technova/shared /srv/technova/private-intern1
sudo chown root:technova /srv/technova/shared
sudo chmod 2770 /srv/technova/shared
sudo chown intern1:intern1 /srv/technova/private-intern1
sudo chmod 700 /srv/technova/private-intern1
id intern1; id intern2
ls -ld /srv/technova/shared /srv/technova/private-intern1
sudo -u intern1 touch /srv/technova/shared/test-intern1.txt && echo OK
sudo -u intern2 ls /srv/technova/private-intern1 || echo "denied as expected"
```

কমান্ড ব্যাখ্যা: `useradd -m` — home ডিরেক্টরি তৈরি করে; `-G technova` — supplementary group; `chmod 2770` — setgid (2) + group rwx (7) + owner rwx (7) + other কিছুই নয় (0): নতুন ফাইল group `technova`-র হবে; `chmod 700` — শুধু মালিকের অধিকার; `sudo -u intern1` — ওই user হিসেবে কমান্ড চালায়।

Expected output (illustrative): `id intern1`-তে `groups=...technova`; `ls -ld`-তে `drwxrws--- root technova` ও `drwx------ intern1 intern1`; শেষ লাইনে `denied as expected`।

Verification: `su - intern1` দিয়ে login করে `touch /srv/technova/shared/myfile` সফল ও `touch /srv/technova/private-intern1/x` ব্যর্থ হওয়া দেখুন।

সাধারণ ব্যর্থতা: `passwd` না দিলে user login করতে পারবে না; `chmod 777` দিলে সবাই সবকিছু করতে পারবে — এটি ভুল, নিরাপত্তা ঝুঁকি। Setgid ভুলে গেলে নতুন ফাইলের group user-এর primary group হবে।

> **সতর্কতা:** কখনোই `chmod -R 777 /srv` বা `/`-তে চালাবেন না। Lab শেষে তৈরি user মুছে ফেলুন।

Cleanup / Reset:

```
sudo rm -rf /srv/technova
sudo userdel -r intern1
sudo userdel -r intern2
sudo groupdel technova
```

Lab 05 — ACLs ও sudo: Least Privilege প্রয়োগ

> **Scenario:** একজন deploy user-কে একটি নির্দিষ্ট অ্যাপ ডিরেক্টরিতে লেখার অধিকার দিতে হবে, কিন্তু তাকে পুরো group-এর সদস্য করা যাবে না। আর একজন junior admin-কে শুধু `systemctl restart nginx` চালানোর অধিকার দিতে হবে, পূর্ণ root নয়।

শেখার উদ্দেশ্য: `getfacl` / `setfacl` ও `sudoers`-এ সীমিত অধিকার দিতে পারবেন।

Prerequisites: অধ্যায় ১১–১২, Lab 04।

Tools / Environment: Ubuntu VM, sudo, `acl` package।

Initial state: `/srv/app` নেই; `deployer` ও `junior` user নেই।

ধাপসমূহ

1. ACL package ও user তৈরি করুন।
2. শুধু `deployer`-কে একটি ডিরেক্টরিতে rwx দিন, অন্যরা যেন না পায়।
3. `junior`-কে শুধু একটি কমান্ডের sudo অধিকার দিন ও পরীক্ষা করুন।

```
sudo apt update && sudo apt install -y acl
sudo useradd -m -s /bin/bash deployer
sudo useradd -m -s /bin/bash junior
sudo mkdir -p /srv/app
sudo chown root:root /srv/app
sudo chmod 750 /srv/app
sudo setfacl -m u:deployer:rwx /srv/app
sudo setfacl -d -m u:deployer:rwx /srv/app
getfacl /srv/app
sudo -u deployer touch /srv/app/deploy.txt && echo "deployer OK"
sudo -u junior touch /srv/app/no.txt || echo "junior denied as expected"
echo "junior ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx" | sudo tee /etc/sudoers.d/
junior-nginx
sudo chmod 440 /etc/sudoers.d/junior-nginx
sudo visudo -c
sudo -u junior sudo -l -U junior
```

কমান্ড ব্যাখ্যা: `setfacl -m u:deployer:rwx` — নির্দিষ্ট user-কে অধিকার দেয়, group বদলায় না; `-d` — default ACL, নতুন ফাইলেও একই অধিকার; `getfacl` — ACL দেখায়; `visudo -c` — sudoers syntax যাচাই করে, ভুল থাকলে sudo ভেঙে যেতে পারে বলে এটি বাধ্যতামূলক; `sudo -l -U junior` — ওই user কী কী sudo করতে পারে তা দেখায়।

Expected output (illustrative): `getfacl`-তে `user:deployer:rwx` ও `default:user:deployer:rwx`; `visudo -c` → `parsed OK`; `sudo -l`-তে শুধু `/usr/bin/systemctl restart nginx` দেখা যায়।

Verification: `sudo -u junior sudo systemctl restart nginx` চেষ্টা করলে (nginx না থাকলে service ত্রুটি আসবে, কিন্তু permission ত্রুটি আসবে না) — আর `sudo -u junior sudo apt update` করলে permission denied আসা উচিত।

সাধারণ ব্যর্থতা: `/etc/sudoers` সরাসরি সম্পাদনা করে syntax ভুল করলে sudo বন্ধ হয়ে যায় — সবসময় `visudo` ব্যবহার করুন। ACL কাজ না করলে filesystem-এ ACL support আছে কি না দেখুন (Ubuntu-র ext4-এ ডিফল্ট চালু থাকে)।

> **সতর্কতা:** `NOPASSWD: ALL` কখনো দেবেন না। প্রতিটি sudo অধিকার একটি নির্দিষ্ট কমান্ডে সীমিত রাখুন — এটাই least privilege।

Cleanup / Reset:

```
sudo rm /etc/sudoers.d/junior-nginx
sudo rm -rf /srv/app
sudo userdel -r deployer
sudo userdel -r junior
sudo visudo -c
```

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 06 — Process Management: ঝুলে যাওয়া প্রসেস খুঁজে থামানো

> **Scenario:** সার্ভার হঠাৎ ধীর। `top`-এ দেখা যাচ্ছে একটি প্রসেস CPU খাচ্ছে। আপনাকে দায়ী প্রসেস শনাক্ত করে নিরাপদে থামাতে হবে, পুরো সার্ভার reboot না করে।

শেখার উদ্দেশ্য: `ps`, `top` / `htop`, `kill`, signal, foreground/background job পরিচালনা করতে পারবেন।

Prerequisites: অধ্যায় ০৫, ১৩।

Tools / Environment: Ubuntu VM, `htop`।

Initial state: সিস্টেম স্বাভাবিক; কোনো test প্রসেস চলছে না।

ধাপসমূহ

1. একটি নিরীহ CPU-খরুচে test প্রসেস background-এ চালান।
2. PID খুঁজে, resource দেখে, প্রথমে নরম signal, প্রয়োজনে জোরালো signal দিয়ে থামান।
3. একটি sleep job দিয়ে `jobs`, `bg`, `fg` অনুশীলন করুন।

```
sudo apt update && sudo apt install -y htop
yes > /dev/null &
echo $!
ps aux | grep -v grep | grep yes
top -b -n 1 | head -20
kill -TERM %1
sleep 1; ps aux | grep -v grep | grep yes || echo "stopped"
sleep 300 &
jobs
bg %1
fg %1
# Ctrl+Z চেপে suspend করুন, তারপর:
# bg %1  — background-এ চালু রাখুন;  kill %1 — থামান
pstree -p | head -20
```

কমান্ড ব্যাখ্যা: `yes > /dev/null &` — `yes` অনবরত output দেয়, `&` background-এ পাঠায়; `$!` — শেষ background job-এর PID; `kill -TERM` (signal 15) — প্রসেসকে ভদ্রভাবে বন্ধ হতে বলে, ফাইল গুছিয়ে নেওয়ার সুযোগ দেয়; `kill -9` (SIGKILL) — জোর করে থামায়, শেষ উপায়; `jobs` — বর্তমান shell-এর job তালিকা; `pstree -p` — parent-child গাছ।

Expected output (illustrative): `ps aux`-তে `yes` প্রসেসের %CPU প্রায় 100; `kill -TERM`-এর পর `ps`-তে আর নেই, `stopped` ছাপা হয়।

Verification: `ps aux | grep yes` খালি; `uptime`-এ load কমে আসে।

সাধারণ ব্যর্থতা: ভুল PID-তে `kill` করলে গুরুত্বপূর্ণ service বন্ধ হতে পারে — `ps -o pid,ppid,cmd -p <PID>` দিয়ে নিশ্চিত হোন। প্রথমেই `kill -9` দেবেন না; data হারাতে পারে। Zombie প্রসেস `kill`-এ যায় না — তার parent-কে দেখতে হয়।

> **সতর্কতা:** `pkill -9` বা `killall` pattern মিলিয়ে চালাবেন না — নামের আংশিক মিলে অন্য প্রসেস মারা যেতে পারে। সবসময় PID দিয়ে, একটিকে লক্ষ্য করে kill করুন।

Cleanup / Reset: `jobs` দেখে বাকি job থাকলে `kill %1` করুন; `ps aux | grep yes` খালি নিশ্চিত করুন।

Lab 07 — APT Package Management: নিরাপদে ইনস্টল, আপডেট ও পরিষ্কার

> **Scenario:** সার্ভারে `nginx` ও `curl` দরকার। কিন্তু গত সপ্তাহে `apt upgrade` অর্ধেক অবস্থায় থেমে গিয়েছিল। আপনাকে package system সুস্থ করে নিরাপদে ইনস্টল করতে হবে।

শেখার উদ্দেশ্য: `apt` / `dpkg`, repository, dependency, ভাঙা package মেরামত করতে পারবেন।

Prerequisites: অধ্যায় ১৪।

Tools / Environment: Ubuntu VM, ইন্টারনেট।

Initial state: `nginx` ইনস্টল নেই।

ধাপসমূহ

1. Package তালিকা হালনাগাদ, package তথ্য দেখুন।
2. `nginx` ও `curl` ইনস্টল করে সংস্করণ ও ফাইল যাচাই করুন।
3. Upgrade অনুকরণ (dry-run) দেখে তারপর আসল upgrade করুন।
4. অপ্রয়োজনীয় package পরিষ্কার করুন।

```
sudo apt update
apt show nginx | head -20
apt-cache policy nginx curl
sudo apt install -y nginx curl
nginx -v
curl --version | head -1
dpkg -l | grep -E "nginx|curl"
which nginx; dpkg -L nginx | head -10
sudo apt upgrade --dry-run | head -30
sudo apt upgrade -y
sudo apt autoremove -y
sudo apt autoclean
sudo apt list --upgradable
```

কমান্ড ব্যাখ্যা: `apt update` — repository থেকে তালিকা আনে, কিছু ইনস্টল করে না; `apt-cache policy` — কোন version, কোন repository থেকে আসবে দেখায়; `dpkg -l` — ইনস্টল করা package তালিকা; `dpkg -L` — package-এর ফাইলগুলো কোথায় বসেছে; `autoremove` — আর দরকার নেই এমন dependency মুছে দেয়; `--dry-run` — না চালিয়ে কী হবে দেখায়।

Expected output (illustrative): `nginx -v` → `nginx/1.24.0 (Ubuntu)` (সংস্করণ আপনার সময় অনুযায়ী ভিন্ন হতে পারে); `curl --version`-এ সংস্করণ; `apt list --upgradable` upgrade-এর পর খালি বা প্রায় খালি।

Verification: `systemctl status nginx --no-pager` active; ব্রাউজার বা `curl http://localhost`-এ Ubuntu nginx welcome page।

সাধারণ ব্যর্থতা: `Could not get lock /var/lib/dpkg/lock` — আরেকটি apt চলছে; অপেক্ষা করুন, lock ফাইল জোর করে মুছবেন না। `dpkg was interrupted` হলে `sudo dpkg --configure -a` তারপর `sudo apt -f install`।

> **Troubleshooting:** `404 Not Found` repository ত্রুটি হলে `sudo apt update` আবার চালান; অজানা PPA যোগ করবেন না — তৃতীয় পক্ষের repository নিরাপত্তা ঝুঁকি।

Cleanup / Reset: Lab 15 পর্যন্ত nginx রেখে দিন (পরে লাগবে); শুধু এই ল্যাবের জন্য মুছতে চাইলে `sudo apt remove --purge nginx -y && sudo apt autoremove -y`।

Lab 08 — systemd: নিজের একটি Custom Service তৈরি

> **Scenario:** TechNova-র একটি ছোট Python স্ক্রিপ্ট সার্ভার চালু হলেই চলতে হবে, থেমে গেলে নিজে থেকে আবার চালু হতে হবে, আর log `journalctl`-এ দেখা যেতে হবে। `nohup` দিয়ে চালানো পেশাদার সমাধান নয় — systemd service দরকার।

শেখার উদ্দেশ্য: unit file লেখা, `systemctl enable/start/status`, `journalctl` দিয়ে service পরিচালনা করতে পারবেন।

Prerequisites: অধ্যায় ১৫, Lab 07।

Tools / Environment: Ubuntu VM, Python 3।

Initial state: `/opt/hello-app` নেই, service নেই।

ধাপসমূহ

1. একটি সহজ Python অ্যাপ ও unit file তৈরি করুন।
2. Daemon reload করে service enable ও start করুন।
3. Log দেখুন, service থামিয়ে restart আচরণ পরীক্ষা করুন।

```
sudo mkdir -p /opt/hello-app
cat > /tmp/hello.py << 'EOF'
import time
while True:
    print("hello from technova app", flush=True)
    time.sleep(10)
EOF
sudo cp /tmp/hello.py /opt/hello-app/hello.py
sudo tee /etc/systemd/system/hello-app.service > /dev/null << 'EOF'
[Unit]
```

```
Description=TechNova Hello App
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/hello-app/hello.py
Restart=on-failure
RestartSec=5
User=nobody
Group=nogroup

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable hello-app.service
sudo systemctl start hello-app.service
sudo systemctl status hello-app.service --no-pager
journalctl -u hello-app.service --no-pager -n 20
sudo systemctl restart hello-app.service
journalctl -u hello-app.service --no-pager -n 10
```

কমান্ড ব্যাখ্যা: `[Unit]` — বিবরণ ও dependency; `After=network.target` — নেটওয়ার্কের পরে শুরু; `[Service]` `ExecStart` — চালানোর কমান্ড; `Restart=on-failure` — crash হলে আবার চালু; `User=nobody` — root নয়, সীমিত user-এ চালানো (least privilege); `daemon-reload` — unit file বদলের পর systemd-কে আবার পড়তে বলে; `enable` — boot-এ চালু হবে, `start` — এখনই চালু।

Expected output (illustrative): `status`-তে `Active: active (running)`; `journalctl`-তে বারবার `hello from technova app` লাইন।

Verification: `systemctl is-enabled hello-app` → `enabled`; `ps aux | grep hello.py`-তে প্রসেস দেখা যায়।

সাধারণ ব্যর্থতা: `daemon-reload` না করলে পুরোনো unit চলবে; `ExecStart`-এ ভুল path হলে `status`-তে `failed (Result: exit-code)` — `journalctl -u hello-app -n 50` দিয়ে কারণ দেখুন। Root-এ অ্যাপ চালাবেন না — `User=` দিন।

> **সতর্কতা:** `Restart=always` ভুল configuration-এ বারবার crash-loop তৈরি করতে পারে — প্রথমে `on-failure` ব্যবহার করুন, log দেখুন।

Cleanup / Reset:

```
sudo systemctl stop hello-app.service
sudo systemctl disable hello-app.service
sudo rm /etc/systemd/system/hello-app.service /opt/hello-app/hello.py
sudo systemctl daemon-reload
sudo rm -rf /opt/hello-app /tmp/hello.py
```

Lab 09 — Disk Management: ডিস্ক ভরা, Partition ও fstab

> **Scenario:** সার্ভারের `/` 92% ভরা। আপনাকে খুঁজে বের করতে হবে কোন ডিরেক্টরি বড়, একটি অতিরিক্ত ভার্চুয়াল ডিস্ক যুক্ত করে mount করতে হবে, যাতে reboot-এর পরও থাকে।

শেখার উদ্দেশ্য: `lsblk`, `df`, `du`, mount, `/etc/fstab` ব্যবহার করতে পারবেন।

Prerequisites: অধ্যায় ১৬। Snapshot নেওয়া থাকা বাধ্যতামূলক।

Tools / Environment: Ubuntu VM; VirtualBox-এ একটি অতিরিক্ত 5 GB VDI (VM বন্ধ করে যুক্ত করুন)।

Initial state: একটি ডিস্ক (`/dev/sda`); অতিরিক্ত ডিস্ক যুক্ত হয়েছে, কিন্তু format/mount হয়নি।

ধাপসমূহ

1. ডিস্ক ও ব্যবহার দেখুন, বড় ডিরেক্টরি খুঁজুন।
2. নতুন ডিস্কে filesystem তৈরি করে `/mnt/data`-তে mount করুন।
3. `fstab`-এ UUID দিয়ে স্থায়ী করুন, reboot ছাড়া যাচাই করুন।

```
lsblk -f
df -h
sudo du -sh /var/* 2>/dev/null | sort -rh | head -10
sudo du -sh /home/* 2>/dev/null | sort -rh | head -10
# নতুন ডিস্কটি চিনুন (সাধারণত /dev/sdb, আকার দিয়ে নিশ্চিত হোন)
lsblk
sudo mkfs.ext4 /dev/sdb
sudo mkdir -p /mnt/data
sudo mount /dev/sdb /mnt/data
df -h /mnt/data
sudo blkid /dev/sdb
# blkid থেকে UUID কপি করে নিচের লাইনে বসান (উদাহরণ UUID বদলে নিন)
echo "UUID=PASTE-UUID-HERE  /mnt/data  ext4  defaults  0  2" | sudo tee -a /etc/fstab
sudo umount /mnt/data
sudo mount -a
df -h /mnt/data
ls -ld /mnt/data
```

কমান্ড ব্যাখ্যা: `lsblk -f` — block device, filesystem ও mountpoint; `du -sh` — ডিরেক্টরির মোট আকার; `mkfs.ext4` — নতুন filesystem তৈরি (আগের সব তথ্য মুছে যায়); `blkid` — UUID দেখায়; `fstab`-এ UUID ব্যবহার করলে device নাম (`sdb`) বদলে গেলেও mount ঠিক থাকে; `mount -a` — `fstab`-এর সব entry আবার mount করে, reboot ছাড়াই পরীক্ষা করা যায়।

Expected output (illustrative): `df -h /mnt/data`-তে 5 GB ডিস্ক, ব্যবহৃত প্রায় 0; `mount -a`-এর পর কোনো ত্রুটি নেই।

Verification: `findmnt /mnt/data` ও `cat /etc/fstab | grep data` মিলিয়ে দেখুন; VM reboot করে `df -h`-তে `/mnt/data` আবার আছে কি না দেখুন।

সাধারণ ব্যর্থতা: ভুল ডিস্কে `mkfs` চালালে মূল ডিস্ক মুছে যাবে — আকার ও `lsblk` দিয়ে তিনবার নিশ্চিত হোন। `fstab`-এ ভুল UUID থাকলে boot আটকে যেতে পারে — `mount -a` দিয়ে আগেই যাচাই করুন, snapshot রাখুন।

> **সতর্কতা:** `mkfs` ও `dd` ধ্বংসাত্মক। শুধু নতুন, খালি ভার্চুয়াল ডিস্কে চালান। `/dev/sda`-তে কখনো নয়।

Cleanup / Reset: `sudo umount /mnt/data`, `fstab` থেকে লাইনটি মুছে `sudo mount -a` করুন; চাইলে VM বন্ধ করে অতিরিক্ত VDI সরিয়ে দিন।

Lab 10 — Resource Monitoring: CPU, RAM, I/O Bottleneck খুঁজে বের করা

> **Scenario:** ব্যবহারকারীরা বলছেন অ্যাপ ধীর। CPU, RAM, নাকি ডিস্ক — কোনটি bottleneck, তা না জেনে RAM বাড়ানো অর্থের অপচয়। আপনাকে তথ্য দিয়ে প্রমাণ করতে হবে।

শেখার উদ্দেশ্য: `uptime`, `free`, `vmstat`, `top` / `htop`, `iostat` দিয়ে bottleneck শনাক্ত করতে পারবেন।

Prerequisites: অধ্যায় ০৫, ১৭, Lab 06।

Tools / Environment: Ubuntu VM, `sysstat` package।

Initial state: সিস্টেম স্বাভাবিক।

ধাপসমূহ

1. Baseline মাপুন (স্বাভাবিক অবস্থা)।
2. ইচ্ছাকৃত load তৈরি করে আবার মাপুন ও তুলনা করুন।
3. কোন resource চাপে আছে, তা লিখে সিদ্ধান্ত নিন।

```
sudo apt update && sudo apt install -y sysstat
uptime
free -h
vmstat 1 5
top -b -n 1 | head -20
iostat -x 1 3
# Load তৈরি (VM-এ, অল্প সময়ের জন্য)
stress --version 2>/dev/null || sudo apt install -y stress
timeout 20 stress --cpu 2 --vm 1 --vm-bytes 256M &
uptime; free -h; vmstat 1 5; iostat -x 1 3
wait
uptime; free -h
```

কমান্ড ব্যাখ্যা: `uptime`-এর load average (1, 5, 15 মিনিট) — CPU core সংখ্যার সাথে তুলনা করুন; `free -h`-তে `available` কলামই আসল ফাঁকা RAM, `free` নয়; `vmstat`-এ `si/so` (swap in/out) বেশি হলে RAM চাপে, `wa` বেশি হলে I/O অপেক্ষা; `iostat -x`-এ `%util` 90%+ হলে ডিস্ক bottleneck; `stress` — পরীক্ষার জন্য কৃত্রিম load।

Expected output (illustrative): Baseline-এ load 0.10-এর নিচে, `available` RAM পর্যাপ্ত, `%util` কম। Load চলাকালে load average 2-এর উপরে, `free`-এ available কমে, `vmstat`-এ CPU `us` বাড়ে। Load শেষে আবার স্বাভাবিক।

Verification: Baseline ও load অবস্থার সংখ্যা পাশাপাশি লিখুন — কোন কলাম বদলেছে, তা দিয়ে bottleneck-এর নাম বলুন।

সাধারণ ব্যর্থতা: শুধু `free` দেখে RAM শেষ ভাবা — Linux cache-এর জন্য RAM ব্যবহার করে, `available` দেখুন। Load average-কে শতাংশ ভাবা — এটি core সংখ্যার সাপেক্ষে বুঝতে হয় (2 core-এ load 2.0 মানে সম্পূর্ণ ব্যস্ত)।

> **Pro Tip:** একবার মেপে সিদ্ধান্ত নেবেন না। `vmstat 1 5` বা `iostat` কয়েকবার চালিয়ে ধারা (trend) দেখুন — এক মুহূর্তের spike বিভ্রান্ত করে।

Cleanup / Reset: `stress` শেষ হলে `ps aux | grep stress` খালি নিশ্চিত করুন; প্রয়োজনে `pkill stress` নয়, PID দিয়ে `kill` করুন।

পরিশিষ্ট A — Guided Labs (চলমান)

Lab 11 — Log Troubleshooting: কেন Service থামল, Log পড়ে খুঁজুন

> **Scenario:** সকালে nginx চালু হচ্ছে না। কেউ বলছে "সার্ভার ডাউন", কিন্তু কারণ কেউ জানে না। আপনাকে অনুমান নয়, log পড়ে কারণ প্রমাণ করতে হবে।

শেখার উদ্দেশ্য: `journalctl`, `/var/log`, `dmesg` দিয়ে service failure-এর মূল কারণ (root cause) খুঁজতে পারবেন।

Prerequisites: অধ্যায় ১৫, ১৮, Lab 07–০৮।

Tools / Environment: Ubuntu VM, nginx ইনস্টল করা।

Initial state: nginx চলছে।

ধাপসমূহ

1. ইচ্ছাকৃতভাবে একটি ভুল configuration তৈরি করে nginx ভাঙুন (VM-এ, নিরাপদে)।
2. Service status, journal ও error log পড়ে কারণ শনাক্ত করুন।
3. ভুল সারিয়ে service আবার চালু করুন।

```
sudo systemctl status nginx --no-pager
sudo cp /etc/nginx/nginx.conf /tmp/nginx.conf.bak
echo "this is not valid nginx config" | sudo tee -a /etc/nginx/nginx.conf
sudo systemctl restart nginx || echo "failed as expected"
sudo systemctl status nginx --no-pager
journalctl -u nginx --no-pager -n 50
sudo tail -50 /var/log/nginx/error.log
sudo nginx -t
sudo cp /tmp/nginx.conf.bak /etc/nginx/nginx.conf
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl status nginx --no-pager
sudo tail -20 /var/log/nginx/error.log
dmesg | tail -20
```

কমান্ড ব্যাখ্যা: `nginx -t` — configuration পরীক্ষা করে, service চালু না করে; `journalctl -u nginx` — শুধু ওই unit-এর log; `tail -50` — ফাইলের শেষ ৫০ লাইন, যেখানে সাম্প্রতিক ত্রুটি থাকে; `dmesg` — kernel ring buffer, hardware/driver ত্রুটি দেখায়। প্রথমে backup নেওয়া হয়েছে বলে ভাঙা অবস্থা থেকে ফেরা যায়।

Expected output (illustrative): ভাঙা অবস্থায় `nginx -t` → `emerg` / syntax error ও লাইন নম্বর; `status` → `failed`; মেরামতের পর `nginx -t` → `syntax is ok`, `test is successful`; `status` → `active (running)`।

Verification: `curl -I http://localhost`-তে `HTTP/1.1 200 OK`; `journalctl -u nginx --since "5 minutes ago"`-তে নতুন কোনো error নেই।

সাধারণ ব্যর্থতা: শুধু `restart` বারবার চালানো — কারণ না জেনে restart করলে একই ত্রুটি ফিরে আসে। Log না পড়ে configuration অনুমান করে বদলানো। `/var/log`-এর ফাইল সরাসরি সম্পাদনা করা — log পড়ার জিনিস, বদলানোর নয়।

> **Troubleshooting:** ত্রুটির লাইনে ফাইল নাম ও লাইন নম্বর দেওয়া থাকে — সেখান থেকেই শুরু করুন। সময় (timestamp) মিলিয়ে দেখুন: failure-এর ঠিক আগের লাইনগুলোই সবচেয়ে গুরুত্বপূর্ণ।

Cleanup / Reset: `rm /tmp/nginx.conf.bak`; `sudo systemctl status nginx` active নিশ্চিত করুন।

Lab 12 — UFW Firewall ও SSH Hardening: সার্ভারের দরজা বন্ধ করা

> **Scenario:** TechNova-র সার্ভার ইন্টারনেটে যাবে। এখন সব port খোলা। আপনাকে শুধু SSH (22) ও HTTP (80) খুলে বাকি সব বন্ধ করতে হবে, আর password দিয়ে SSH login বন্ধ করে key-based login চালু করতে হবে।

শেখার উদ্দেশ্য: UFW দিয়ে firewall নিয়ম, SSH key তৈরি, `sshd_config` নিরাপদ করা ও `ss` দিয়ে listening port যাচাই করতে পারবেন।

Prerequisites: অধ্যায় ১৯, Lab 01, Lab 07।

Tools / Environment: Ubuntu VM, `ufw`, `openssh-server`।

Initial state: UFW inactive; SSH password login চালু।

ধাপসমূহ

1. বর্তমান listening port দেখুন।
2. SSH অনুমতি দিয়ে তারপর UFW চালু করুন (ক্রমটি গুরুত্বপূর্ণ)।
3. SSH key জোড়া তৈরি করে key login পরীক্ষা করুন, তারপর password login বন্ধ করুন।

```
sudo ss -tulpn
sudo ufw status verbose
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw enable
sudo ufw status verbose
sudo ss -tulpn
ssh-keygen -t ed25519 -C "nahid@ubuntu-lab" -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ls -l ~/.ssh/
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
sudo grep -E "PasswordAuthentication|PermitRootLogin" /etc/ssh/sshd_config
```

কমান্ড ব্যাখ্যা: `ss -tulpn` — listening TCP/UDP port, প্রসেস ও PID; `ufw allow OpenSSH` — SSH profile অনুযায়ী port 22 খোলে; `ufw enable`-এর আগে SSH allow না করলে নিজেই lockout হয়ে যাবেন — এই Lab-এ VM

console আছে বলে ঝুঁকি কম, কিন্তু আসল সার্ভারে এটি সবচেয়ে সাধারণ ভুল; `ssh-keygen -t ed25519` — আধুনিক, ছোট ও নিরাপদ key; `authorized_keys`-এ public key থাকলে ওই private key দিয়ে password ছাড়া login হয়; private key (`id_ed25519`) কখনো কাউকে দেবেন না, শেয়ার করবেন না।

Expected output (illustrative): `ufw status`-তে `Status: active`, `22/tcp ALLOW`, `80/tcp ALLOW`; `ss`-তে `0.0.0.0:22` ও `0.0.0.0:80` listening; `ls -l ~/.ssh/`-তে `authorized_keys` 600।

Verification: নতুন terminal থেকে `ssh -i ~/.ssh/id_ed25519 nahid@localhost` (VM-এ) password ছাড়া ঢোকে কি না দেখুন। `sudo ufw status numbered` দিয়ে নিয়মের ক্রম দেখুন।

সাধারণ ব্যর্থতা: `~/.ssh` 700 ও `authorized_keys` 600 না হলে SSH key উপেক্ষা করে — permission-ই কারণ। `sshd_config` বদলে `systemctl reload ssh` না করলে পরিবর্তন কার্যকর হয় না।

> **সতর্কতা:** এই ল্যাবে `PasswordAuthentication no` সরাসরি চালু করবেন না — প্রথমে key login পরীক্ষা করে নিশ্চিত হোন। Key হারালে ও password বন্ধ থাকলে সার্ভারে আর ঢোকা যাবে না। আসল সার্ভারে বদলের আগে `sshd_config.bak` রাখুন ও console access নিশ্চিত করুন। Private key কখনো Git, চ্যাট বা স্ক্রিনশটে দেবেন না।

Cleanup / Reset: Lab শেষে VM-এ UFW চাইলে চালু রাখুন; পরীক্ষার key মুছতে চাইলে `rm ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub` ও `authorized_keys` থেকে লাইনটি সরান। `sshd_config.bak` থেকে ফিরতে `sudo cp /etc/ssh/sshd_config.bak /etc/ssh/sshd_config && sudo systemctl reload ssh`।

Lab 13 — Backup ও Restore: tar ও rsync দিয়ে তথ্য ফিরিয়ে আনা

> **Scenario:** TechNova-র `/srv/technova`-তে গুরুত্বপূর্ণ project ফাইল আছে। ডিস্ক নষ্ট হলে বা ভুলে মুছে গেলে তথ্য ফিরিয়ে আনতে হবে। Backup নেওয়া হয়নি — এমন কথা শোনার আগেই backup ও restore পরীক্ষা করতে হবে।

শেখার উদ্দেশ্য: `tar` দিয়ে archive, `rsync` দিয়ে incremental backup, checksum যাচাই ও restore করতে পারবেন।

Prerequisites: অধ্যায় ২০, Lab 04।

Tools / Environment: Ubuntu VM, `rsync`।

Initial state: `/srv/technova`-তে কিছু নমুনা ফাইল আছে (না থাকলে তৈরি করুন)।

ধাপসমূহ

1. নমুনা তথ্য তৈরি করে tar backup নিন ও checksum লিখে রাখুন।
2. rsync দিয়ে দ্বিতীয় backup নিন, একটি ফাইল বদলে আবার rsync চালিয়ে incremental আচরণ দেখুন।
3. মূল ফাইল মুছে (VM-এ, নমুনা তথ্য) backup থেকে restore করুন।

```
sudo apt update && sudo apt install -y rsync
sudo mkdir -p /srv/technova/docs
echo "Project plan v1" | sudo tee /srv/technova/docs/plan.txt
echo "Client notes" | sudo tee /srv/technova/docs/notes.txt
sudo mkdir -p /backup
sudo tar -czvf /backup/technova-2026-11-08.tar.gz -C /srv technova
ls -lh /backup/
```

```
sha256sum /backup/technova-2026-11-08.tar.gz | sudo tee /backup/technova-2026-11-08.sha256
tar -tzvf /backup/technova-2026-11-08.tar.gz | head -20
sudo rsync -av /srv/technova/ /backup/technova-rsync/
echo "Project plan v2 - updated" | sudo tee /srv/technova/docs/plan.txt
sudo rsync -av /srv/technova/ /backup/technova-rsync/
# Restore পরীক্ষা (নমুনা তথ্য, VM-এ)
sudo rm -rf /srv/technova/docs
sudo mkdir -p /tmp/restore-test
sudo tar -xzvf /backup/technova-2026-11-08.tar.gz -C /tmp/restore-test
ls -R /tmp/restore-test
cat /tmp/restore-test/technova/docs/plan.txt
sha256sum -c /backup/technova-2026-11-08.sha256
```

কমান্ড ব্যাখ্যা: `tar -czvf` — c=create, z=gzip compress, v=verbose, f=file; `-C /srv technova` — `/srv`-তে গিয়ে `technova` archive করে, absolute path এড়ায়; `sha256sum` — backup অক্ষত আছে কি না যাচাইয়ের fingerprint; `rsync -av` — a=archive (permission, সময় ধরে রাখে), v=verbose; দ্বিতীয়বার rsync শুধু বদলে যাওয়া ফাইল কপি করে — এটাই incremental-এর মূল ধারণা; `tar -tzvf` — না খুলে ভেতরে কী আছে দেখা।

Expected output (illustrative): tar ফাইল কয়েক KB; `tar -tzvf`-তে `technova/docs/plan.txt` দেখা যায়; দ্বিতীয় rsync-এ শুধু `plan.txt` কপি হয়; restore-এর পর `plan.txt`-তে `Project plan v1` ফিরে আসে; `sha256sum -c` → `OK`।

Verification: Restore করা ফাইলের বিষয়বস্তু মূলের সাথে মিলিয়ে দেখুন; `diff -r /tmp/restore-test/technova /backup/technova-rsync` (v2 পার্থক্য থাকবে — কেন, তা বুঝুন: tar backup v1 সময়ের, rsync v2)।

সাধারণ ব্যর্থতা: Backup নেওয়া কিন্তু restore পরীক্ষা না করা — সবচেয়ে বড় ভুল; backup ফাইল ও মূল তথ্য একই ডিস্কে রাখা — ডিস্ক নষ্ট হলে দুটোই যায়; `rsync`-এ trailing slash (`/srv/technova/` বনাম `/srv/technova`) ভুল হলে গন্তব্যে অতিরিক্ত ডিরেক্টরি তৈরি হয়।

> **সতর্কতা:** `rm -rf` শুধু এই ল্যাবের নমুনা ডিরেক্টরিতে চালান। আসল তথ্যে restore পরীক্ষা করার আগে backup-এর checksum যাচাই করুন।

Cleanup / Reset: `sudo rm -rf /tmp/restore-test /backup/technova-*`; চাইলে `/srv/technova` রেখে দিন Project B-র জন্য।

Lab 14 — Disk Full Investigation: হঠাৎ ডিস্ক ভরে গেলে কী করবেন

> **Scenario:** সকালে অ্যাপ লিখতে পারছে না — `No space left on device`। সার্ভার reboot নয়, প্রথমে কোথায় জায়গা শেষ হয়েছে তা খুঁজতে হবে, তারপর নিরাপদে জায়গা খালি করতে হবে।

শেখার উদ্দেশ্য: Disk full-এর কারণ শনাক্ত, নিরাপদ cleanup, log rotation ধারণা প্রয়োগ করতে পারবেন।

Prerequisites: অধ্যায় ১৬, ১৮, Lab 09।

Tools / Environment: Ubuntu VM।

Initial state: ডিস্ক স্বাভাবিক; পরীক্ষার জন্য একটি বড় নমুনা ফাইল তৈরি করা হবে।

ধাপসমূহ

1. একটি বড় নমুনা ফাইল তৈরি করে ডিস্ক চাপ অনুকরণ করুন (ছোট আকারে, VM-এ)।
2. `df` ও `du` দিয়ে দায়ী ডিরেক্টরি খুঁজুন।
3. নিরাপদে মুছে জায়গা ফিরিয়ে আনুন; কী মুছবেন না, তা লিখুন।

```
df -h /
df -i /
sudo du -sh /* 2>/dev/null | sort -rh | head -10
sudo du -sh /var/log/* 2>/dev/null | sort -rh | head -10
sudo du -sh /tmp/* ~/.* 2>/dev/null | sort -rh | head -10
# অনুকরণ: /tmp-এ 500 MB নমুনা ফাইল (আসল ডিস্ক ভরবেন না)
fallocate -l 500M /tmp/big-test-file
df -h /
ls -lh /tmp/big-test-file
# সবচেয়ে বড় ফাইলগুলো খুঁজুন
sudo find /tmp /var/log -type f -size +10M -exec ls -lh {} \; 2>/dev/null | head -20
sudo journalctl --disk-usage
sudo journalctl --vacuum-size=100M
rm /tmp/big-test-file
df -h /
sudo apt autoremove -y; sudo apt autoclean
```

কমান্ড ব্যাখ্যা: `df -i` — inode শেষ হয়েছে কি না দেখায় (জায়গা থাকলেও inode শেষ হলে ফাইল তৈরি হয় না); `du -sh /*` — root-এর প্রতিটি ডিরেক্টরির আকার; `fallocate` — দ্রুত বড় ফাইল তৈরি করে, পরীক্ষার জন্য; `journalctl --vacuum-size=100M` — journal log 100 MB-এ নামিয়ে আনে; `find -size +10M` — বড় ফাইল খুঁজে বের করে।

Expected output (illustrative): `du`-তে `/tmp/big-test-file` 500M দেখা যায়; মুছে ফেলার পর `df -h`-তে Available আবার বাড়ে; `journalctl --disk-usage`-তে journal আকার দেখা যায়।

Verification: মুছে ফেলার আগে ও পরে `df -h /` লিখে তুলনা করুন; `touch /tmp/test-write && rm /tmp/test-write` দিয়ে লেখা যায় কি না পরীক্ষা করুন।

সাধারণ ব্যর্থতা: চলমান প্রসেসের খোলা log ফাইল মুছলে জায়গা ফেরে না — প্রসেসটি ফাইল handle ধরে রাখে; তখন service restart বা `truncate` দরকার (এই ল্যাবে `journalctl --vacuum` নিরাপদ পথ)। `/var/log`-এর সব ফাইল মুছে ফেলা — তদন্তের প্রমাণ নষ্ট হয়; logrotate-এর নিয়ম বুঝে cleanup করুন। Inode শেষ হলে ছোট ছোট অসংখ্য ফাইল (যেমন cache/session) দায়ী হতে পারে — `find` দিয়ে সংখ্যা গুনুন।

> **সতর্কতা:** `rm -rf /var/log/*` বা `/tmp`-এর সবকিছু অন্ধভাবে মুছবেন না। কী মুছছেন, কেন মুছছেন — আগে `ls` দিয়ে দেখুন।

Cleanup / Reset: `rm -f /tmp/big-test-file /tmp/test-write`; `df -h /` স্বাভাবিক নিশ্চিত করুন।

Lab 15 — Final Server Health Check: একটি সার্ভারের সম্পূর্ণ স্বাস্থ্য পরীক্ষা

> **Scenario:** আপনি TechNova-র নতুন junior sysadmin। সোমবার সকালে সাপ্তাহিক health check রিপোর্ট দিতে হবে: সার্ভার চালু আছে কি না, ডিস্ক/RAM/CPU ঠিক আছে কি না, service চলছে কি না, log-এ ত্রুটি আছে কি না, backup হয়েছে কি না, update বাকি আছে কি না।

শেখার উদ্দেশ্য: আগের সব ল্যাবের দক্ষতা একত্র করে একটি পুনরাবৃত্তিযোগ্য health-check রুটিন তৈরি ও রিপোর্ট লিখতে পারবেন।

Prerequisites: Lab 01–১৪ সবকটি (অন্তত Lab 01, 07, 08, 10, 11, 13)।

Tools / Environment: Ubuntu VM, সব আগের service।

Initial state: সার্ভার চলছে; nginx ও hello-app (যদি থাকে) চলছে।

ধাপসমূহ

1. একটি health-check স্ক্রিপ্ট তৈরি করুন।
2. চালিয়ে রিপোর্ট ফাইলে সংরক্ষণ করুন।
3. প্রতিটি অংশের ফল নিজে ব্যাখ্যা করে একটি সংক্ষিপ্ত সিদ্ধান্ত লিখুন: OK / Watch / Action needed।

```
cat > ~/health-check.sh << 'EOF'
#!/bin/bash
echo "=== TechNova Server Health Check: $(date) ==="
echo "--- Uptime & Load ---"
uptime
echo "--- Disk ---"
df -h /
df -i /
echo "--- Memory ---"
free -h
echo "--- Services ---"
systemctl is-active nginx ssh
systemctl --failed --no-pager
echo "--- Updates ---"
apt list --upgradable 2>/dev/null | head -10
echo "--- Recent Errors ---"
journalctl -p err --since "24 hours ago" --no-pager -n 20
echo "--- Listening Ports ---"
sudo ss -tulpn
echo "--- Backup ---"
ls -lh /backup/ 2>/dev/null || echo "No /backup directory found"
EOF
chmod +x ~/health-check.sh
~/health-check.sh | tee ~/health-report-$(date +%F).txt
cat ~/health-report-*.txt | head -60
```

কমান্ড ব্যাখ্যা: `systemctl --failed` — ব্যর্থ service তালিকা করে; `journalctl -p err --since "24 hours ago"` — গত ২৪ ঘণ্টার error-level log; `tee` — output একই সাথে স্ক্রিনে ও ফাইলে পাঠায়; `$(date +%F)` — ফাইলের নামে আজকের তারিখ (যেমন 2026-11-08) বসায়; স্ক্রিপ্টের প্রথম লাইন `#!/bin/bash` (shebang) বলে কোন interpreter দিয়ে চালাতে হবে।

Expected output (illustrative): প্রতিটি অংশে শিরোনামসহ ফলাফল; `systemctl is-active`-তে `active`; `--failed` খালি (বা শুধু জানা ব্যর্থ service); error log-এ নতুন গুরুতর ত্রুটি নেই; `/backup/`-তে Lab 13-এর ফাইল দেখা যায়।

Verification: রিপোর্ট ফাইলটি খুলে প্রতিটি অংশে হাতে লিখুন: OK / Watch / Action needed। অন্তত একটি Action খুঁজে বের করুন (যেমন বাকি update, বা journal আকার) ও তা করুন।

সাধারণ ব্যর্থতা: স্ক্রিপ্টে `chmod +x` না করলে Permission denied; `sudo` দরকার এমন অংশ (যেমন `ss -p`) স্ক্রিপ্টে sudo ছাড়া চালালে তথ্য অসম্পূর্ণ থাকবে — কোন অংশে sudo দরকার, তা নোট করুন। রিপোর্ট তৈরি করে কাউকে না পাঠানো — health check-এর উদ্দেশ্য সিদ্ধান্ত, শুধু ফাইল নয়।

> **Pro Tip:** এই স্ক্রিপ্টকে সাপ্তাহিক cron job-এ পরিণত করা যায়, কিন্তু প্রথমে হাতে চালিয়ে প্রতিটি লাইনের অর্থ বুঝুন। Automation বোঝার বিকল্প নয়।

Cleanup / Reset: রিপোর্ট ফাইল `~/health-report-*.txt` সংরক্ষণ করুন (Project B-র deliverable হিসেবে লাগবে); স্ক্রিপ্টটি `~/health-check.sh`-এ রেখে দিন।

পরিশিষ্ট B — Independent Assignments (স্বাধীন অ্যাসাইনমেন্ট)

নিচের ১০টি অ্যাসাইনমেন্টের সমাধান এখানে দেওয়া হয়নি — সমাধান আলাদা Answer Guide-এ আছে। প্রথমে নিজে চেষ্টা করুন, আটকে গেলে অধ্যায় ও ল্যাব দেখুন, তারপর Answer Guide মিলিয়ে নিন। প্রতিটি অ্যাসাইনমেন্ট VM-এ করুন, নিজের কাজের ধাপ ও ফলাফল একটি নোট ফাইলে লিখে রাখুন।

> **নিয়ম:** Beginner (১–৪) একা করা যাবে। Intermediate (৫–৭) করার আগে সংশ্লিষ্ট ল্যাব শেষ করুন। Advanced (৮–১০) সময় নিয়ে, log পড়ে, ধাপে ধাপে করুন — সরাসরি সমাধান খুঁজবেন না।

Beginner — অ্যাসাইনমেন্ট ১–৪

Assignment 01 — আমার সার্ভারের পরিচয়পত্র

নিজের VM-এ একটি `system-info.txt` ফাইল তৈরি করুন, যেখানে থাকবে: Ubuntu version, kernel version, CPU মডেল ও core সংখ্যা, মোট RAM, ডিস্ক আকার ও ফাঁকা জায়গা, hostname, IP ঠিকানা, uptime। প্রতিটি তথ্যের পাশে কোন কমান্ড দিয়ে পেয়েছেন, তা লিখুন। ফাইলটি `~/assignments/` ডিরেক্টরিতে রাখুন।

Assignment 02 — Log থেকে গল্প বের করা

Lab 03-এর মতো নিজে একটি ২০ লাইনের নমুনা access log তৈরি করুন (অন্তত ৩টি IP, ৩টি URL, 200/404/500 status মিশিয়ে)। তারপর shell কমান্ড দিয়ে বের করুন: (ক) মোট request, (খ) প্রতিটি status code কতবার, (গ) সবচেয়ে বেশি request করা IP, (ঘ) সবচেয়ে জনপ্রিয় URL। ফলাফল `log-report.txt`-এ লিখুন ও প্রতিটি কমান্ডের ব্যাখ্যা এক লাইনে দিন।

Assignment 03 — নিরাপদ ফাইল অপারেশন

`~/practice` ডিরেক্টরিতে ১০টি ফাইল তৈরি করুন (নামে তারিখ ও ধরন মিশিয়ে, যেমন `report-2026-11-08.txt`)। তারপর: (ক) `.txt` ফাইলগুলো `~/practice/text/`-এ সরান, (খ) একটি ফাইলের hard link ও একটি symbolic link তৈরি করে পার্থক্য পরীক্ষা করুন (মূল ফাইল মুছে কী হয়?), (গ) `tar` দিয়ে পুরো `practice` ডিরেক্টরির backup নিন, (ঘ) ভুল করে `rm` চালানোর ঝুঁকি এড়াতে আপনি কী সতর্কতা নেবেন, তা তিন লাইনে লিখুন।

Assignment 04 — User ও Permission পরিকল্পনা

একটি ছোট দলের জন্য পরিকল্পনা লিখুন (কাগজে বা Markdown ফাইলে): ২ জন developer, ১ জন designer, ১ জন intern। কে কোন group-এ থাকবে, শেয়ার্ড ফোল্ডার `/srv/team`-এর permission কেমন হবে (সংখ্যা ও অক্ষরে), intern কী দেখতে পারবে না — তা যুক্তিসহ লিখুন। তারপর VM-এ পরিকল্পনাটি বাস্তবায়ন করে `ls -ld` ও `id` output দিয়ে যাচাই করুন।

Intermediate — অ্যাসাইনমেন্ট ৫–৭

Assignment 05 — Package ও Service রুটিন

VM-এ `htop` ও `curl` ইনস্টল আছে কি না দেখুন; না থাকলে ইনস্টল করুন। তারপর: (ক) `apt update` ও `apt list --upgradable` চালিয়ে কয়টি update বাকি, তা লিখুন, (খ) `nginx` ইনস্টল করে চালু করুন ও `curl http://localhost` দিয়ে যাচাই করুন, (গ) nginx-কে boot-এ চালু হওয়ার জন্য enable করুন ও `systemctl is-enabled nginx` দিয়ে প্রমাণ করুন, (ঘ) একটি package-এর version ও repository `apt-cache policy` দিয়ে দেখান। প্রতিটি ধাপের output সংরক্ষণ করুন।

Assignment 06 — নিজের Service লেখা

Lab 08-এর ধাঁচে, কিন্তু ভিন্ন অ্যাপ দিয়ে একটি systemd service তৈরি করুন: একটি Bash স্ক্রিপ্ট প্রতি ৩০ সেকেন্ডে তারিখ ও `uptime` একটি ফাইলে লিখবে (`/var/log/myapp.log` নয় — প্রথমে `/tmp/myapp.log` ব্যবহার করুন, কেন তা ব্যাখ্যা করুন)। Service-টি `nobody` user-এ চালান, boot-এ enable করুন, `journalctl` দিয়ে log দেখান, service থামিয়ে আবার চালু করে আচরণ লিখুন।

Assignment 07 — Disk ও Resource রিপোর্ট

নিজের VM-এর একটি এক পৃষ্ঠার resource রিপোর্ট তৈরি করুন: load average, `free -h` (available ব্যাখ্যাসহ), সবচেয়ে বড় ৫টি ডিরেক্টরি (`du` দিয়ে), সবচেয়ে বেশি CPU ও RAM ব্যবহার করা ৫টি প্রসেস, ডিস্ক ব্যবহার শতাংশ, inode ব্যবহার। শেষে এক অনুচ্ছেদে লিখুন — এই সার্ভারে এখনই কোনো সমস্যা আছে কি না, থাকলে কী করবেন।

Advanced Troubleshooting — অ্যাসাইনমেন্ট ৮–১০

Assignment 08 — "ওয়েবসাইট খুলছে না" — ধাপে ধাপে তদন্ত

VM-এ nginx চালু থাকা অবস্থায় ইচ্ছাকৃতভাবে একটি সমস্যা তৈরি করুন (যেমন ভুল configuration, বা service বন্ধ, বা firewall-এ port 80 বন্ধ — যেকোনো একটি)। তারপর ধরুন আপনি কিছুই জানেন না: `systemctl status`, `journalctl`, `ss -tulpn`, `curl`, `ufw status` — এই ক্রমে তদন্ত করে মূল কারণ লিখুন, মেরামত করুন, আবার `curl` দিয়ে প্রমাণ করুন। তদন্তের প্রতিটি ধাপে কী দেখলেন ও কী সিদ্ধান্ত নিলেন, তা লিখুন।

Assignment 09 — "ডিস্ক ভরে গেছে" — নিরাপদ উদ্ধার

Lab 14-এর ধাঁচে একটি বড় ফাইল বা অসংখ্য ছোট ফাইল দিয়ে ডিস্ক চাপ তৈরি করুন। তারপর: (ক) `df -h` ও `df -i` — কোনটি শেষ হয়েছে, তা নির্ণয় করুন, (খ) `du` ও `find` দিয়ে দায়ী জায়গা খুঁজুন, (গ) কী মুছবেন ও কী মুছবেন না — তার তালিকা যুক্তিসহ লিখুন, (ঘ) নিরাপদে জায়গা খালি করে `touch` দিয়ে লেখা যায় কি না প্রমাণ করুন। ভুল cleanup-এর ঝুঁকি কী, তা এক অনুচ্ছেদে লিখুন।

Assignment 10 — "SSH-এ ঢোকা যাচ্ছে না" — Access সমস্যার তদন্ত

VM-এ SSH access-এ একটি সমস্যা তৈরি করুন (যেমন `authorized_keys` permission বদলে দেওয়া, বা UFW-তে SSH বন্ধ করা, বা ভুল user দিয়ে চেষ্টা — যেকোনো একটি, VM console access হাতে রেখে)। তারপর console থেকে তদন্ত করুন: service চলছে কি না, port listening আছে কি না, firewall নিয়ম, `~/.ssh` permission, `/var/log/auth.log`-এ কী লেখা আছে। মূল কারণ, মেরামতের ধাপ ও ভবিষ্যতে এড়ানোর উপায় লিখুন।

> **সতর্কতা:** Assignment ৮–১০-এ ইচ্ছাকৃত সমস্যা তৈরির আগে snapshot নিন। আসল সার্ভারে বা অন্য কারও সিস্টেমে এসব পরীক্ষা করবেন না।

পরিশিষ্ট C — Major Projects (তিনটি বড় প্রজেক্ট)

তিনটি প্রজেক্টই VM-এ করবেন। প্রতিটি প্রজেক্টের deliverable একটি ফোল্ডার বা রিপোর্ট — শুধু "করেছি" বললে হবে না, প্রমাণ (কমান্ড output, configuration ফাইল, স্ক্রিনশটের বিবরণ) জমা দিতে হবে।

Project A — Linux Workstation: নিজের Linux পরিবেশ তৈরি

> **Business Scenario:** আপনি একজন junior software engineer। আপনার দৈনন্দিন কাজ — কোড লেখা, Git, terminal, ডকুমেন্টেশন — সব Linux-এ করতে চান। একটি নির্ভরযোগ্য personal workstation দরকার, যা আপনি নিজে ইনস্টল, সাজিয়েছেন ও রক্ষণাবেক্ষণ করেন।

Requirements: Ubuntu Server 24.04 LTS বা Ubuntu Desktop 24.04 LTS VM; নিজের user account (sudo সহ); প্রয়োজনীয় tool ইনস্টল করা (Git, curl, Python 3, একটি text editor); home ডিরেক্টরির সুসংগঠিত কাঠামো; নিয়মিত update রুটিন; backup পরিকল্পনা।

Architecture — Project A

Host Laptop (Windows/macOS) → VirtualBox → Ubuntu VM (2 GB RAM, 20 GB Disk) → User: nahid (sudo) → Home: ~/code, ~/docs, ~/backup → Tools: Git, Python 3, curl, htop → Backup: ~/backup (rsync, সাপ্তাহিক)

Tasks ও Milestones:

1. Milestone 1 — VM ইনস্টল ও প্রথম configuration (Lab 01 অনুযায়ী)।
2. Milestone 2 — প্রয়োজনীয় software ইনস্টল ও সংস্করণ নথিভুক্ত করা।
3. Milestone 3 — Filesystem কাঠামো: `~/code`, `~/docs`, `~/downloads`, `~/backup` তৈরি ও উদ্দেশ্য লেখা।
4. Milestone 4 — Shell পরিবেশ: `.bashrc`-এ দরকারী alias (যেমন `ll`) যোগ করা, প্রতিটির ব্যাখ্যা লেখা।
5. Milestone 5 — Backup: `~/docs`-এর rsync backup ও restore পরীক্ষা।
6. Milestone 6 — Health-check রিপোর্ট (Lab 15-এর স্ক্রিপ্ট ব্যবহার)।

Deliverables: (১) `workstation-report.md` — system তথ্য, ইনস্টল করা software ও সংস্করণ, filesystem কাঠামো, alias তালিকা; (২) backup ও restore প্রমাণ; (৩) health-check রিপোর্ট।

Acceptance Criteria: VM reboot-এর পর সব service ও configuration অক্ষত থাকে; `git --version`, `python3 --version` চলে; backup থেকে একটি ফাইল restore করে দেখানো যায়; রিপোর্টে প্রতিটি সিদ্ধান্তের কারণ লেখা আছে।

Verification Checklist:

- Ubuntu 24.04 LTS প্রমাণ (`lsb_release -a`) আছে কি?
- সব software-এর সংস্করণ লেখা আছে কি?
- Home কাঠামো ও alias ব্যাখ্যাসহ আছে কি?
- Backup ও restore পরীক্ষা হয়েছে কি?

- Health-check রিপোর্টে OK/Watch/Action লেখা আছে কি?

Evaluation Rubric:

মানদণ্ড	Excellent	Good	Needs Work
System documentation	সব তথ্য, সংস্করণ ও কারণসহ সম্পূর্ণ	তথ্য আছে, কিছু কারণ অনুপস্থিত	তথ্য অসম্পূর্ণ বা ভুল
Filesystem ও shell	সুসংগঠিত, alias ব্যাখ্যাসহ	কাঠামো আছে, ব্যাখ্যা আংশিক	এলোমেলো, ব্যাখ্যা নেই
Backup ও restore	Restore পরীক্ষাসহ প্রমাণিত	Backup আছে, restore পরীক্ষা হয়নি	Backup নেই বা ভুল
Health check	সিদ্ধান্তসহ সম্পূর্ণ রিপোর্ট	রিপোর্ট আছে, সিদ্ধান্ত আংশিক	রিপোর্ট নেই

Optional Advanced Challenges: SSH key দিয়ে password-ছাড়া login; একটি cron job দিয়ে সাপ্তাহিক backup স্বয়ংক্রিয় করা (প্রথমে হাতে পরীক্ষা করে); `ufw` চালু করে শুধু প্রয়োজনীয় port খোলা।

Project B — Linux Server Administration: একটি সার্ভারের পূর্ণ দায়িত্ব

> **Business Scenario:** TechNova-র একটি Ubuntu সার্ভারে ৩ জন developer ও ১ জন intern কাজ করবেন। সার্ভারে একটি ওয়েব অ্যাপ (nginx), নিয়মিত backup, log পর্যবেক্ষণ ও নিরাপত্তা — সব আপনার দায়িত্ব।

Requirements: Ubuntu Server 24.04 LTS VM; ৪ জন user ও ২টি group; nginx service; UFW firewall; `/srv/technova` শেয়ার্ড ডিরেক্টরি; সাপ্তাহিক backup; health-check রুটিন।

Architecture — Project B

Internet → UFW (22, 80 only) → Ubuntu Server 24.04 → nginx (80) → /srv/technova (developers group, 2770) → Users: dev1, dev2, designer, intern1 → Logs: journalctl + /var/log/nginx → Backup: /backup (tar + rsync, সাপ্তাহিক) → Admin: health-check.sh

Tasks ও Milestones:

1. Milestone 1 — User/group তৈরি, শেয়ার্ড ডিরেক্টরি ও permission (Lab 04–০৫)।
2. Milestone 2 — nginx ইনস্টল, enable, `curl` দিয়ে যাচাই (Lab 07)।
3. Milestone 3 — Custom service (Lab 08) — অ্যাপ log `journalctl`-এ দেখা যায়।
4. Milestone 4 — UFW ও SSH hardening (Lab 12) — শুধু 22 ও 80 খোলা।
5. Milestone 5 — Backup ও restore পরীক্ষা (Lab 13)।
6. Milestone 6 — Health-check রিপোর্ট ও maintenance checklist (Lab 15)।

Deliverables: (১) `server-admin-report.md` — user/group তালিকা, permission ব্যাখ্যা, service তালিকা, firewall নিয়ম, backup পরিকল্পনা; (২) health-check রিপোর্ট; (৩) restore পরীক্ষার প্রমাণ।

Acceptance Criteria: প্রতিটি user শুধু তার অধিকার অনুযায়ী access পায় (intern শেয়ার্ড ফোল্ডারে লিখতে পারে, অন্যের private ফোল্ডারে নয়); nginx reboot-এর পরও চলে; UFW-তে শুধু 22 ও 80 ALLOW; backup থেকে একটি ফাইল restore করা যায়; কোনো failed service নেই।

Verification Checklist:

- `id` ও `ls -ld /srv/technova` পরিকল্পনা অনুযায়ী কি?
- `systemctl status nginx` active ও enabled কি?
- `ufw status`-তে শুধু প্রয়োজনীয় নিয়ম কি?
- `journalctl -u nginx -n 20`-তে নতুন error আছে কি?
- Backup checksum OK ও restore পরীক্ষা হয়েছে কি?

Evaluation Rubric:

মানদণ্ড	Excellent	Good	Needs Work
Users ও permissions	Least privilege, যাচাইসহ সম্পূর্ণ	কাঠামো ঠিক, যাচাই আংশিক	777 বা অতিরিক্ত অধিকার আছে
Services	সব service active, enabled, log পরিষ্কার	চলছে, কিন্তু enable/log আংশিক	Service ব্যর্থ বা বন্ধ
Security	UFW + SSH key, নিয়ম নথিভুক্ত	Firewall আছে, SSH আংশিক	Firewall বন্ধ বা সব port খোলা
Backup ও monitoring	Restore পরীক্ষাসহ, health রিপোর্ট সিদ্ধান্তসহ	Backup আছে, restore/রিপোর্ট আংশিক	Backup বা রিপোর্ট নেই

Optional Advanced Challenges: `logrotate` configuration বুঝে একটি log-এর rotation পরীক্ষা; fail2ban-এর ধারণা পড়ে SSH brute-force সুরক্ষা নিয়ে এক পৃষ্ঠা লেখা (ইনস্টল বাধ্যতামূলক নয়); disk quota-র ধারণা পড়ে `/srv/technova`-র ব্যবহার সীমা নিয়ে পরিকল্পনা লেখা।

Project C — Production Server Challenge: TechNova Production Server

> **Business Scenario:** TechNova একটি ছোট সফটওয়্যার কোম্পানি। আপনি তার junior Linux systems administrator। কোম্পানির Ubuntu সার্ভারে কোম্পানির ওয়েবসাইট (nginx) ও একটি internal অ্যাপ চলে। সোমবার সকালে একসাথে কয়েকটি সমস্যা ধরা পড়েছে — service থেমে গেছে, ডিস্ক প্রায় ভরা, CPU বেশি, একজন কর্মী SSH-এ ঢুকতে পারছেন না। আপনাকে সার্ভার প্রস্তুত, সুরক্ষিত, পর্যবেক্ষণ ও রক্ষণাবেক্ষণ করতে হবে, সমস্যাগুলোর তদন্ত করে backup থেকে recovery দেখাতে হবে।

Requirements: Ubuntu Server 24.04 LTS VM (Project B-র সার্ভার ব্যবহার করা যাবে); nginx + একটি custom service; UFW; backup (`/backup`); health-check স্ক্রিপ্ট; incident লগ (প্রতিটি সমস্যা, কারণ, মেরামত, সময়)।

Architecture — Project C

Users / Internet → UFW (22, 80) → Ubuntu Server 24.04 → nginx (website, port 80) + hello-app (systemd service) → /srv/technova (team data) → /var/log + journalctl (monitoring) → /backup (tar + rsync, off-disk কপির পরিকল্পনাসহ) → Admin (health-check.sh, incident log)

Tasks ও Milestones:

1. Milestone 1 — Baseline: সার্ভার প্রস্তুত করুন (user, service, firewall, backup) ও baseline health রিপোর্ট লিখুন।
2. Milestone 2 — Simulated Incident 1: nginx service failure — log পড়ে কারণ খুঁজে মেরামত করুন, incident লগে লিখুন।
3. Milestone 3 — Simulated Incident 2: Disk pressure — `df` / `du` দিয়ে কারণ খুঁজে নিরাপদ cleanup করুন।
4. Milestone 4 — Simulated Incident 3: Resource pressure — `top` / `vmstat` দিয়ে CPU/RAM bottleneck শনাক্ত করে দায়ী প্রসেস সামলান।
5. Milestone 5 — Simulated Incident 4: Access issue — একজন user SSH-এ ঢুকতে পারছেন না; permission, firewall ও `auth.log` দেখে মেরামত করুন।
6. Milestone 6 — Backup ও Recovery: একটি ফাইল মুছে backup থেকে restore করে প্রমাণ করুন; recovery সময় লিখুন।
7. Milestone 7 — Final Report: পুরো ঘটনা, প্রতিটি incident-এর root cause, মেরামত, ভবিষ্যতে এড়ানোর উপায় ও maintenance checklist।

Deliverables: (১) Baseline health রিপোর্ট; (২) Incident log (৪টি incident, প্রতিটিতে লক্ষণ, তদন্তের ধাপ, root cause, মেরামত, যাচাই); (৩) Backup/restore প্রমাণ; (৪) Final report — maintenance checklist সহ।

Acceptance Criteria: চারটি incident-এর প্রতিটির root cause log-এর প্রমাণসহ লেখা; প্রতিটি মেরামতের পর service `curl` /login দিয়ে যাচাই করা; backup থেকে restore সফল; final report-এ ভবিষ্যৎ প্রতিরোধের ব্যবস্থা (যেমন log rotation, monitoring, backup যাচাই) লেখা; কোনো ধ্বংসাত্মক কমান্ডের প্রমাণ নেই।

Verification Checklist:

- Baseline ও final health রিপোর্ট তুলনা করা যায় কি?
- প্রতিটি incident-এ `journalctl` /log-এর উদ্ধৃতি আছে কি?
- nginx ও custom service শেষে active কি?
- ডিস্ক ব্যবহার নিরাপদ সীমায় (80%-এর নিচে) কি?
- SSH login পরীক্ষা সফল কি?
- Restore করা ফাইলের checksum/বিষয়বস্তু মিলেছে কি?

Evaluation Rubric:

মানদণ্ড	Excellent	Good	Needs Work
		কারণ পাওয়া গেছে, প্রমাণ আংশিক	

Troubleshooting পদ্ধতি	ধাপে ধাপে, log-প্রমাণসহ, অনুমান নয়		অনুমান বা বারবার restart
Root cause	৪টি incident-এর সঠিক মূল কারণ	৩টি সঠিক	২ বা তার কম
Backup ও recovery	Restore পরীক্ষাসহ, সময় লেখা	Restore হয়েছে, সময়/প্রমাণ আংশিক	Restore হয়নি
Final report	সম্পূর্ণ, প্রতিরোধের ব্যবস্থাসহ	সম্পূর্ণ, প্রতিরোধ আংশিক	অসম্পূর্ণ

Optional Advanced Challenges: প্রতিটি incident-এর জন্য একটি করে "যদি এটি production-এ রাত ৩টায় হতো" — প্রথম ১৫ মিনিটের action plan লেখা; health-check স্ক্রিপ্টে disk 80% ছাড়ালে সতর্কবার্তা যোগ করা; backup-এর একটি কপি VM-এর বাইরে (যেমন হোস্টে) রাখার পরিকল্পনা লেখা।

পরিশিষ্ট D — Final Examination (চূড়ান্ত পরীক্ষা)

এই পরীক্ষা বইয়ের সব অংশের সমন্বিত মূল্যায়ন। সময়: ৩ ঘণ্টা (Practical ১০টি কাজ: ৯০ মিনিট; Short-answer: ৪৫ মিনিট; Troubleshooting case: ৪৫ মিনিট)। VM-এ practical অংশ করবেন; উত্তরপত্রে প্রতিটি কাজের কমান্ড ও ফলাফল লিখুন। পাস মানে শুধু নম্বর নয় — নিচের rubric-এ "Genuine Understanding" স্তরে পৌঁছানো।

> **নিয়ম:** Practical কাজে snapshot নিন। কোনো কাজে আটকে গেলে পরের কাজে যান, শেষে ফিরে আসুন। Short-answer-এ শুধু সংজ্ঞা নয়, "কেন" ব্যাখ্যা করুন।

Practical Final Exam — ১০টি কাজ (প্রতিটি ৫ নম্বর, মোট ৫০)

1. VM-এ login করে `system-info.txt` তৈরি করুন: Ubuntu version, kernel, CPU, RAM, disk, hostname, IP, uptime — প্রতিটির কমান্ডসহ।
2. `~/exam`-এ ৫টি ফাইল তৈরি করে `grep` ও `awk` দিয়ে একটি ফাইল থেকে নির্দিষ্ট pattern-এর লাইন বের করে `result.txt`-এ লিখুন।
3. `examuser` নামে user ও `examteam` group তৈরি করে `/srv/exam` ডিরেক্টরিতে 2770 permission দিন; `ls -ld` ও `id` দিয়ে যাচাই করুন।
4. `examuser`-কে শুধু `/srv/exam`-তে ACL দিয়ে rwx দিন (group সদস্য না করে) ও `getfacl` দিয়ে দেখান।
5. `curl` ইনস্টল আছে কি না দেখুন, না থাকলে ইনস্টল করুন; `apt-cache policy curl` ও `dpkg -l | grep curl` output লিখুন।
6. একটি সহজ systemd service তৈরি করুন যা প্রতি মিনিটে একটি ফাইলে তারিখ লেখে; `systemctl status` ও `journalctl` দিয়ে চালু প্রমাণ করুন।
7. `df -h`, `du -sh /var/*`, `free -h` চালিয়ে সবচেয়ে বড় ডিরেক্টরি ও available RAM চিহ্নিত করে এক অনুচ্ছেদে ব্যাখ্যা করুন।
8. nginx ইনস্টল করে চালু করুন, `curl -I http://localhost` দিয়ে 200 OK প্রমাণ করুন, তারপর ইচ্ছাকৃতভাবে configuration ভুল করে `nginx -t` দিয়ে ত্রুটি দেখান ও মেরামত করুন।
9. UFW চালু করে শুধু SSH ও HTTP অনুমতি দিন; `ufw status verbose` ও `ss -tulpn` দিয়ে যাচাই করুন।
10. `/srv/exam`-এর tar backup নিয়ে checksum লিখুন, একটি ফাইল মুছে backup থেকে restore করে প্রমাণ করুন।

Short-Answer — ১০টি প্রশ্ন (প্রতিটি ৩ নম্বর, মোট ৩০)

1. Program, process ও thread-এর পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন।
2. Kernel space ও user space আলাদা কেন? System call কীভাবে তাদের সংযুক্ত করে?
3. Virtual memory কী? RAM কম থাকলে swap কী করে, আর কেন swap বেশি ব্যবহার খারাপ লক্ষণ?
4. `chmod 754` বলতে কী বোঝায়? Owner, group ও other প্রত্যেকে কী করতে পারবে?
5. Hard link ও symbolic link-এর পার্থক্য কী? মূল ফাইল মুছলে প্রতিটিতে কী হয়?
6. systemd-তে `enable` ও `start`-এর পার্থক্য কী? `daemon-reload` কখন দরকার?
7. Load average 4.0 বলতে কী বোঝায়? ২-core ও ৮-core সার্ভারে এর অর্থ কি একই?

8. APT-তে `update` ও `upgrade`-এর পার্থক্য কী? `update` না করে `upgrade` করলে কী সমস্যা হতে পারে?
9. SSH key-based login password login-এর চেয়ে নিরাপদ কেন? Private key-এর permission 600 হওয়া কেন জরুরি?
10. Backup নেওয়া হয়েছে — তবু restore পরীক্ষা না করলে কেন তা নিরাপদ নয়? একটি উদাহরণ দিন।

Troubleshooting Case Studies — ৫টি কেস (প্রতিটি ৪ নম্বর, মোট ২০)

1. **Case 1 — Service down:** `systemctl status nginx`-তে `failed` দেখা যাচ্ছে। আপনি কোন ক্রমে কী কী পরীক্ষা করবেন? প্রতিটি ধাপে কী দেখলে কী সিদ্ধান্ত নেবেন, লিখুন।
2. **Case 2 — Disk full:** অ্যাপ লিখতে পারছে না, `df -h`-তে `/` 100%। `df -i` স্বাভাবিক। আপনি কীভাবে দায়ী জায়গা খুঁজবেন ও নিরাপদে জায়গা খালি করবেন? কী মুছবেন না, কেন?
3. **Case 3 — High CPU:** `uptime`-এ load average 6.0 (২-core সার্ভার), `top`-এ একটি প্রসেস 180% CPU খাচ্ছে। আপনি কী করবেন — প্রথমে কী দেখবেন, কখন `kill` করবেন, কখন করবেন না?
4. **Case 4 — Permission denied:** একজন developer `/srv/technova/shared/report.txt` সম্পাদনা করতে পারছেন না — `Permission denied`। `ls -l`, `id`, `getfacl` দিয়ে কীভাবে কারণ খুঁজবেন? সম্ভাব্য তিনটি ভিন্ন কারণ লিখুন।
5. **Case 5 — SSH lockout:** UFW চালু করার পর SSH-এ ঢোকা যাচ্ছে না। VM console আছে। আপনি কীভাবে তদন্ত করে ফিরে আসবেন, আর ভবিষ্যতে এই ভুল কীভাবে এড়াবেন?

Assessment Rubric — আসল বোঝাপড়া কীভাবে মাপবেন

মোট ১০০ নম্বর: Practical ৫০ + Short-answer ৩০ + Troubleshooting ২০। পাস: ৬০+। কিন্তু নম্বরের পাশাপাশি স্তর দেখুন — নিচের স্তরে না পৌঁছালে বিষয়টি আবার পড়ুন।

স্তর	বৈশিষ্ট্য	আপনি কোথায় আছেন?
Memorisation (মুখস্থ)	কমান্ডের নাম মনে আছে, কিন্তু কেন তা করে বলতে পারেন না; output পড়ে অর্থ বুঝতে পারেন না	Short-answer-এ "কেন" লিখতে না পারলে এই স্তরে আছেন — অধ্যায়ের theory আবার পড়ুন
Application (প্রয়োগ)	নির্দেশনা অনুযায়ী ল্যাব করতে পারেন; output-এর অর্থ ব্যাখ্যা করতে পারেন	Practical ৩৫+ পেলে এই স্তরে আছেন
Genuine Understanding (আসল বোঝাপড়া)	নতুন সমস্যায় নিজে তদন্তের ক্রম সাজাতে পারেন; ভুল হলে log পড়ে কারণ খুঁজতে পারেন; নিরাপত্তা ও ঝুঁকি বিবেচনা করে সিদ্ধান্ত নিতে পারেন	Troubleshooting-এ ১৪+ ও Practical-এ ৪০+ পেলে এই স্তরে আছেন — আপনি ছোট সার্ভারের দায়িত্ব নিতে প্রস্তুত
Professional Judgement (পেশাদার বিচার)	কী করবেন না, তা জানেন; backup/restore পরীক্ষা ছাড়া দাবি করেন না; snapshot ছাড়া ঝুঁকিপূর্ণ কাজ করেন না; অন্যকে বুঝিয়ে বলতে পারেন	Project C-র rubric-এ Excellent পেলে এই স্তরের দিকে এগোচ্ছেন — এটি বই পড়ে নয়, বারবার অনুশীলন করে আসে

> **মনে রাখবেন:** এই বই পড়া শেষ মানেই expert হওয়া নয়। Expert হওয়া মানে — ভুল করা, log পড়ে শেখা, আবার করা। এই পরীক্ষায় কম নম্বর পেলে হতাশ হবেন না; কোন স্তরে আটকে আছেন তা চিহ্নিত করে সেই অধ্যায় ও ল্যাবে ফিরে যান।

পরিশিষ্ট E — Glossary (শব্দকোষ)

ইংরেজি শব্দ অনুযায়ী বর্ণানুক্রমিক। প্রতিটি শব্দের বাংলা অর্থ সংক্ষেপে দেওয়া হয়েছে; বিস্তারিত ব্যাখ্যার জন্য সংশ্লিষ্ট অধ্যায় দেখুন।

Term	বাংলা অর্থ / সংজ্ঞা	অধ্যায়
Access Control List (ACL)	নির্দিষ্ট user বা group-কে ফাইলে অতিরিক্ত অধিকার দেওয়ার তালিকা; সাধারণ permission-এর বাইরে সূক্ষ্ম নিয়ন্ত্রণ	১২
APT (Advanced Package Tool)	Debian/Ubuntu-তে package ইনস্টল, আপডেট ও অপসারণের টুল; repository থেকে package আনে	১৪
Archive	একাধিক ফাইল একত্র করে একটি ফাইলে সংরক্ষণ; tar দিয়ে করা হয়	১০, ২০
Backup	তথ্যের অনুলিপি সংরক্ষণ, যাতে হারিয়ে গেলে ফিরিয়ে আনা যায়	২০
BIOS	কম্পিউটার চালু হওয়ার সময় hardware পরীক্ষা ও bootloader খোঁজার প্রাথমিক firmware	০৭
Binary	০ ও ১ দিয়ে প্রকাশিত তথ্য; কম্পিউটারের মৌলিক ভাষা	০১
Block Device	ডিস্কের মতো device, যেখানে তথ্য block আকারে পড়া-লেখা হয়	১৬
Bootloader	কম্পিউটার চালুর সময় kernel load করা ছোট প্রোগ্রাম; যেমন GRUB	০৭
Boot Process	Power on থেকে login prompt পর্যন্ত ধাপগুলো: firmware, bootloader, kernel, init/systemd	০৭
Cache	ঘন ঘন ব্যবহৃত তথ্য দ্রুত পাওয়ার জন্য রাখা অস্থায়ী, দ্রুত স্মৃতি; CPU cache, disk cache	০২
Checksum	ফাইলের বিষয়বস্তু থেকে হিসাব করা সংক্ষিপ্ত মান; ফাইল বদলেছে কি না যাচাই করতে ব্যবহৃত হয়	২০
CLI (Command Line Interface)	কমান্ড টাইপ করে কম্পিউটার চালানোর উপায়; terminal ও shell-এর মাধ্যমে	০৯
Clock Speed	CPU প্রতি সেকেন্ডে কত চক্র কাজ করে, GHz এককে মাপা হয়	০২
Command	Shell-কে দেওয়া নির্দেশ; যেমন ls, cd	০৯
Context Switch	CPU এক প্রসেস থেকে আরেক প্রসেসে কাজ বদলানোর প্রক্রিয়া	০৪, ০৫
Core	CPU-র ভেতরে স্বাধীনভাবে কাজ করা একটি processing unit	০২
CPU (Central Processing Unit)	কম্পিউটারের মূল প্রসেসর; নির্দেশনা কার্যকর করে	০২
Daemon	পেছনে চলমান background service প্রসেস; যেমন sshd, nginx	১৫

Dependency	একটি package চালাতে যেসব অন্য package দরকার	১৪
Device Driver	Kernel-কে নির্দিষ্ট hardware ব্যবহার করতে শেখানো সফটওয়্যার	০৪
Directory	ফাইল ও অন্য ডিরেক্টরি ধারণকারী ফোল্ডার	০৮
Disk Usage	ডিস্কের কতটুকু ব্যবহৃত ও কতটুকু ফাঁকা, তার হিসাব	১৬
Distribution (Distro)	Linux kernel + GNU টুল + package manager + configuration-এর সম্পূর্ণ প্যাকেজ; যেমন Ubuntu, Fedora	০৬
dpkg	Debian/Ubuntu-র নিম্ন-স্তরের package টুল; .deb ফাইল পরিচালনা করে	১৪
Environment Variable	Shell ও প্রোগ্রামের জন্য সংরক্ষিত নাম-মান জোড়া; যেমন PATH, HOME	০৯
Execute Permission	ফাইল চালানো বা ডিরেক্টরিতে ঢোকার অধিকার (x)	১১
File Descriptor	খোলা ফাইল বা I/O চ্যানেলের সংখ্যাসূচক পরিচয় (0 stdin, 1 stdout, 2 stderr)	০৯
File System	ডিস্কে ফাইল সংরক্ষণ ও খোঁজার নিয়ম; যেমন ext4	০৩, ০৮
Filesystem Hierarchy Standard (FHS)	Linux-এ ডিরেক্টরি কাঠামোর মানদণ্ড: /etc, /var, /home ইত্যাদির ভূমিকা	০৮
Firewall	নেটওয়ার্ক traffic নিয়ন্ত্রণকারী ব্যবস্থা; কোন port খোলা থাকবে তা ঠিক করে	১৯
Firmware	Hardware-এর ভেতরে স্থায়ীভাবে থাকা নিম্ন-স্তরের সফটওয়্যার	০৭
Foreground Process	Terminal-এ সরাসরি চলা প্রসেস, যা শেষ না হলে prompt ফেরে না	১৩
Free Memory	ব্যবহৃত নয় এমন RAM; available-এর সাথে গুলিয়ে ফেলবেন না	১৭
fstab	/etc/fstab ফাইল; boot-এ কোন filesystem কোথায় mount হবে তার তালিকা	১৬
GID (Group ID)	প্রতিটি group-এর সংখ্যাসূচক পরিচয়	১১
GNU	Free software প্রকল্প, যার টুলগুলো (bash, gcc) Linux distribution-এর অংশ	০৬
GRUB	Linux-এ বহুল ব্যবহৃত bootloader	০৭
Hard Link	একই inode-র আরেকটি নাম; মূল ফাইল মুছলেও তথ্য থাকে	০৮
Heap	প্রসেসের memory-র অংশ, যেখানে চলাকালীন বরাদ্দ করা তথ্য থাকে	০৫
Home Directory	প্রতিটি user-এর ব্যক্তিগত ডিরেক্টরি; যেমন /home/nahid	০৮
htop	Interactive process ও resource পর্যবেক্ষণ টুল; top-এর উন্নত সংস্করণ	১৩, ১৭
Incremental Backup	শুধু বদলে যাওয়া তথ্যের backup; পূর্ণ backup-এর পর নেওয়া হয়	২০
Inode	Filesystem-এ প্রতিটি ফাইলের metadata (মালিক, permission, অবস্থান) সংরক্ষণকারী কাঠামো	১৬

Input/Output (I/O)	কম্পিউটার ও বাইরের জগতের (ডিস্ক, নেটওয়ার্ক, keyboard) মধ্যে তথ্য আদান-প্রদান	০১, ০২
Interrupt	Hardware বা software থেকে CPU-কে পাঠানো জরুরি সংকেত	০৪
IP Address	নেটওয়ার্কে একটি device-এর সংখ্যাসূচক ঠিকানা	০৭
Journal	systemd-র কেন্দ্রীয় log সংগ্রহ; journalctl দিয়ে পড়া হয়	১৫, ১৮
journalctl	systemd journal থেকে log দেখার কমান্ড	১৫, ১৮
Kernel	Operating system-এর মূল অংশ; hardware, process ও memory পরিচালনা করে	০৩, ০৪
Kernel Space	Kernel-এর জন্য সংরক্ষিত সুরক্ষিত memory অঞ্চল; সাধারণ প্রোগ্রাম সরাসরি ঢুকতে পারে না	০৪
Kill Signal	প্রসেসকে থামাতে বা আচরণ বদলাতে পাঠানো সংকেত; যেমন SIGTERM (15), SIGKILL (9)	১৩
Least Privilege	প্রতিটি user বা প্রোগ্রামকে শুধু তার কাজের জন্য যতটুকু অধিকার দরকার, ততটুকুই দেওয়ার নীতি	১২, ১৯
Load Average	নির্দিষ্ট সময়ে CPU-র জন্য অপেক্ষমাণ প্রসেসের গড় সংখ্যা; সিস্টেমের চাপের সূচক	১৭
Log	সিস্টেম ও অ্যাপের ঘটনার লিখিত বিবরণ; তদন্তের প্রধান উৎস	১৮
Log Rotation	পুরোনো log ফাইল সংকুচিত বা মুছে নতুন ফাইল শুরু করার ব্যবস্থা	১৮
Man Page	Linux কমান্ড ও configuration-এর আনুষ্ঠানিক নির্দেশিকা; man কমান্ড দিয়ে পড়া হয়	০৯
Mount	একটি filesystem-কে ডিরেক্টরি গাছের একটি জায়গায় যুক্ত করা	০৮, ১৬
Mount Point	যে ডিরেক্টরিতে একটি filesystem mount করা হয়	০৮, ১৬
Multi-user Target	systemd-র একটি target; নেটওয়ার্কসহ সাধারণ multi-user অবস্থা	১৫
Nice Value	প্রসেসের scheduling অগ্রাধিকারের মান; কম মান মানে বেশি অগ্রাধিকার	১৩
NVMe	দ্রুত SSD-র জন্য তৈরি storage interface; SATA SSD-র চেয়ে অনেক দ্রুত	০২
Open Source	যে সফটওয়্যারের source code সবার জন্য উন্মুক্ত ও পরিবর্তনযোগ্য	০৬
Operating System (OS)	Hardware ও application-এর মধ্যে কাজ করা সফটওয়্যার; resource পরিচালনা করে	০৩
Orphan Process	যার parent প্রসেস শেষ হয়ে গেছে, কিন্তু নিজে চলছে এমন প্রসেস	০৫

Package	ইনস্টলযোগ্য সফটওয়্যারের সংকলিত একক; যেমন .deb ফাইল	১৪
Package Manager	Package ইনস্টল, আপডেট ও অপসারণের ব্যবস্থা; যেমন APT	১৪
Page / Paging	Virtual memory-র নির্দিষ্ট আকারের খণ্ড; RAM ও ডিস্কের মধ্যে আদান-প্রদান হয়	০৫
Partition	ডিস্কের আলাদা করা অংশ; প্রতিটি অংশে আলাদা filesystem থাকতে পারে	০৭, ১৬
PATH	Shell কমান্ড খোঁজার ডিরেক্টরিগুলোর তালিকা; environment variable	০৯
Permission	ফাইল বা ডিরেক্টরিতে কে পড়তে, লিখতে বা চালাতে পারবে, তার নিয়ম (rwx)	১১
PID (Process ID)	প্রতিটি চলমান প্রসেসের অনন্য সংখ্যাসূচক পরিচয়	০৫, ১৩
Pipe (|)	একটি কমান্ডের output আরেকটি কমান্ডের input হিসেবে পাঠানোর ব্যবস্থা	০৯
Port	নেটওয়ার্কে একটি service-এর সংখ্যাসূচক দরজা; যেমন SSH 22, HTTP 80	১৯
Process	চলমান প্রোগ্রাম; memory, PID ও অবস্থাসহ একটি সক্রিয় সত্তা	০৫, ১৩
RAM (Random Access Memory)	চলমান প্রোগ্রাম ও তথ্যের জন্য ব্যবহৃত দ্রুত, অস্থায়ী স্মৃতি	০২
Read Permission	ফাইলের বিষয়বস্তু পড়ার অধিকার (r)	১১
Redirection	কমান্ডের input/output ফাইল বা অন্য জায়গায় পাঠানো (>, >>, <)	০৯
Register	CPU-র ভেতরে অত্যন্ত দ্রুত, ছোট স্মৃতি; গণনার সময় তথ্য রাখে	০২
Repository	Package সংরক্ষণের অনলাইন ভাণ্ডার; APT সেখান থেকে package আনে	১৪
Restore	Backup থেকে তথ্য ফিরিয়ে আনার প্রক্রিয়া	২০
Root (User)	Linux-এ সর্বোচ্চ অধিকারসম্পন্ন user (UID 0); সবকিছু করতে পারে	১১
Root Directory (/)	Filesystem গাছের শীর্ষ ডিরেক্টরি	০৮
rsync	দুটি জায়গার মধ্যে শুধু বদলে যাওয়া অংশ কপি করে sync করার টুল; backup-এ ব্যবহৃত হয়	২০
Scheduling	CPU কোন প্রসেসকে কখন চালাবে, তা ঠিক করার প্রক্রিয়া	০৫
Script	একাধিক কমান্ড এক ফাইলে লিখে চালানোর ব্যবস্থা; যেমন Bash script	০৯
Service	পেছনে চলমান প্রোগ্রাম, যা নির্দিষ্ট কাজ দেয়; systemd unit দিয়ে পরিচালিত হয়	১৫
Setgid	ডিরেক্টরিতে দিলে নতুন ফাইল ডিরেক্টরির group পায়; ফাইলে দিলে group অধিকারে চলে	১১
Setuid	ফাইলে দিলে প্রোগ্রাম ফাইলের মালিকের অধিকারে চলে; সতর্কতার সাথে ব্যবহার করতে হয়	১১

Shell	ব্যবহারকারীর কমান্ড পড়ে kernel-কে নির্দেশ দেওয়া প্রোগ্রাম; যেমন Bash	০৯
Signal	প্রসেসকে পাঠানো বার্তা; যেমন থামা, আবার শুরু করা	১৩
Snapshot	VM বা সিস্টেমের একটি সময়ের সম্পূর্ণ অবস্থার সংরক্ষিত অনুলিপি; ফিরে আসার জন্য	০৭
SSD (Solid State Drive)	Flash memory-ভিত্তিক দ্রুত storage; চলমান অংশ নেই	০২
SSH (Secure Shell)	দূরবর্তী সার্ভারে নিরাপদে (encrypted) login ও কমান্ড চালানোর protocol	১৯
Stack	প্রসেসের memory-র অংশ; function call ও স্থানীয় variable রাখে	০৫
Standard Error (stderr)	ত্রুটির বার্তার জন্য আলাদা output চ্যানেল (file descriptor 2)	০৯
Standard Input (stdin)	কমান্ডের input চ্যানেল (file descriptor 0)	০৯
Standard Output (stdout)	কমান্ডের সাধারণ output চ্যানেল (file descriptor 1)	০৯
Sticky Bit	ডিরেক্টরিতে দিলে শুধু ফাইলের মালিকই তার ফাইল মুছতে পারে; যেমন /tmp	১১
Storage Hierarchy	Register, cache, RAM, SSD/HDD — গতি ও আকার অনুযায়ী স্মৃতির স্তরবিন্যাস	০২
sudo	সাধারণ user-কে সীমিতভাবে root অধিকারে কমান্ড চালানোর অনুমতি দেওয়ার ব্যবস্থা	১২
sudoers	/etc/sudoers ফাইল; কে কোন কমান্ড sudo দিয়ে চালাতে পারবে, তার নিয়ম	১২
Swap	RAM ভরে গেলে ডিস্কের অংশ ব্যবহার করে memory বাড়ানোর ব্যবস্থা; RAM-এর চেয়ে ধীর	০২, ১৭
Symbolic Link (Symlink)	আরেকটি ফাইল বা ডিরেক্টরির দিকে নির্দেশ করা বিশেষ ফাইল	০৮
Syscall (System Call)	প্রোগ্রাম থেকে kernel-এর সেবা চাওয়ার নিয়ন্ত্রিত উপায়; যেমন read, write	০৪
systemd	আধুনিক Linux-এর init system ও service manager; boot, service ও log পরিচালনা করে	০৭, ১৫
Terminal	কমান্ড টাইপ ও output দেখার text interface	০৯
Thread	একটি প্রসেসের ভেতরে স্বাধীনভাবে চলা ছোট কাজের একক; memory ভাগ করে নেয়	০৫
UFW (Uncomplicated Firewall)	Ubuntu-র সহজ firewall টুল; iptables/nftables-এর উপরে কাজ করে	১৯
UID (User ID)	প্রতিটি user-এর সংখ্যাসূচক পরিচয়	১১
umask	নতুন ফাইল ও ডিরেক্টরির ডিফল্ট permission থেকে কী বাদ যাবে, তা ঠিক করা মান	১১
Unit (systemd)	systemd-র পরিচালনার একক; service, timer, target ইত্যাদি	১৫

Uptime	সিস্টেম শেষ reboot-এর পর থেকে কতক্ষণ চলছে	১৭
User Space	সাধারণ প্রোগ্রামগুলো চলার memory অঞ্চল; kernel space থেকে আলাদা	০৪
Virtual Machine (VM)	সফটওয়্যার দিয়ে তৈরি কৃত্রিম কম্পিউটার; হোস্টের ভেতরে আলাদা OS চালায়	০৭
Virtual Memory	প্রতিটি প্রসেসকে দেওয়া আলাদা ঠিকানার জগৎ; RAM ও ডিস্ক মিলিয়ে কাজ করে	০২, ০৫
Write Permission	ফাইল বদলানো বা ডিরেক্টরিতে ফাইল তৈরি/মোছার অধিকার (w)	১১
Zombie Process	শেষ হয়ে গেছে, কিন্তু parent এখনো তার অবস্থা পড়েনি বলে তালিকায় রয়ে গেছে এমন প্রসেস	০৫

পরিশিষ্ট F — References (তথ্যসূত্র)

নিচের সব সূত্রই আনুষ্ঠানিক (official) প্রকল্পের ডকুমেন্টেশন। কোনো সূত্র বানানো হয়নি। URL-গুলো ৮ নভেম্বর ২০২৬-এ যাচাইকৃত আনুষ্ঠানিক ডোমেইনে নির্দেশ করে; কোনো নির্দিষ্ট পৃষ্ঠা বদলে গেলে সংশ্লিষ্ট প্রকল্পের ডকুমেন্টেশন সূচি থেকে খুঁজে নিন।

> **তিন ধরনের তথ্যের পার্থক্য:** (১) **Tested facts** — এই বইয়ের ল্যাবে Ubuntu Server 24.04 LTS-এ হাতে চালিয়ে যাচাই করা কমান্ড ও আচরণ; (২) **General principles** — Linux ও computer systems-এর স্থায়ী ধারণা (যেমন kernel/user space, permission মডেল), যা সংস্করণ বদলালেও মূলত একই থাকে; (৩) **Version-dependent behaviour** — package সংস্করণ, kernel সংস্করণ, ডিফল্ট configuration — এগুলো Ubuntu 24.04 LTS-এর জন্য লেখা, অন্য সংস্করণে ভিন্ন হতে পারে। সন্দেহ হলে নিজের সিস্টেমে `man` ও আনুষ্ঠানিক ডকুমেন্টেশন দেখুন।

Ubuntu ও Debian

- Ubuntu Server Documentation — https://ubuntu.com/server/docs
- Ubuntu 24.04 LTS Release Notes — https://discourse.ubuntu.com/t/noble-numbat-release-notes/42726
- Ubuntu Installation Guide — https://ubuntu.com/server/docs/installation
- Ubuntu Security Documentation — https://ubuntu.com/security
- Debian Administrator's Handbook — https://debian-handbook.info/browse/stable/
- Debian Documentation — https://www.debian.org/doc/

Linux Kernel ও GNU

- The Linux Kernel Documentation — https://www.kernel.org/doc/html/latest/
- The Linux Kernel Archives — https://www.kernel.org/
- GNU Operating System Documentation — https://www.gnu.org/doc/
- GNU Bash Reference Manual — https://www.gnu.org/software/bash/manual/
- GNU Coreutils Manual — https://www.gnu.org/software/coreutils/manual/

systemd

- systemd Documentation — https://systemd.io/
- systemd Manual Pages — https://www.freedesktop.org/software/systemd/man/latest/
- systemctl Manual — https://www.freedesktop.org/software/systemd/man/latest/systemctl.html
- journalctl Manual — https://www.freedesktop.org/software/systemd/man/latest/journalctl.html

Man Pages ও Command Reference

- Linux Man Pages (man7.org) — https://man7.org/linux/man-pages/
- Ubuntu Manpages — https://manpages.ubuntu.com/
- Linux Documentation Project — https://tldp.org/

Fedora / RHEL পরিবার ও অন্যান্য

- Fedora Documentation — https://docs.fedoraproject.org/
- Red Hat Enterprise Linux Documentation — https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9
- Red Hat System Administration Guide — https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_basic_system_settings/
- Arch Wiki (সাধারণ Linux ধারণার জন্য সহায়ক) — https://wiki.archlinux.org/

নিরাপত্তা ও অন্যান্য আনুষ্ঠানিক সূত্র

- OpenSSH Manual Pages — https://www.openssh.com/manual.html
- UFW Documentation (Ubuntu) — https://help.ubuntu.com/community/UFW
- Ubuntu Server Guide: Security — https://ubuntu.com/server/docs/security
- rsync Documentation — https://rsync.samba.org/documentation.html
- VirtualBox User Manual — https://www.virtualbox.org/manual/

একটি সতর্কতা: ব্লগ পোস্ট, ফোরাম উত্তর বা AI-এর দেওয়া কমান্ডকে তথ্যসূত্র হিসেবে ব্যবহার করবেন না — সেগুলো পুরোনো, ভুল বা আপনার সংস্করণের জন্য অনুপযুক্ত হতে পারে। সবসময় উপরের আনুষ্ঠানিক ডকুমেন্টেশন ও নিজের সিস্টেমের `man` পৃষ্ঠাকে চূড়ান্ত মানুন। কোনো কমান্ড বা পদ্ধতি এই বইয়ে যাচাই করা যায়নি বলে মনে হলে, তা পরীক্ষা না করে production সার্ভারে চালাবেন না।

প্রকাশনা তথ্য: DEVOPS ENGINEERING MASTERY, Volume 01 — Computer Systems & Linux Engineering। প্রথম সংস্করণ, প্রকাশ: ৮ নভেম্বর ২০২৬। প্রধান পরিবেশ: Ubuntu Server 24.04 LTS (kernel 6.8 series, systemd, APT)। এই back-matter-এর ল্যাব, অ্যাসাইনমেন্ট, প্রজেক্ট, পরীক্ষা, শব্দকোষ ও তথ্যসূত্র একই সংস্করণের অংশ।